# 100ASK_STM32MP157

Dual-core ARM Cortex-A7 + Cortex-M4
Embedded Development Platform

## M4 User Manual

Rev. 1.1

2020/08/15

0755-86200561
www.100ask.net
Shenzhen, Guangdong, China
Shenzhen 100ask Technology Co. Ltd.

# 注意事项与售后维修

## 1. 注意事项

- 使用产品之前，请仔细阅读本手册，并妥善保管，以备将来参考；
- 请注意和遵循标注在产品上的所有警示和指引信息；
- 请使用配套电源适配器，以保证电压、电流的稳定；
- 请在凉爽、干燥、清洁的地方使用本产品；
- 请勿在冷热交替环境中使用本产品，避免结露损坏元器件；
- 请勿在湿气过重、温度过高或过低环境中使用本产品，使用时注意产品的通风；
- 请勿将任何液体泼溅在本产品上，禁止使用有机溶剂或腐蚀性液体清洗本产品；
- 请勿在多尘、脏乱的环境中使用本产品，如果长期不使用，请包装好本产品；
- 请勿在振动过大的环境中使用，任何跌落、敲打或剧烈晃动都可能损坏线路及元器件；
- 请勿在通电情况下，插拔核心板及外围模块(特别是串口模块)；
- 请勿自行维修、拆解本产品，如产品出现故障应及时联系本公司进行维修；
- 请勿自行修改或使用未经授权的配件，由此造成的损坏将不予保修；

## 2. 售后维修

1) 保修期限

- 底板、核心板：三个月（非人为损坏）
- 显示屏：七天（非人为损坏）

2) 保修说明

- 7 天内：产品（底板、核心板、屏幕）非人为损坏，本公司免费更换/维修，并承担来回运费；
- 7 天至 3 个月内：底板、核心板非人为损坏，本公司免费维修，并承担来回运费（屏幕不提供维修）；
- 3 个月至 1 年：底板、核心板非人为损坏或人为轻微损坏，只收更换元器件费用，免费维修，买家承担来回运费；
- 起始时间以快递签收日为准；

3) 联系方式

官方网站：www.100ask.net
淘宝网站：100ask.taobao.com

地　址：广东省深圳市龙岗区布吉南湾街道平吉大道建昇大厦 B1505
联系人：售后维修部
电　话：0755-86200561
邮　编：518114

邮寄须知：保修期限内，寄回本产品请预先垫付邮费，公司不接收任何到付快递。

# 技术支持与开发定制

## 1. 技术支持范围

1) 本公司提供的各类开发软件的安装，入门使用，环境搭建；
2) 本公司提供的所有裸机代码的烧写验证；
3) 本公司发布的操作系统的编译、烧写；
4) 本公司发布产品的工控板、模块的硬件原理；
5) 本公司发布的各种外设模块驱动及源码；
6) 本公司发布的配套手册在使用过程中遇到的问题；
7) 本公司产品的故障诊断及售后维修服务；

## 2. 技术讨论范围

由于嵌入式系统知识范围广泛，涉猎种类繁多，我们无法保证对各种问题都能一一解答，以下内容无法供技术支持，只能提供建议。

1) 本公司发布的教程之外的知识；
2) 非本公司发布的 U-Boot、Linux 内核的编译和移植；
3) 非本公司发布的工控板的各类驱动支持；
4) 非本公司发布的外设模块的硬件原理和驱动设计；

## 3. 技术支持方式

1) 官方论坛发帖提问(推荐)：bbs.100ask.net
2) 官方淘宝通过阿里旺旺咨询：100ask.taobao.com
3) QQ 群咨询（QQ 群号咨询淘宝客服，需提供淘宝购买订单号验证加入）；
4) 技术支持邮箱：weidongshan@qq.com
5) 联系电话：0755-86200561

## 4. 技术支持时间

星期一到星期五;上午 9:00—12:00;下午 14:00—17:30;

公司按照国家法定节假日安排休息，在此期间无法提供技术支持，请将问题发送至技术支持邮箱或在论坛对应板块发帖，我们将在工作日尽快给您回复。

## 5. 投诉和建议

如果您对我们有不满意或者建议，可发送邮件到 weidongshan@qq.com 进行反馈，也可拨打 0755-86200561 取得联系，我们将不断改进。

## 6. 定制开发服务

本公司提供嵌入式操作系统底层驱动、硬件板卡的有偿定制开发服务，以缩短您的产品开发周期。请将需求发送邮件到 weidongshan@qq.com。

# 资料获取与后续更新

## 1. 资料的获取

1) 百度网盘下载

百度网盘里面有本产品的所有配套资料，包括原理图、发布的 U-Boot、内核镜像和源码、所需的开发软件、工具等等。

进入 www.100ask.net，在导航栏选择“资料下载”，点击“到百度网盘下载”，跳转到百度网盘后，找到对应的文件夹即可。

2) 视频配套教程

后续会为该工控板录制一套裸机、Linux 驱动、应用的配套付费教学视频，有需要的客户可以进入官方淘宝 100ask.taobao.com 选购。

3) 维基百科教程

维基百科里面会有视频配套的笔记，进入 wiki.100ask.net，选择对应的板块查看。

## 2. 后续更新

后续文档、视频等资料的更新，为了确保您的资料是最新状态，请密切关注我们的动态，我们将会通过微信公众号和 QQ 群公告推送，购买了本产品的客户请添加 QQ 群（QQ 群号咨询淘宝客服，需提供淘宝购买订单号验证加入）或关注微信公众号。

# 版权声明

**百问科技©2019**

深圳百问网科技有限公司版权所有，并保留对本手册及声明的一切权力。

未得到本公司的书面许可，任何单位和个人不得以任何方式或形式对本手册内的任何部分进行复制、摘录、备份、修改、传播、翻译成其他语言、将其全部或部分用于商业用途。

# 更新记录

| | |
|---|---|
| **类别** | 100ASK_ STM32MP157 系列文档 |
| **文档名** | 100ASK_ STM32MP157 M4 用户手册 |
| **当前版本** | 1.1 |
| **适用型号** | 100ASK_ STM32MP157 |
| **编辑** | 百问科技文档编辑团队 |
| **审核** | 韦东山 |

| **修改日志** | | |
|---|---|---|
| 版本 | 修改时间 | 更改说明 |
| 1.0 | 2019.10.30 | 初始版本，黄成 |
| 1.1 | 2020.08.15 | 发布版本，黄成 |
| | | |
| | | |
| | | |
| | | |
| | | |

# 目录

注意事项与售后维修 ..... I
技术支持与开发定制 ..... II
资料获取与后续更新 ..... III
版权声明 ..... IV
更新记录 ..... V
目录 ..... 1
前言 ..... 3
第 1 章 开发平台介绍 ..... 4
1.1 固件库 ..... 4
1.2 STM32CubeMX ..... 5
1.3 STM32CubeIDE ..... 6
1.4 STM32Cube 生态 ..... 7
1.5 开发流程 ..... 9
1.6 M4 调试 ..... 33
1.7 工程结构分析 ..... 42
第 2 章 M4 基础开发示例 ..... 50
2.1 资源分配 ..... 50
2.2 闪烁 LED 灯 ..... 53
2.3 系统时钟—HSI ..... 58
2.4 系统时钟—HSE ..... 68
2.5 按键扫描 ..... 73
2.6 按键中断 ..... 78
2.7 串行通信—串口打印 ..... 87
2.8 串行通信—RS485 ..... 98
2.9 DMA—MemoryToMemory ..... 103
2.10 通用定时器—PWM ..... 110
2.11 通用定时器—us 延时 ..... 114
2.12 SysTick 定时器-us 延时 ..... 118
2.13 ADC—使用 DMA ..... 122
2.14 ADC—内部温度 ..... 129
2.15 DAC—软件触发 ..... 132
2.16 I2C—软件模拟 ..... 137
2.17 I2C—硬件模式 ..... 150

2.18 SPI—软件模拟 ..... 153
2.19 SPI—硬件模式 ..... 162
2.20 CAN 通信 ..... 164
2.21 WWDG—窗口看门狗 ..... 179
第 3 章 M4 高级开发示例 ..... 184
3.1 FreeRTOS ..... 184
3.2 A7_M4 用户态通信 ..... 188
3.3 A7_M4 内核态通信 ..... 203

# 前言

STM32 系列 MCU(Microcontroller Unit)在中国的受众非常广，许多高校学生将 STM32 系列 MCU 作为嵌入式入门的首选，许多 MCU 工程师也将 STM32 系列 MCU 作为方案首选。许多用户在对 STM32 系列 MCU 熟悉之后，想进一步学习运行 Linux 的 MPU(Microprocessor Unit)时，不得不选择其它厂商的 MPU，重新熟悉一套开发套件，加大了前期学习的负担。

不含 MMU(Memory Management Unit)的 MCU，主要用于一些对实时性要求较高的场景，比如航空航天、汽车电子等。而带有 MMU 的 MPU，可以通过 MMU 来管理虚拟内存空间，从而可以支持诸如 Linux 的操作系统，打开操作系统的大门。

意法半导体(STMicroelectronics)稳固 STM32 系列在 MCU 市场中的重要地位后，首次以 STM32MP15 平台进入 MPU 市场。该平台采用 MCU+MPU 的方案，旨在为工业制造、消费电子、医疗保健、智能家居等多个市场领域提供高性能、硬实时、低功耗、更安全的解决方案。

在 STM32MP157x 上开发 Cortex-M4（后面简称 M4）的操作流程几乎和开发 STM32 单片机一致。本手册将先介绍开发使用的平台，然后介绍开发流程，最后依次介绍 M4 资源进行基础开发，以及特色的核之间通信等。

# 第1章 开发平台介绍

本章将先对开发软件及其生态介绍，然后介绍开发流程，使用 STM32CubeMX 生成基于 HAL 库工程模板，再使用 STM32CubeIDE 编辑工程代码，最后再介绍如何对 STM32MP157x 的 M4 调试、运行。

## 1.1 固件库

接触过单片机的读者，都应该知道**开发者是通过修改单片机寄存器来实现对单片机的控制**。如果是简单的 51 单片机，寄存器比较少，一般都是直接操作寄存器控制单片机。对于 STM32 系列单片机，至少都几百个寄存器，一个一个操作效率太低。于是 ST 官方就提供了固件库，用户无需再直接操作寄存器，而是使用库里面提供的函数控制外设，极大的提高了开发效率，这也是 STM32 流行的一个重要因素。

STM32 先后推出了两套主要固件库：标准外设库（Standard Peripherals Library）和 HAL 库（Hardware Abstraction Layer）。标准外设库是 STM32 最早发布的固件库，对 STM32 芯片进行了完整的封装，包含所有外设的性能特征，每个外设驱动都由一组函数组成，这组函数覆盖了该外设所有功能。因为发布比较早，教程比较多，目前仍有不少开发者使用。标准外设库是针对某一系列 STM32 芯片而设计的，没有可移植性，目前该库已将停止了更新、研发，最近几年发布的新 STM32 芯片已经不再支持。

HAL 库是 ST 公司这几年极力推荐的官方库，目前支持 STM32 全系产品，可以说 HAL 库就是用来取代之前的标准外设库的。相比标准外设库，HAL 库具有更高的抽象整合水平，API 集中关注外设的公共函数功能，使得可以轻松实现从一个 STM32 产品移植到另一个不同的 STM32 系列产品。此外还有 LL 库（Low Layer），相比其它固件库更接近硬件层，一般和 HAL 库结合使用，可以看作是 HAL 库的补充。寄存器操作、标准固件库、HAL 库和 LL 库之间的差异如表 1.1.1 所示。

表 1.1.1 寄存器操作、标准固件库、HAL 库和 LL 库之间的差异

| | 学习难度 | 使用难度 | 移植性 | 执行效率 | 代码大小 | 支持范围 | 建议 |
|---|---|---|---|---|---|---|---|
| **寄存器操作** | 难 | 难 | 难 | **高** | **小** | 小 | 不推荐 |
| **标准固件库** | **适中** | **适中** | 难 | 适中 | 适中 | 适中 | 不推荐 |
| **STM32Cube(HAL)** | **适中** | **适中** | **容易** | 稍低 | 稍大 | **大** | **推荐** |
| **STM32Cube(LL)** | **适中** | **适中** | 难 | **高** | **小** | 目前小 | 一般 |

有了以上对比，相信大家对 STM32 固件库有了大致的了解。目前 ST 官方只提供 STM32MP157x 的 HAL 库，STM32CubeMX 生成的都是基于 HAL 库工程模板，因此本手册后面将使用 HAL 讲解 M4 的开发。但无论使用哪个固件库，最终都是对寄存器进行操作，因此虽然不建议使用寄存器写代码，但还是必须会查手册、看寄存器。

# 1.2 STM32CubeMX

STM32CubeMX 是 ST 意法半导体推出的 STM32 系列芯片图形可视化配置工具，用户可以通过图形化向导为 Cortex-M 系列 MCU 生成初始化代码工程模板，以及为 Cortex-A 系列 MPU 生成用于 Linux 的部分设备树。

对于 STM32MP157x，可以为整个系统配置 GPIO 和时钟，并以交互的方式将外围设备分配给 Cortex-M4 或 Cortex-A7。并且提供一些实用功能，比如 DDR 配置和调整，使 STM32 MPU 入门变得更加容易。

STM32CubeMX 特性如下：

- 直观的 STM32 系列 MCU 和 MPU 的选型
- 易于使用的丰富图形化用户界面，可以进行以下配置：
  - 自动处理引脚冲突
  - 动态验证 Cortex-M 内核外围设备和中间件功能的参数约束
  - 动态验证配置的时钟树
  - 查看上电顺序、预测功耗
- 生成适合 IAR、Keil 和 GCC 编译器的 Cortex-M4 初始化 C 代码工程项目
- 生成适合 Linux 设备的 Cortex-A 部分设备树
- 支持 Windows、Linux、macOS 上独立运行，或者作为 Eclipse 插件运行

STM32CubeMX 支持的 STM32 如图 1.2.1 所示。

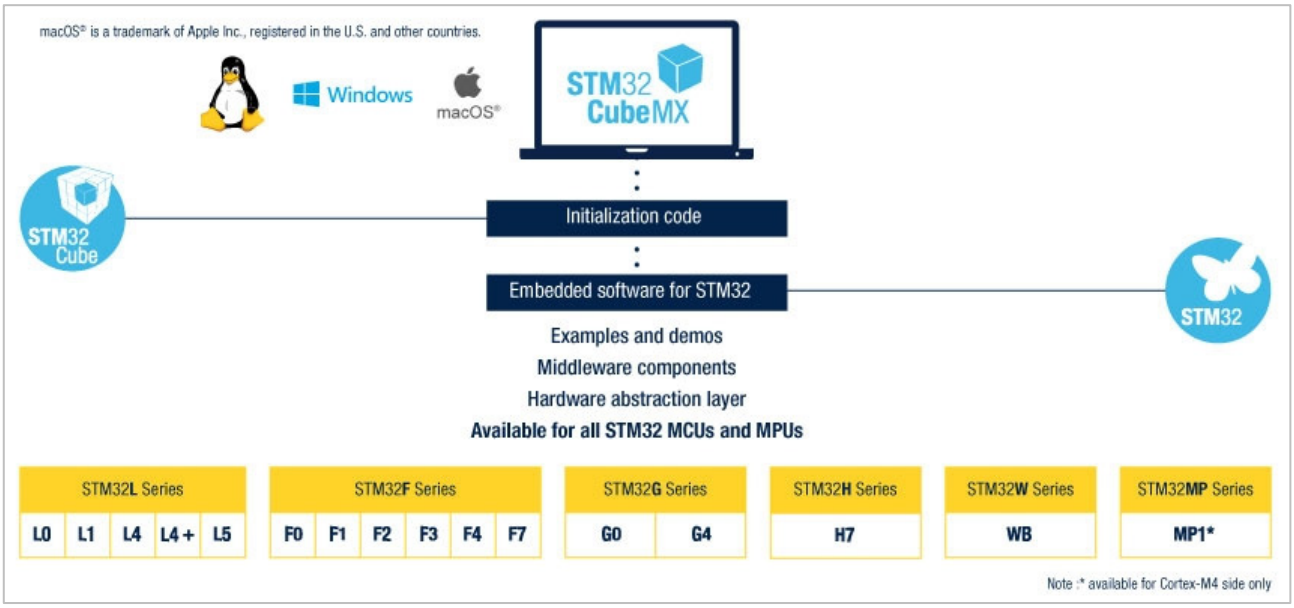


图 1.2.1 STM32CubeMX 支持 STM32 全系列

# 1.3 STM32CubeIDE

支持 STM32 开发的集成开发环境（IDE,Integrated Development Environment）有很多，国内熟知的有 Keil MDK-ARM 和 IAR EWAR。这两个 IDE 都很好用，但它们都是商业软件，免费或评估版要么有器件型号限制，要么有程序容量限制。于是出现了免费的 Eclipse+GNU GCC 来搭建 STM32 开发环境，但搭建过程繁琐、版本差异大导致教程不统一，对新手很不友好。

STM32CubeIDE 是 ST 公司基于 Eclipse/CDT 框架和 GUN GCC 工具链制作的免费 IDE，并集成了 STM32CubeMX。可以实现 STM32 系列芯片的外围设备配置、代码生成、代码编辑、代码编译、在线调试，并且支持数百个 Eclipse 现有插件。**在实际使用体验中，经过数次版本更新，Bug 已经比较少了，至少完成本手册内容，是完全没问题的。除此之外，打开软件比较慢，有时候还会卡一下，期待后续版本优化，做到像 Keil 那么流畅。**

STM32CubeIDE 特性如下：

- 集成 STM32CubeMX，可提供以下服务:
  - STM32 MCU 和 MPU 的选择
  - 引脚分配、时钟、外围设备和中间件配置
  - 项目创建和初始化代码的生成
- 基于 Eclipse/CDT，并支持 Eclipse 插件，使用 ARM GNU C/C++工具链和 GDB 调试器
- 其他高级调试功能，包括：
  - CPU 核心、外设寄存器和内存视图
  - 实时变量观看视图
  - 系统分析和实时跟踪(SWV)
  - CPU 故障分析工具
- 支持 ST-LINK（STMicroelectronics）和 J-Link（SEGGER）调试
- 支持从 Atollic TrueSTUDIO 和 System Workbench for STM32（SW4STM32）导入项目
- 多操作系统支持：Windows、Linux 和 macOS（仅支持 64 位版本）

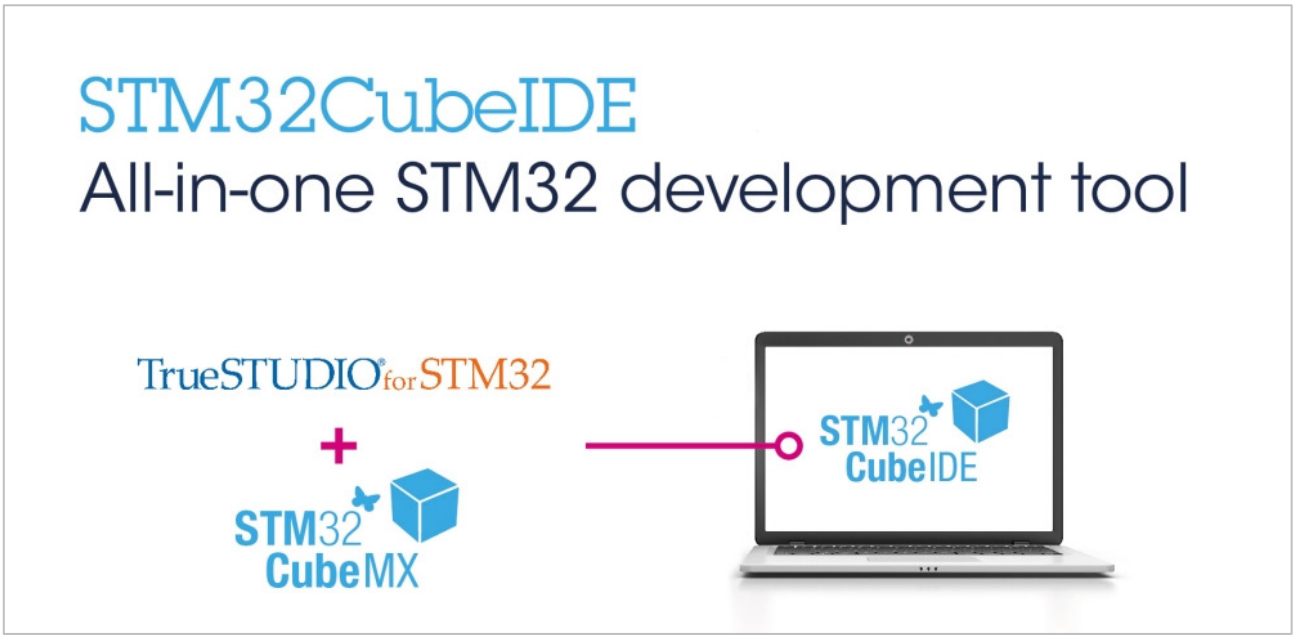


图 1.3.1 多合一 STM32 开发工具——STM32CubeIDE

# 1.4 STM32Cube 生态

最后再聊一聊 STM32Cube 生态。ST 公司最近几年一直在推 STM32Cube，现在基本已经成型，用户可以使用 ST 公司提供的工具、软件包，快速、高效、免费的完成开发。

STM32Cube 生态系统就是**面向 STM32 系列芯片的完整软件解决方案**。软件解决方案包含软件工具套件和嵌入式软件。

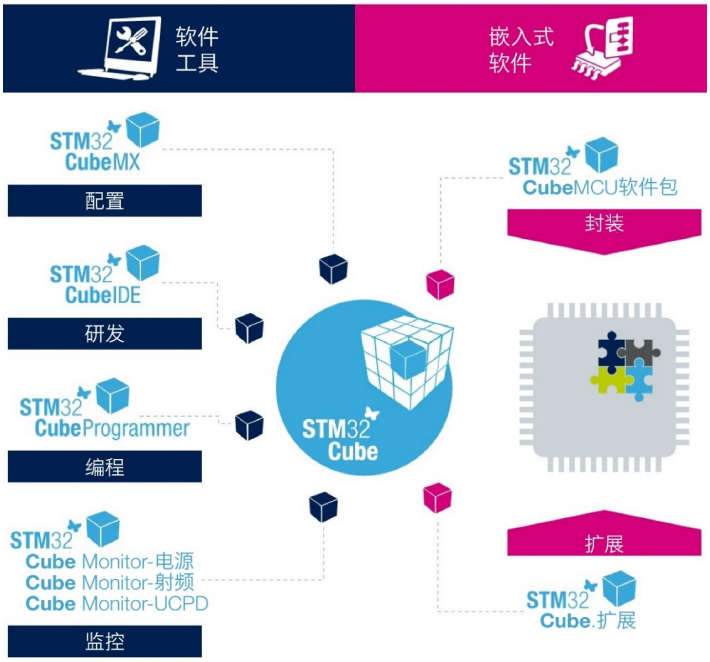


图 1.4.1 STM32Cube 生态系统

STM32Cube 软件工具套件为开发人员提供软件工具，包含:

- STM32CubeMX, 面向所有 STM32 芯片的配置工具。简单易用的图形用户界面为 Cortex-M 内核生成初始化 C 代码，并为 Cortex-A 内核生成 Linux 设备树源码。
- STM32CubeIDE, 一个集成开发环境。该 IDE 基于 Eclipse 或 GNU C/C++工具链等开源解决方案，包括编译报告功能和高级调试功能。它还集成了其他工具，如 STM32CubeMX（本身包含在 STM32CubeIDE 中）。
- STM32CubeProgrammer, 一种编程工具。它通过多种多样可用的通信媒介（JTAG、SWD、UART、USB DFU、I2C、SPI、CAN 等）为读取、写入和验证设备和外部存储器等操作提供简单易用且高效的环境。
- STM32CubeMonitor 系列工具。功能强大的监控工具，可帮助开发人员实时微调其应用程序的行为和性能。

STM32Cube 嵌入式软件帮助开发人员加快项目开发，包含:

- STM32Cube MCU 和 MPU 软件包, 专用于每个 STM32 系列产品，提供操作可用的 STM32 外设集所需的全部嵌入式软件模块。它们包括驱动程序（HAL、底层等）、中间件，以及各种实际用例中使用的大量示例代码。
- STM32Cube 扩展包, 适用于面向应用的解决方案。STM32 扩展包来自 ST 或核准的合作伙伴，通过附加的嵌入式软件模块对 STM32Cube MCU 软件包进行补充和扩展，旨在围绕 STM32 创建广泛的可扩展嵌入式软件产品。

可以看出，ST 公司已经针对 STM32 系列芯片提供了完整解决方案。STM32CubeMX 完成芯片配置和工程代码初始化。STM32CubeIDE 完成代码编辑、功能实现。STM32CubeProgrammer 完成代码烧写。STM32CubeMonitor 完成监控、功能验证。

图 1.4.2 STM32Cube 软件工具完整解决方案

STM32Cube 软件工具套件支持功能列表。

| 产品编号 | 配置 | 初始化代码生成 | 代码编辑 | 代码编译 | 调试 | 二进制编程 | 监控 |
|---|---|---|---|---|---|---|---|
| STM32CubeMX | ✔ | ✔ | | | | | |
| STM32CubeIDE | ✔ (集成STM32CubeMX) | ✔ (集成STM32CubeMX) | ✔ | ✔ | ✔ | ✔ | |
| STM32CubeProgrammer | | | | | | ✔ | |
| STM32CubeMonitor-RF | | | | | | | ✔ |
| STM32CubeMonitor-UCPD | | | | | | | ✔ |
| STM32CubeMonitor-Power | | | | | | | ✔ |

STM32Cube 嵌入式软件的 MCU 软件包支持功能列表。

| 产品编号 | Low-Layer API | HAL API | FreeRTOS RTOS | FatFS文件系统 | Graphics STemWin | Graphics TouchGFX | USB主机ST库 | USB设备ST库 | LwIP TCP/IP mbedTLS | 触摸感应ST库 | USBPD库 | OpenAMP库 | BLE堆栈 | OpenThread堆栈 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| STM32CubeF0 | ✔ | ✔ | ✔ | ✔ | ✔ | | | ✔ | | ✔ | | | | |
| STM32CubeF1 | ✔ | ✔ | ✔ | ✔ | ✔ | | ✔ | ✔ | ✔ | | | | | |
| STM32CubeF2 | ✔ | ✔ | ✔ | ✔ | ✔ | | ✔ | ✔ | ✔ | | | | | |
| STM32CubeF3 | ✔ | ✔ | ✔ | ✔ | ✔ | | | ✔ | | ✔ | | | | |
| STM32CubeF4 | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | | | | | |
| STM32CubeF7 | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | | | | | |
| STM32CubeH7 | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | ✔ | | | ✔ | | |
| STM32CubeG0 | ✔ | ✔ | ✔ | ✔ | | | | | | | ✔ | | | |
| STM32CubeG4 | ✔ | ✔ | ✔ | ✔ | | | | ✔ | | | ✔ | | | |
| STM32CubeL0 | ✔ | ✔ | ✔ | ✔ | | | | ✔ | | ✔ | | | | |
| STM32CubeL1 | ✔ | ✔ | ✔ | ✔ | ✔ | | ✔ | ✔ | | ✔ | | | | |
| STM32CubeL4 | ✔ | ✔ | ✔ | ✔ | ✔ | ✔* | ✔ | ✔ | | ✔ | | | | |
| STM32CubeWB | ✔ | ✔ | ✔ | ✔ | | | | ✔ | | ✔ | | | ✔ | ✔ |
| STM32CubeMP1 | ✔ | ✔ | ✔ | | | | | | | | | ✔ | | |

* H2 2019

# 1.5 开发流程

## 1.5.1 软件安装

了解开发平台后，接着就安装软件。因为 STM32CubeIDE 集成了 STM32CubeMX，同时支持二进制代码烧写，因此 M4 的开发，只需要安装 STM32CubeIDE 即可。

- **从官网下载**

读者可以从 ST 官网下载 STM32CubeIDE，但操作流程比较麻烦。因此可以跳过本方法，使用后面的“从网盘下载”方法。

从官方下载，读者首先需要先在 ST 官网注册一个账户，然后浏览器打开 STM32CubeIDE 下载页面(https://www.st.com/zh/development-tools/stm32cubeide.html#get-software)，这里选择 Windows 为例，默认为当前最新版本 1.4.0（2020.08），如图 1.5.1 所示。

图 1.5.1 STM32CubeIDE 软件下载界面

随后在弹出界面选择“接受”许可协议，并填入前面注册的账户信息，如图 1.5.2 所示

图 1.5.2 填入注册账户信息

最后点击“下载”，将会直接下载或收到带有下载链接的邮件，打开邮件下载即可。

- **从网盘下载**

从 ST 官网下载软件，需要注册比较麻烦，读者可以从百问网的网盘下载。

打开百问网下载中心（http://download.100ask.net/），点击左侧的“100ask_stm32mp157_pro 开发板”，找到“sdk 完整包下载”，里面有百度网盘的链接和密码，如图 1.5.3 所示。

图 1.5.3 百度云下载链接位置

进入网盘后，可见如图 1.5.4 所示的目录，其中“10_M4 专题”目录就是配套本手册的目录。里面包含三个文件夹，如下：

- “源码”：里面是本手册配套的所有源码；
- “工具”：里面是本手册用到的 STM32CubeIDE 压缩包；

- “文档：里面是本手册，后面文档更新，勘误将更新本目录；

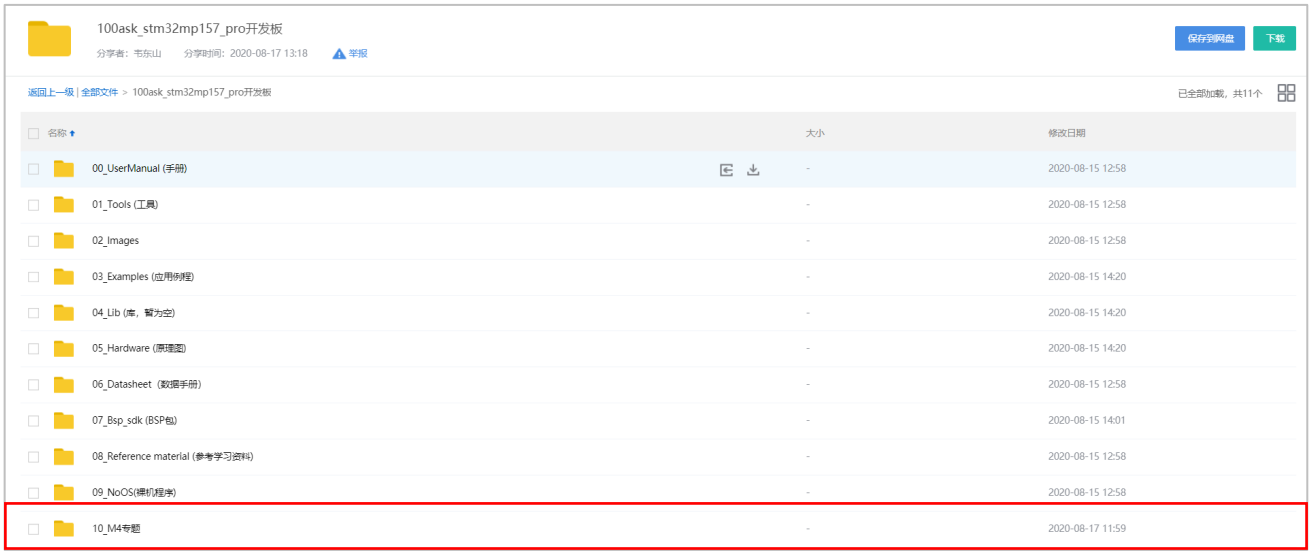


图 1.5.4 STM32CubeIDE 和配套资料

通过前面任一方式下载到 STM32cubeIDE 压缩文件包后，解压可得到 “en.st-stm32cubeide_1.4.0_7511_20200720_0928_x86_64_exe”，选中它，再鼠标右键选择“以管理员身份运行”（安装文件**不能放在中文路径**，否则提示“NSIS Error”错误）。

运行程序“en.st-stm32cubeide_1.4.0_7511_20200720_0928_x86_64_exe”，在安装向导界面选择“Next”，如图 1.5.5 所示。在用户许可协议界面选择“I Agree”，如图 1.5.6 所示。

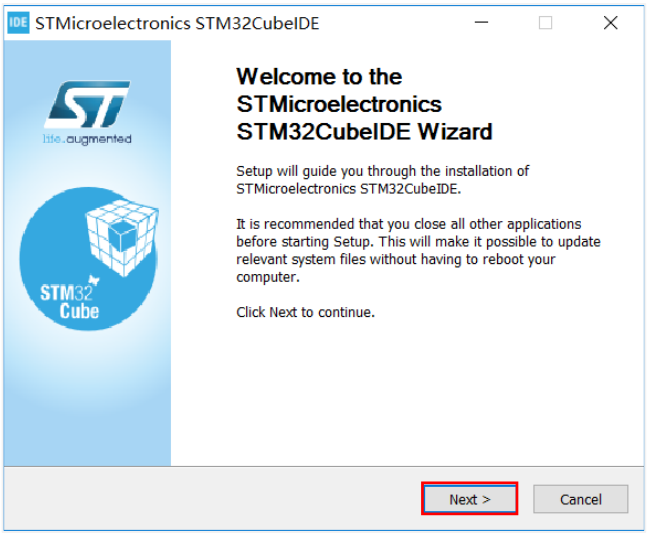


图 1.5.5 安装向导界面

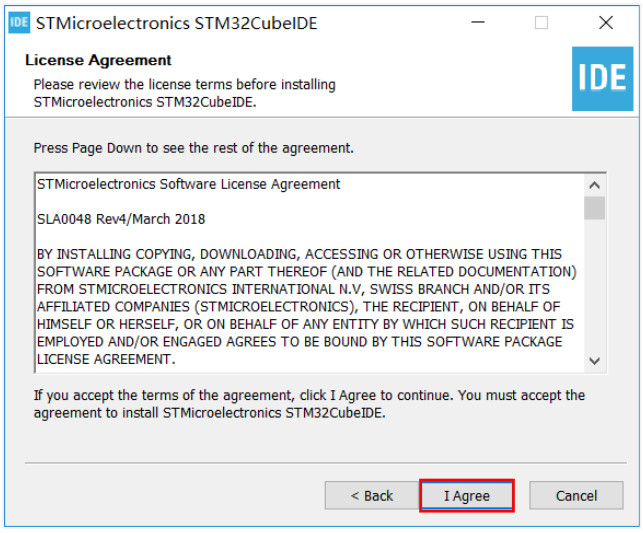


图 1.5.6 用户许可协议界面

接着选择软件安装路径，如果无特殊需求，默认即可，如图 1.5.7 所示。然后选择安装组件，默认勾选安装 J-Link 和 ST-Link 驱动，如图 1.5.8 所示。

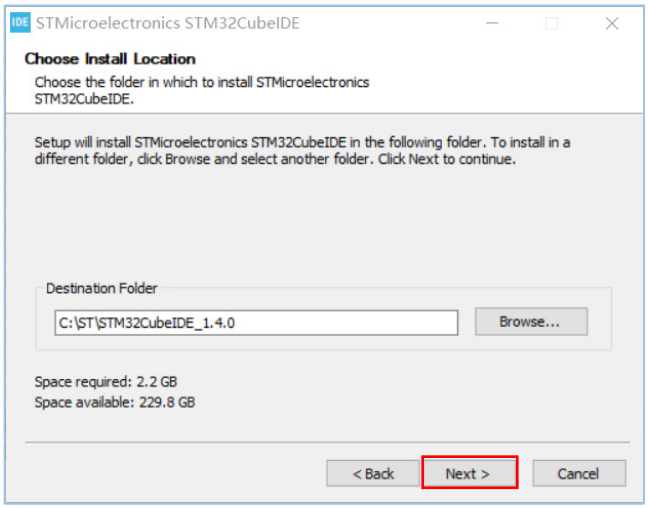


图 1.5.7 安装路径选择界面

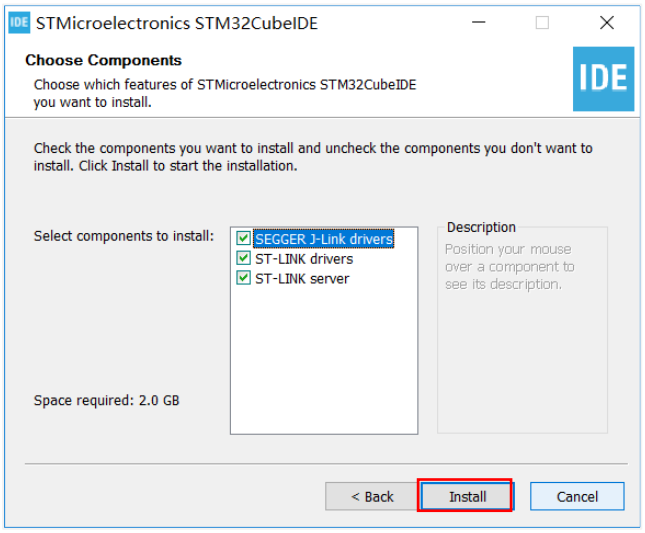


图 1.5.8 安装组件选择界面

随后弹出如图 1.5.9 所示的驱动安装提示界面，选择“安装”即可。

图 1.5.9 驱动安装提示界面

待安装进度条完成，如图 1.5.10 所示时，点击“Next”。最后在安装完成界面，点击如图 1.5.11 所示的“Finish”完成安装。

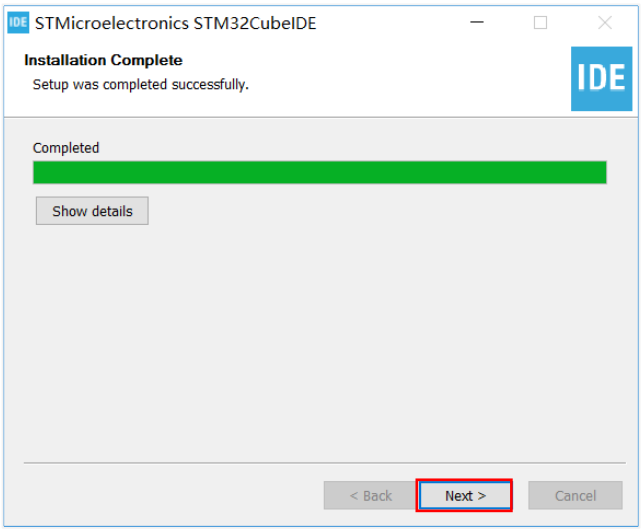


图 1.5.10 安装进度提示界面

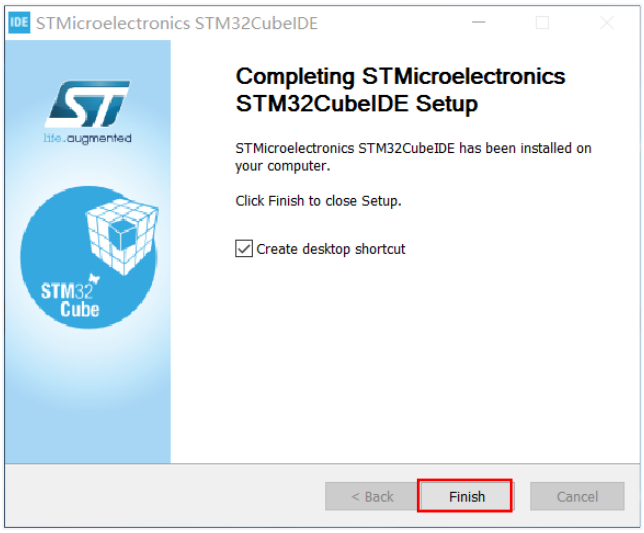


图 1.5.11 安装完成界面

## 1.5.2 创建模板

打开安装好的 STM32CubeIDE，首先会弹出工作空间设置界面，无特殊需求保持默认即可（如果需要自定义其它目录，**注意路径不要包含中文**），随后点击“Launch”，如图 1.5.12 所示。

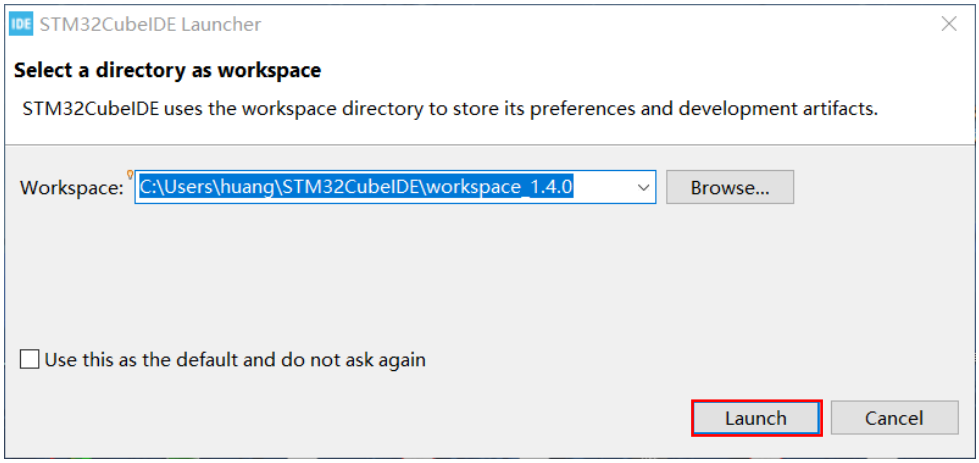


图 1.5.12 STM32CubeIDE 工作空间设置界面

接着依次点击左上角“File”->“New” ->“STM32 Project”来创建 STM32 工程，如图 1.5.13 所示。

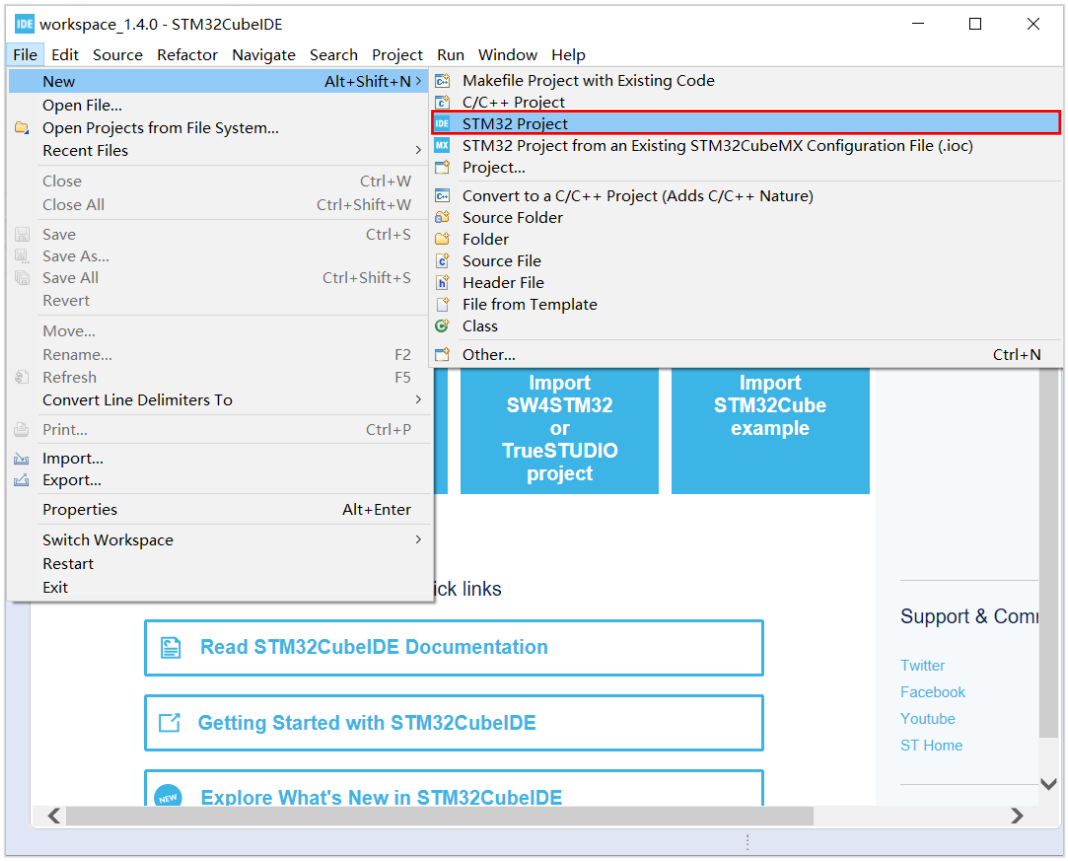


图 1.5.13 创建 STM32 工程

随后出现 STM32CubeMX 界面，在左边“芯片选项区”的搜索框里填入“STM32MP157D”，右上方将出现该芯片的特性介绍等资料，右下方出现该芯片的各种封装选型、主要参数，这里选择“TFBGA361”，如图 1.5.14 所示。

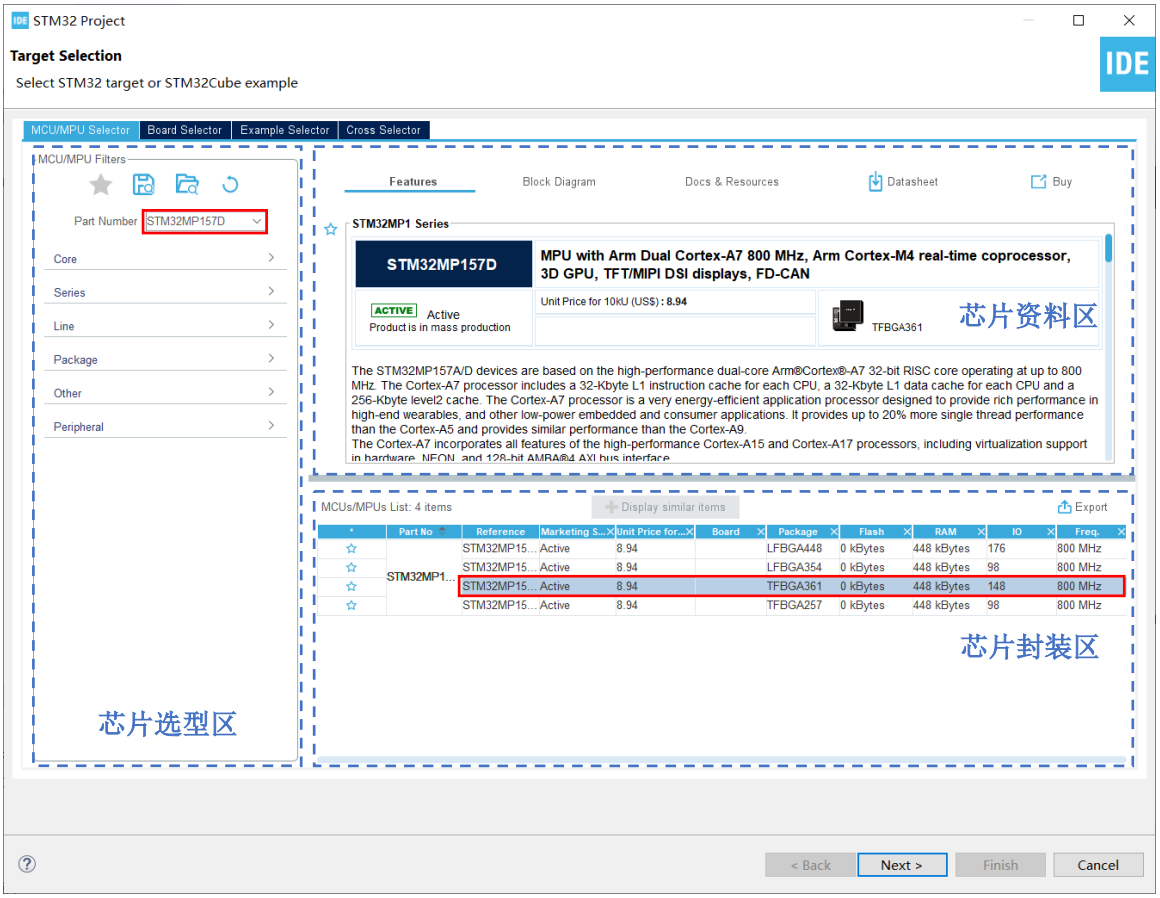


图 1.5.14 STM32CubeMX 选择 STM32MP157D

值得一提的是在芯片资料区，读者切换到“Docs & Resources”标签栏，可以直接下到 STM32MP157D 的数据手册（Datasheet）、参考手册（Reference manual）、编程手册（Programming manual）等资料，如图 1.5.15 所示。

- 数据手册（Datasheet）
  包含 STM32MP157D 的整体功能概述、引脚描述、封装特性等。其中引脚描述部分，有引脚复用查询表格，在原理图设计或编写代码时，可以方便查询使用。
- 参考手册（Reference manual）
  包含 STM32MP157D 的全部资源、寄存器详细说明，多达 4000 多页。由于一些资源是 MCU 和 MPU 共享的，所以无论是 MCU 编程，还是 MPU 编程，都要用到该手册。
- 编程手册（Programming manual）
  包含 STM32MP157D 的编程手册和 GPU 应用手册。编程手册包含一些 M4 处理器的指令、核心外设详细说明。GPU 应用手册包含一些特性介绍、应用编程建议。

最后点击“Next”，完成芯片选择。

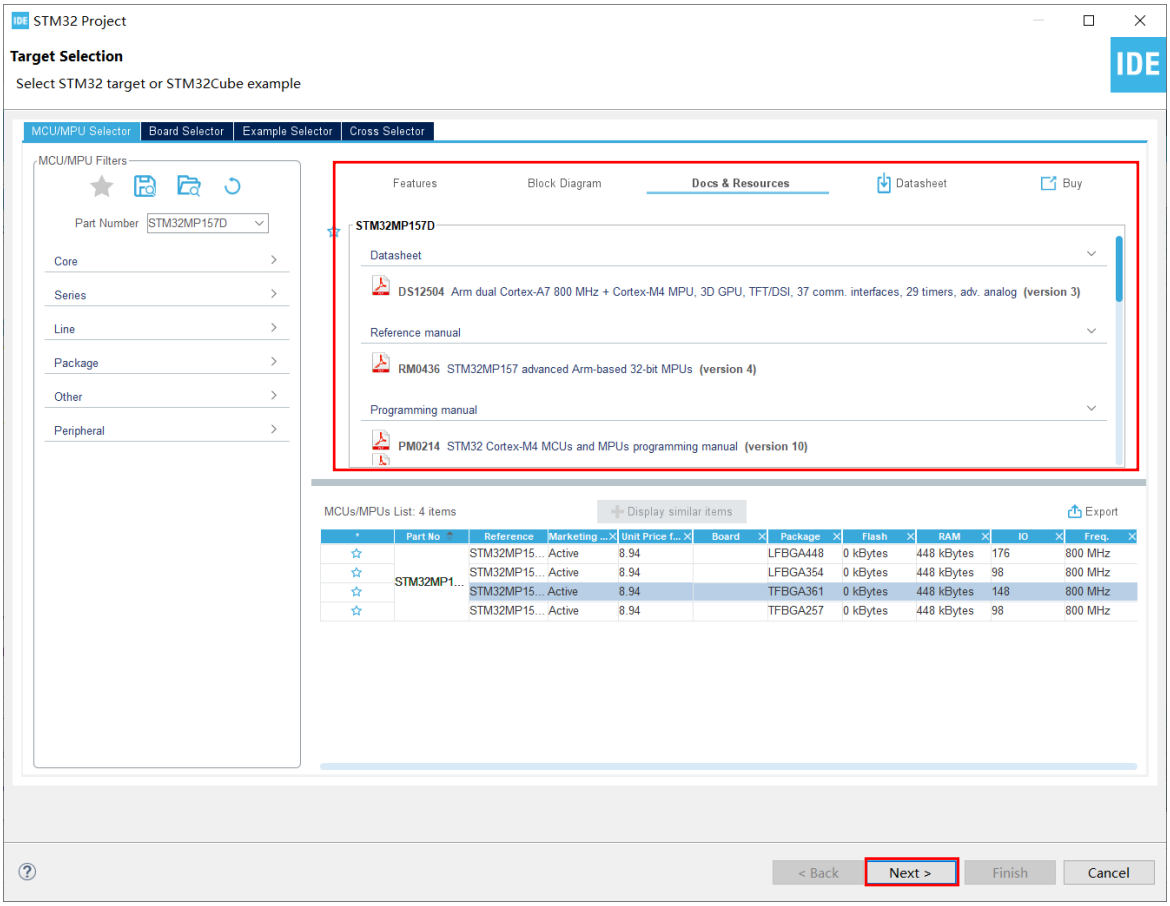


图 1.5.15 STM32CubeMX 的资料手册

紧接着需要在工程设置界面设置工程名字，这里取名为“ProjectDemo”，其它保持默认即可，如图图 1.5.16 所示。随后点击“Finish”,将出现提示“是否切换到 STM32CubeMX 关联视图”，如图 1.5.17 所示，点击“Yes”即可。

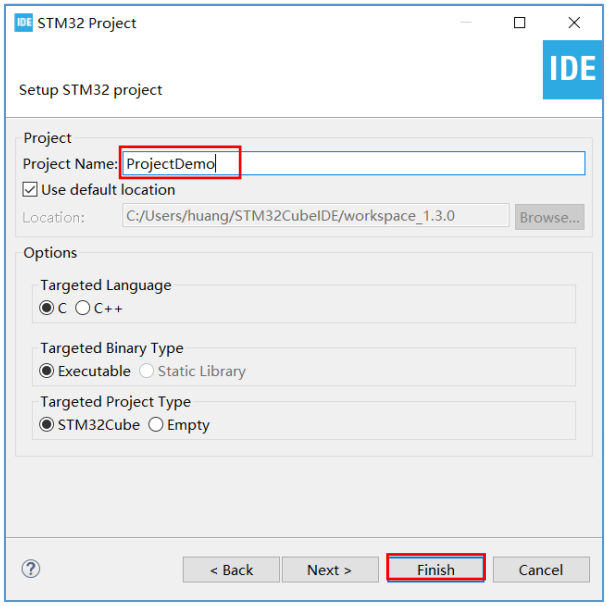


图 1.5.16 工程设置界面

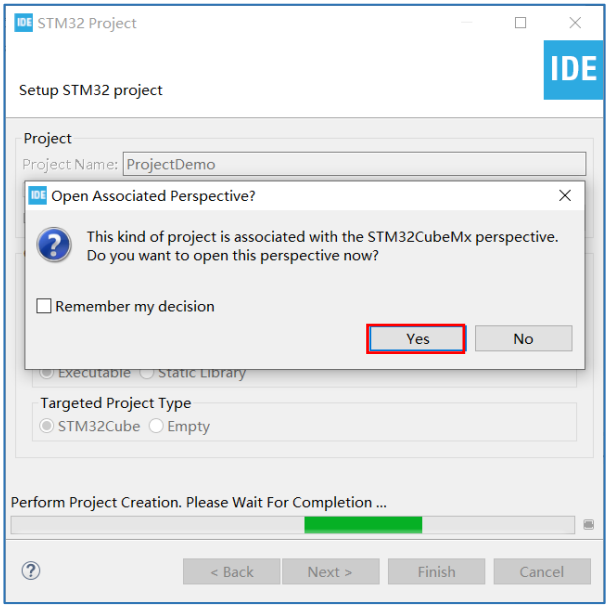


图 1.5.17 关联视图切换

随后便会自动下载 STM32MP1 的 HAL 固件，如图 1.5.18 所示。

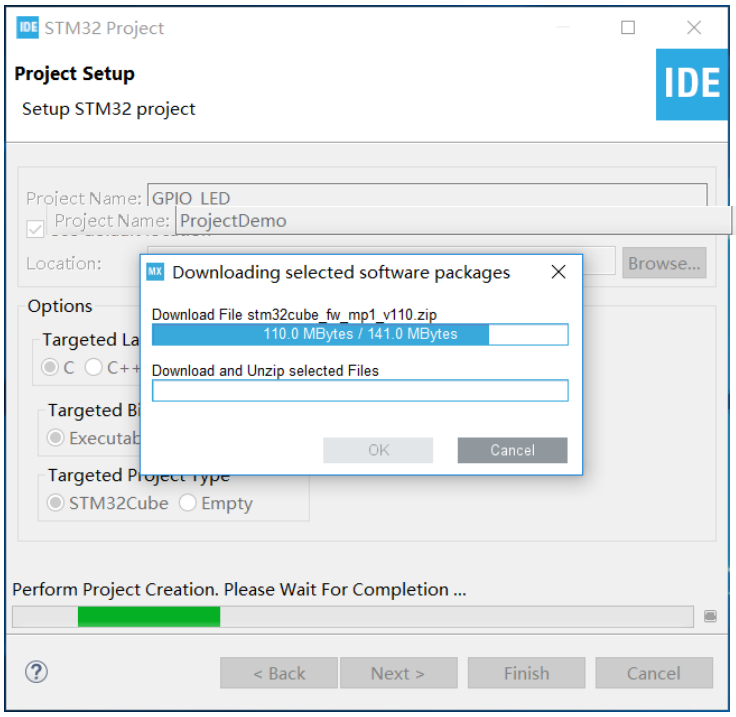


图 1.5.18 下载 STM32MP1 的 HAL 固件

待前面 STM32MP1 的 HAL 库下载完成后，将自动进入 STM32CbueMX 界面，在芯片下方的搜索框输入“PG8”（即开发板黄色 LED 所接引脚，后面开发示例再详细讲解），随后在芯片示意图上出现一个闪烁的点，这个点就是代表引脚 PG8。然后鼠标左键点击该点，在弹出的功能复用菜单里选择“GPIO_Output”，如图 1.5.19 所示。

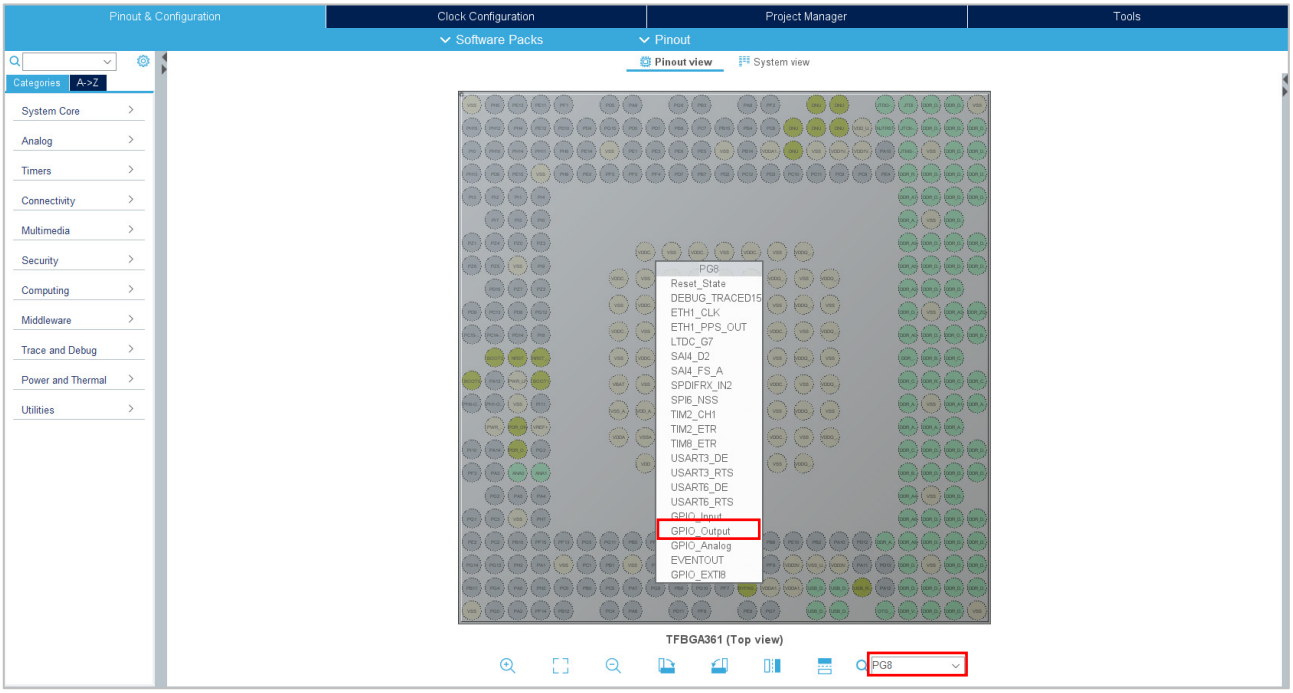


图 1.5.19 STM32CubeMX 配置引脚复用功能

这里需要注意一下，和其它 STM32 单片机不同，还需要设置“Pin Reservation”给“Cortex-M4”，**否者 STM32CubeMX 不会生成 GPIO 初始化相关代码**。如图 1.5.20 所示，在刚才选择的引脚上，鼠标右键选择“Pin Reservation”->“Cortex-M4”。

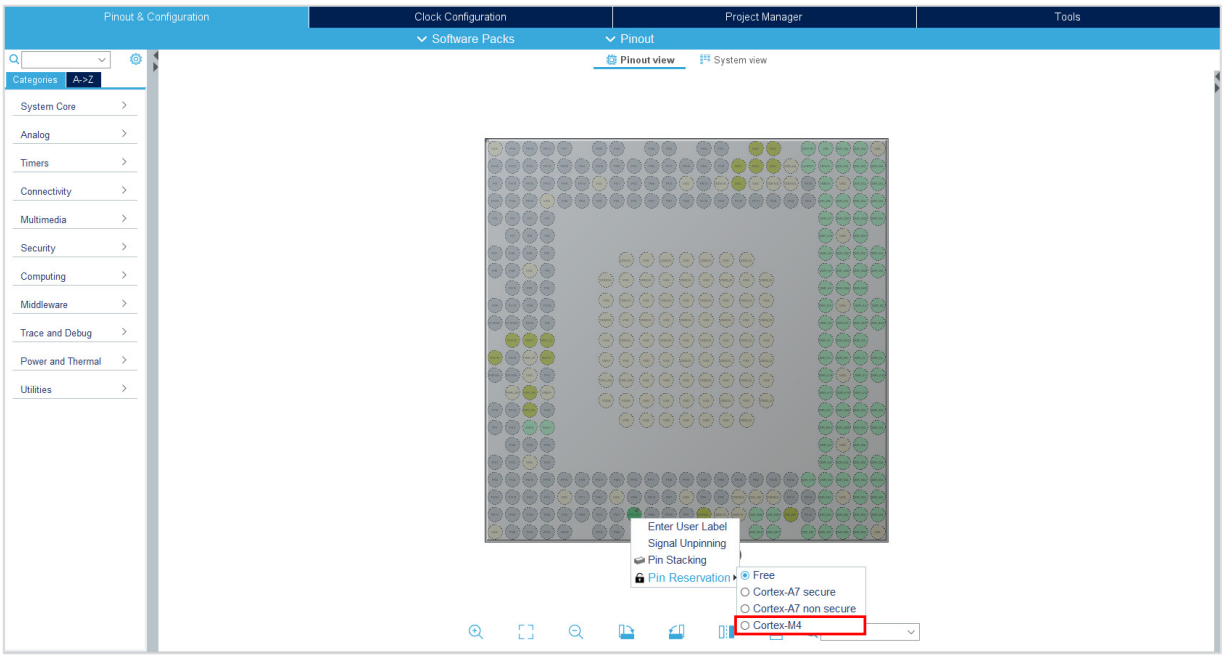


图 1.5.20 STM32CubeMX 配置引脚给 M4

随后，再点击左边的“System Core”标签，选中“GPIO”，再选中刚才配置的“PG8”，下方将会显示该 GPIO 的详细配置。这里分别选择“Output Push Pull”、“Pull-up”、“Very High”，再填入自定义名字“LED_YELLOW”，如图 1.5.21 所示。关于这些选项的详细含义，将放在后面的开发示例详细介绍，这里只熟悉操作流程。

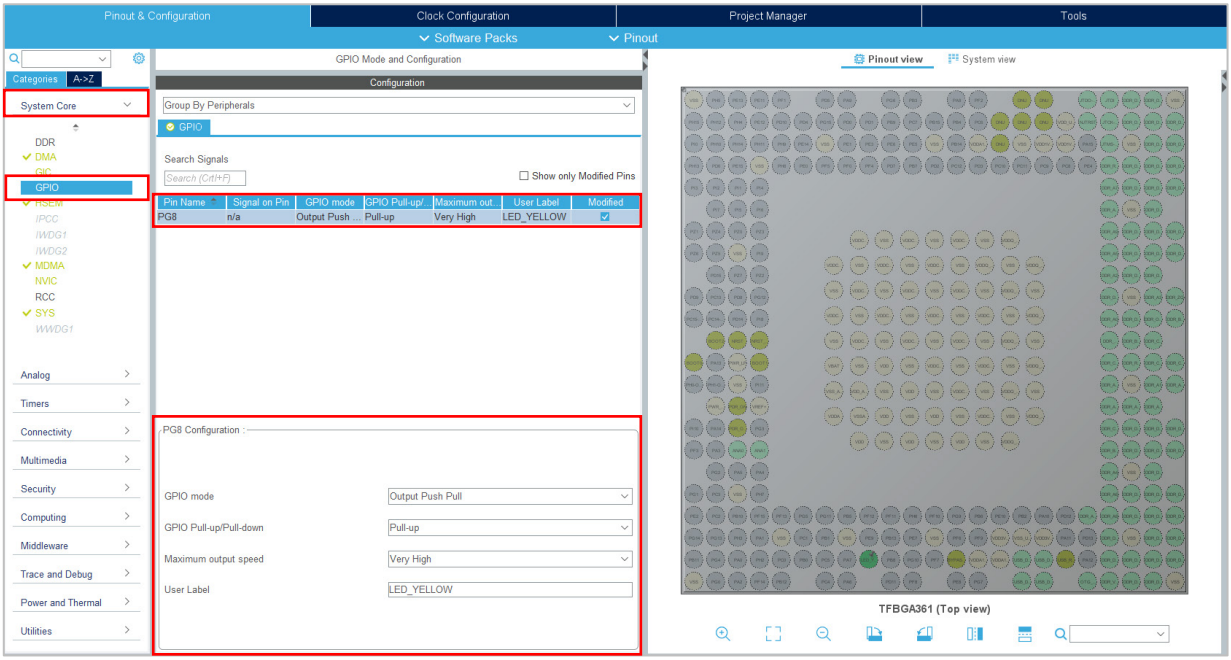


图 1.5.21 STM32CubeMX 配置引脚详细参数

为了让外设初始化代码单独生成在“.c/.h”文件中，切换到“Project Manager”标签，选择“Code Generator”,勾选“Generate peripheral initialization as a pair of '.c/.h' files per peripheral”，如图 1.5.22 所示。

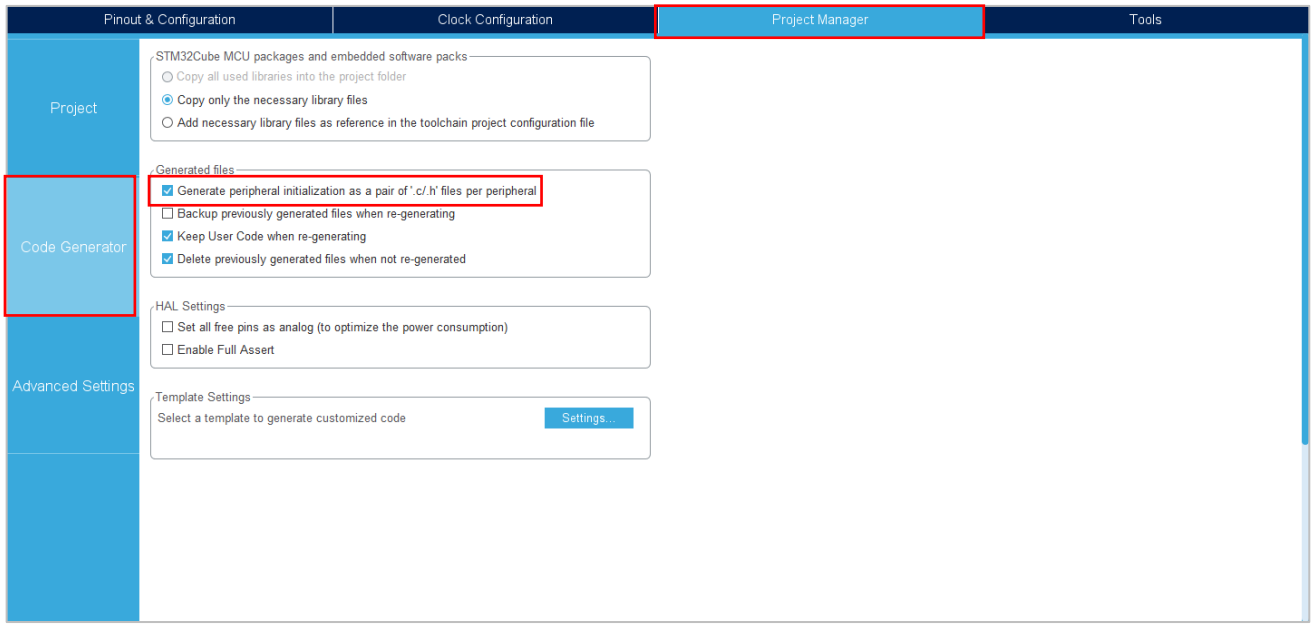


图 1.5.22 设置外设初始化单独生成

接着点击“Project”，选择“Grenerate Code”产生初始化代码，如图 1.5.23 所示。

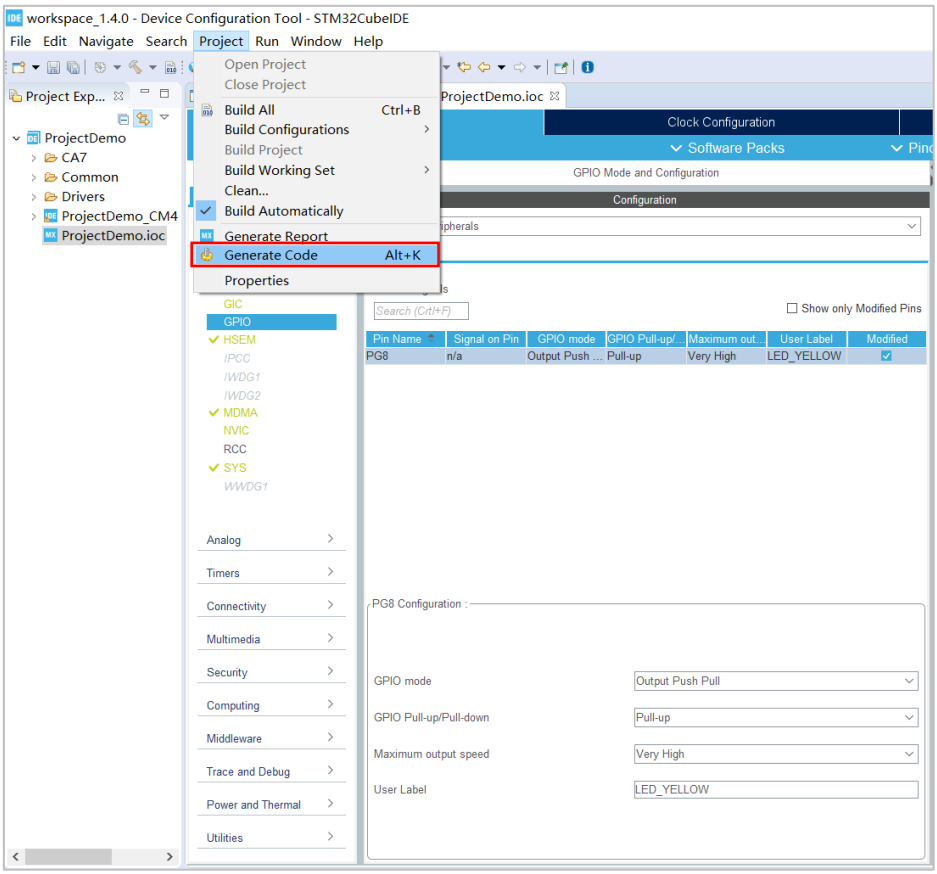


图 1.5.23 STM32CubeMX 生成初始化代码

随后在左边的“Project Explorer”工程浏览器找到“main.c”。路径为“ProjectDemo”->“ProjectDemo_CM4”->“Core”->“Src”->“main.c”，如图 1.5.24 所示。

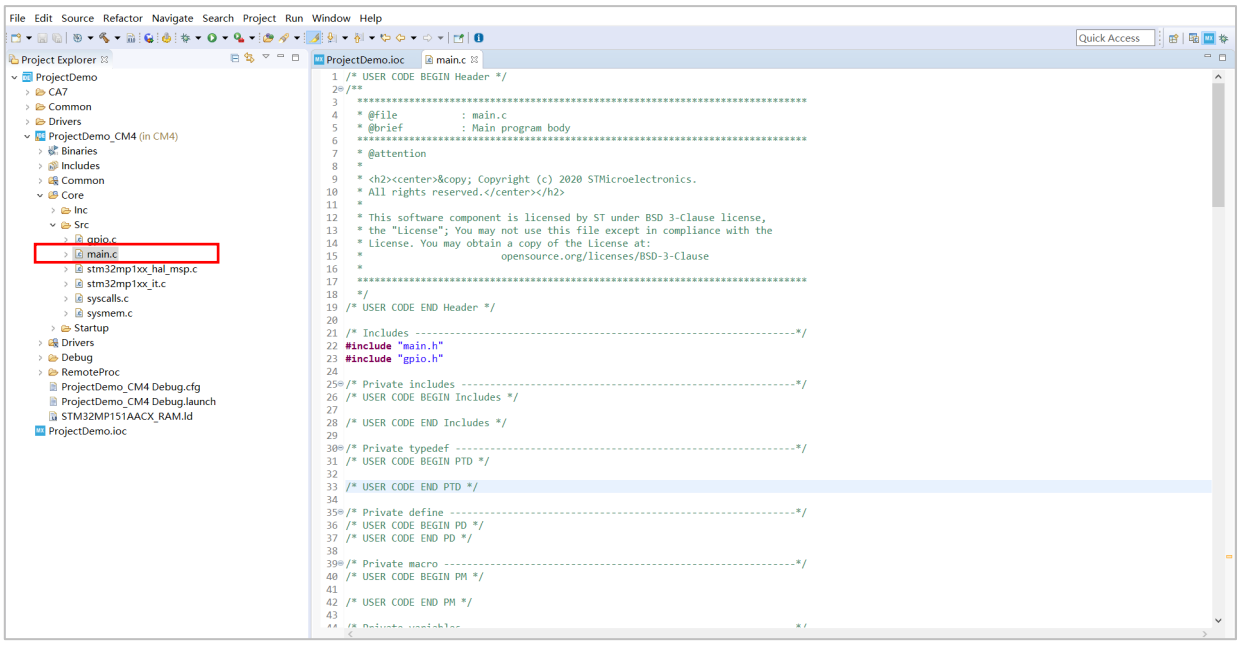


图 1.5.24 工程主函数

现在工程模块就创建完了，以后只需先打开工程的保存路径，如前面示例的“C:\Users\huang\STM32CubeIDE\workspace_1.4.0\”,再打开对应的项目名文件,如前面的“ProjectDemo”,找到“.project”便可再次打开，如图 1.5.25 所示。

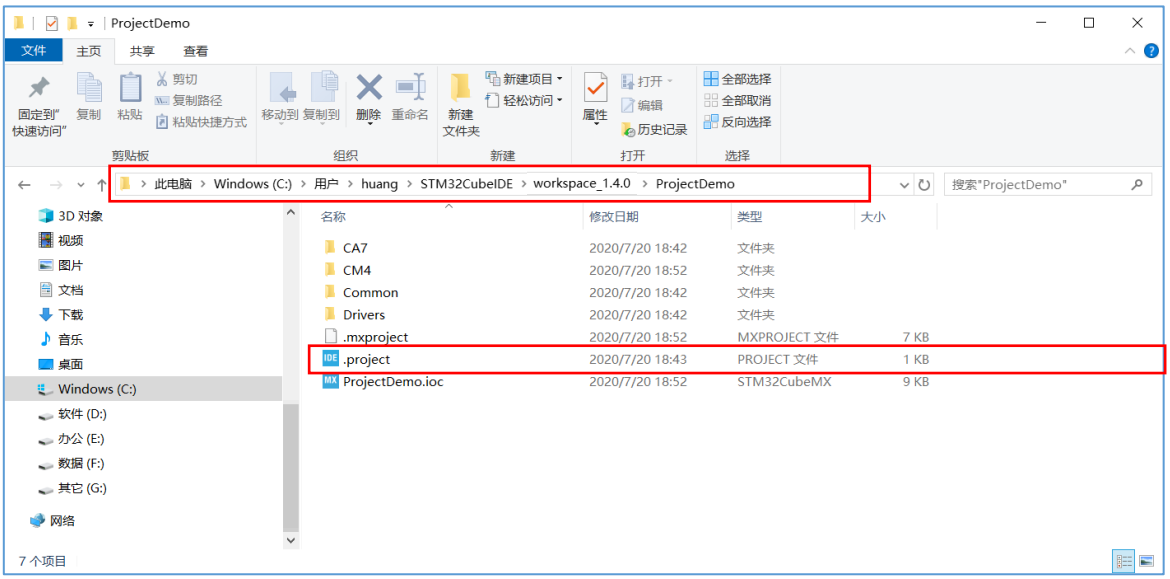


图 1.5.25 工程所在路径

STM32CubMX 生成的工程模板，只是做了一些初始操作。以本模板工程为例，在“main.c”里的主函数里面，依次进行了 HAL 初始化、系统时钟配置、GPIO 初始化，如图 1.5.26 所示。

```
  */
int main(void)
{
  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */


  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  /* USER CODE BEGIN 2 */

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
```

图 1.5.26 工程主函数代码

将鼠标光标移到“MX_GPIO_Init();”位置，按下“Ctrl”+“鼠标左键”，将跳到函数定义位置。或者在函数上，点击鼠标右键，选择“Open Declaration”跳到函数定义位置，如图 1.5.27 所示。

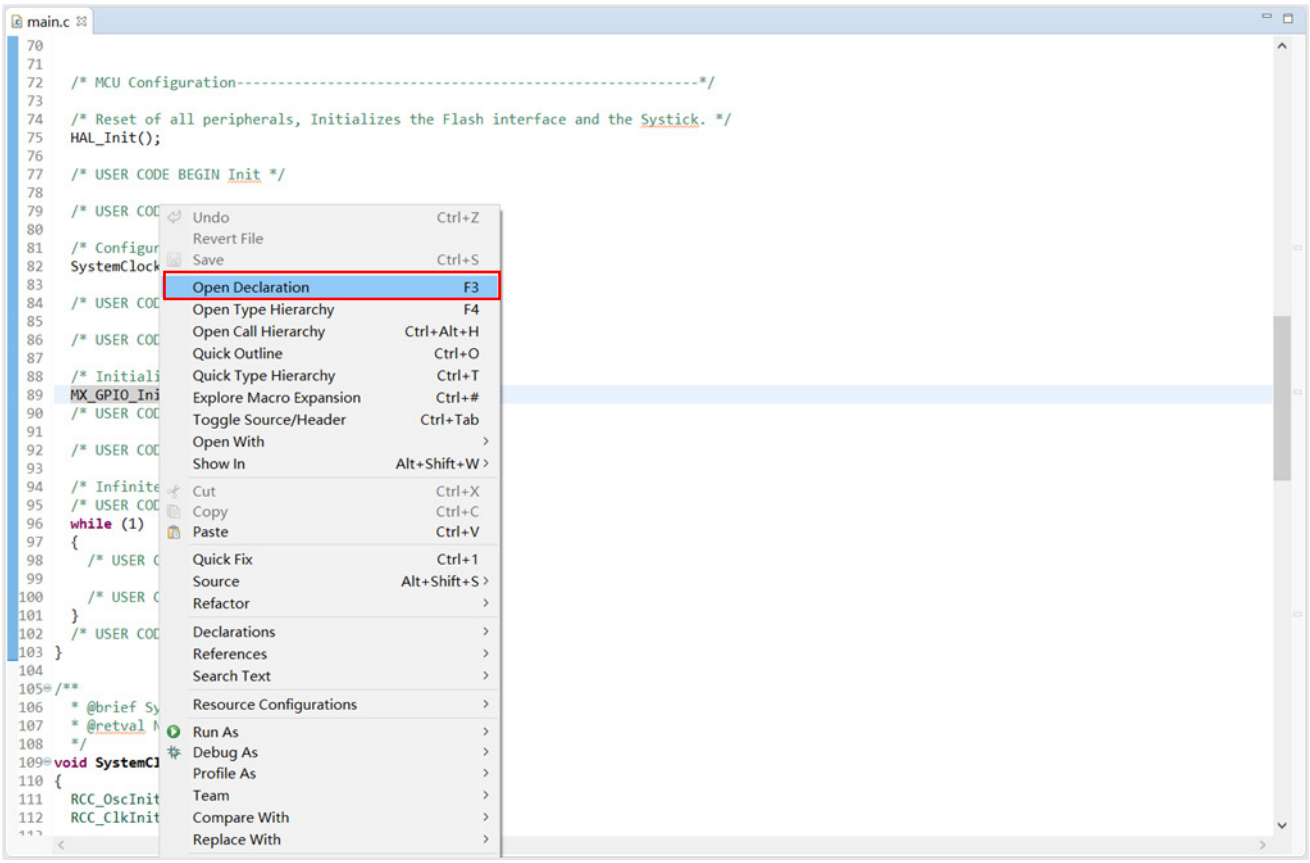


图 1.5.27 跳到函数定义位置

可以看到“MX_GPIO_Init()”在“gpio.c”中的内容如下：

```
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOG_CLK_ENABLE();

/*Configure GPIO pin : PtPin */
  GPIO_InitStruct.Pin = LED_YELLOW_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
  HAL_GPIO_Init(LED_ YELLOW_GPIO_Port, &GPIO_InitStruct);
}
```

可以看到使能了 GPIOG 的时钟和配置了 LED_YELLOW_Pin 引脚。

“LED_YELLOW_Pin”:对应 STM32CubeMX 里“User Label”定义的“LED_YELLOW”,如果跳转到定义位置，可以看到值为“GPIO_PIN_8”。末尾的“LED_YELLOW_GPIO_Port”对应的是“GPIOG”；

“GPIO_MODE_OUTPUT_PP”:对应 STM32CubeMX 里“GPIO mode”选择的“Output Push Pull”；

“GPIO_PULLUP”:对应 STM32CubeMX 里“GPIO Pull-up/Pull-down”选择的“Pull-up”；

“GPIO_SPEED_FREQ_VERY_HIGH”：对应STM32CubeMX里“Maximum output speed”选择的“Very High”；

可以看到“MX_GPIO_Init()”的内容基本就是在STM32CubeMX设置的内容，但仅仅是做了相关的初始化配置，具体的控制逻辑，还需要自己使用HAL库编写。

### 1.5.3 使用HAL库

这里假设需要控制LED_YELLOW灯间隔一秒交替亮灭，前面STM32CubeMX自动初始化了该引脚，还需要自己使用HAL库进行控制。

使用HAL库，就是调用ST提供的API函数。现在还没有STM32MP1系列的HAL库用户手册，但STM32MP1的优势之一就是可以复用STM32 MCU生态。也就是说，HAL库的用法和其他STM32 MCU兼容，很多代码都可以互相复制。因此我们可以直接参考STM32F4系列的《UM1725 Description of STM32F4 HAL and LL drivers》手册使用HAL库。亦或者直接看HAL库源码，在左侧边的工程浏览器依次展开“ProjectDemo”->“ProjectDemo_CM4”->“Drivers”->“STM32MP1xx_HAL_Driver”->“Src”->“stm32mp1xx_hal_gpio.c”，可以看到HAL库提供的API函数。根据名字，可以大致猜测其功能，比如要控制GPIO的值，则应该使用“HAL_GPIO_WritePin()”，双击该函数，即可跳到对应的定义位置，如图 1.5.28 所示。

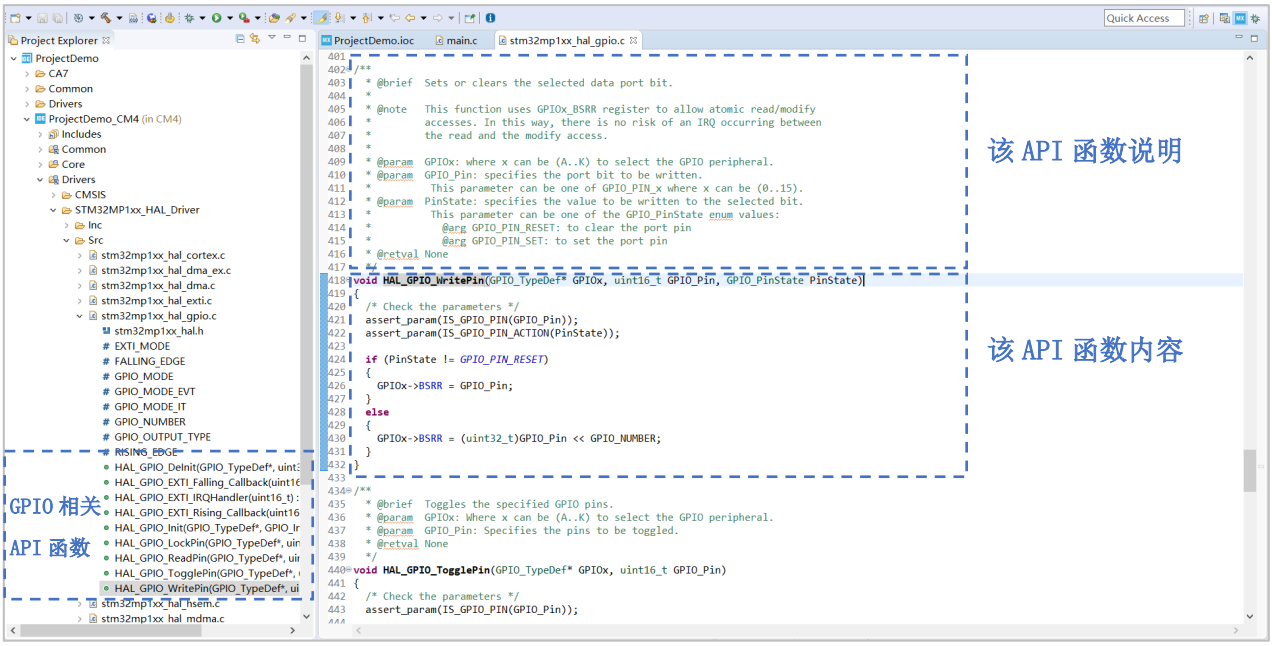


图 1.5.28 HAL库里GPIO相关源码

仔细的看下该API函数说明，如果英文不好，借助翻译软件，翻译出来大致如下：

@简介：设置或清除选中端口位的数据
@注意：该函数使用GPIOx_BSRR寄存器来允许原子读/写访问。这样，就不会在读写访问之间发生IR中断
@参数GPIOx：其中x可以是（A到K）以选择GPIO外设端口。
@参数GPIO_Pin：指定要写入的端口位。此参数可以是GPIO_PIN_x之一，其中x可以是0到15。
@参数PinState：指定要写入所选位的值。此参数可以是GPIO_PinState枚举值之一：
@GPIO_PIN_RESET：清除端口引脚
@GPIO_PIN_SET：设置端口引脚
@返回值：无

即该函数就是设置/清除指定 GPIO 的值，需要三个参数，第一个为端口号，这里为“GPIOG”，或者前面提到对应宏定义的“LED_YELLOW_GPIO_Port”；第二个为该端口号的哪位，这里为“GPIO_PIN_8”，或前面提到对应宏定义的“LED_YELLOW_Pin”；第三个为设置的值，如果输出低着设置“GPIO_PIN_RESET”，如果设置输出高则为“GPIO_PIN_SET”。因此如果要控制 LED_YELLOW 则编写如下代码即可：

```
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_8, GPIO_PIN_SET);  //设置 GPIOG 8 引脚输出高电平
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_8, GPIO_PIN_RESET); //设置 GPIOG 8 引脚输出低电平
或
HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_SET);
HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_RESET);
```

通过以上代码就可以控制 GPIO 引脚接的 LED 灯了，但这样亮灭切换很快，肉眼观察不到，需要在切换之间添加个延时。仿照前面的思维，在“stm32mp1xx_hal.c”这个提供 HAL 初始化、工具函数的 HAL 源文件里，可以找到“HAL_Delay()”，该函数的相关介绍见图 1.5.29 所示。

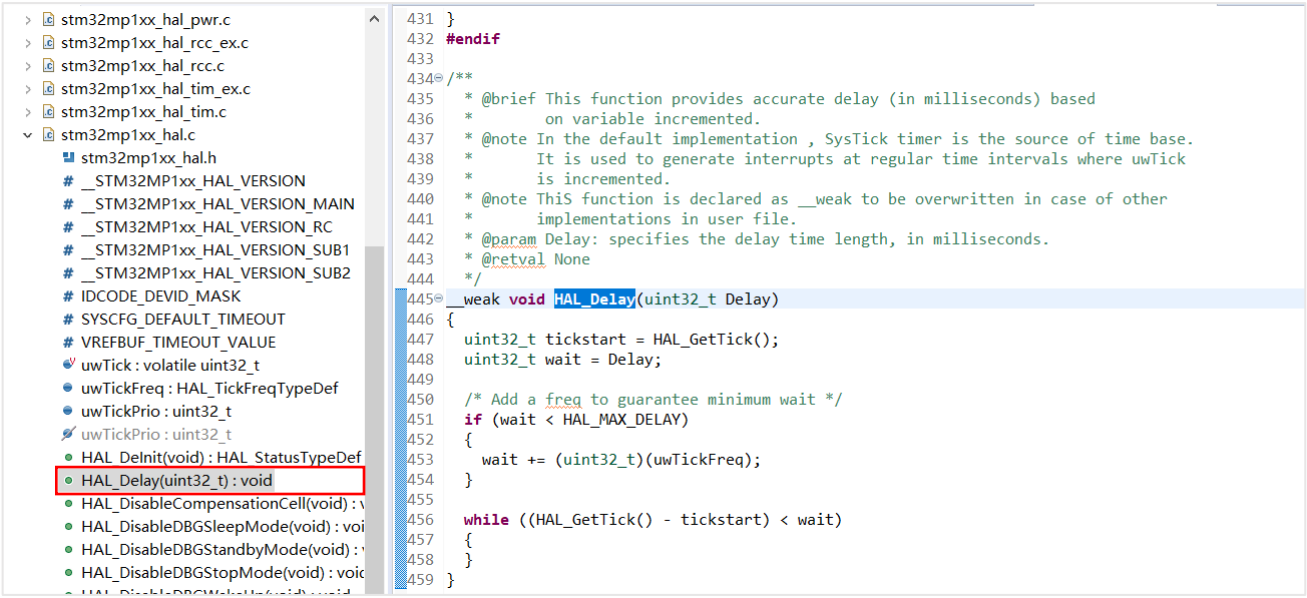


图 1.5.29 HAL 库提供的延时函数源码

可以看到“HAL_Delay()”只需要一个参数，参数为延时时间，单位为毫秒。因此完善前面的代码如下：

```
HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_SET); //设置 GPIOG 8 引脚输出高电平
HAL_Delay(1000); //延时 1000 毫秒，即 1 秒
HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_RESET); //设置 GPIOG 8 引脚输出低电
HAL_Delay(1000); //延时 1000 毫秒，即 1 秒
```

STM32CubeMX 支持动态配置，比如用户第一次设置 GPIOG 8 为 LED_YELLOW 生成了工程模板，用户添加控制代码后，又想再添加其它引脚生成工程模板，为了避免第二次生成的工程模板覆盖之前的用户自己添加的代码，因此需要把用户代码放置在对应合适的位置。如图 1.5.30 所示，将编写的 GPIO 代码放在“/* USER CODE BEGIN 3 */”和“/* USER CODE END 3 */”之间。

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
  HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_SET); //设置GPIOG 8引脚输出高电平
  HAL_Delay(1000); //延时1000毫秒，即1秒
  HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_RESET); //设置GPIOG 8引脚输出低电平
  HAL_Delay(1000); //延时1000毫秒，即1秒

}
/* USER CODE END 3 */
```

图 1.5.30 用户代码添加位置

至此，一个简单的 LED 控制程序就编写完成了。

## 1.5.4 编译上传

编写完代码，还需编译、烧写、运行才能看到效果。选中工程“ProjectDemo_CM4”，鼠标右键选择“Build Project”即可完成编译，如图 1.5.31 所示。

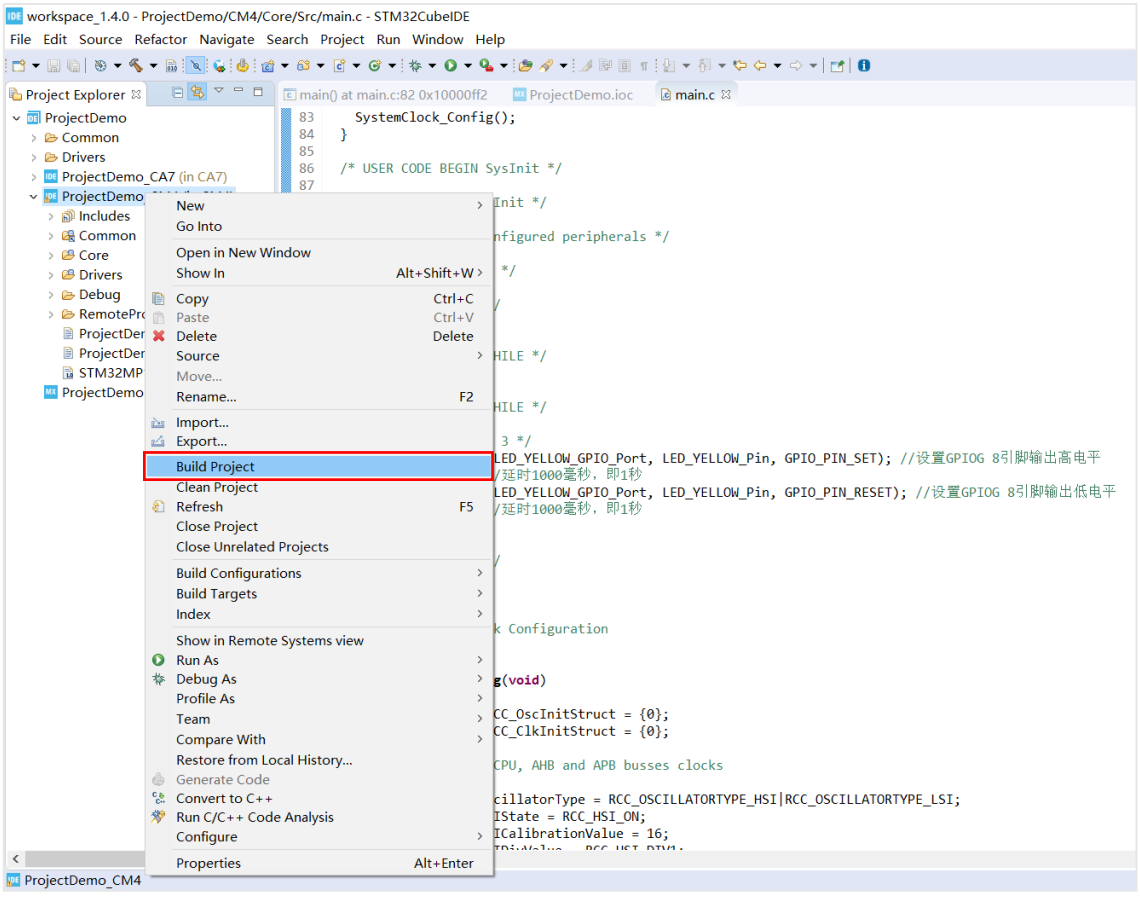


图 1.5.31 编译工程

编译完成后在工程项目下会新生成一个“Debug”目录，里面的“ProjectDemo_CM4.elf”就是二进制目标文件，如图 1.5.32 所示。

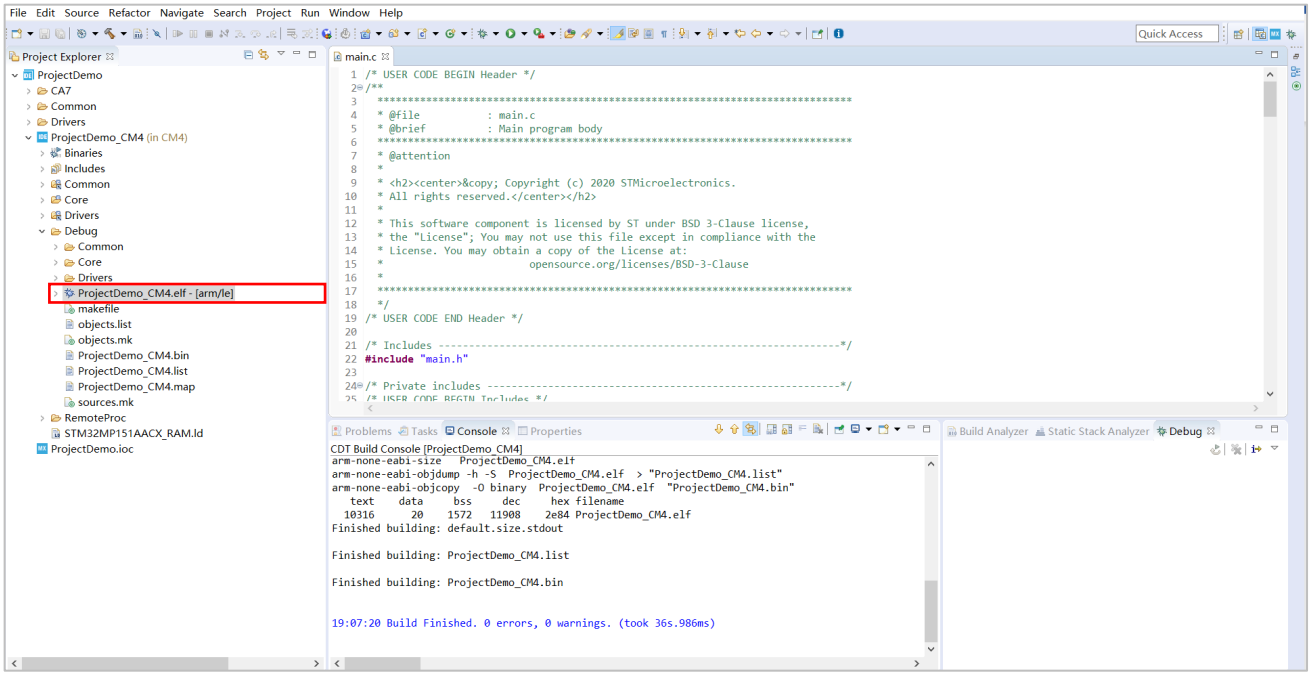


图 1.5.32 二进制目标文件

接触过 STM32 单片机的读者应该熟悉 STM32 的烧写方法，通常使用 J-Link 或 ST-Link 将二进制文件烧写到单片机的 Flash 里，然后单片机上电后自动读取 Flash 的二进制文件到 RAM 里实现运行。

STM32MP157x 系列里的 M4 启动运行的方式略有差异，**必须先启动 A7 才能启动 M4**。因此需要先让 A7 运行 Bootloadr 或 Linux，再加载 ELF 二进制文件启动 M4。出厂的开发板默认烧写有 Linux 系统，读者可以先直接使用。

想要 Linux 加载前面生成的“ProjectDemo_CM4.elf”二进制文件，还需要将文件先上传到 Linux。用户可以用网线连接开发板，让电脑和开发板处于同一个网段，再通过 FTP/SFTP 上传文件到 Linux。但这个方法条件相对苛刻，很多时候电脑和开发板无法处于同一个网段，因此在开发板 Linux 系统里将 USB OTG 模拟为虚拟网卡，只要用户将 USB OTG 连接到电脑，就如同实现了局域网联接，之后便可以很方便的上传文件。

先如所示，连接开发板电源、设备好拨码开关(eMMC 启动)，连接调试串口和 USB OTG 到电脑 USB 接口。

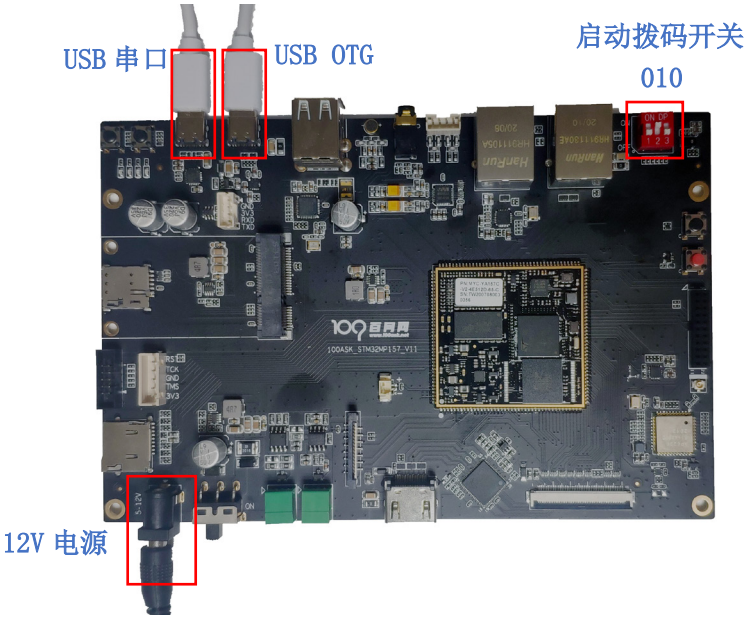


图 1.5.33 开发板实物连接图

然后使用 MobaXterm(https://mobaxterm.mobatek.net/download.html)之类的串口工具连接开发板。电脑打开 MobaXterm，如图 1.5.34 所示，新建会话“Session”，选择串口“Serial”，选择端口 COMx（以读者实际看到为准），波特率 115200，最后点击“OK”即可。设置完成，就可启动开发板电源，此时会有一系列打印，直到打印停止，则表示进入了系统。

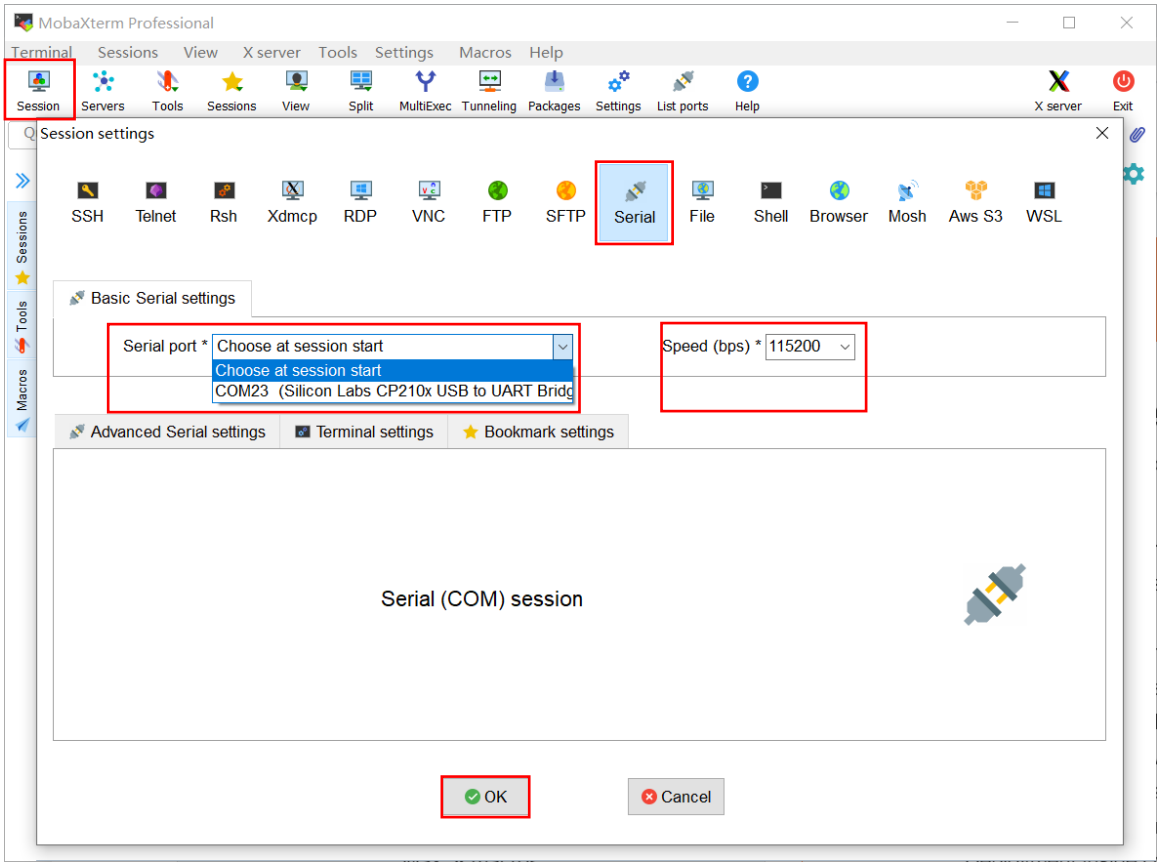


图 1.5.34 串口设置

进入开发板的 Linux 系统后，输入“ifconfig”命令查看网络情况，如图 1.5.35 所示，可以看到名为“usb0”的网卡，IP 地址为“192.168.7.1”。

```
root@100ask:~# ifconfig
eth1      Link encap:Ethernet  HWaddr 92:E3:7E:90:92:07
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:372 errors:0 dropped:0 overruns:0 frame:0
          TX packets:372 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:23104 (22.5 KiB)  TX bytes:23104 (22.5 KiB)

usb0      Link encap:Ethernet  HWaddr 56:7B:EE:30:75:F4
          inet addr:192.168.7.1  Bcast:192.168.7.255  Mask:255.255.255.0
          inet6 addr: fe80::547b:eeff:fe30:75f4/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1602 errors:0 dropped:0 overruns:0 frame:0
          TX packets:8 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:232971 (227.5 KiB)  TX bytes:1977 (1.9 KiB)
```

图 1.5.35 开发板网卡状态

接着打开电脑“控制面板”->“网络和 Internet”->“网络连接”，可以看到一个备注为“Remote NDIS Compatible Device”的网络设置，如图 1.5.36 所示，这个设备就是识别到的开发板 USB OTG 模拟的网卡设备。如果未出现该设备，尝试先拔掉 USB OTG 线，重新启动 Linux，待进入 Linux 终端后，再稍等一会插入 USB OTG 线。

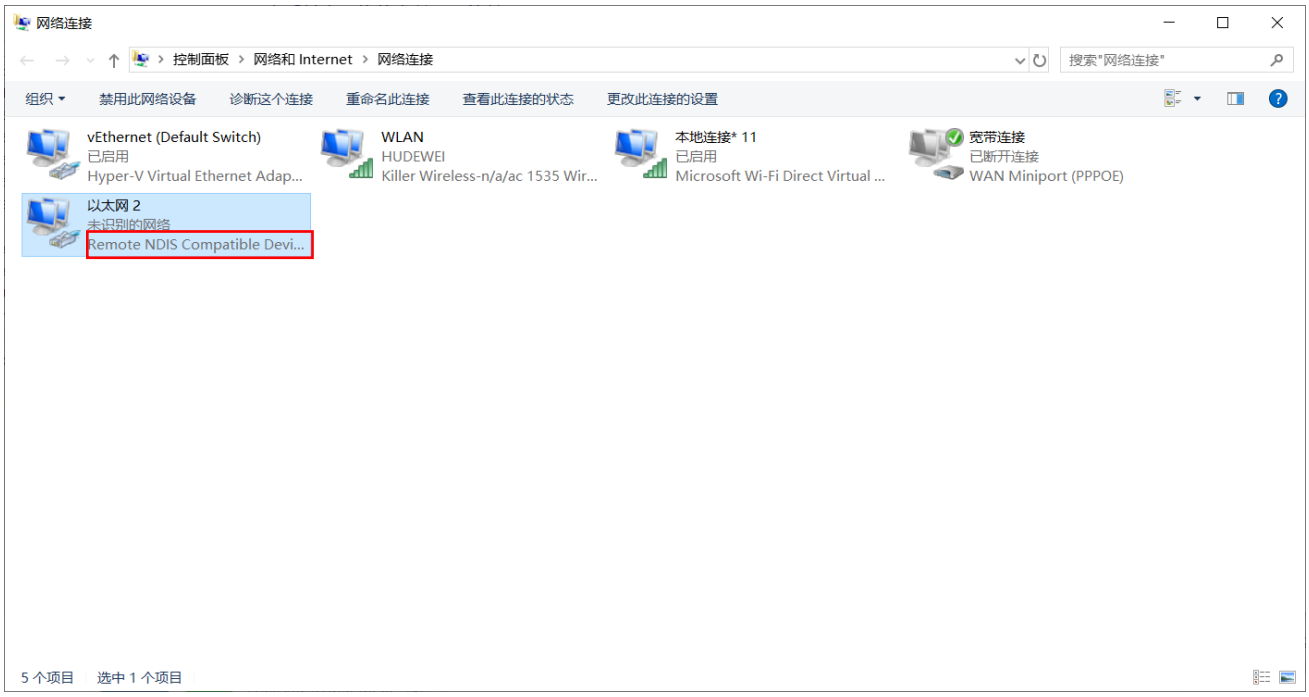


图 1.5.36 电脑网络连接设备

选中该设备，点击鼠标右键，选择“属性”，接着会弹出该网络设备的属性窗口，如图 1.5.37 所示。在里面选中“Internet 协议版本 4(TCP/IPv4)”，再点击“属性”按钮。

再弹出的属性页面，勾选“使用下面的 IP 地址(S):”，IP 地址设置为与开发板 USB 虚拟网卡同一网段即可，这里假设设置为“192.168.7.10”，子网掩码为“255.255.255.0”，最后点击确定。

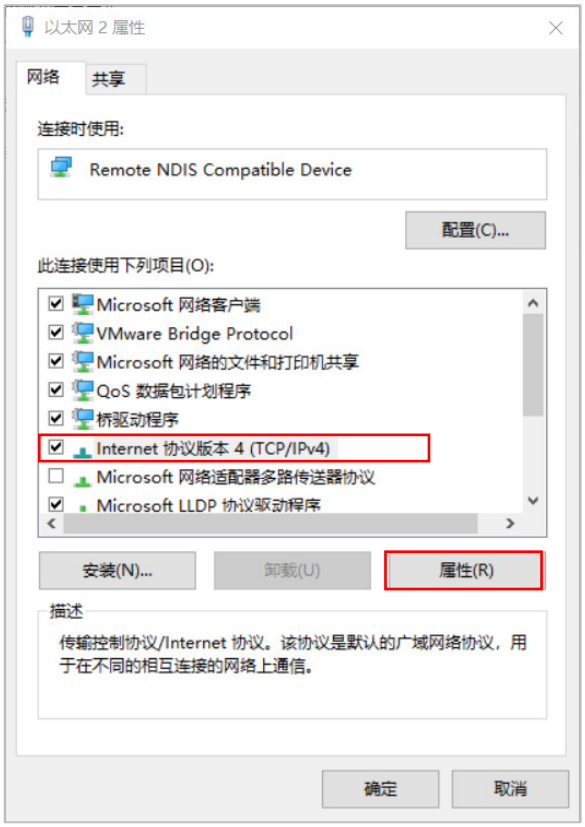


图 1.5.37 网络属性面板

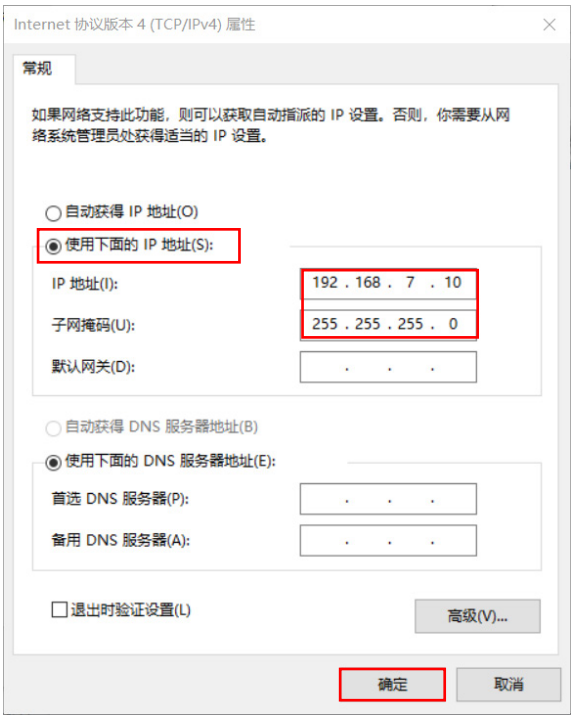


图 1.5.38 设置电脑 IP

设置完后，电脑就和开发板类似处于同一个局域网，这里可以使用“MobaXterm”里的远程登陆软件，选择“SSH”连接，输入开发板 IP 地址“192.168.7.1”，点击确定便可远程登陆开发板，如图 1.5.39 所示设置。

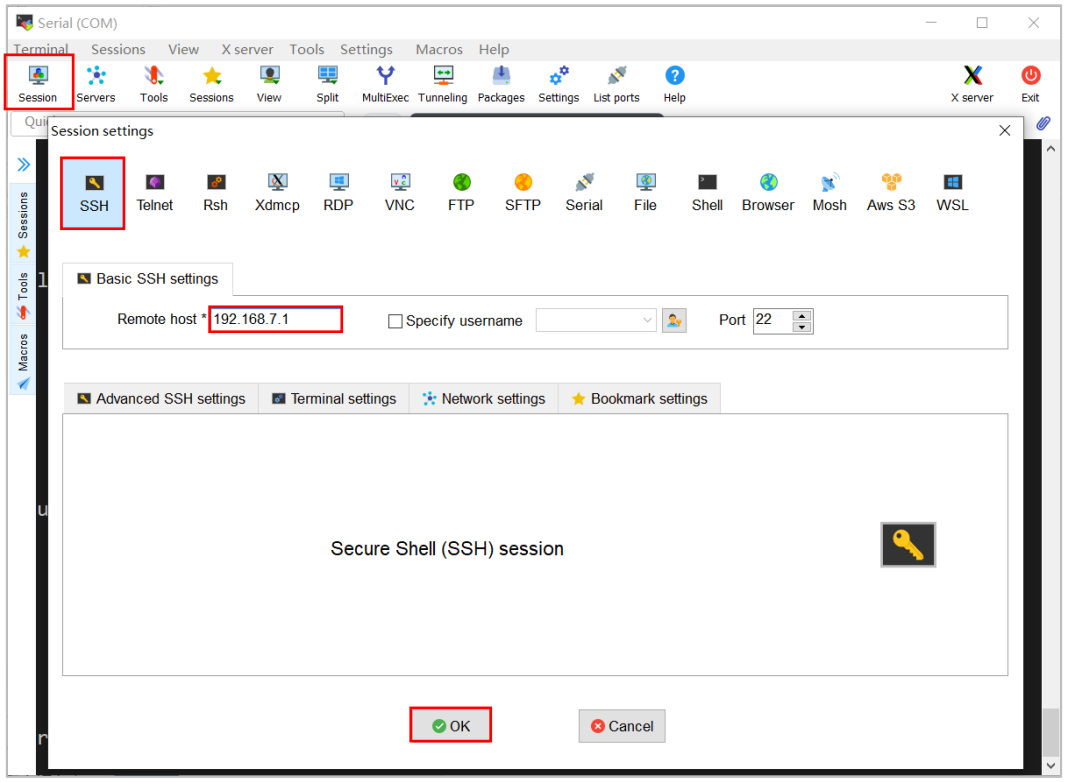


图 1.5.39 SSH 远程登陆设置

接着要求输入登陆账户，输入“root”，回车即可，如图 1.5.40 所示。

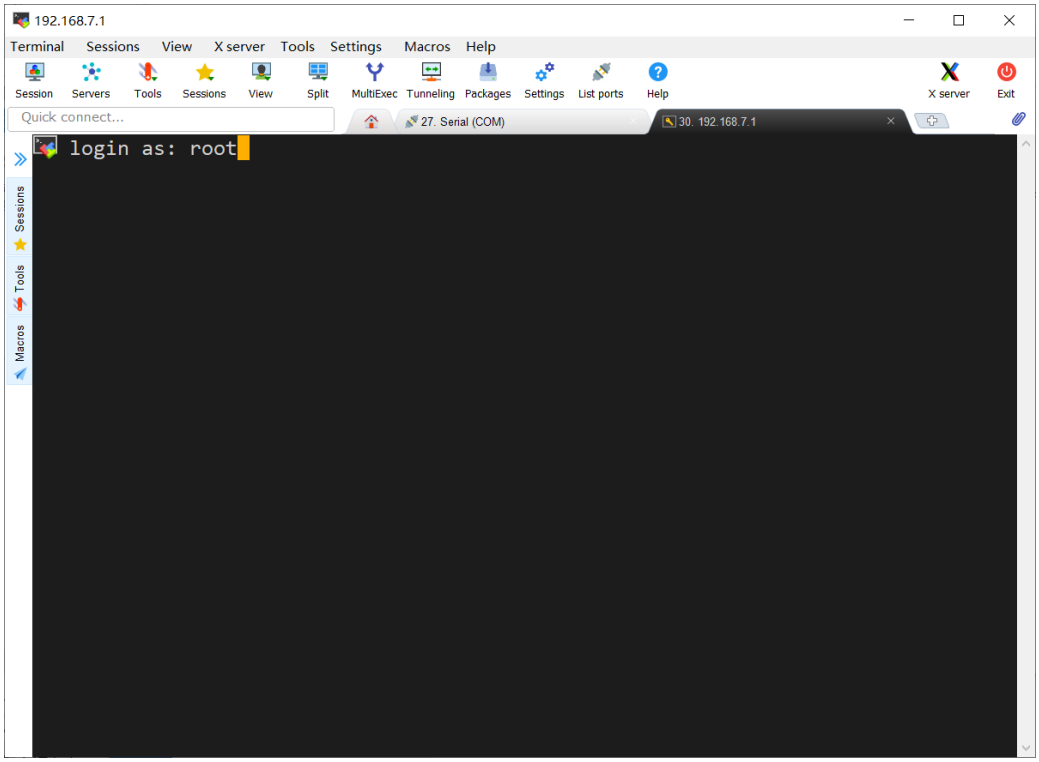


图 1.5.40 输入 Linux 账户名

登陆进去后，就可以在右边黑色终端区域输入命令操作 Linux，左边选择“Scp”标签按钮，即可切换到该标签页面，在此页面拖进拖出文件，完成电脑和开发板文件的互传，如图 1.5.41 所示。下方有个“Follow terminal folder”可以勾选上，这样在右边操作终端进入不同目录，左边的文件路径也会对应切换。

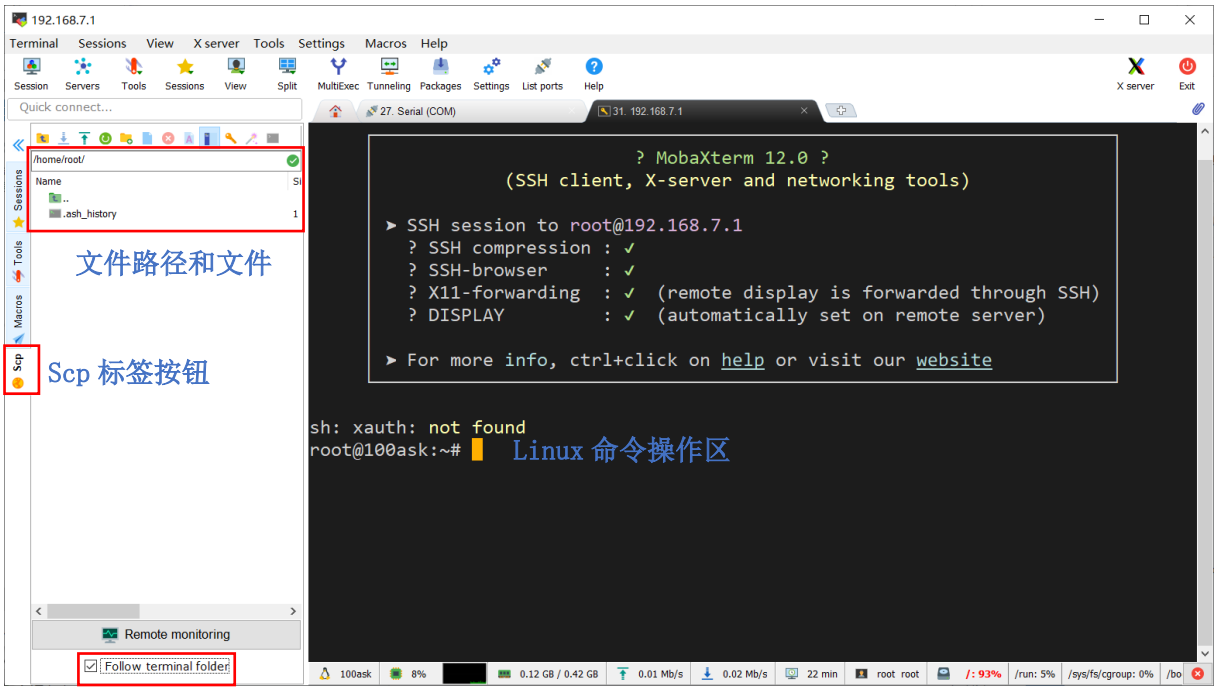


图 1.5.41 MobaXterm 操作界面

现在，在右边输入“cd /lib/firmware”命令，进入“/lib/firmware”目录，同时左边的 Scp 标签里的文件路径也自动切换到该目录（如果没有自动切换，用户手动输入路径即可）。然后将前面生成的“ProjectDemo_CM4.elf”拖到左边窗口，实现了文件的上传，如图 1.5.42 所示。

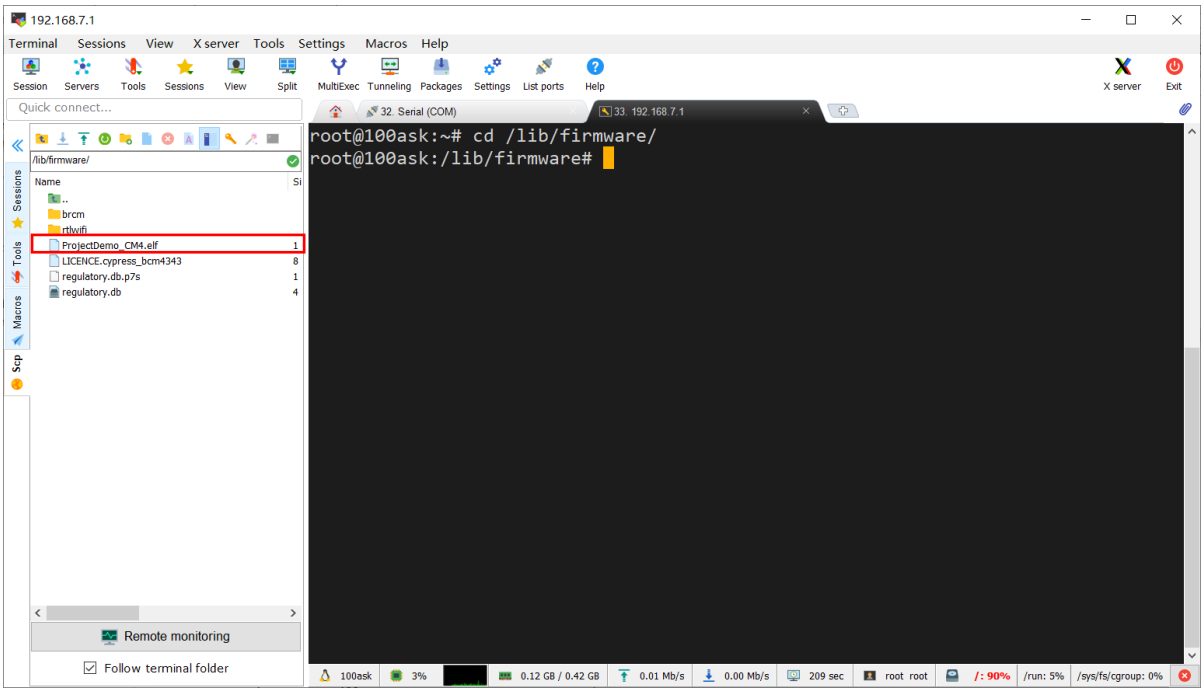


图 1.5.42 上传 M4 固件

### 1.5.5 加载运行

随后执行以下命令，加载固件，启动 M4 运行固件。

```
[root@100ask:~]$ echo ProjectDemo_CM4.elf > /sys/class/remoteproc/remoteproc0/firmware //加载固件
[root@100ask:~]$ echo start > /sys/class/remoteproc/remoteproc0/state  //启动M4
```

启动 M4 时，内核会有一堆打印信息（在串口终端执行才会打印内核信息，SSH 执行需要修改打印等级才会打印），如图 1.5.43 所示。

```
root@100ask:~# echo ProjectDemo_CM4.elf > /sys/class/remoteproc/remoteproc0/firmware
root@100ask:~# echo start > /sys/class/remoteproc/remoteproc0/state
[  178.142314] remoteproc remoteproc0: powering up m4
[  178.274011] remoteproc remoteproc0: Booting fw image ProjectDemo_CM4.elf, size 1799352
[  178.280636] remoteproc remoteproc0: header-less resource table
[  178.286520] remoteproc remoteproc0: no resource table found for this firmware
[  178.293757] remoteproc remoteproc0: header-less resource table
[  178.299386] remoteproc remoteproc0: remote processor m4 is now up
root@100ask:~#
```

图 1.5.43 M4 启动时 Linux 内核打印

此时观察开发板，原本只有绿色 LED 灯闪烁，现在黄色 LED 灯也在闪烁，如图 1.5.44 所示。

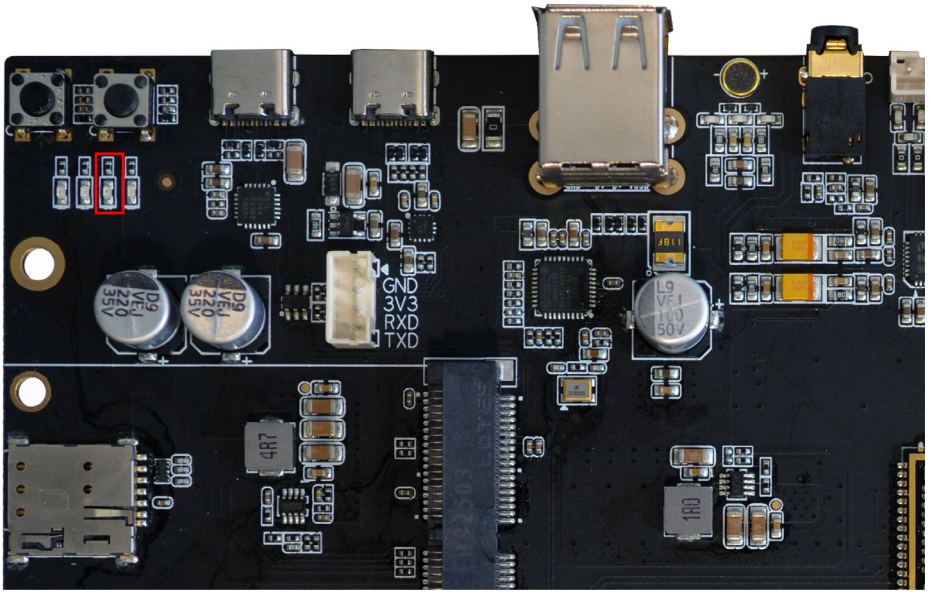


图 1.5.44 开发板黄色 LED 灯

执行以下命令可以停止 M4 运行。

```
[root@100ask:~]$ echo stop > /sys/class/remoteproc/remoteproc0/state  //停止M4
```

执行后，黄色 LED 灯停止闪烁，内核打印 M4 停止相关信息，如图 1.5.45 所示。

```
root@100ask:~# echo stop > /sys/class/remoteproc/remoteproc0/state
[  263.102777] remoteproc remoteproc0: warning: remote FW shutdown without ack
[  263.108404] remoteproc remoteproc0: stopped remote processor m4
root@100ask:~#
```

图 1.5.45 M4 停止时 Linux 内核打印

# 1.6 M4 调试

M4 的调试方式有两种：工程模式（Engineering Mode）和量产模式（Production Mode）。

- 工程模式:类似 STM32 单片机烧写到 RAM，M4 启动，无需 A7 加载。因为烧写到 RAM 里，掉电程序消失，一般用于调试。
- 量产模式:A7 启动后，通过 Remoteproc 服务启动 M4，固件先被存放在 Flash，掉电不会消失，但需要 A7 先启动，引导 M4 启动，用于实际部署。

工程模式连线、操作简单，作为前期调试最方便不过了。量产模式连线较多，需要 USB OTG 模拟网卡，相对没有那么稳定。虽然量产模式有那么多不足，但有些情况不得不使用量产模式，比如在调试核之间通信时,必须 A7 和 M4 都启动运行才能调试。

后面实验，多使用工程模式，两种模式对比如表 1.6.1 所示。

表 1.6.1 两种调试模式对比

| | 工程模式 | 量产模式 |
|---|---|---|
| 是否需要<br>J-Link/ST-link | 需要 | 需要 |
| 启动<br>方式 | BOOT[0:2]为001 | 正常eMMC/SD Card启动Linux |
| 是否需要<br>运行Linux | 不需要 | 需要板子运行Linux<br>且Linux与Windows网络连通 |
| 使用<br>建议 | 日常调试M4 | 需要A7参与的实验 |

## 1.6.1 工程模式调试方法

以前面“ProjectDemo”工程为例，介绍工程模式调试方法。首先将 ST-Link 一端连着电脑 USB，另一端经过 JTAG 转接板插在开发板 JTAG 口，如图 1.6.1 所示，一切连接完后再打开电源开关。

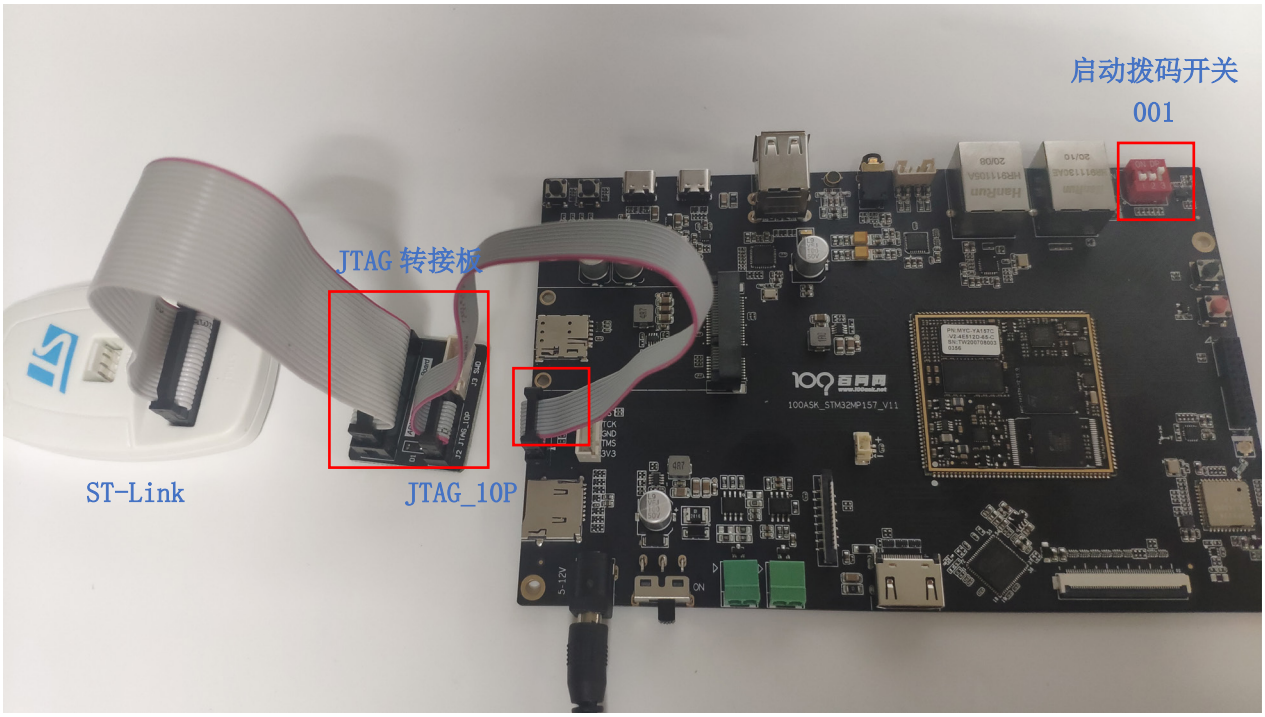


图 1.6.1 工程模式连接实物图

打开 STM32CubeIDE，在菜单栏点击“Run”->“Debug Configurations”，如图 1.6.2 所示。

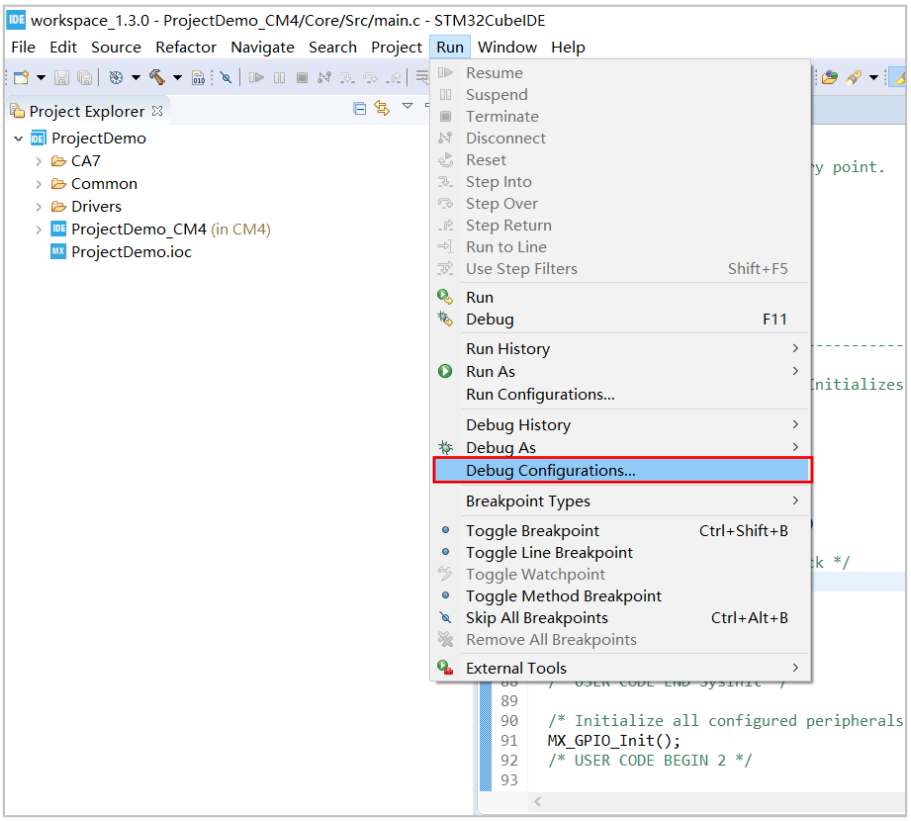


图 1.6.2 Debug 设置

随后弹出设置框，先选中图 1.6.3①处的“STM32 Cortex-M C/C++ Application”，然后点击左上角②处的新建调试按钮“New launch configuration”，右边的配置视图窗口，③处会默认选择之前编译生成的 ELF 文件（需先编译工程）。

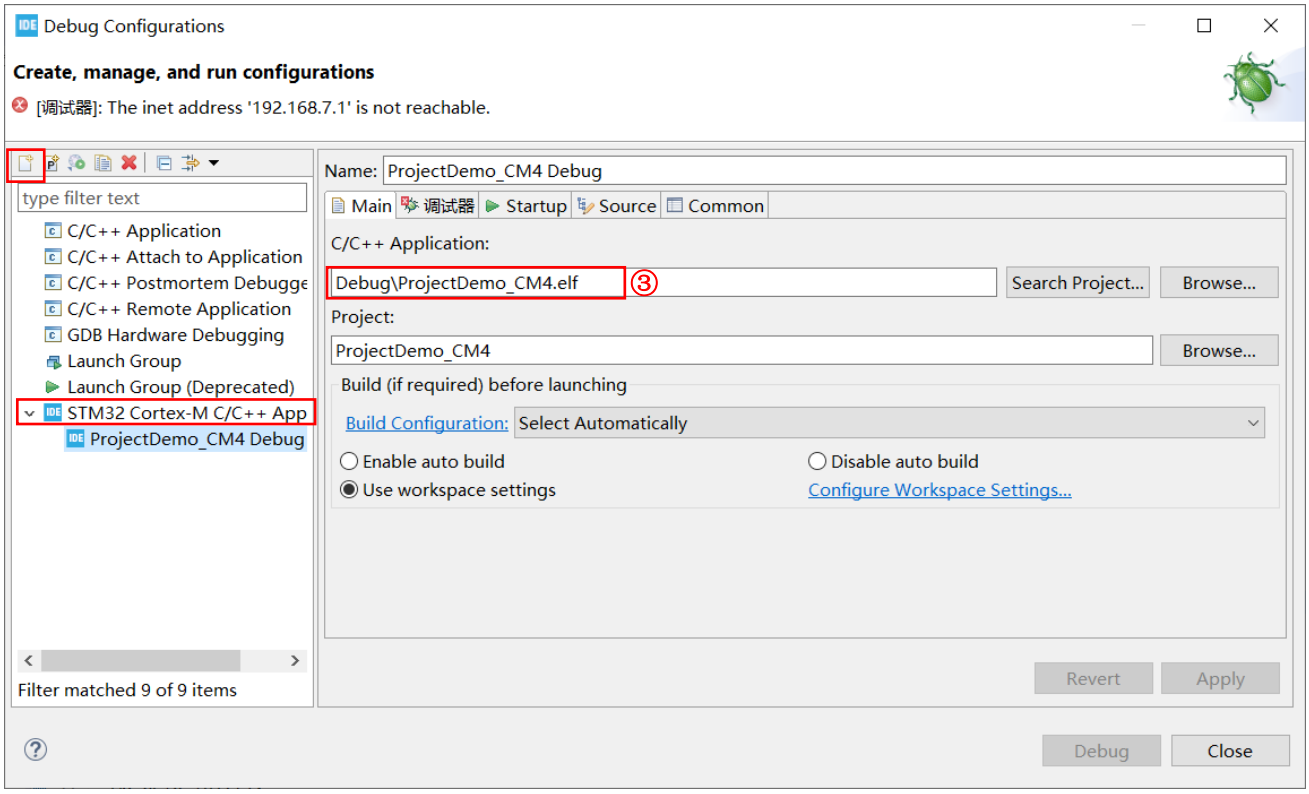


图 1.6.3 新建调试配置

然后切到“调试器”标签页，在“Load Mode”单选框里选择“thru JTAG/SWD link(Engineering mode)”，也就是工程模式，调试探头选择默认的“ST-LINK(OpenOCD)”，最后点击“Debug”开始调试，如图 1.6.4 所示。

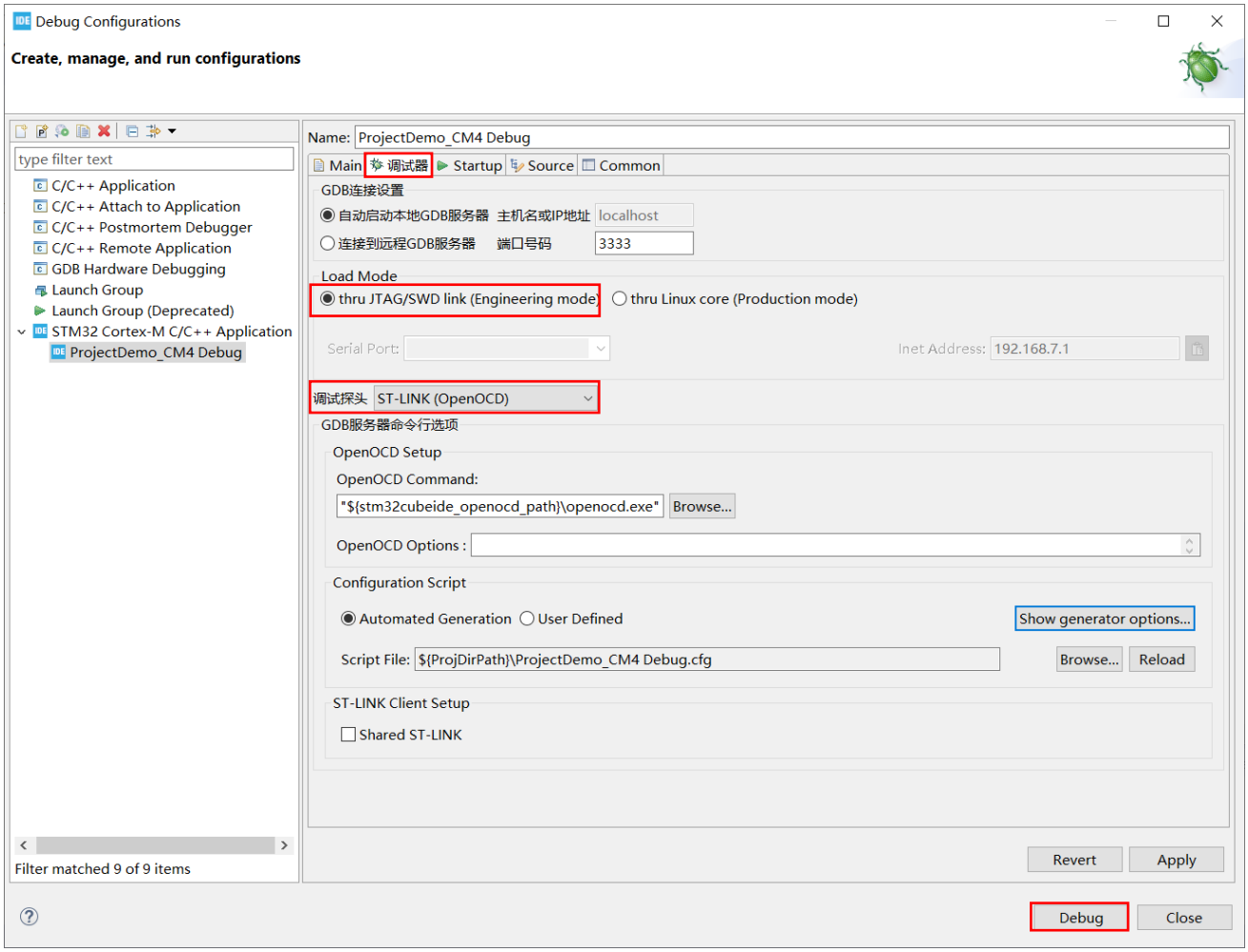


图 1.6.4 工程模式调试器设置

随后重新编译，进入调试视图，调试视图常用按钮如图 1.6.5 所示。

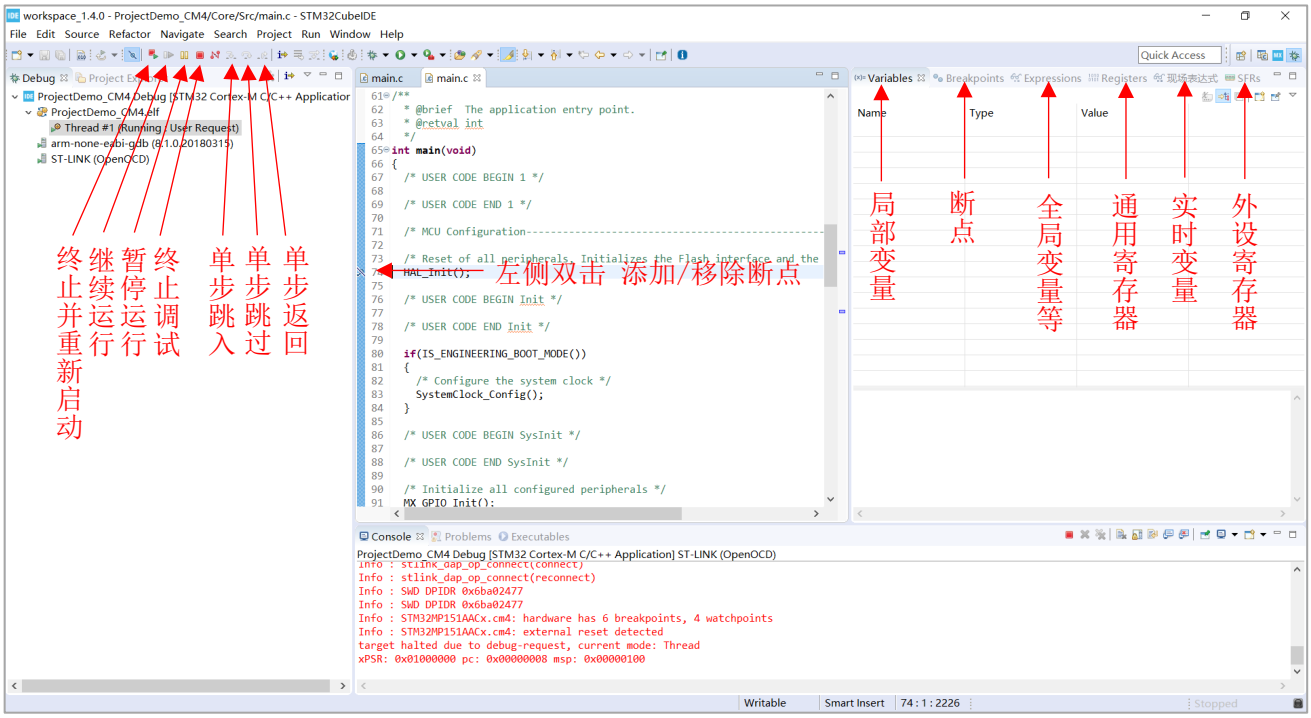


图 1.6.5 常用调试按钮

此时开发板黄色 LED 闪烁，使用工程模式可以很方便的对 M4 进行调试、验证。

## 1.6.2 量产模式调试方法

以前面“ProjectDemo”工程为例，介绍量产模式调试方法。首先将 ST-Link 一端连着电脑 USB，另一端插在开发板 JTAG 口。再使用 Type-C 线，将开发板的 USB OTG 接口与电脑相连（用于模拟 USB 网络连接），最后还需使用 Type-C 线，将开发板的 USB 串口与电脑相连（用于自动获取 USB 模拟网卡的 IP），连接如所示。

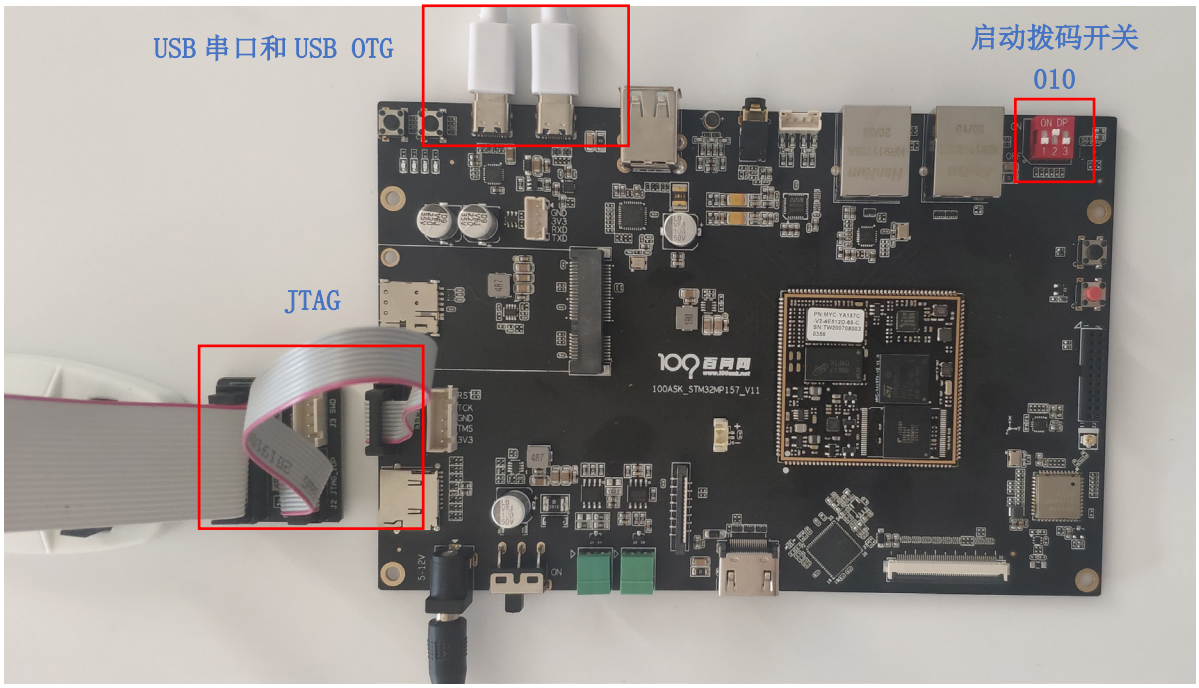


图 1.6.6 量产模式接线实物图

因为 STM32CubeIDE 使用串口获取 USB 虚拟网卡 IP，又通过 USB 网卡组成的局域网上传固件，因此无论是 USB 串口还是 USB OTG 出错，都会导致量产模式无法继续。为了防止串口端口被占用，建议此时关闭 MobaXterm 等串口助手工具。另外，STM32CubeIDE 对串口设备有筛选，默认只识别“FTDI”和“STMicroelectronics”设备，100ASK_STM32MP157 使用的 USB 转串口芯片为 CP2104，需要修改偏好设置，如图 1.6.7 所示，选择“Windows”->“Preferences”。

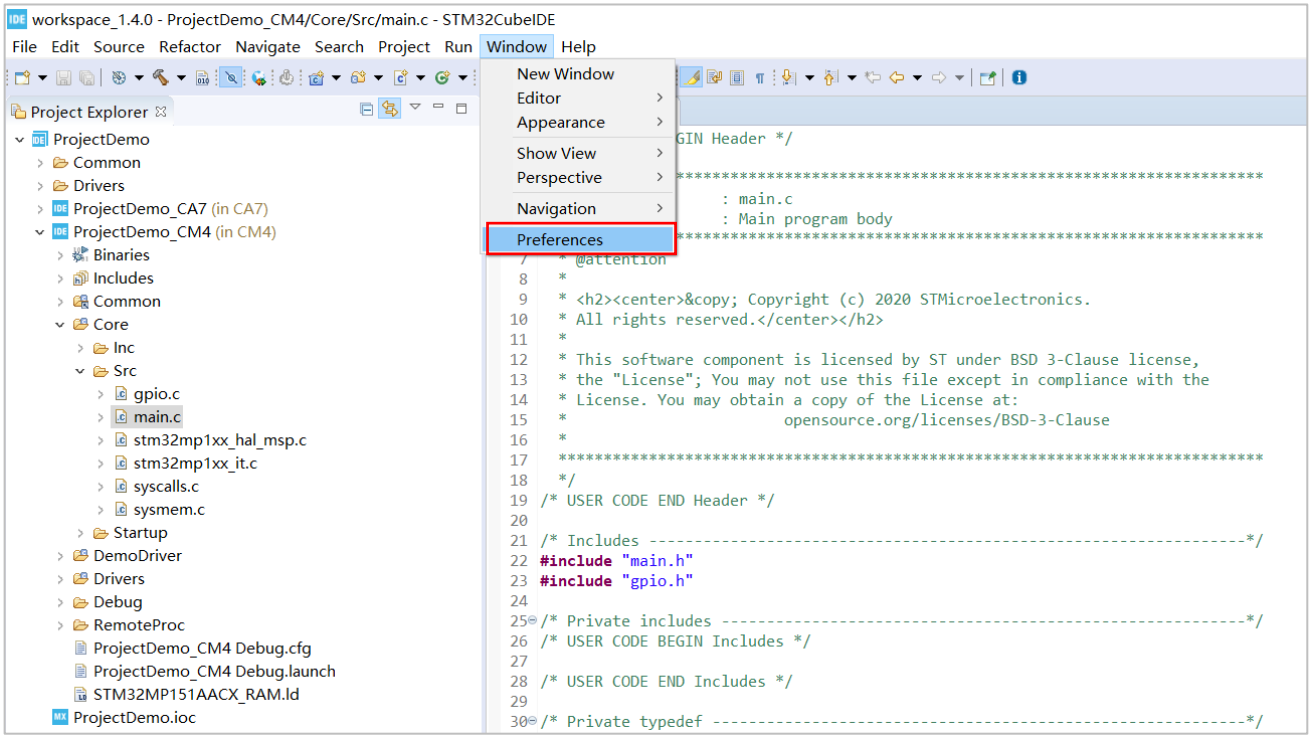


图 1.6.7 偏好设置

展开“STM32Cube”标签，选择“MPU Serial”，勾选右边的移除筛选“Remove filter”，在下拉框中选择当前开发板的端口号（以读者自己实际端口号为准，如果存在多个，需要去设备管理器查看 CP210X 对应的端口号），如图 1.6.8 所示。

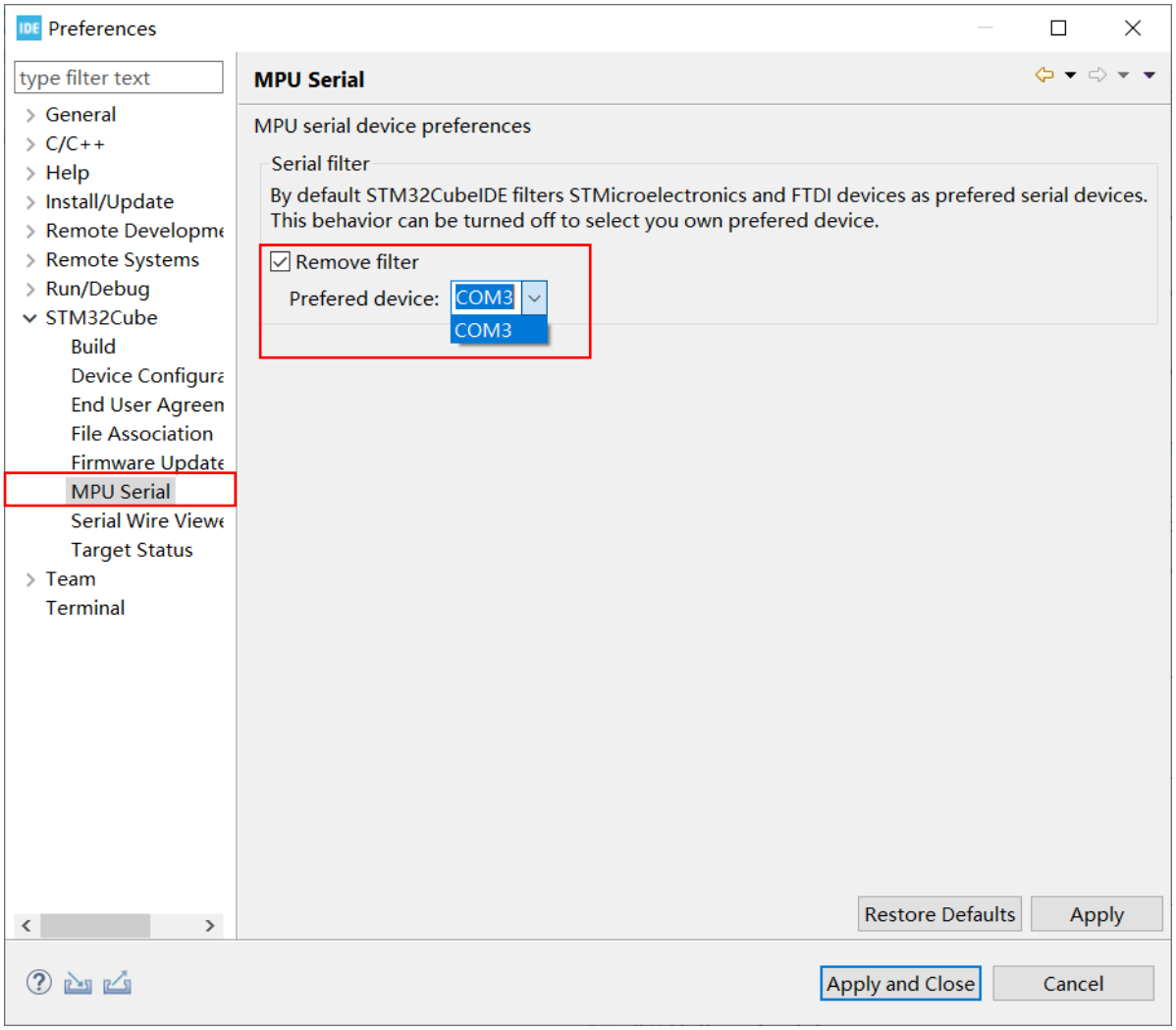


图 1.6.8 移除筛选端口号

再次打开“调试器”标签页，在“Load Mode”单选框里选择“thru JTAG/SWD link(Engineering mode)”，也就是工程模式，调试探头选择默认的“ST-LINK(OpenOCD)”，可以看到软件自动识别了串口和 USB 虚拟网卡的 IP，最后点击“Debug”开始调试，如图 1.6.4 所示。如果未识别到 IP，检查下电脑是否识别到“Remote NDIS Compatible Device”设备。如果未出现该设备，尝试先拔掉 USB OTG 线，重新启动 Linux，待进入 Linux 终端后，再稍等一会插入 USB OTG 线。

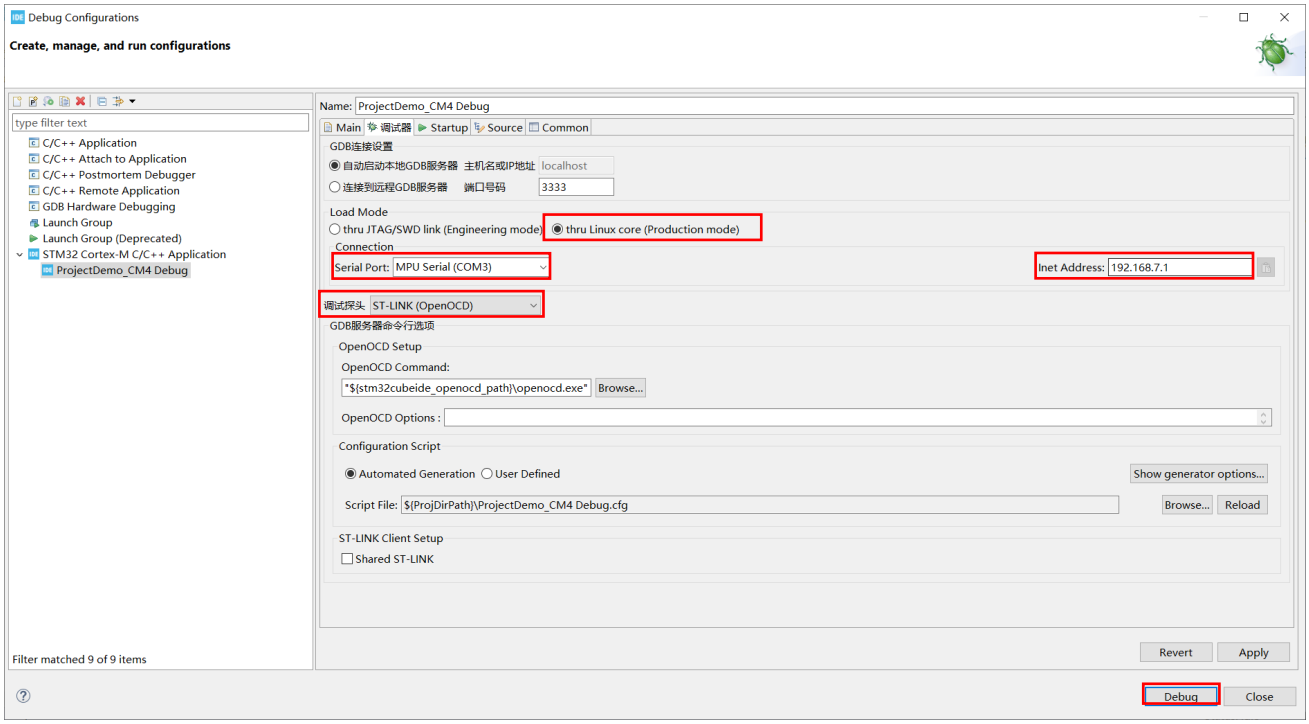


图 1.6.9 量产模式调试器设置

随后进入调试视图，后续调试操作和前面工程模式的调试方式一样，如图 1.6.10 所示。

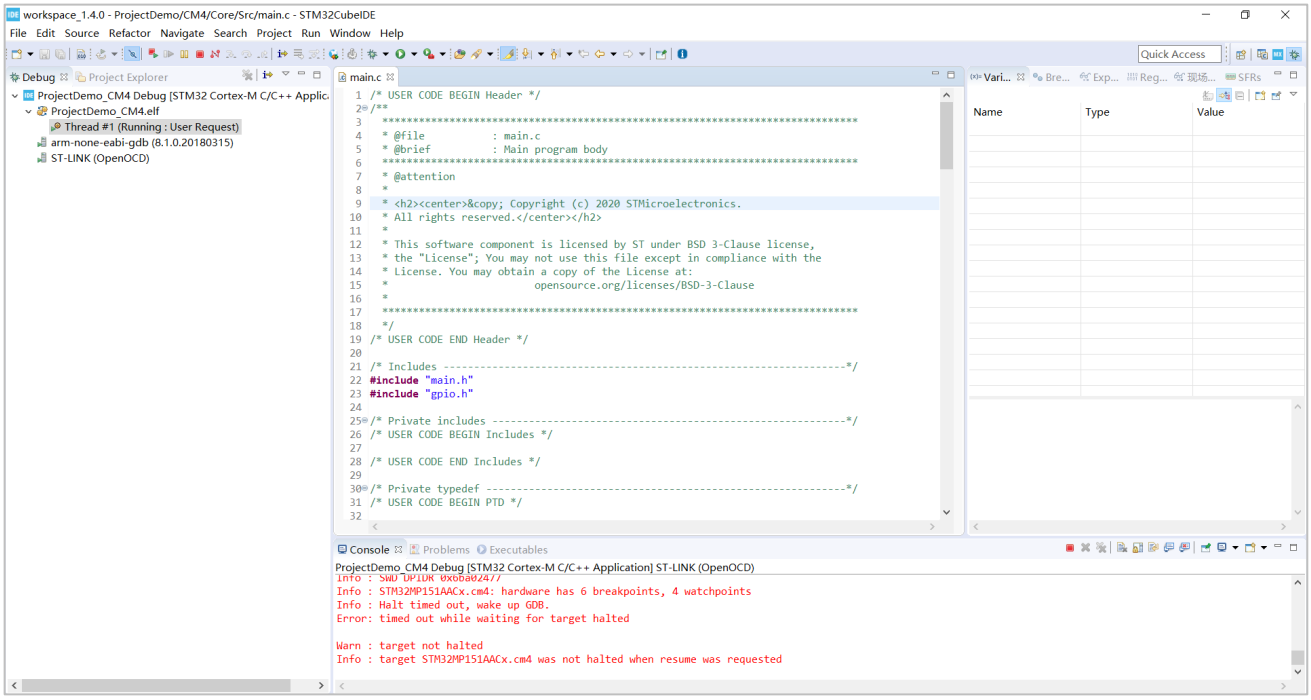


图 1.6.10 量产模式调试视图

# 1.7 工程结构分析

本节将对 STM32CubeIDE 生成的工程模板框架进行讲解，了解这些文件的作用、包含关系，为后面的学习打下基础。

一个默认生成的工程模板源文件结构如图 1.7.1 所示。

- CA7：里面包含 A7 处理器的各个阶段(TF-A、U-Boot、Kernel)的可能用到的设备树文件。之所以需要生成设备树文件，是因为 STM32MP157x 上的一些资源，只能 A7 或 M4 独享，因此需要在设备树里提前指定谁占用该资源，这个在后续的 M4 开发实例中会有用到。
- CM4：M4 工程的核心，里面有一个 M4 的工程项目，后面讲解的重点。
- Common：CMSIS 设备外设访问层，会被包含到 M4 工程里面。
- Driver：CMSIS 库、HAL 库，会被包含到 M4 工程里面。
- .mxproject 和 ProjectDemo.ioc：STM32CubeMX 工程入口和配置信息。
- .project：本 STM32CubeIDE 工程入口

图 1.7.1 STM32CubeMX 工程模板目录

可以看出，STM32CubeIDE 工程 = STM32CubeMX 工程 + STM32CubeIDE M4 工程 + A7 设备树文件。用 STM32CubeIDE 打开工程，在工程浏览器可以清楚的看到工程层级关系，如图 1.7.2 所示。

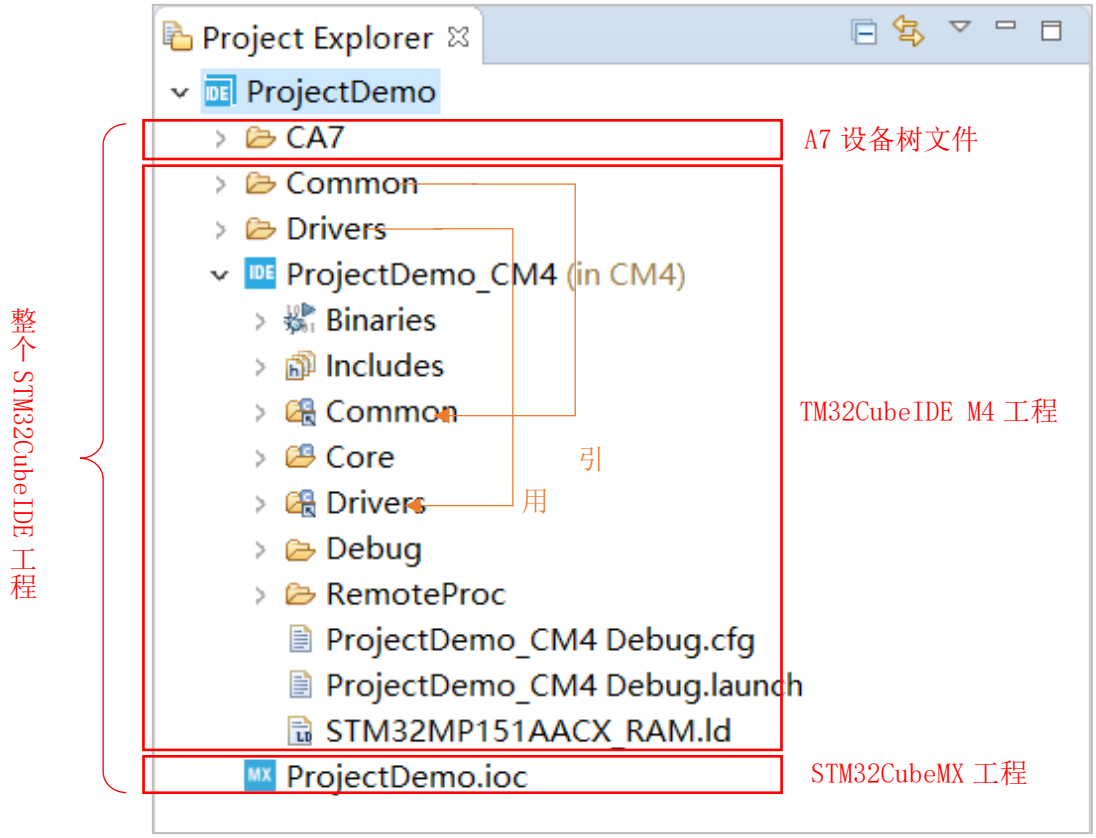


图 1.7.2 STM32CubeIDE 工程组成

STM32CubeIDE M4 是我们主要关心的内容，前面图 1.7.2 是一个经过编译、调试后的工程，内容结构介绍如下：

- Binaries：指向生成的目标 ELF 二进制文件。
- Includes：指向所有涉及的头文件。包括交叉编译工具链头文件和后面“Core”、“Driver”文件夹里相关文件的头文件。
- Common：CMSIS 设备外设访问层，里面有个“system_stm32mp1xx.c”文件，提供系统初始化函数“SystemInit()”、系统核心时钟更新函数“SystemCoreClockUpdate()”、系统核心时钟频率全局变量“SystemCoreClock”。
- Core：“Inc”文件夹里是相关头文件；“Src”文件夹里包含主函数所在文件“main.c”、资源列表文件“rsc_table.c”、MCU 特定包(MCU Specific Package)初始化文件“stm32mp1xx_hal_msp.c”、中断服务程序“stm32mp1xx_it.c”、最小系统调用文件“syscalls.c”、最小系统内存调用文件“sysmem.c”；“Startup”文件夹里是“startup_stm32mp15xx.s”，是 MCU 上电后运行的第一个程序，会初始化栈、中断向量表等，最后跳到“main()”函数。
- Drivers:“CMSIS”文件夹里是 CMSIS 设备外设访问层相关的头文件。“STM32MP1xx_HAL_Driver”文件夹就是 STM32 提供的 HAL 库源文件，每个文件的内容如表 1.7.1 所示。

表 1.7.1 HAL 库文件说明

| STM32MP1xx_HAL_Driver | |
|---|---|
| stm32mp1xx_hal_cortex.c | Cortex HAL模块驱动。包含中断优先级NVIC配置、Systick配置等 |
| stm32mp1xx_hal_dma_ex.c | DMA扩展HAL模块驱动。提供管理DMA扩展外围设备功能。 |

| | |
|---|---|
| stm32mp1xx_hal_dma.c | 直接内存访问DMA HAL模块驱动。提供初始化DMA、DMA控制等操作。 |
| stm32mp1xx_hal_exti.c | 外部中断EXTI HAL模块驱动。提供外部中断初始化、管理等操作。 |
| stm32mp1xx_hal_gpio.c | 通用输入输出GPIO HAL模块驱动。提供GPIO初始化、输入输出、中断配置等。 |
| stm32mp1xx_hal_hsem.c | 硬件信号量HSEM HAL模块驱动。提供信号量的获取、检测、清除、释放、中断处理等， |
| stm32mp1xx_hal_mdma.c | 主直接内存访问MDMA HAL模块驱动。提供初始化MDMA、MDMA控制、状态获取等操作。 |
| stm32mp1xx_hal_pwr_ex.c | 外部电源控制PWR HAL模块驱动。提供外部电源的配置、使能、休眠唤醒等。 |
| stm32mp1xx_hal_pwr.c | 电源控制PWR HAL模块驱动。提供外部电源的配置、使能、功耗切换等。 |
| stm32mp1xx_hal_rcc_ex.c | 外部复位和时钟控制RCC HAL模块驱动。提供外设时钟配置、使能、获取等。 |
| stm32mp1xx_hal_rcc.c | 复位和时钟控制RCC HAL模块驱动。提供时钟配置、使能、获取等。 |
| stm32mp1xx_hal_tim_ex.c | 定时器TIM HAL模块驱动。提供定时器扩展外设配置、控制等。 |
| stm32mp1xx_hal_tim.c | 定时器TIM HAL模块驱动。提供定时器初始化、PWM、输入捕获等。 |
| stm32mp1xx_hal.c | HAL模块驱动。包含HAL通用的API，比如延时、外设初始化等。 |

- Debug：执行编译或 debug 生成的文件，其中就包含 ELF 二进制文件；
- RemoteProc 文件夹：里面是 ST 制作的一个 Linux 脚本，用于自动运行 ELF 文件；
- ProjectDemo_CM4 Debug.*：debug 配置信息文件；
- STM32MP157AACX_RAM.ld：工程的链接脚本，指定 ELF 文件的空间布局；

在实际开发中，习惯将外设操作，定义在相应的.c 文件和.h 文件里，然后添加到工程中来使用。这样可以方便工程管理，同时也不用担心 STM32CubeMX 修改配置后更新导致自己代码丢失。

继续以“ProjectDemo”工程为例，在“CM4”目录里，新建一个“DemoDriver”目录，如图 1.7.3 所示。

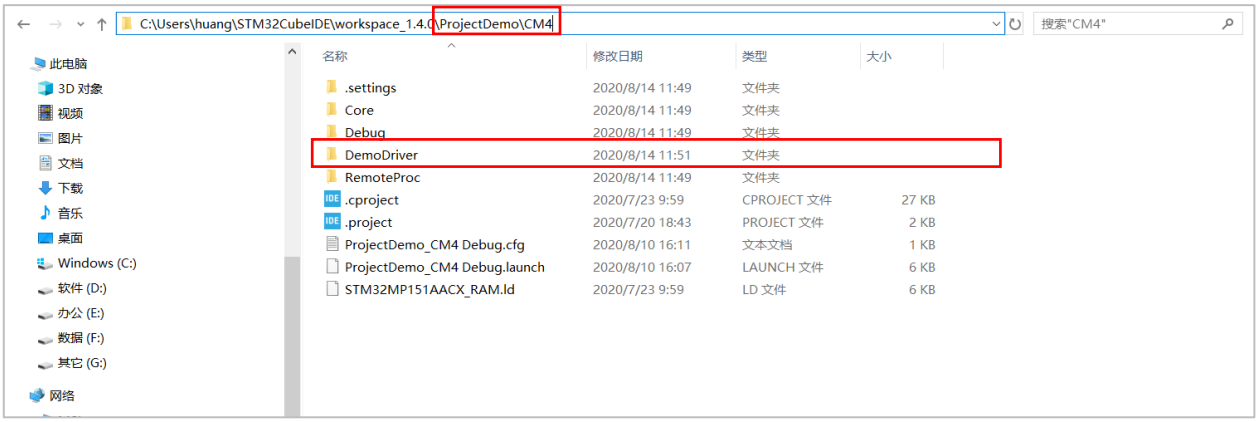


图 1.7.3 添加自定义目录

然后在 STM32CubeIDE 里选中 M4 工程“ProjectDemo”，鼠标右键选择“Properties”，如图 1.7.4 所示。

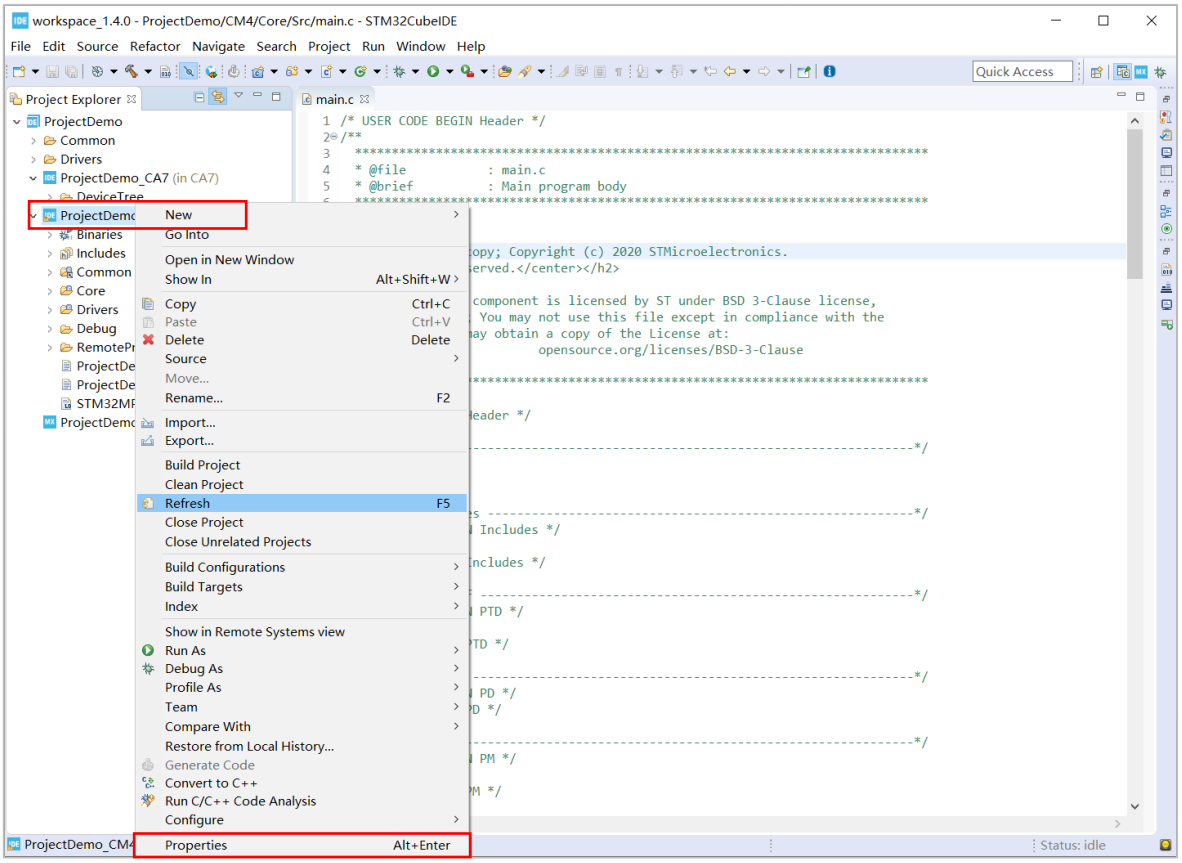


图 1.7.4 设置 M4 工程属性

接着在“C/C++ General”下选择“Paths and Symbols”，切到“Includes”标签，点击右边的“Add…”，输入刚才新建的文件夹名“DemoDriver”，最后点击应用并关闭窗口“Apply and Close”，如图 1.7.5 所示。

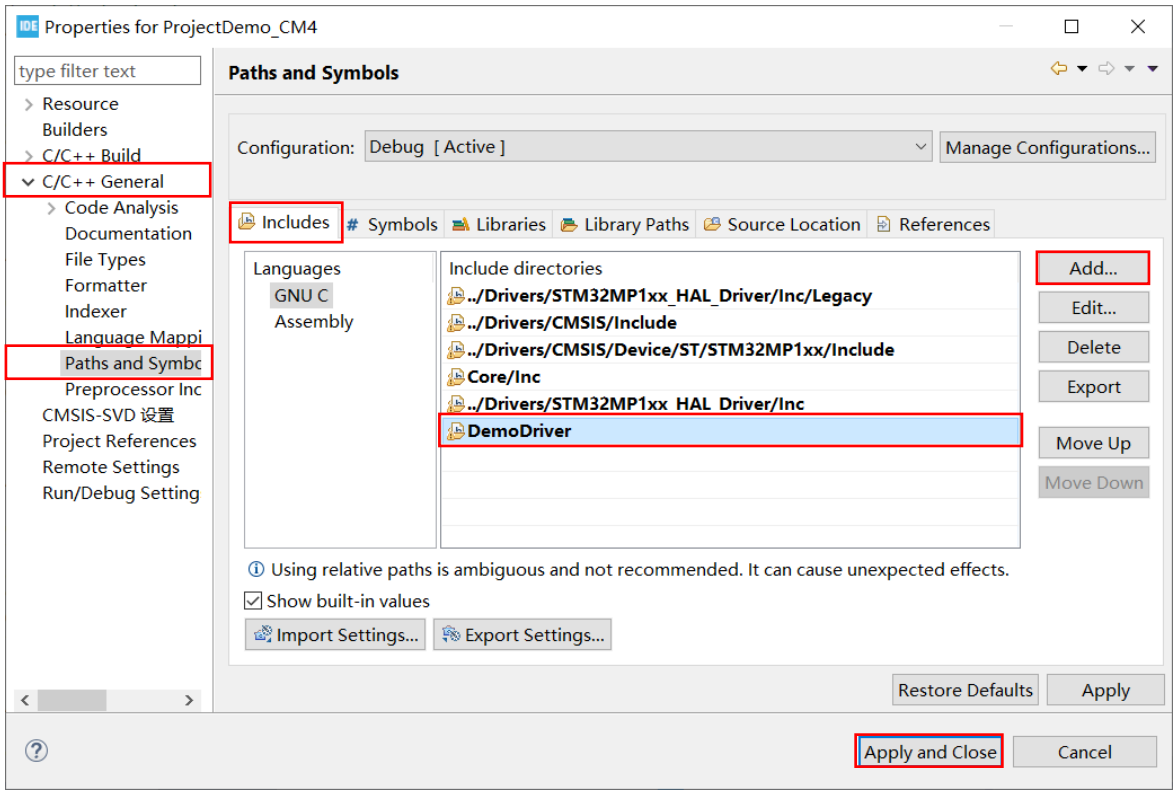


图 1.7.5 添加自定义头文件路径

再次选中 M4 工程“ProjectDemo”，鼠标右键选择刷新工程“Refresh”，便可在工程看到“DemoDriver”目录，如图 1.7.6 所示。

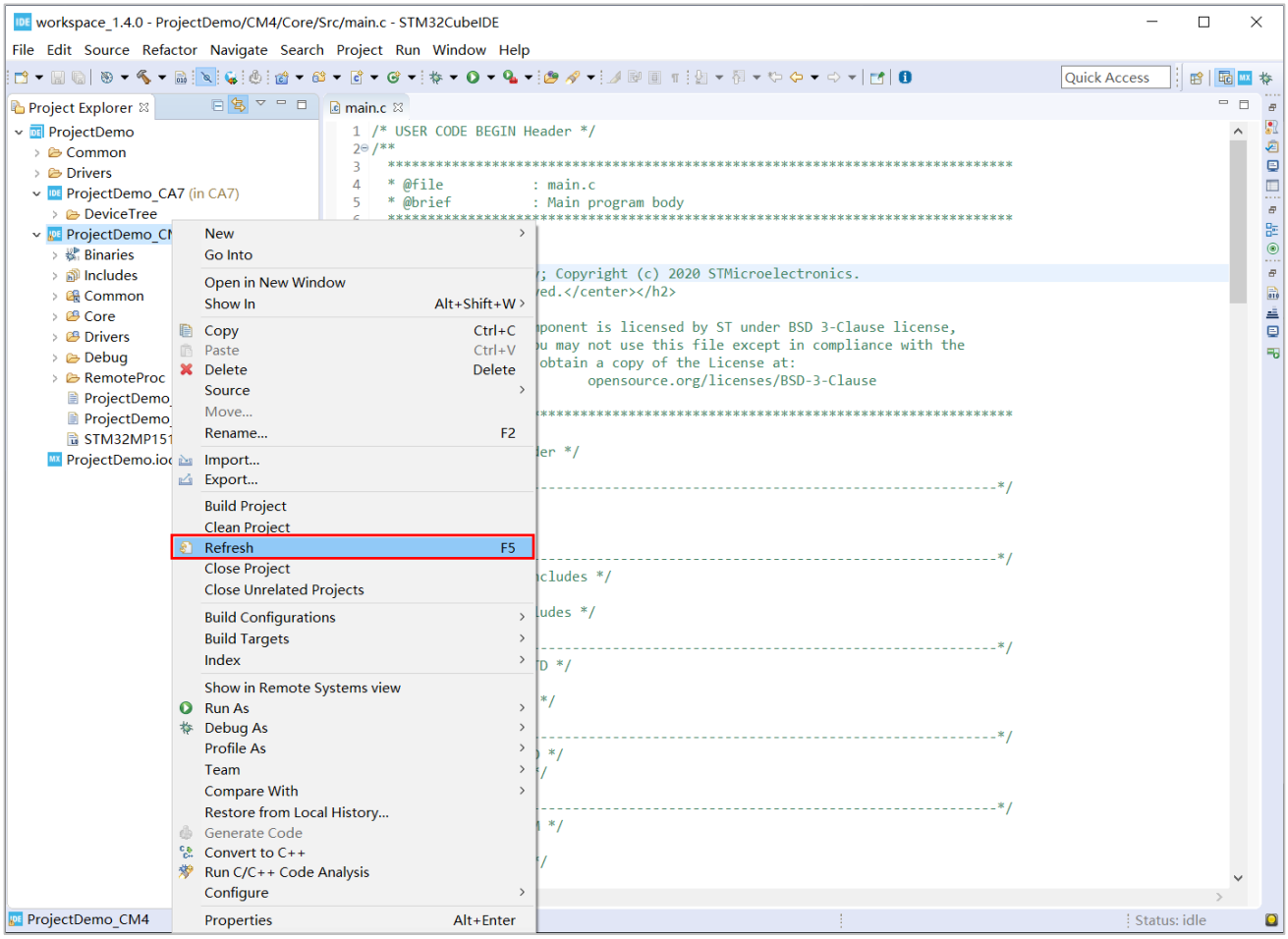


图 1.7.6 刷新工程

重新进入属性“Properties”窗口，切换到“Source Location”标签，点击右边的“Add Folder”，在弹出的窗口中选中“DemoDriver”，最后点击“Apply and Close”，如图 1.7.7 所示。

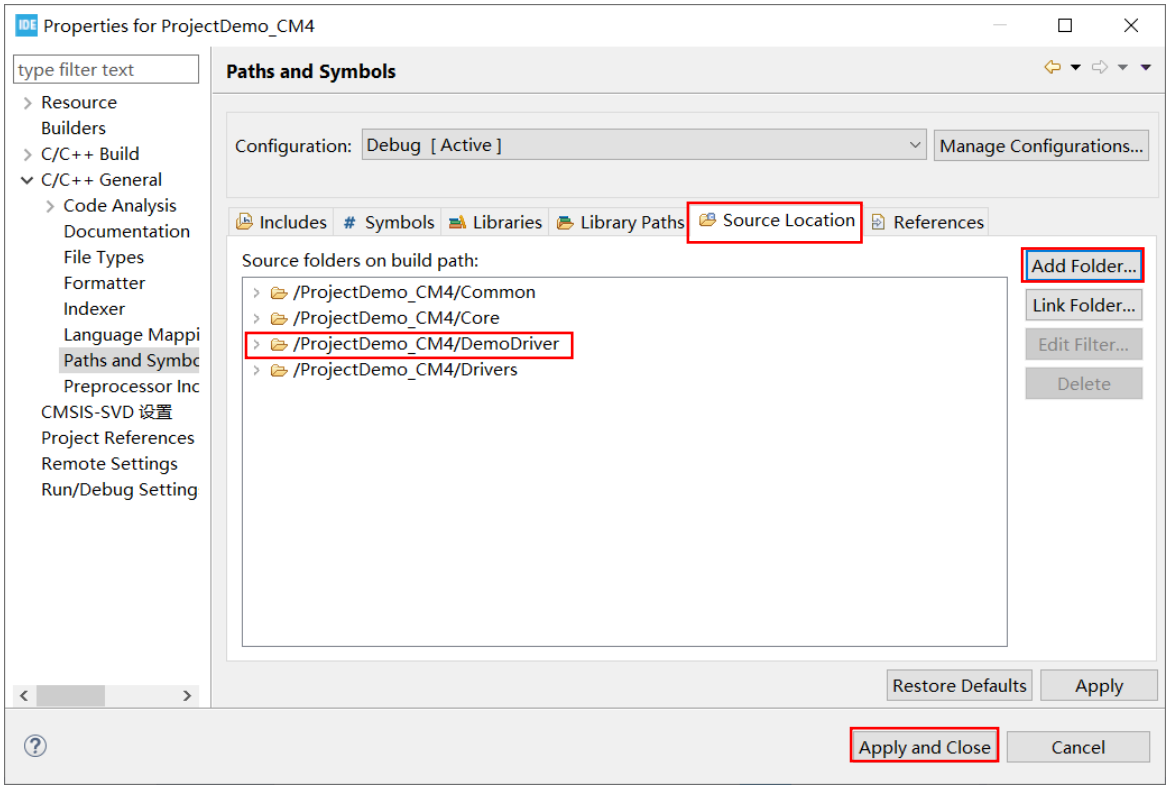


图 1.7.7 添加自定义代码文件路径

以后将外设操作文件放在“DemoDriver”目录，具体效果参考后面示例。

另外，需要一提的是，STM32CubeIDE 默认编码为 UTF-8，如果有中文注释，STM32CubeMX 动态重新生成初始化代码时，中文会部分乱码。后面示例的代码，会有中文注释，因此需要把编码格式改为 GBK，如图 1.7.8 所示，进入属性“Properties”窗口，选择“Resource”标签，在“Text file encoding”下“Other”手动输入“GBK”。这样其实也有弊端，比如串口打印中文就会乱码，用户根据需求自行取舍。

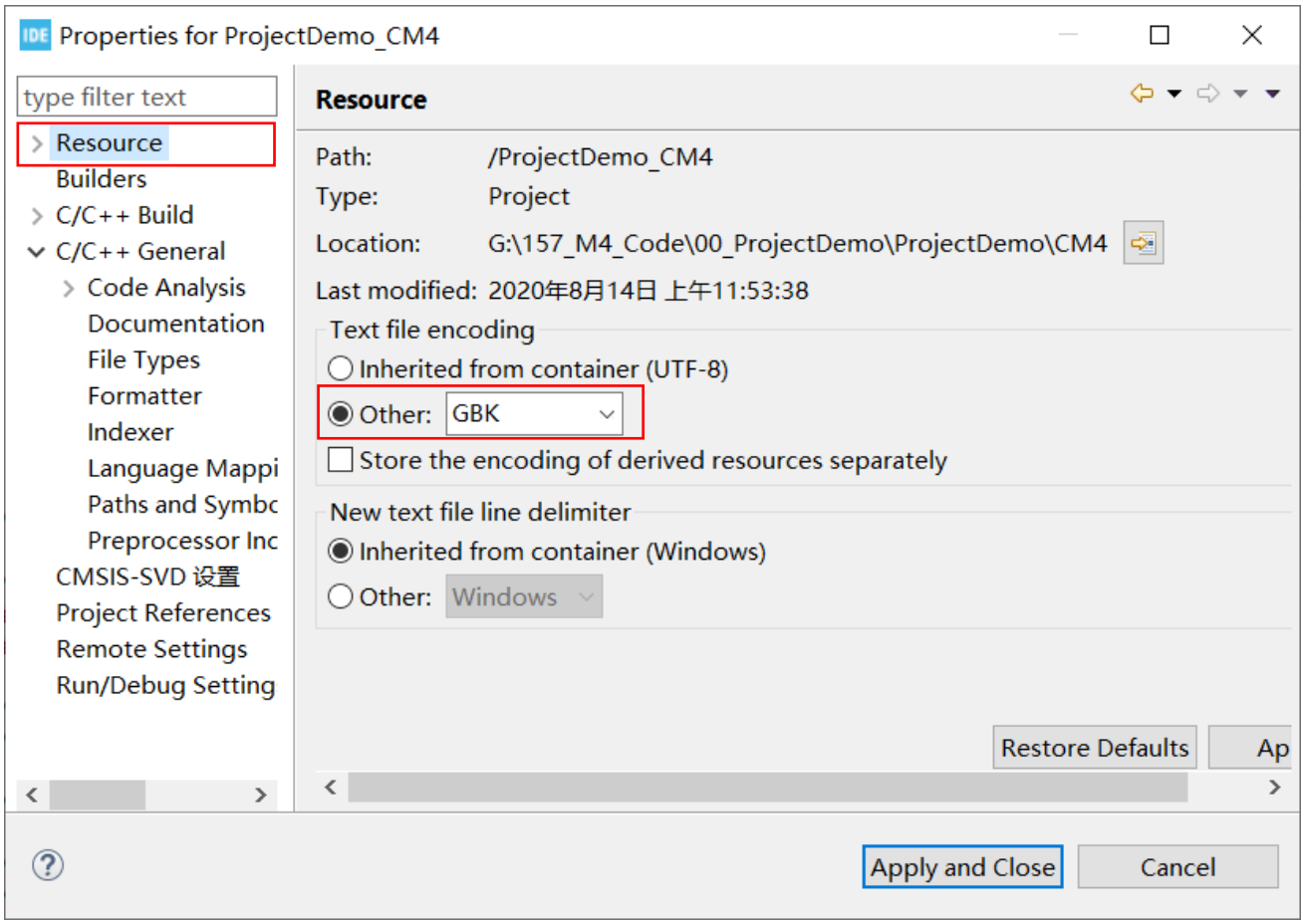


图 1.7.8 设置编码格式为 GBK

# 第2章 M4 基础开发示例

## 2.1 资源分配

STM32MP157x 片上有很多功能模块，每个功能模块/资源/外设通常称作 IP(Intelligent Property)，这些 IP 有些只能分配给 A7，有些只能分配给 M4，有些只能同时分配给其中一个，有些则可以两者同时使用。各 IP 分配的详细示意如图 2.1.1 所示。

本手册主要针对 M4 开发，因此对只能分配给 A7 的功能模块将跳过。对于同时分配给其中一个的 IP，需要配合设备树分配 IP 进行实验。对于只能分配给 M4 或共享的 IP 则直接操作进行实验。

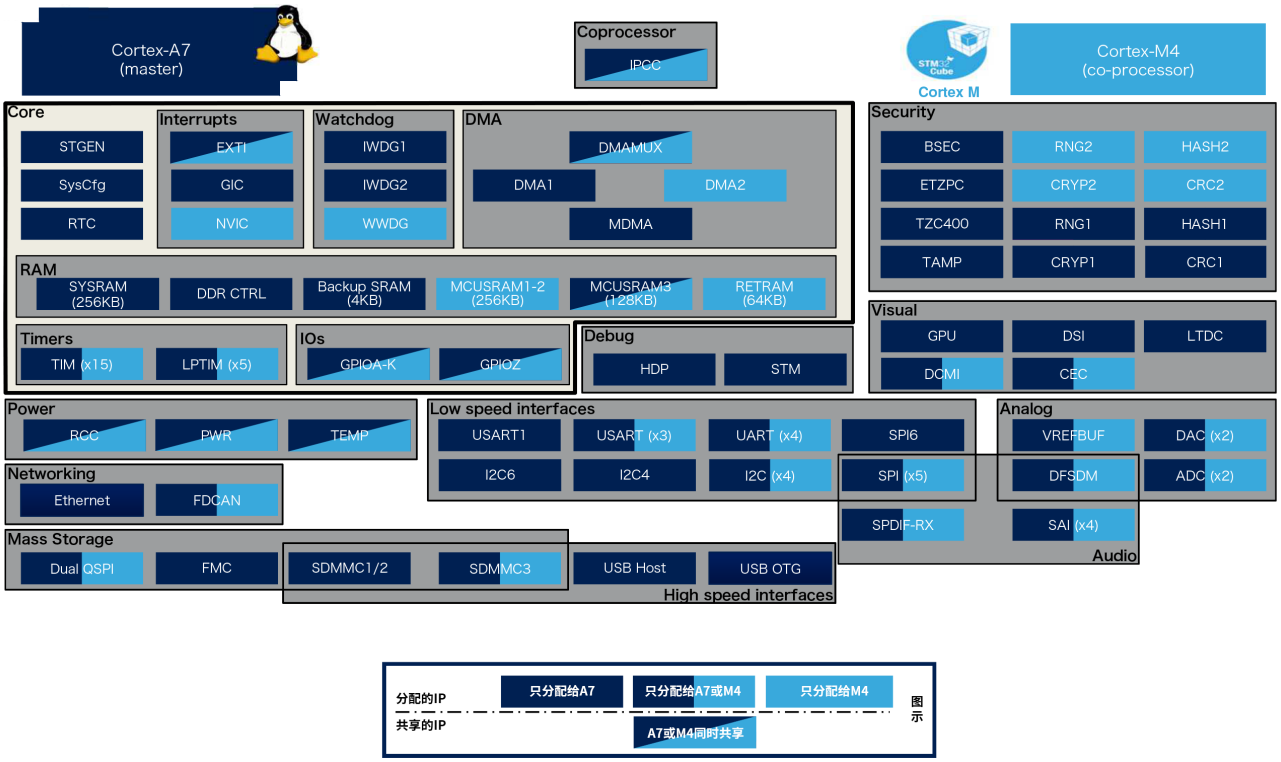


图 2.1.1 STM32MP1 的 IP 分配

为了方便大家使用，这里提供 DTBO（Device Trees Overlays）方案，动态的共享资源。比如 I²C1,启动系统后默认分给了 A7，在系统运行的某一个时刻，想启动 M4，同时把 I²C1 分配给 M4，以往的方法是修改设备树，编译替换，重新启动，比较麻烦。采用 DTBO 后，可以在 Linux 运行时把 I²C1 分配给 M4，无需重启系统，下面来体验一下动态分配流程。

在本手册配套的源码里，有个“IP_ALLOC”目录，目录里三个子文件，如下：

- “0_DTB”：里面是设备树编译出来的 DTB 文件，该 DTB 支持设备树覆盖，因此要用它替换目前系统的 DTB 文件，该操作只需执行一次；
- “1_DTBO”：里面是本文档涉及到的外设共享资源，以 IP 为最小单位，比如“100ask_stm32mp1_i2c1_m4.dtbo”就是指 I²C1 分配给 M4。此外还有 dts 源文件，供读者参考；
- “2_ELF”：存放本示例所有生成的 ELF 文件，供读者直接使用验证；

启动开发板，在 Linux 上执行“i2cdetect -l”，可以看到识别到三个 I²C 控制器。

```
[root@100ask:~]$ i2cdetect -l
i2c-1   i2c             STM32F7 I2C(0x40013000)                 I2C adapter
i2c-2   i2c             STM32F7 I2C(0x5c002000)                 I2C adapter
i2c-0   i2c             STM32F7 I2C(0x40012000)                 I2C adapter
```

利用 USB 模拟网卡组成的局域网，通过 MobaXterm SSH 登陆开发板，再将“0_DTB”目录的“stm32mp157c-100ask-512d-lcd-v1.dtb”上传到开发板任意目录，然后拷贝到“/boot”目录下，再重启开发板，如图 2.1.2 所示。

```
[root@100ask:~]$ cp stm32mp157c-100ask-512d-lcd-v1.dtb /boot
[root@100ask:~]$ reboot
```

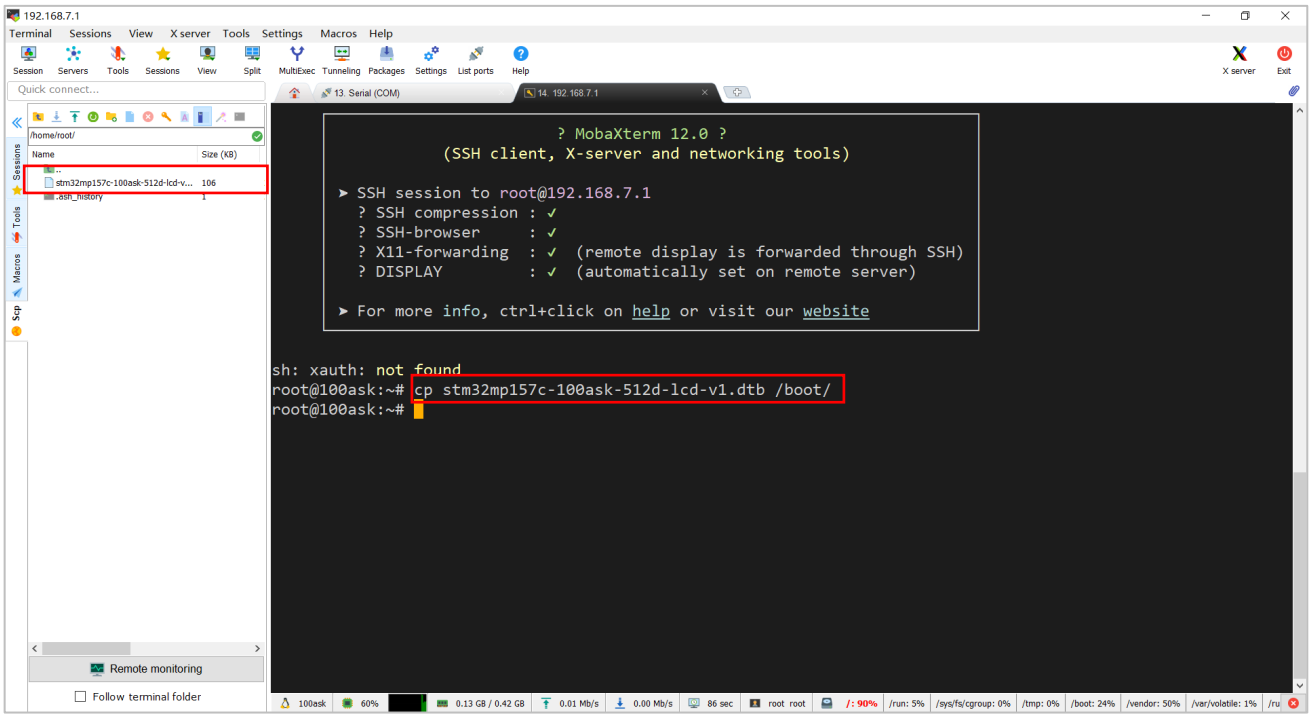


图 2.1.2 上传、替换 DTB

重启后，再上传“100ask_stm32mp1_i2c1_m4.dtbo”，然后执行以下命令，覆盖设备树。

```
[root@100ask:~]$ export DTBO_NAME=100ask_stm32mp1_i2c1_m4
[root@100ask:~]$ mkdir -p /sys/kernel/config/device-tree/overlays/$DTBO_NAME
[root@100ask:~]$ cat $DTBO_NAME.dtbo > /sys/kernel/config/device-tree/overlays/$DTBO_NAME/dtbo
[root@100ask:~]$ echo 1 > /sys/kernel/config/device-tree/overlays/$DTBO_NAME/status
```

第一行是添加个临时环境变量，简化后面命令，后面用“$DTBO_NAME”即可表示“100ask_stm32mp1_i2c1_m4”；第二行是创建文件节点，自动生成相关文件；将 dtbo 写入节点；第四行是生效覆盖。执行完后可以看到大量打印（串口终端才能默认看到内核打印信息），再执行“执行“i2cdetect -l”，可以发现少了一个 Linux 少了一个 I²C 控制器，如图 2.1.3 所示。

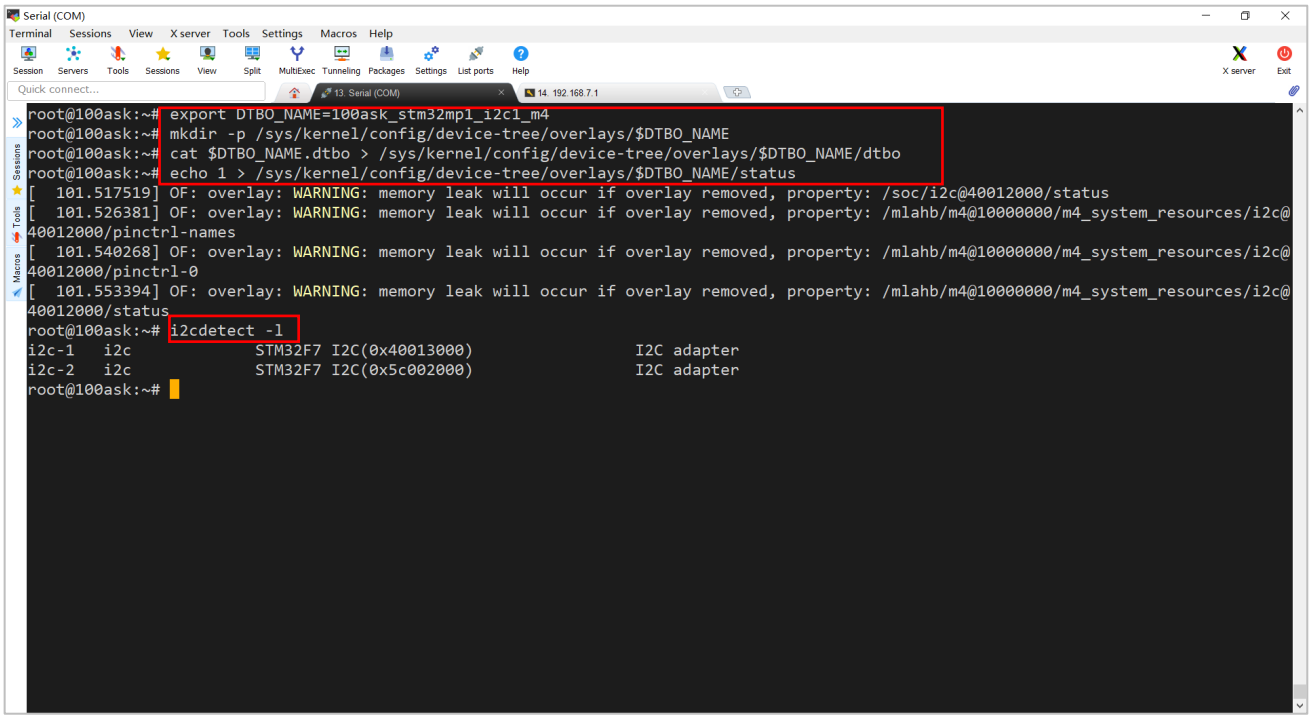


图 2.1.3 设备树覆盖效果

后面实验如果有 IP 只能分配给 A7 或 M4 其中一个，调试时使用工程模式，不涉及资源分配操作。如果是实际项目部署，或者需要 A7 和 M4 同时运行的实验，就按此操作动态覆盖设备树分配 IP 资源。

# 2.2 闪烁 LED 灯

- 设计需求

前面一节，以 LED 为例熟悉了开发流程，本小节将从实际开发的角度示例 LED 灯。
这里假设需求为间隔 1 秒，交替点亮、熄灭两个 LED 灯。

## 2.2.1 基础知识

LED(light-emitting diode)，即发光二极管，俗称LED灯。LED灯作为二极管，具有单向导通性，需要正极端电压高于负极端约3V才能导通，亮度则由通过的电流大小决定，一般5-10mA即可。

MCU有很多引脚，大部分引脚都可以供用户使用。引脚可以作为功能引脚或GPIO，功能引脚就是作为特殊功能的引脚，比如用于串口传输引脚；GPIO就是输出高低电平或者获取外部输入的电平高低。如果将LED灯与引脚相连，然后设置引脚为GPIO功能，并输出高低电平就可以控制亮灭LED灯了。

通常LED灯与MCU引脚的方式有如图 2.2.1所示的两种方式。方式一的LED灯正极与电源正极连接，LED灯负极与MCU的GPIO连接，此时LED灯正极将一直为高电平，灯的亮灭将由LED负极所连的GPIO电平决定。GPIO输出高电平，LED灯两端都为高电平，灯熄灭。GPIO输出低电平，LED灯正级端电压高于负极端电压约3.3V，LED灯将点亮。理解方式一后，再看方式二，LED负极端连的电源地，将一直为低电平，LED灯的亮灭将由正级端所连接的GPIO决定，GPIO为高时LED灯将点亮，反之则熄灭。

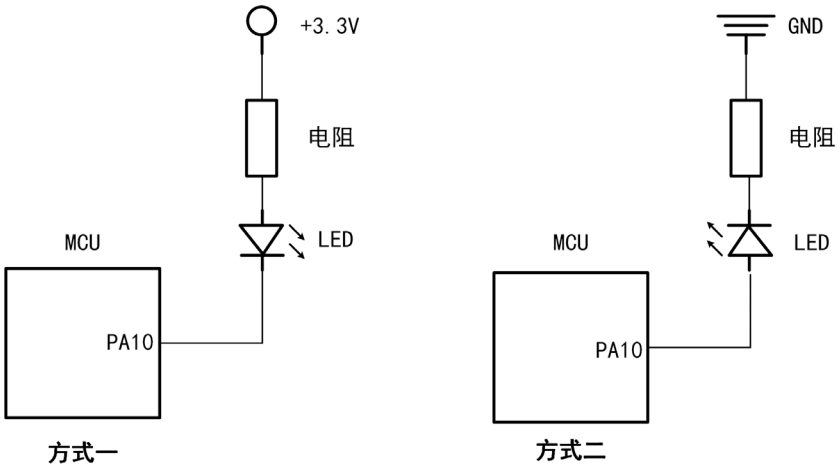


图 2.2.1 LED 与 MCU 两种常见连接方式

由此，可以看出，LED的亮灭可以由GPIO输出的电平高低决定，但是输出高电平点亮LED还是熄灭LED灯，需要看原理图是如何设计的。此外，还需要看原理图LED所接在哪个引脚，从而控制对应引脚的电平。

## 2.2.2 硬件设计

打开配套资料里底板的原理图文件《100ASK_STM32MP157_PRO_V11.pdf》，LED原理图部分如图 2.2.2所示。

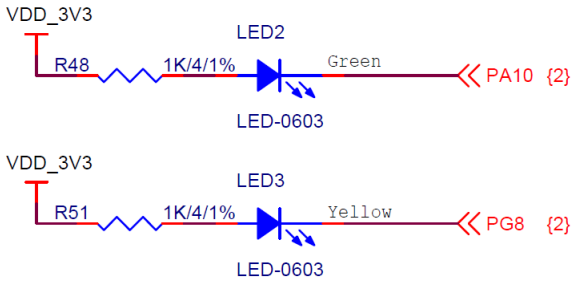


图 2.2.2 开发板 LED 原理图

可以看到，这里采用的是前面的介绍的方式一连接方式。由原理图，可以得到两个重要信息：

1) LED 使用哪个引脚：
LED Green 接在 PA10 上，LED Yellow 接在 PG8 上。
2) 怎么控制引脚才能点亮/熄灭 LED：
两个灯都采用方式一连接，GPIO 输出高电平时，灯熄灭；GPIO 输出低电平时，灯点亮。

### 2.2.3 MX 设置

参考前面开发流程里的创建模板，使用STM32CubeIDE里的STM32CubeMX创建工程代码，前面图1.5.21STM32CubeMX配置引脚详细参数图里，有好几个选项，这里依次介绍每个选项的含义。

- GPIO mode：用于设置输出模式。可选有“Output Push Pull”(推挽输出)、“Output Open Drain”(开漏输出)。在不考虑上拉电阻的情况下，需要较强驱动能力的场合，选择推挽输出，需要电平转换、线与功能的场合，选择开漏输出。
- GPIO Pull-up/Pull-down：用于设置IO口上下拉模式。可选有“Pull-up”(上拉)、“Pull-dwon”(下拉)、“No pull-ip and no pull-down”(没有上下拉)。上/下拉的主要目的是将GPIO信号在不确定状态有个确定的电平。
- Maximum output speed：用于设置GPIO高低切换的速度。可选有“Low”(低速)、“Medium”(中速)、“High”(高速)、“Very High”(超高速)。切换速度越高,性能越好，但功耗越大。
- User Label：用来设置GPIO的别名，体现在代码里就是宏。比如这里设置为“LED_GREEN”,在代码里可以用“LED_GREEN”代替“GPIO_PIN_10”，用“LED_GREEN_GPIO_Port”代替“GPIOA”，让代码有更强的可读性。

两个 LED 灯配置如图 2.2.3 所示。

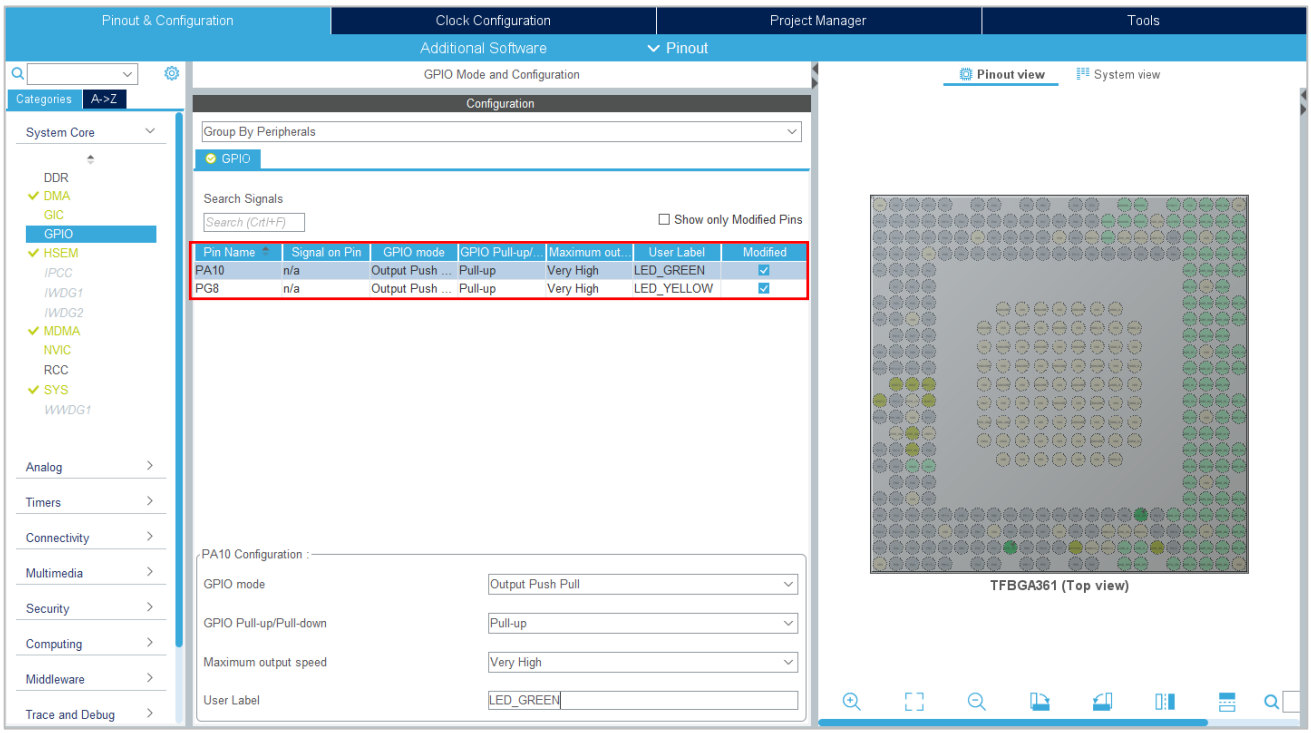


图 2.2.3 开发板 LED 灯 MX 配置

## 2.2.4 代码设计

参考前面工程结构分析，在在“CM4”目录里，新建一个“DemoDriver”目录，同时创建“driver_led.c”和“driver_led.h”，如图 2.2.4 所示。

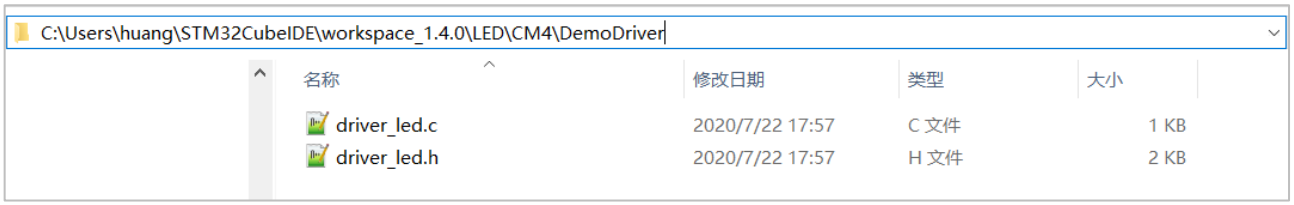


图 2.2.4 创建 LED 操作文件

前面示例使用的“HAL_GPIO_WritePin()”来操作GPIO，现在可以在自定义添加的文件里，再简化一点，使用宏定义代替函数操作“driver_led.h”内容如下：

```
#include "main.h"
#include "stm32mp1xx_hal.h"

/********  LED 硬件相关宏定义 ********/
#define LED_GREEN_ON()      HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_RESET)
#define LED_GREEN_OFF()     HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_SET)

#define LED_YELLOW_ON()     HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_RESET)
#define LED_YELLOW_OFF()    HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_SET)

/********  LED 函数声明 ********/
extern void DemoLedInit(void);
```

```
#endif
```

“driver_led.c”可以实现一些控制逻辑，主函数只需要调用即可，“driver_led.c”代码如下：

```
#include "driver_led.h"

void DemoLedInit(void)
{
  /* LED 对应的引脚的初始化,已经由 STM32CubeMX 在 main.c 自动生成 */

  /* 两个灯初始化状态为亮 */
  LED_GREEN_ON();
  LED_YELLOW_ON();
}

void LedBlinking(void)
{
  /* LED2(GREEN)和 LED3(YELLOW)间隔 1S 闪烁  */
  LED_GREEN_ON();
  LED_YELLOW_ON();
  HAL_Delay(1000);

  LED_GREEN_OFF();
  LED_YELLOW_OFF();
  HAL_Delay(1000);
}
```

在“driver_led.c”实现需要的控制效果后，“main.c”调用即可：

```
  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  /* USER CODE BEGIN 2 */
  DemoLedInit(); // 自定义引脚初始化
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
LedBlinking(); // LED 闪烁
  }
  /* USER CODE END 3 */
```

### 2.2.5 实验效果

因为GPIO是共享IP，可以同时A7和M4同时访问，因此无需替换设备树。且使用工程模式调试，无需Linux运行，如图 2.2.5所示效果。

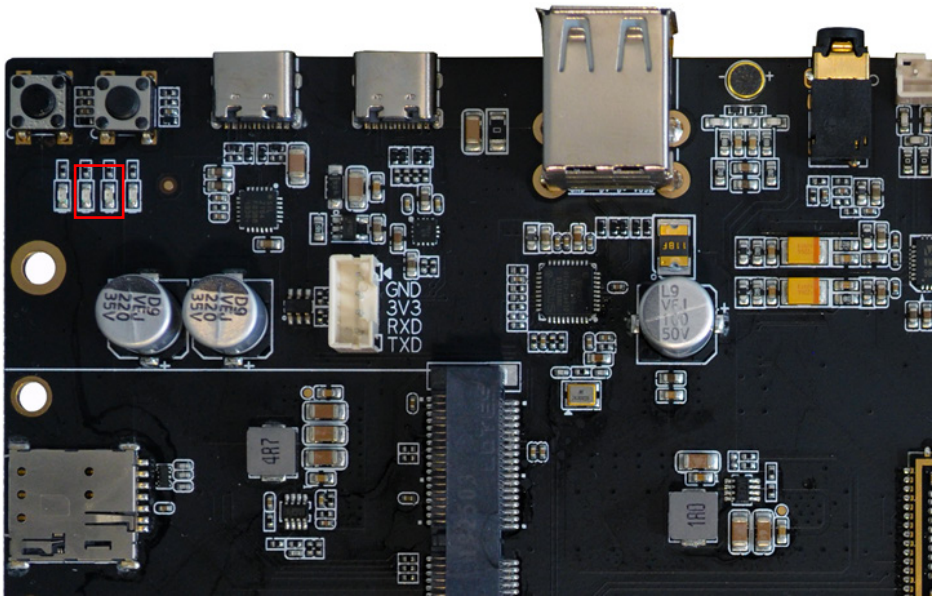


图 2.2.5 M4 闪烁 LED 灯实验效果

# 2.3 系统时钟—HSI

- 设计需求

这里假设需求为使用内部时钟 HSI，让 STM32MP157 的 M4 全速运行在最高 209MHz。

## 2.3.1 基础知识

时钟是处理器运行的基础，时钟信号推动芯片内部各部分执行相应的指令。时钟如同人体内部的心脏一样，心脏跳动一下，血液就流动一下。时钟产生一次，推动处理器就执行一下指令。除了 CPU，芯片上所有的外设都需要时钟(GPIO、$I^2C$、SPI 等)，由此可见时钟的重要性。

在芯片运行的时钟频率里，频率越高，芯片处理的速度越快，但同时功耗也越高。为了功耗和性能兼顾，微处理器一般有多个时钟源，同时还将时钟分频为多个大小，适配不同需求的外设。

STM32MP157 的 M4，拥有五个时钟源：

- HSI(High-speed internal oscillator)：

HIS 是内部的高速 RC 振荡器，频率 64MHz。因为是内部提供，可以降低成本，缺点是精度较差。

- HSE(High-speed external oscillator)：

HSE 是外部的高速振荡器，需要外部电路晶振，输入频率范围要求为 4-48MHz。因为需要外部电路提供，成本会增加，但精度较好。

- LSI(Low-speed internal oscillator)：

LSI 是内部的低速 RC 振荡器，频率 32KHz。一般用于看门狗、RTC 实时时钟、DAC 等。

- LSE(Low-speed external oscillator)：

LSE 是外部的低速振荡器，需要外部电路晶振，输入频率范围要求为 0-1000KHz，常使用 32.768KHz。一般用于 RTC 实时时钟。

- CSI(Low-power internal oscillator)：

CSI 是内部的低功耗 RC 振荡器，频率 4MHz。主要用于低功耗，需要低功耗时代替 64MHz 的 HSI。

有了多个时钟源后，需要经过选择器选择使用其中哪个时钟源，选择器（MUX）的示意图如图 2.3.1 所示。可以看到有三个时钟源可选:HIS、CSI、HSE，最终只会选择其中一个，比如这里选择的 HSI。

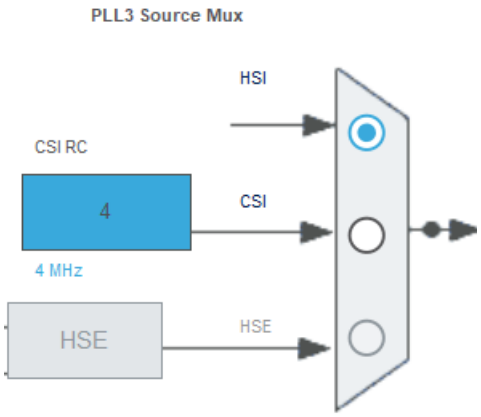


图 2.3.1 时钟源选择器图示

M4 最高频率达 209MHz，前面提到的任一时钟源，都没有这么高。因此芯片里面还会有一个 PLL(锁相环)用来倍频，是频率翻倍到指定值。锁相环倍频器示意图如图 2.3.2 所示，图中左下角有“PLL1”注释，且有“X25”字样，说明会倍频 25 倍。此外，里面还有“/2”字样，表示二分频，将频率减半。

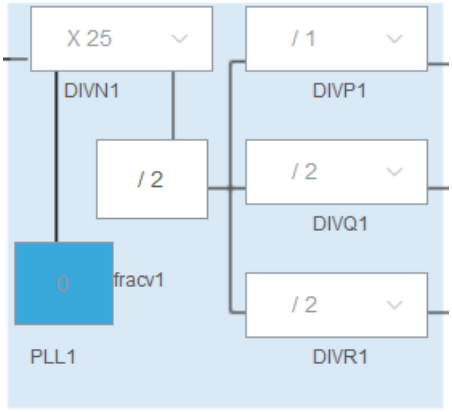


图 2.3.2 PLL 锁相环图示

有了这前面的知识，就可以看懂 STM32CubeMX 里面的时钟设置了，STM32MP157 的时钟如所示。

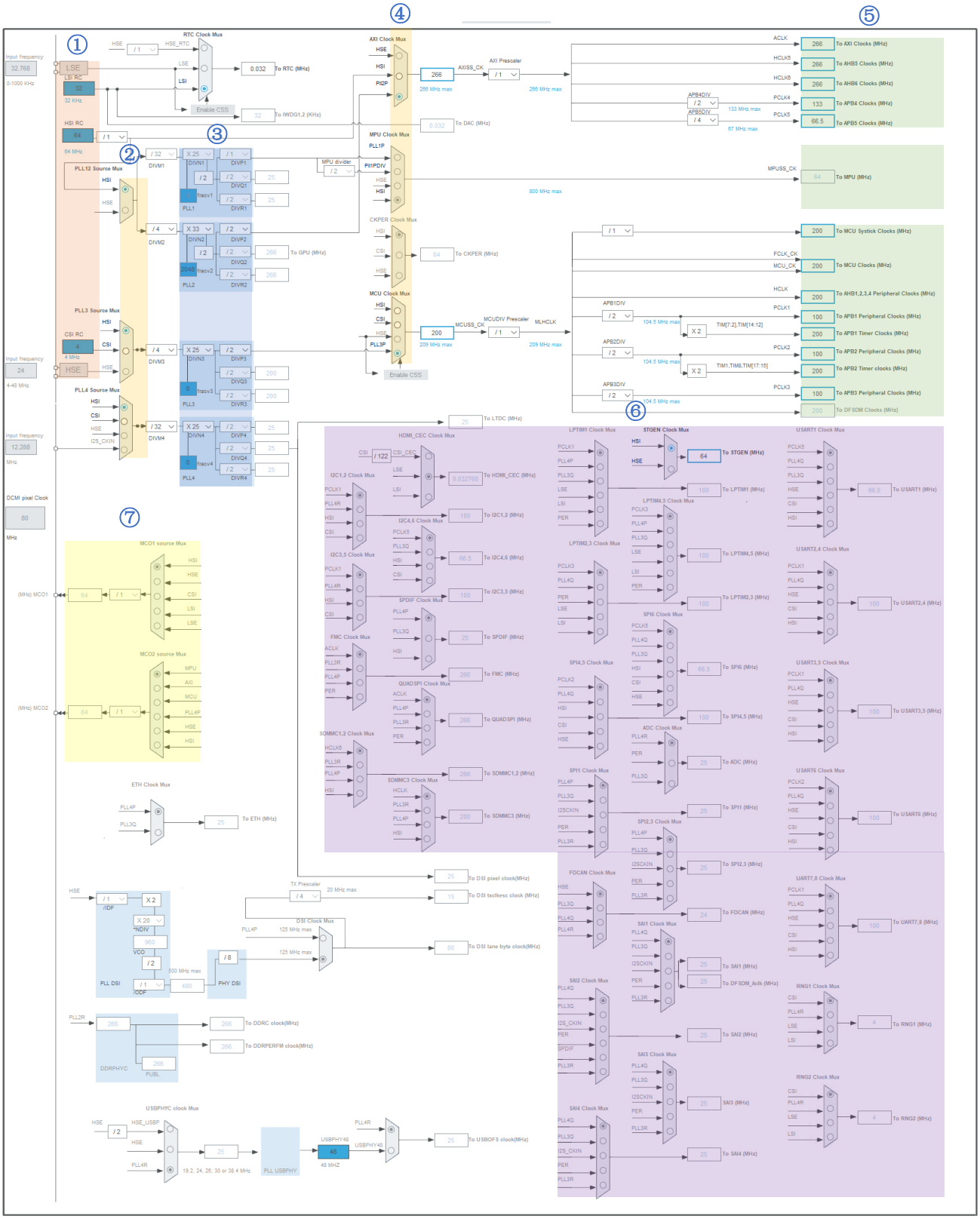


图 2.3.3 STM32MP157 时钟

STM32MP157 的时钟图，看起比较复杂，实际主要的就图中示意的几块，下面分别介绍。

① 五个时钟源

三个内部时钟源：HIS、LSI、CSI；
两个外部时钟源：HSE、LSE；

**② 四个 PLL 锁相环**

PLL12 Source Mux 是选择哪个时钟源提供给 PLL1 和 PLL2；
PLL3 Source Mux 是选择哪个时钟源提供给 PLL3；
PLL4 Source Mux 是选择哪个时钟源提供给入 PLL4；

**③ 三个锁相环时钟源选择器**

PLL1 为 MPU Clock Mux 输入时钟；
PLL2 为 AXI Clock Mux 输入时钟；
PLL3 为 MCU Clock Mux 输入时钟；
PLL4 为 DSI 等外设提供时钟；

**④ 三个锁相环时钟源选择器**

AXI Clock Mux 选择哪路时钟提供给 AXI、AHB5、AHB6、APB4、APB5；
MPU Clock Mux 选择哪路时钟提供给 MPU(A7 最高 800MHz)；
CKPER Clock Mux 选择哪路时钟提供给 CKPER；
MCU Clock Mux 选择哪路时钟提供给 MCU(M4 最高 209MHz)以及 AHB1-4、APB1-3 等；

**⑤ AXI、AHB、APB、MPU、MCU 时钟**

该部分可以分成三部分：
上面的 AXI、AHB5、AHB6、APB4、APB5 提供给 A7 外设使用;
中间的 MPU Clock 提供给 A7 核心使用;
下面的 MCU Clock 提供给 M4 核心使用，以及 AHB1-4、APB1-3 提供给 M4 外设使用，
AXI、AHB、APB 和外设的关系，在后面介绍。

**⑥ 各个外设时钟选择器**

这部分包含 STM32MP157 所有外设时钟选择，只要在 STM32CubeMX 使能了外设，对应部分的时钟就会由灰色变成蓝色。

**⑦ 时钟输出两个选择器**

两个时钟输出信号选择器，用于选择把哪个时钟源输出供外部使用。

经过图中①至⑥这几步，就将时钟源传到了各个外设。其中第⑤-⑥的关系，参考图 2.3.4 所示。

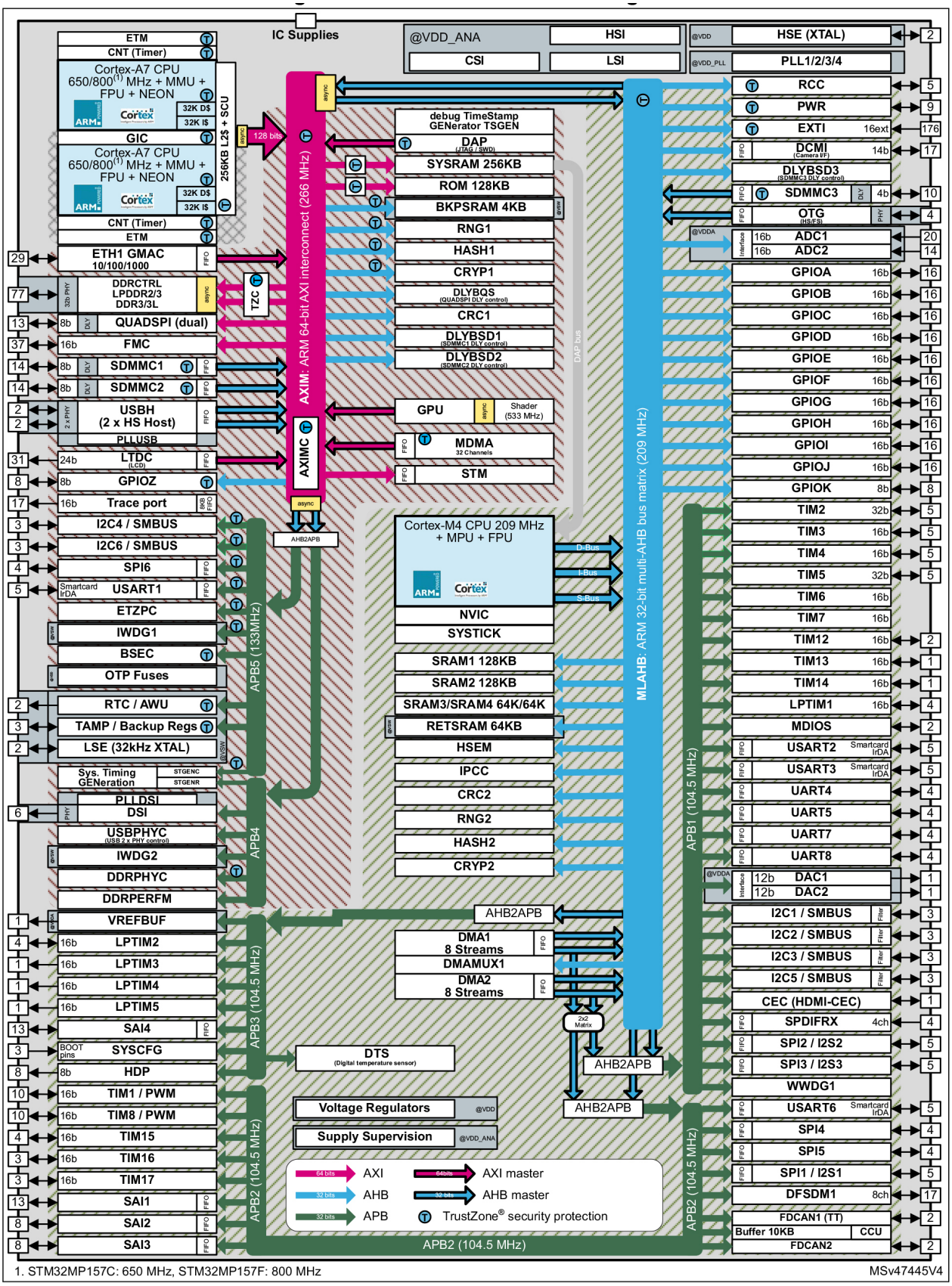


图 2.3.4 STM32MP157 总线

图中红色为 AXI(Advanced eXtensible Interface)高级可拓展接口，蓝色为 AHB(Advanced High-performance Bus)高级高性能总线，绿色为 APB(Advanced Peripheral Bus)高级外围总线。

红色的 AXI 这里就先不用关心，它主要是给 A7 外设使用。图中右半部分是 M4 的总线结构，可以看到蓝色的 AHB 总线(Max 209MHz)上，连接有 GPIO、ADC 这些高速外设等。三条绿色的 APB 总线(Max 104.5Mhz)，连接有 $I^2C$、SPI、USART 等低速外设，比如窗口看门狗 WWDG1，就挂在 APP1 上。

## 2.3.2 硬件设计

本实验实验内部时钟，不涉及硬件设计。

## 2.3.3 MX 设置

使用 STM32CubeIDE 创建新工程，打开 STM32CubeMX,切换到时钟配置界面，如图 2.3.5 所示。

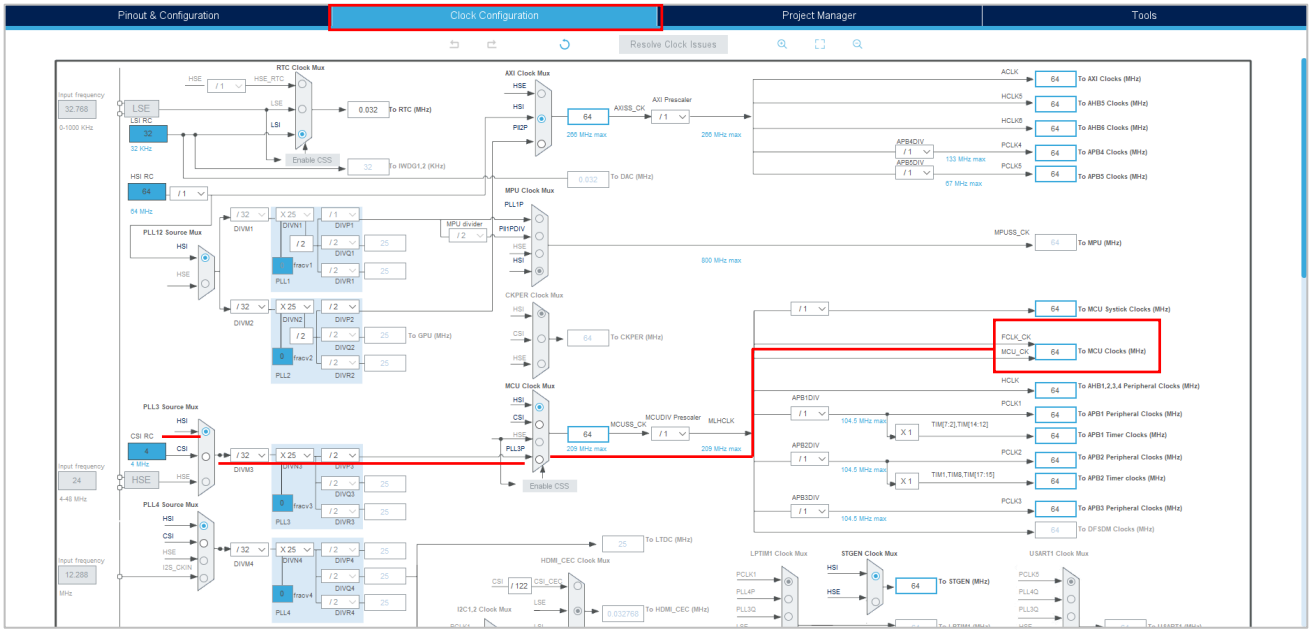


图 2.3.5 STM32MP157 默认时钟

①是 MCU 的时钟频率，默认是 64MHz，我们需求为 209MHz；

②处为一个分频器，默认为一分频，可以看作没有该分频器；

③处为 MCU 时钟选择器，默认为 HSI 输入，有前面可知，系统默认的 HSI 为 64 MHz，这里直接使用 HIS 是达不到 209 MHz 的。第二个 CSI 为 4MHz，也是不够的。第三个 HSE，因为外部时钟还没使能，这里还是灰色的，不能选择。第四个 PLL3P，有前面 PLL3 倍频得到，可能满足需求；

④处为 PLL 锁相环，可以将输入的时钟倍频；

⑤处为 PLL3 的输入时钟选择器，可选的有 HIS、CSI、HSE，这里实验为使用内部 RC 振荡器，因此选择默认的 HIS 即可。

分析完后，可以发现③处是需要修改的，需要选择 PLL3P 作为输入，同时希望得到 209MHz。因此修改③的选择输入为 PLL3P，同时在后面的框中输入 209，按下回车，稍等一会，STM32CubeMX 会自动计算④处的参数，使得③处得到 209MHz，得到如图 2.3.6 所示。

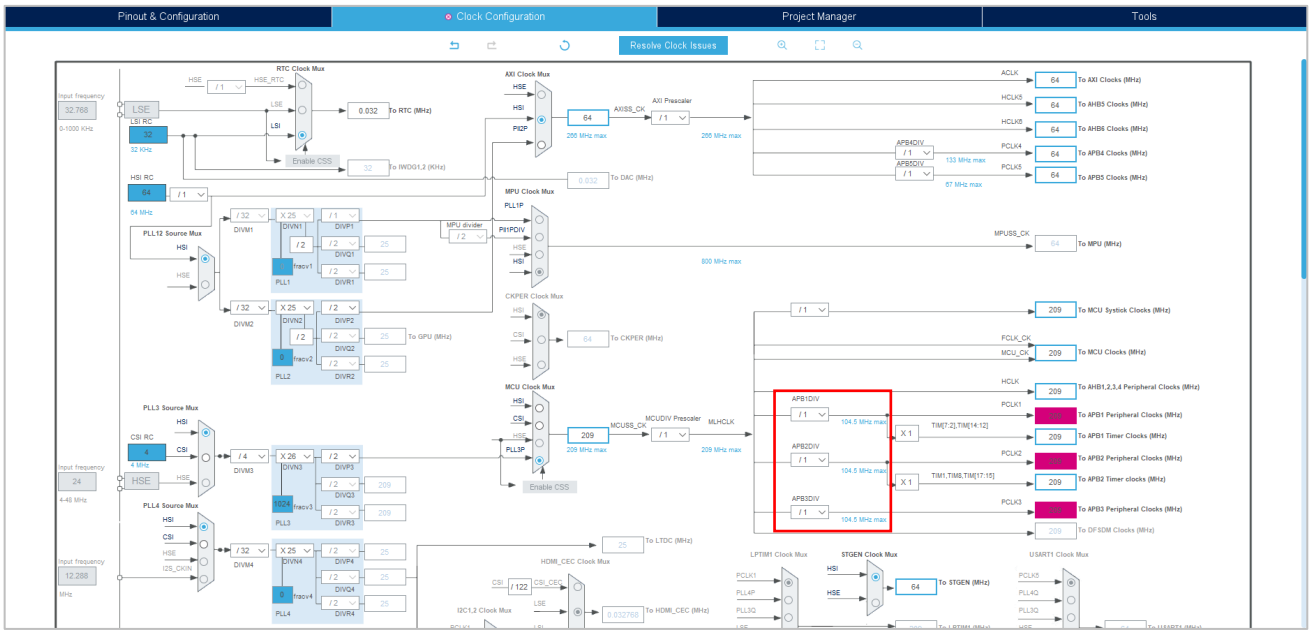


图 2.3.6 设置 MCU Clocks 为 209MHz 后报错

软件计算完成后，可以看到 MCU Clocks 成功变为 209MHz，但同时也多了三个酒红色提示。那是因为酒红色部分超过了最高允许的频率。酒红色部分全为 209MHz，而前面红框处的蓝色提示显示最大频率为 104.5MHz，因此需要将前面的分频器设置为“/2”即可，如图 2.3.7 所示。

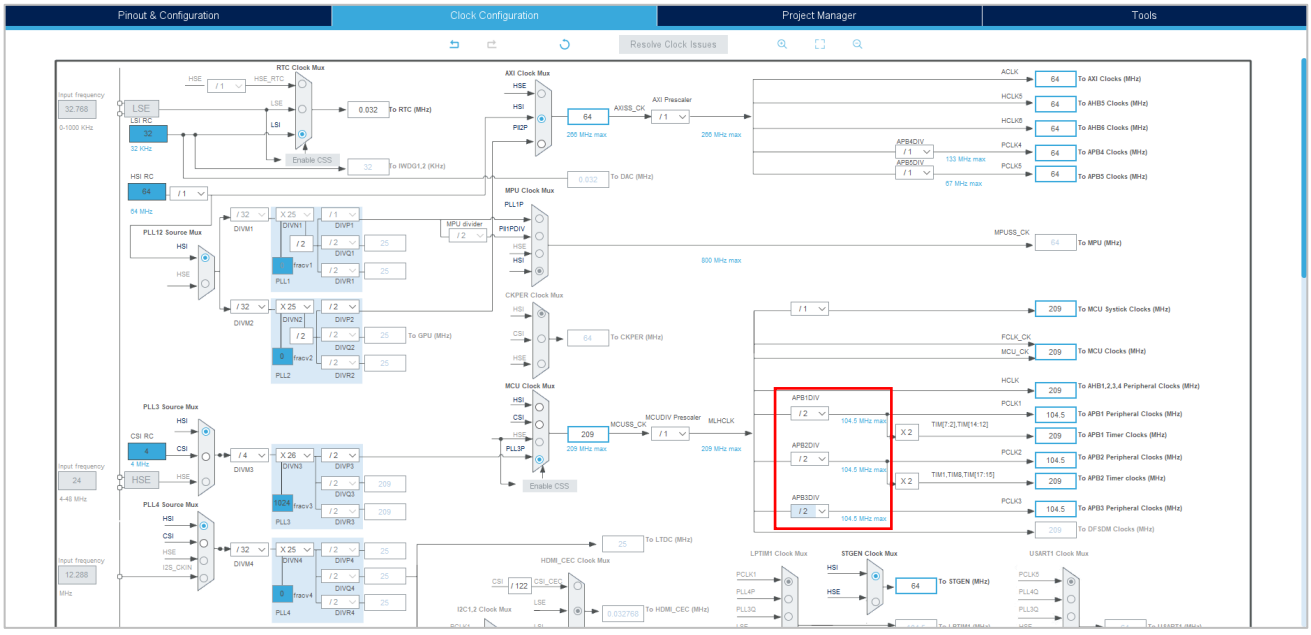


图 2.3.7 设置 MCU Clocks 为 209MHz 后正常

最后，参考前面示例生成初始化工程。

### 2.3.4 代码设计

HAL 库中有个获取系统核心时钟的函数，通过此函数获取系统时钟与我们设置的时钟对比，即可验证设置是否正确。首先定义一个全局变量保存系统时钟值，方便调试时查看，然后在“SystemClock_Config()”函数之后，调用“HAL_RCC_GetSystemCoreClockFreq()”获取设置后的新系统时钟。

```
/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  /* USER CODE BEGIN 1 */
  static uint32_t sys_freq = 0; // 定义一个全局变量获取系统时钟值，验证配置的时钟是否正确
  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  if(IS_ENGINEERING_BOOT_MODE())
  {
    /* Configure the system clock */
    SystemClock_Config();
  }

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  /* USER CODE BEGIN 2 */
  sys_freq = HAL_RCC_GetSystemCoreClockFreq(); // 调用库函数获取系统时钟频率
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */
```

```
    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
```

## 2.3.5 实验效果

由于现在还没实现串口打印，因此只能在调试窗口查看变量值。开始调试时，程序自动卡在“HAL_Init()”，此时右边变量窗口显示“sys_freq”的值为 0，然后在获取系统时钟后的位置设置断点，以查看变量变化，如图 2.3.8 所示。

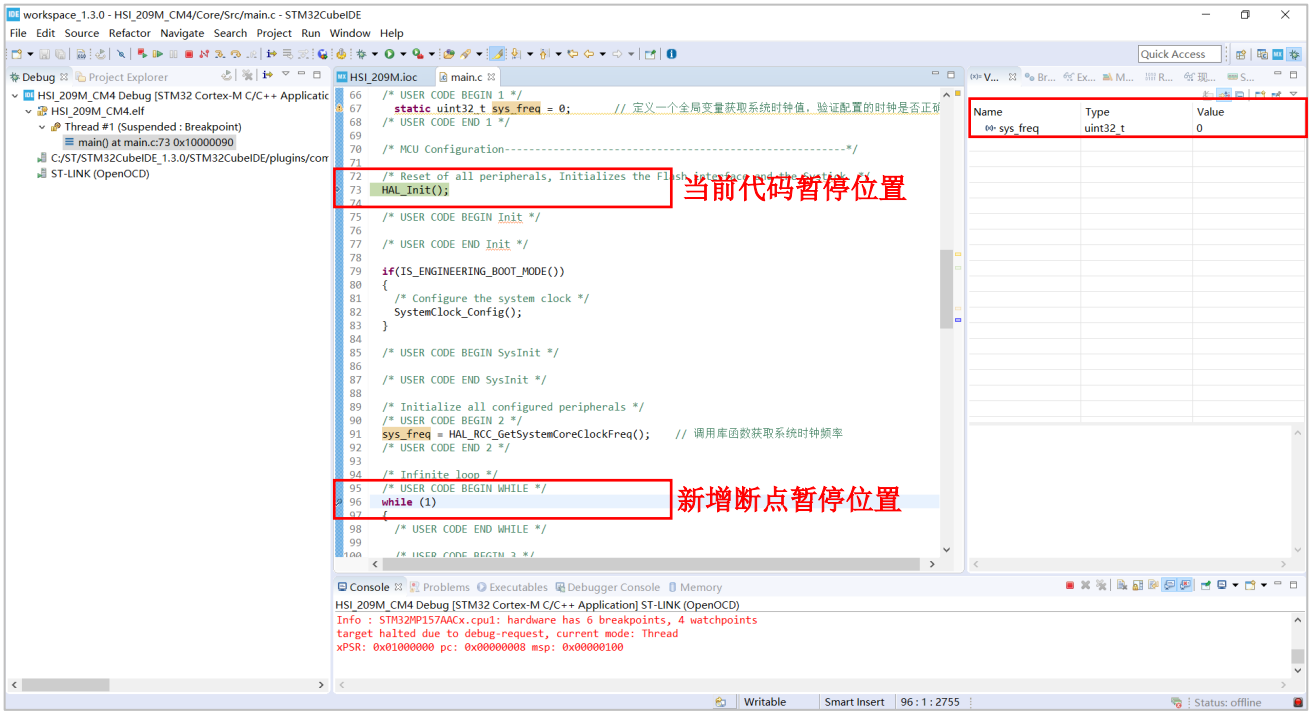


图 2.3.8 HSI 时钟设置前 sys_freq 变量值

随后点击继续运行按钮，将会在断点位置暂停，此时右边 sys_freq 值已经变为“209000128”，即 209MHz，说明设置正确，如图 2.3.9 所示。

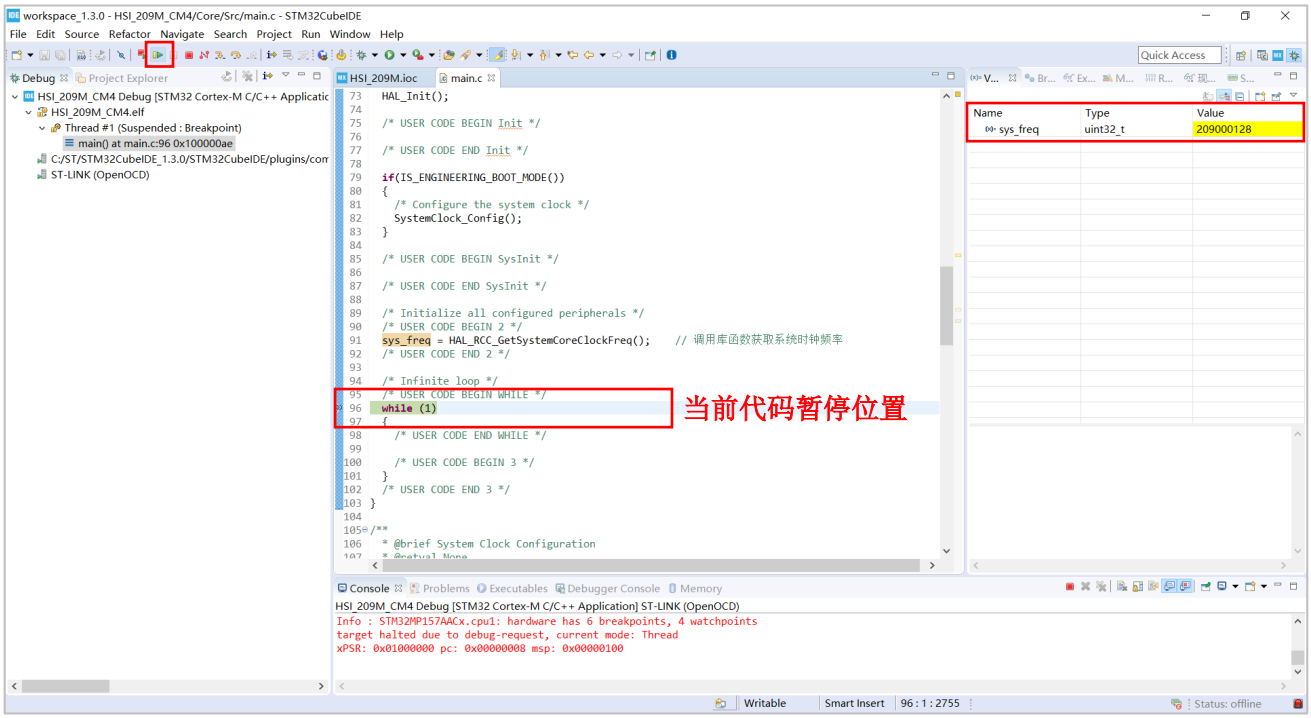


图 2.3.9 HSI 时钟设置后 sys_freq 变量值

# 2.4 系统时钟—HSE

- 设计需求

这里假设需求为使用外部时钟 HSE，让 STM32MP157 的 M4 全速运行在最高 209MHz。

## 2.4.1 基础知识

参考上一实验。

## 2.4.2 硬件设计

打开开发板核心板原理图第 2 页，可以看到外部晶振输入为 24MHz，外部晶振电路如图 2.4.1 所示。

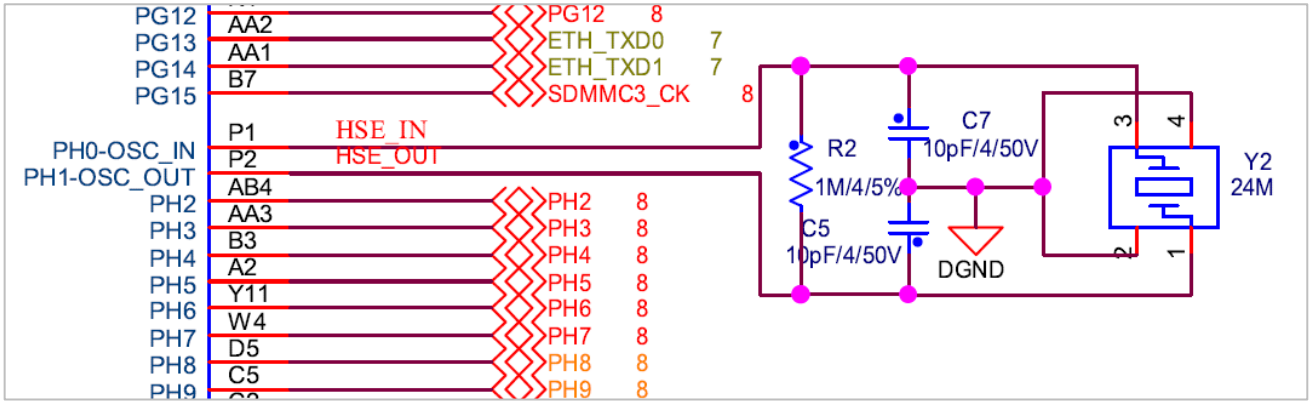


图 2.4.1 开发板外部晶振电路

## 2.4.3 MX 设置

新建 STM32CubeIDE 工程，随后打开 STM32CubeMX，点击“RCC”，在“High Speed Clock(HSE)”的复选框选择“Crystal/Ceramic Resonator”，也就是晶体/陶瓷晶振类型，如图 2.4.2 所示。

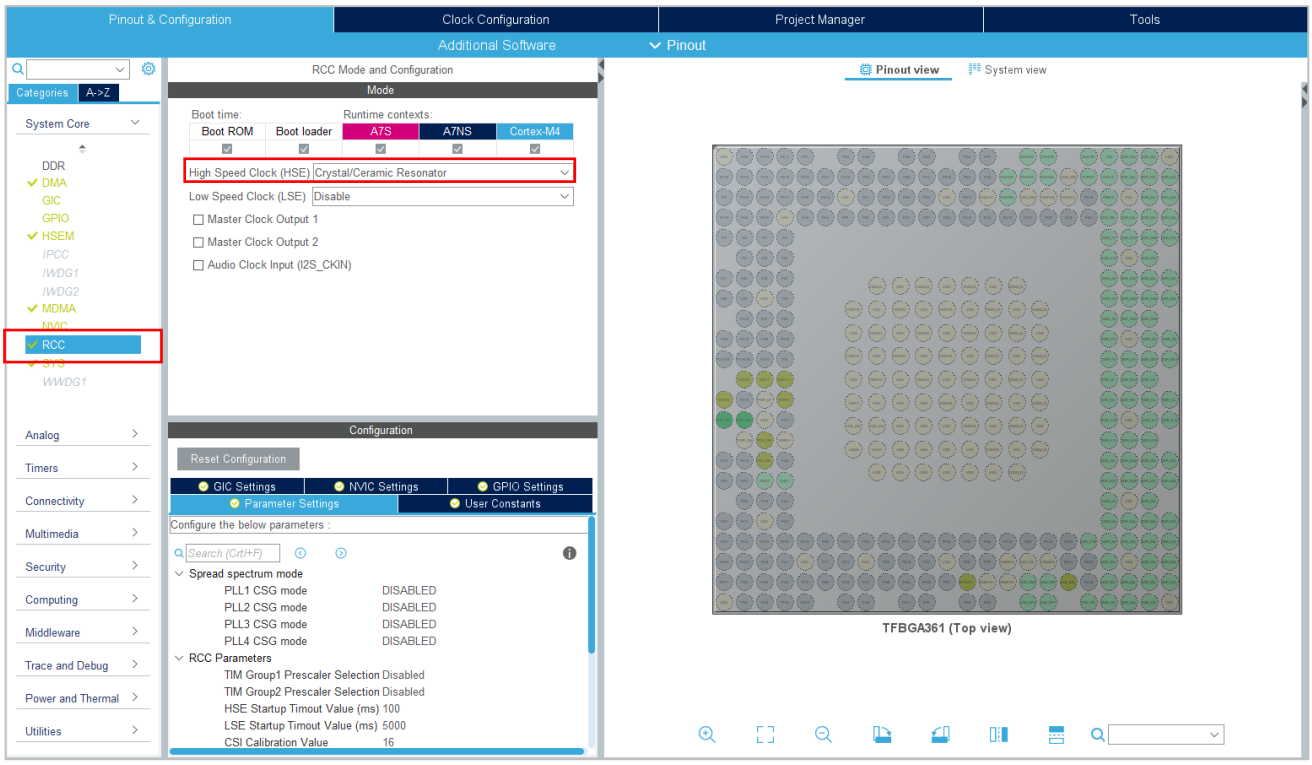


图 2.4.2 选择外部时钟输入类型

随后切换到“Clock Configuration”，设置如图 2.4.3 所示。

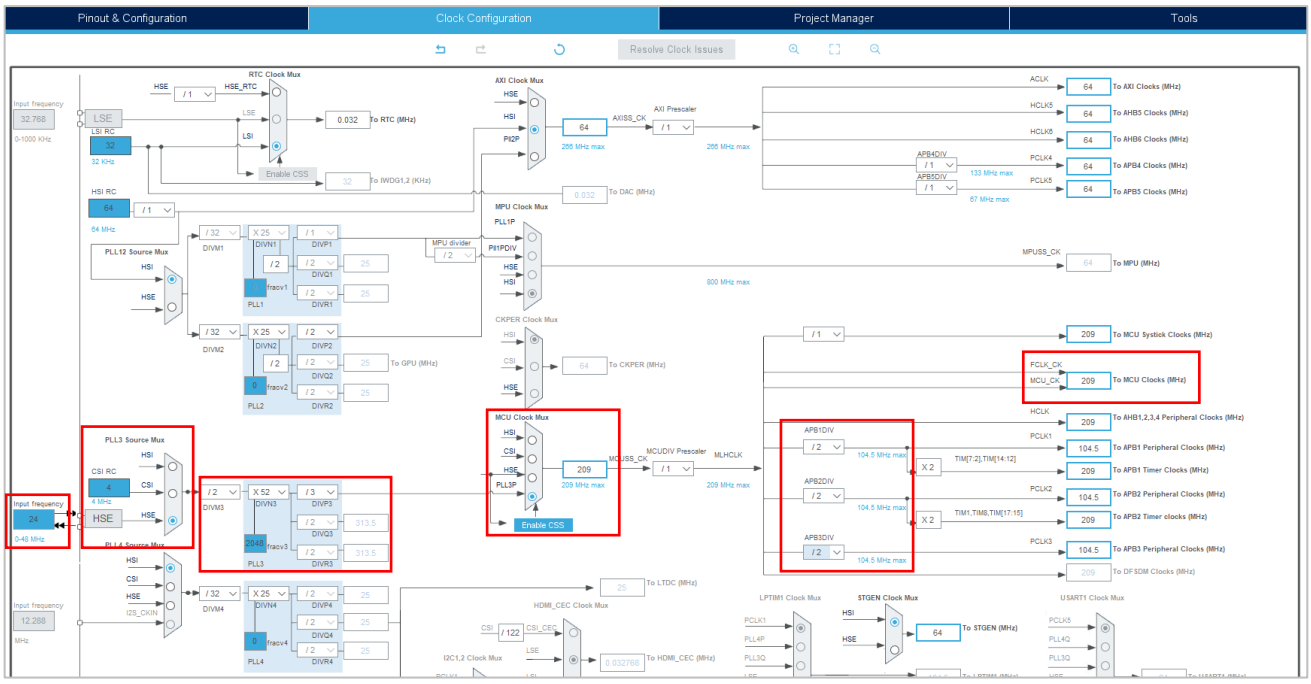

图 2.4.3 HSE 时钟配置

①为外部晶振输入，有原理图可知，输入晶振频率为 24HMz；
②为 PLL3 的选择器，因为使用 HSE 作为时钟源，这里选中 HSE；
③为 PLL3 锁相环，里面参数暂时无需设置，可以后面软件自动计算生成；
④为 MCU 时钟选择器，这里选择 PLL3P，然后在后面输入需要的 209MHz，按下回车，即可自动计算③中的参数值。注意选择器下方有个蓝色的“Enable CSS”，表示使能 CSS(Clock Security System)时钟安全系统功能，即一旦检测到 HSE 时钟故障，则就会切换到内部 RC 振荡器作为时钟信号输入；
⑤为分频器，设置为“/2”防止超过外设最大时钟限制；
⑥为 MCU 系统时钟，已显示为 209MHz；

## 2.4.4 代码设计

同上一个实验一样，调用“HAL_RCC_GetSystemCoreClockFreq()”获取设置后的新系统时钟。

```
/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  /* USER CODE BEGIN 1 */
  static uint32_t sys_freq = 0; // 定义一个全局变量获取系统时钟值，验证配置的时钟是否正确
  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  if(IS_ENGINEERING_BOOT_MODE())
  {
    /* Configure the system clock */
    SystemClock_Config();
  }

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
```

```
  MX_GPIO_Init();
  /* USER CODE BEGIN 2 */
  sys_freq = HAL_RCC_GetSystemCoreClockFreq(); // 调用库函数获取系统时钟频率
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
```

## 2.4.5 实验效果

同上一实验方法一样，通过调试发现 HSE 时钟设置前，sys_freq 变量初始值为 0，如图 2.4.4 所示。

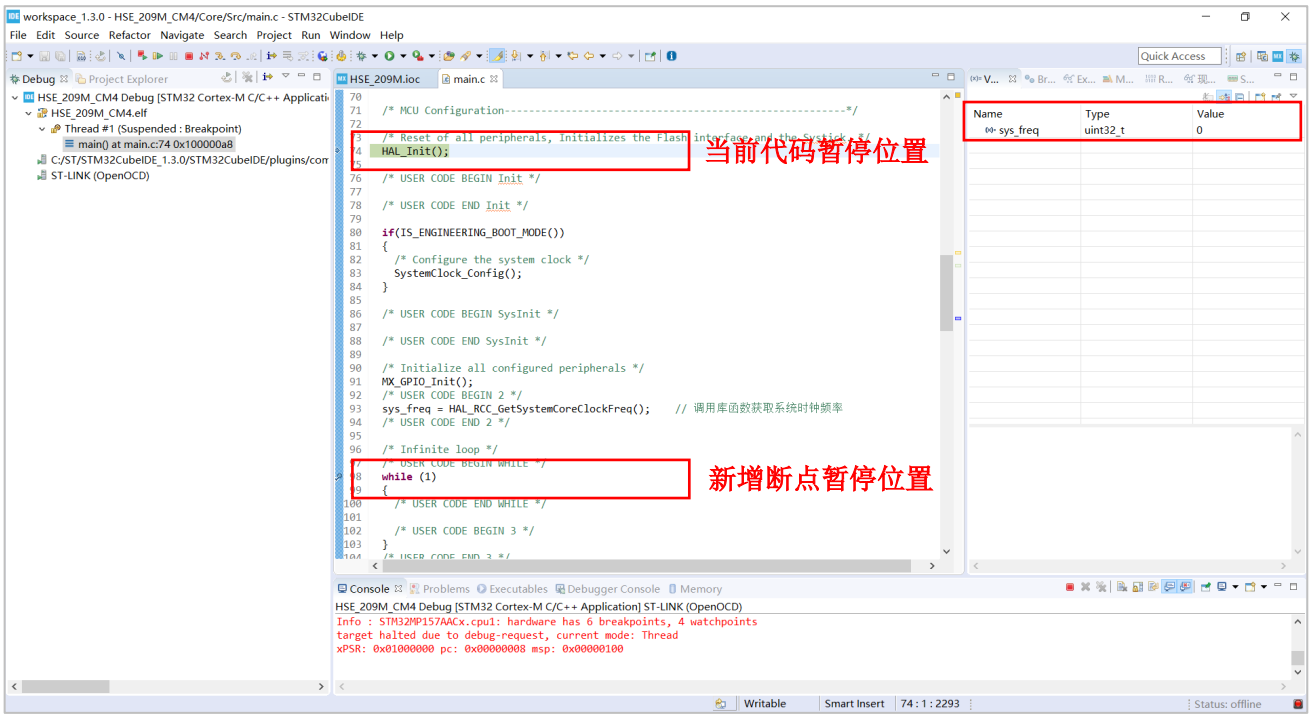


图 2.4.4 HSE 时钟设置前 sys_freq 变量值

待系统时钟配置完成后，发现 sys_freq 值已经变为“209000128”，即 209MHz，说明设置正确，如图 2.4.5 所示。

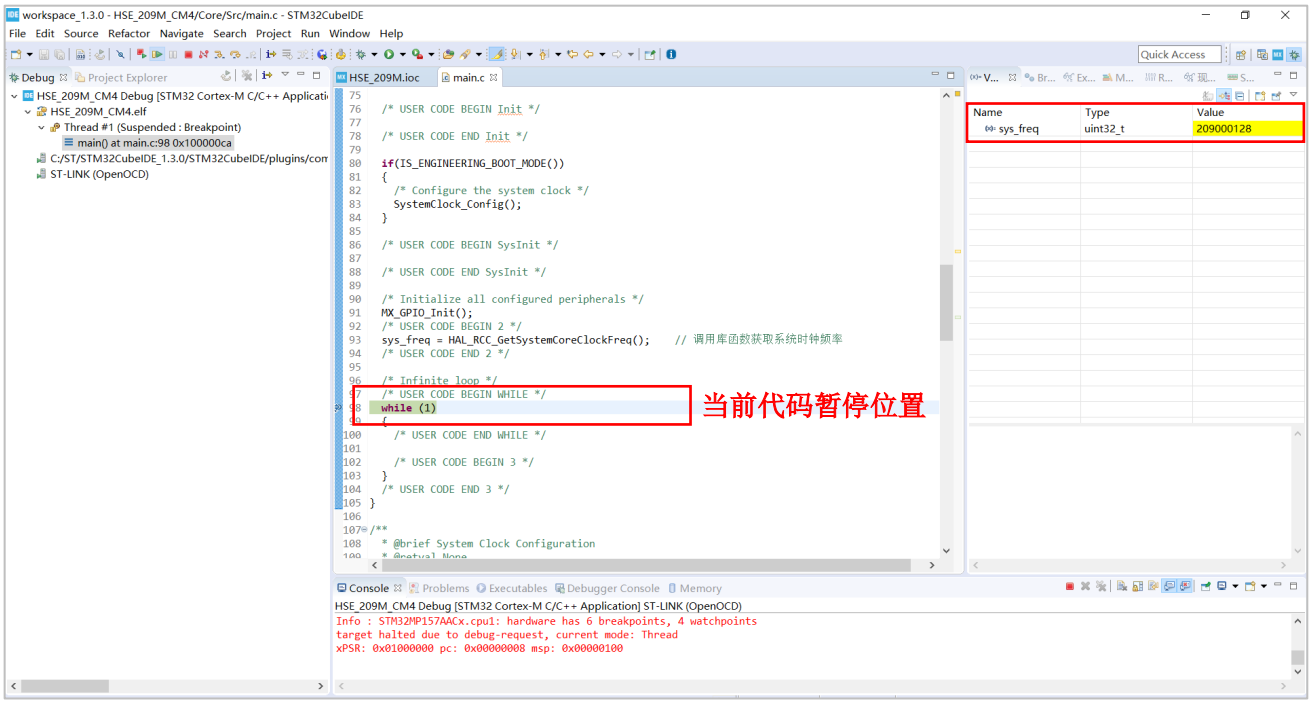


图 2.4.5 HSE 时钟设置后 sys_freq 变量值

此外，读者也可尝试将 CSI 作为系统时钟，让 M4 运行在 209MHz。

# 2.5 按键扫描

- 设计需求

这里假设需求为通过按键扫描的方式实现，按下 KEY_USER1（KEY1）按键，点亮 LED_GREEN，再次按下熄灭 LED_ GREEN；按下 KEY_USER2（KEY2）按键，点亮 LED_YELLOW，再次按下熄灭 LED_YELLOW。

## 2.5.1 基础知识

前面 LED 灯是控制 GPIO 输出，而按键则是读取 GPIO 电平，从而获知用户是否按下按键。

按键监测一般有两种：按键扫描和按键中断。按键扫描是间隔很短时间反复查询 GPIO 状态，从而的值是否有按键动作，这种方式代码简单，但比较耗资源。按键中断而是通过按键产生中断信号，从而实现按键的检测，这种方式需要使用到中断机制，需要对 MCU 了解深入一点，效果是最好的。

本节先介绍按键扫描，理解按键的基本原理，下一节再介绍按键中断，同时了解 M4 的中断机制。

按键一般占用一个 GPIO 口，通过监测该 GPIO 的电平变化得知按键操作，典型的电路如图 2.5.1 所示。当所需按键比较多时，则可以采用矩阵按键减少 GPIO 的占用。矩阵按键需要通过编程扫描等方式实现对多个按键的监控，这里以最简单的独立按键为基础进行介绍。

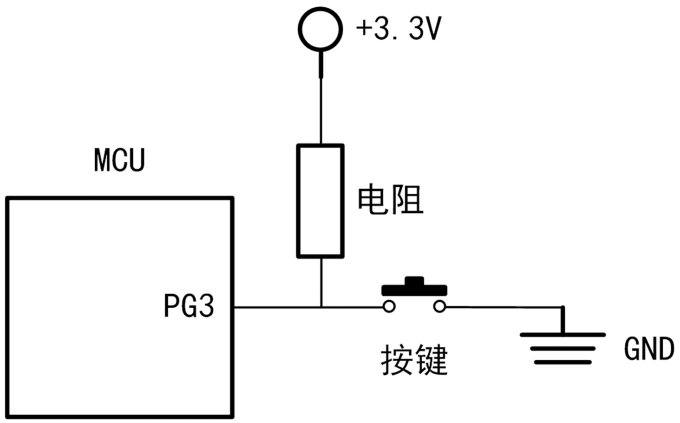


图 2.5.1 按键与 MCU 的连接方式示意图

可以看到，在没有按下按键时，电源 3.3V 通过电阻连接到 MCU 的 PG3 脚上，此时 MCU 读取 PG3 的电平就是 3.3V 的高电平。在按键按下时，电源 3.3V 经过电阻，再经过按键连接到了地，此时 PG3I 连接到接地的一端，读到的电平就是 0V 的低电平。由此，MCU 就可用过读取对应引脚的电平值，得知按键的变化。

常用的按键都是机械触点式按键，机械式按键在按下或释放的过程中，由于机械弹性作用的影响，会伴随机械抖动，如图 2.3.6 所示。

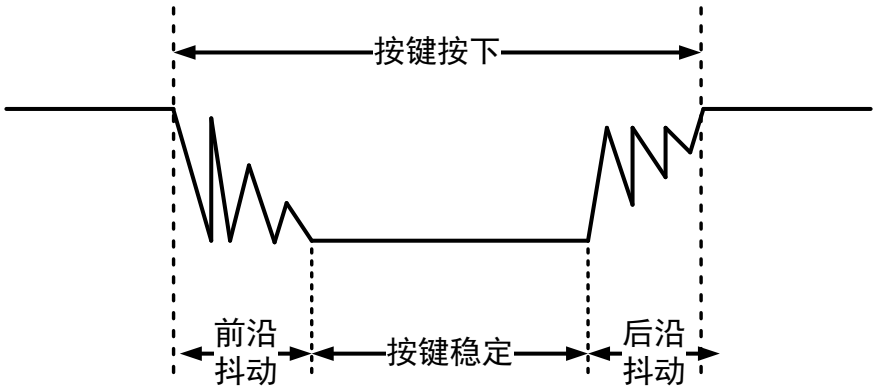


图 2.5.2 机械按键抖动示意图

抖动的时长与机械开关特性相关，一般为 5-10ms。在这抖动过程中，会产生多次高低电平，导致被识别为多次按键操作。为了避免机械触点按键检测误判，必须消抖处理。按键消抖可以硬件上处理，即在按键旁并联电容，吸收抖动的电平。也可以软件处理，即通过延时，避开抖动。

由此，首先获取对应引脚的电平得知按键状态，再硬件或软件消除抖动。

## 2.5.2 硬件设计

打开配套资料里开发板底板原理图文件，按键原理图部分如所示。

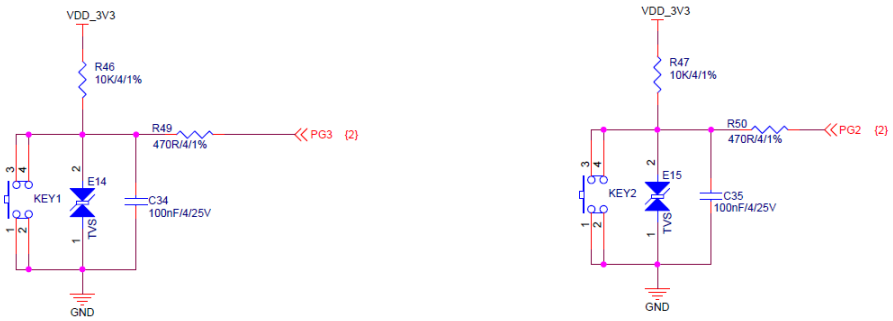


图 2.5.3 开发板按键部分原理图

开发板上有四个按键，其中两个是唤醒按键、复位按键，不在本次介绍范围内。另外两个为用户按键如上图所示，按键KEY1接在了MCU的PG3引脚上，同时并联了一个电容C34用于硬件去抖，软件上可以根据实际效果再决定是否软件去抖，还并联了一个TVS二极管防静电。同理,KEY2接在了PG2上。

## 2.5.3 MX 设置

创建一个新工程，参考前面LED灯工程，设置两个LED灯，参考前面设置任一时钟源作为系统时钟。然后添加PG3、PG2，设置为输入模式“GPIO_Input”。注意按键输入引脚也要设置“Pin Reserved”->“Cortex-M4”，如图 2.5.4所示。

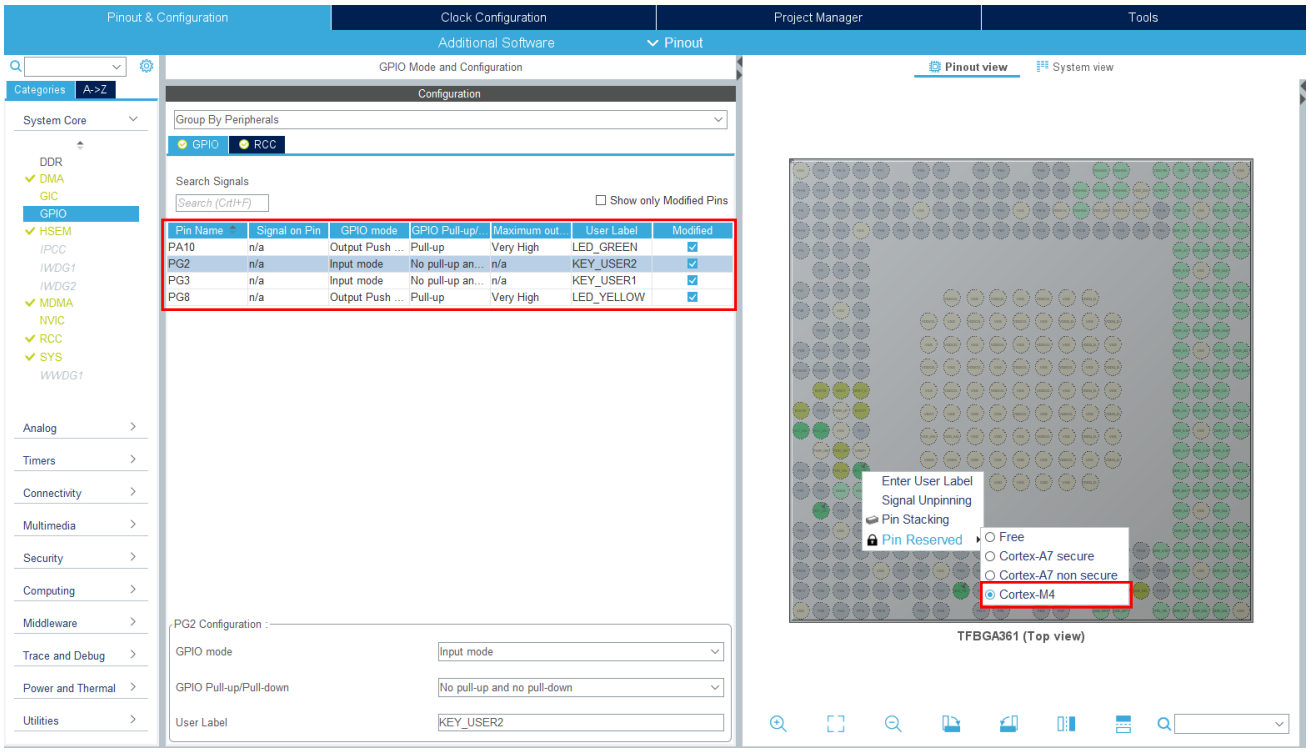


图 2.5.4 设置按键——按键扫描

## 2.5.4 代码设计

参考前面 LED 工程，在“DemoDriver”目录创建“driver_key.c”和“driver_key.h”。“driver_key.h”里定义引脚状态和读取函数，方便使用。

```
/**********************
 * 按键引脚状态定义
**********************/
#define PUSH_DOWN                   GPIO_PIN_RESET
#define SPRING_UP                   GPIO_PIN_SET

/**********************
 * 引脚宏定义
**********************/

/**********************
 * 按键状态读取函数宏定义
**********************/
#define READ_KEY1                   HAL_GPIO_ReadPin(KEY_USER1_GPIO_Port, KEY_USER1_Pin)
#define READ_KEY2                   HAL_GPIO_ReadPin(KEY_USER2_GPIO_Port, KEY_USER2_Pin)
```

“driver_key.c”创建实现函数，读取按键 GPIO 状态和控制 LED 灯。

```
/*
 *  函数名：void Key1Polling(void)
 *  输入参数：无
```

```
 *  输出参数：无
 *  返回值：无
 *  函数作用：使用轮询方式查询键是否按下，通过按下控制灯亮灭
*/
static bool key1_flag = false;
void Key1Polling(void)
{
    if(READ_KEY1 == PUSH_DOWN)      // 如果检测到键被按下
    {
        HAL_Delay();                //延时 5ms 防按键抖动
        if(READ_KEY1 == PUSH_DOWN) // 如果防抖动后按键依然是处于被按下的状态，就认为按键被按下过
        {
            key1_flag = !key1_flag; // 用一个标志位来判断按键被按下次数
            if(key1_flag)
            {
                LED_GREEN_ON();
            }
            else
            {
                LED_GREEN_OFF();
            }
        }
    }
}

/*
 *  函数名：void Key2Polling(void)
 *  输入参数：无
 *  输出参数：无
 *  返回值：无
 *  函数作用：使用轮询方式查询键是否按下，通过按下控制灯亮灭
*/
static bool key2_flag = false;
void Key2Polling(void)
{
    if(READ_KEY2 == PUSH_DOWN)      // 如果检测到键被按下
    {
        HAL_Delay(5);               // 延时 5ms 防按键抖动
        if(READ_KEY2 == PUSH_DOWN) // 如果防抖动后按键依然是处于被按下的状态，就认为按键被按下过
        {
            key2_flag = !key2_flag; // 用一个标志位来判断按键被按下次数
            if(key2_flag)
            {
                LED_YELLOW_ON();
```

```
            }
            else
            {
                LED_YELLOW_OFF();
            }
        }
    }
}
```

以“Key1Polling()”为例，宏 READ_KEY1 实质是使用 HAL 函数“HAL_GPIO_ReadPin()”读取按键引脚状态，如果是按下状态 PUSH_DOWN，延时 5ms 去抖动，再次检测，如果防抖动后按键依然是处于被按下的状态，则本次按键有效。使用 key1_flag 标志位来判断按键被按下次数，每次取反，使 LED 点亮、熄灭交替。

最后在主函数的循环里，一直执行“Key1Polling()”和“Key2Polling()”，以检测按键是否按下。

### 2.5.5 实验效果

编译程序，然后进入工程模式调试，按下 KEY_USER1（KEY1）按键，点亮 LED_GREEN，再次按下熄灭 LED_GREEN；按下 KEY_USER2（KEY2）按键，点亮 LED_YELLOW，再次按下熄灭 LED_YELLOW。

# 2.6 按键中断

- 设计需求

这里假设需求为通过按键中断的方式实现，按下 KEY_USER1（KEY1）按键，点亮 LED_GREEN，再次按下熄灭 LED_ GREEN；按下 KEY_USER2（KEY2）按键，点亮 LED_YELLOW，再次按下熄灭 LED_YELLOW。

## 2.6.1 基础知识

- 中断的基本概念

正常情况下，微处理器根据代码内容，按顺序执行指令。执行过程中，如果遇到其它紧急的事件需要处理，则先暂停当前任务，执行紧急事件，待紧急事件处理完后，再恢复到刚才暂停的地方继续执行。这个产生的紧急事件就叫做中断，如图 2.6.1 所示。

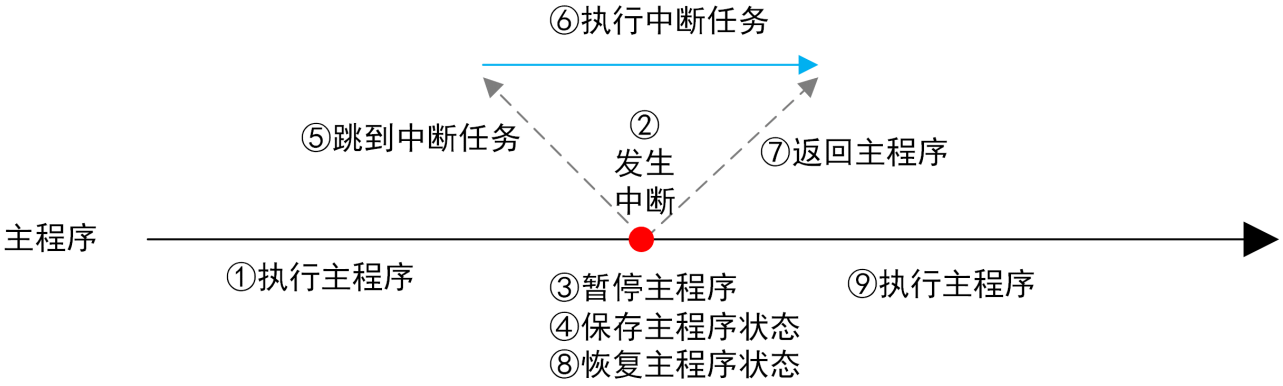


图 2.6.1 中断处理示意图

- 中断的分类

CPU 在执行指令时，检测到非法指令，比如除零、地址访问越界等，会产生中断，这种中断属于内部中断，也叫系统异常。由 CPU 外部设备引起的外部事件，比如 GPIO 中断、USART 中断等，属于外部中断。

ARM CM4 内核可以支持 256 个中断，包括 16 个内部中断和 240 个外部中断。对于 STM32MP157 的 M4,没有用到 CM4 内核的所有资源，只是用到了一部分，只有 10 个内部中断和 150 个外部中断，总计 160 个中断。如图 2.6.2 所示，上半部分是 10 个内部中断，下半部分是 150 个外部中断(中间部分省略)，一般用到的都是外部中断。

| Position | Priority | Type of priority | Acronym | Description | Address | EXTI event (1) |
|---|---|---|---|---|---|---|
| - | - | - | - | Reserved | 0x0000 0000 | - |
| - | -3 | Fixed | Reset | Reset | 0x0000 0004 | - |
| - | -2 | Fixed | NMI | Reserved | 0x0000 0008 | - |
| - | -1 | Fixed | HardFault | All class of fault | 0x0000 000C | - |
| - | 0 | Fixed | MemManage | Memory management | 0x0000 0010 | - |
| - | 1 | Settable | BusFault | Pre-fetch fault, memory access fault | 0x0000 0014 | - |
| - | 2 | Settable | UsageFault | Undefined instruction or illegal state | 0x0000 0018 | - |
| - | - | - | - | Reserved | 0x0000 001C - 0x0000 002B | - |
| - | 3 | Settable | SVCall | System service call via SWI instruction | 0x0000 002C | - |
| - | 4 | Settable | Debug Monitor | Debug monitor | 0x0000 0030 | - |
| - | - | - | - | Reserved | 0x0000 0034 | - |
| - | 5 | Settable | PendSV | Pendable request for system service | 0x0000 0038 | - |
| - | 6 | Settable | Systick | System tick timer | 0x0000 003C | - |
| 0 | 7 | Settable | WWDG1_IT | Window watchdog 1 early wakeup interrupt | 0x00000040 | - |
| 1 | 8 | Settable | PVD_AVD | PVD & AVD detector through EXTI | 0x00000044 | 16 |
| 2 | 9 | Settable | TAMP | Tamper interrupt (include LSECSS interrupts) | 0x00000048 | (18) |
| 3 | 10 | Settable | RTC_WKUP_ALARM | RTC wakeup timer and alarms (A and B) interrupt | 0x0000004C | (19) |
| 4 | 11 | Settable | - | - | 0x00000050 | - |
| 5 | 12 | Settable | RCC | RCC global interrupt (rcc_mcu_irq) | 0x00000054 | - |
| 6 | 13 | Settable | EXTI0 | EXTI line 0 interrupt | 0x00000058 | 0 |
| 7 | 14 | Settable | EXTI1 | EXTI line 1 interrupt | 0x0000005C | 1 |
| 8 | 15 | Settable | EXTI2 | EXTI line 2 interrupt | 0x00000060 | 2 |
| 9 | 16 | Settable | EXTI3 | EXTI line 3 interrupt | 0x00000064 | 3 |
| 10 | 17 | Settable | EXTI4 | EXTI line 4 interrupt | 0x00000068 | 4 |
| ■ ■ ■ ■ ■ ■ | | | | | | |
| 147 | 154 | Settable | DTS | Digital temperature sensor interrupt | 0x0000028C | (72) |
| 148 | 155 | Settable | - | - | 0x00000290 | - |
| 149 | 156 | Settable | MCU_WAKEUP_PIN | Interrupt for all 6 wakeup enabled by MCU | 0x00000294 | (55, 56, 57, 58, 59, 60) |

图 2.6.2 STM32MP157 M4 中断列表(部分)

- 中断的优先级

M4 拥有 160 个中断，这就引入了一些新问题。比如两个中断同时发生，应该先执行哪个中断任务？又比如一个中断发生了，又来了一个更紧急的中断，是该继续执行原来的中断，还是执行新的紧急中断？

针对这些问题，STM32 的每个中断源都需要被指定抢占优先级（pre-emption priority）级别和子优先级（subpriority）级别。

假设中断 A 的抢占优先级比中断 B 的抢占优先级高，两个中断同时发生，那么中断 A 优先执行。

假设中断 A 的抢占优先级和中断 B 的抢占优先级一样，两个中断同时发生，那么子优先级高的中断优先执行。

假设中断 A 的抢占优先级比中断 B 的抢占优先级高，中断 B 先发生，随后 A 也发生，那么将暂停中断 B，先执行中断 A，A 执行完后，再回来执行中断 B，最后执行主程序，这种效果即中断嵌套。

假设中断 A 的抢占优先级比中断 B 的抢占优先级一样，中断 A 的子优先级比中断 B 的子优先级高，中断 B 先发生，随后 A 也发生，那么中断 A 将等待中断 B 执行完后，才会执行中断 A，即子优先级不能中断嵌套。

假设中断 A 的抢占优先级和中断 B 的抢占优先级一样，且子优先级也一样，两个中断同时发生，那么根据前面图 2.6.2 中断列表顺序，排在前面的先执行。

总结中断是否会优先执行依据：首先是抢占先式优先级等级，其次是子优先级等级，只有抢占先式优先级才才可能出现中断嵌套;

前面图 2.6.2 中蓝色框部分为“Fixed”，表示这几个中断优先级是固定的，不可软件编程调整，剩下的 156 个中断都是可以编程调整优先级。ARM CM4 内核原本设计了一个 8 位寄存器来控制优先级，STM32 没用到那么多，提供了一个 NVIC（Nested vectored interrupt controller），即嵌套向量中断控制器来设置中断的优先级，只使用其高四位。

- 中断优先级的分组

现在问题又来了，前面提到一个中断由抢占优先级和子优先级组成。不同的场合有不同的需求，有些时候需要抢占优先级的等级多一点，有时候又需要子优先级的等级多一点，那如何满足不同的需求呢？STM32 又提供一个新概念，中断优先级分组，如表 2.6.1 所示。

表 2.6.1 中断优先级分组

| 组号 | 高四位分配情况[7:4] | 分配结果 |
|---|---|---|
| NVIC_PriorityGroup_0 | 0:4 | 0级抢占优先级，0-15级子优先级 |
| NVIC_PriorityGroup_1 | 1:3 | 0-1级抢占优先级，0-7级子优先级 |
| NVIC_PriorityGroup_2 | 2:2 | 0-3级抢占优先级，0-3级子优先级 |
| NVIC_PriorityGroup_3 | 3:1 | 0-7级抢占优先级，0-1级子优先级 |
| NVIC_PriorityGroup_4 | 4:0 | 0-15级抢占优先级，0级子优先级 |

有了中断优先级分组后，用户就可以根据自己实际需求设置分组，从而得到不同效果。比如一个项目需要子优先级分级比较多，即可考虑设置为 NVIC_PriorityGroup_0，这样就有 16 个等级的子优先级可设置。注意等级的值越小，优先级越高。

- GPIO 的中断

STM32MP157 有 PA-PI、PZ 共 10 组 GPIO，每组 GPIO 又有 0-15 共 16 个 GPIO。数字编号相同的 GPIO(PA0、PB0、PC0……PI0、PZ0)，共享一个中断源，如图 2.6.3 所示。由 EXTI（Extended interrupt and event controller）来决定是哪个 GPIO 接入中断，检测哪一种类型（上升沿、下降沿、双边沿）事件产生中断。

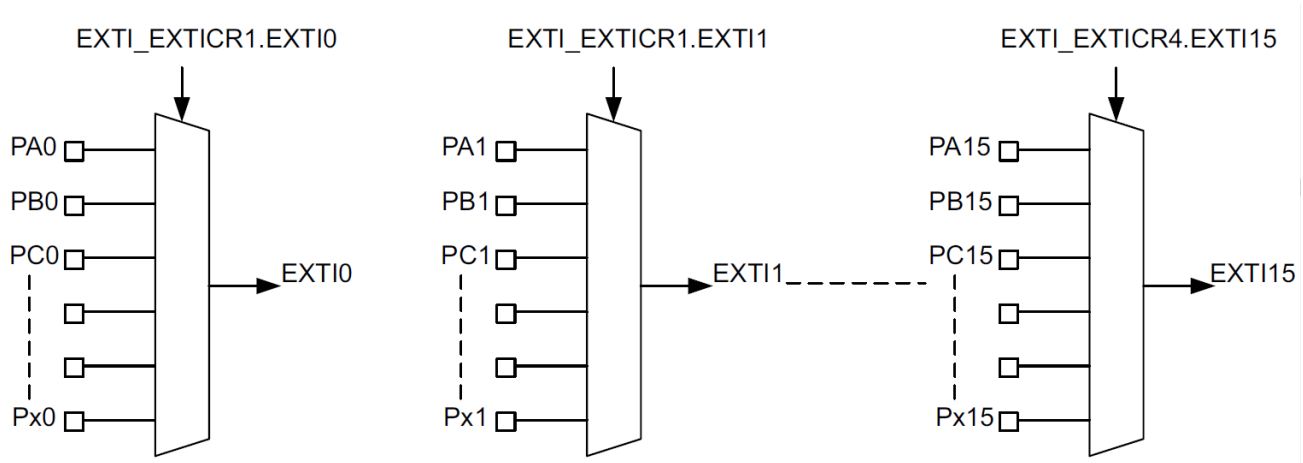


图 2.6.3 外部中断 GPIO 选择示意图

## 2.6.2 硬件设计

同前面按键扫描部分。

## 2.6.3 MX 设置

新建 STM32CubeIDE 工程，打开 STM32CubeMX，参考前面配置好 LED 灯和时钟。选择 PG3,选择引脚复用功能为“GPIO_EXTI3”，如图 2.6.4 所示。

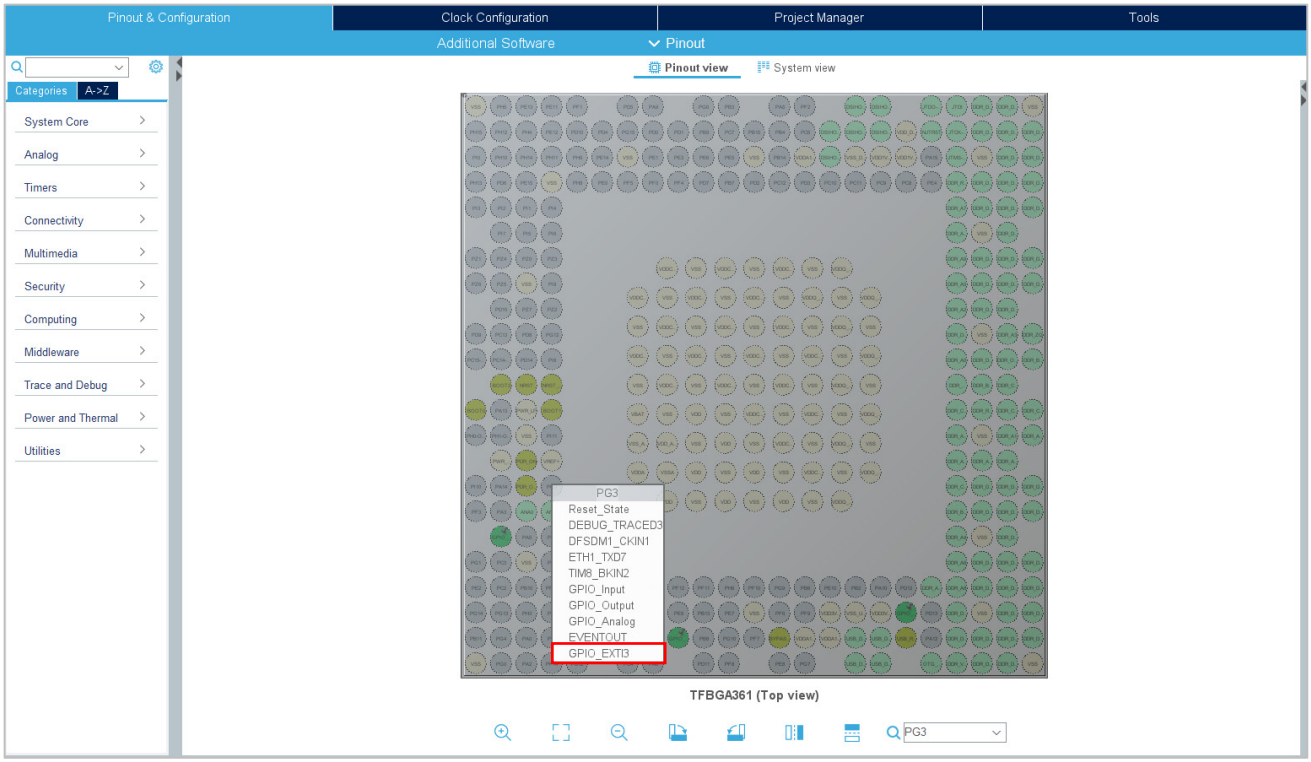


图 2.6.4 设置 KEY1 GPIO 为中断

然后记得设置“Pin Reserved”给“Cortex-M4”，在下方“PG3 Configuration”的“GPIO mode”设置“External Interrupt Mode with Rising edge trigger detection”，同理设置 PG2，最终如图 2.6.5 所示。

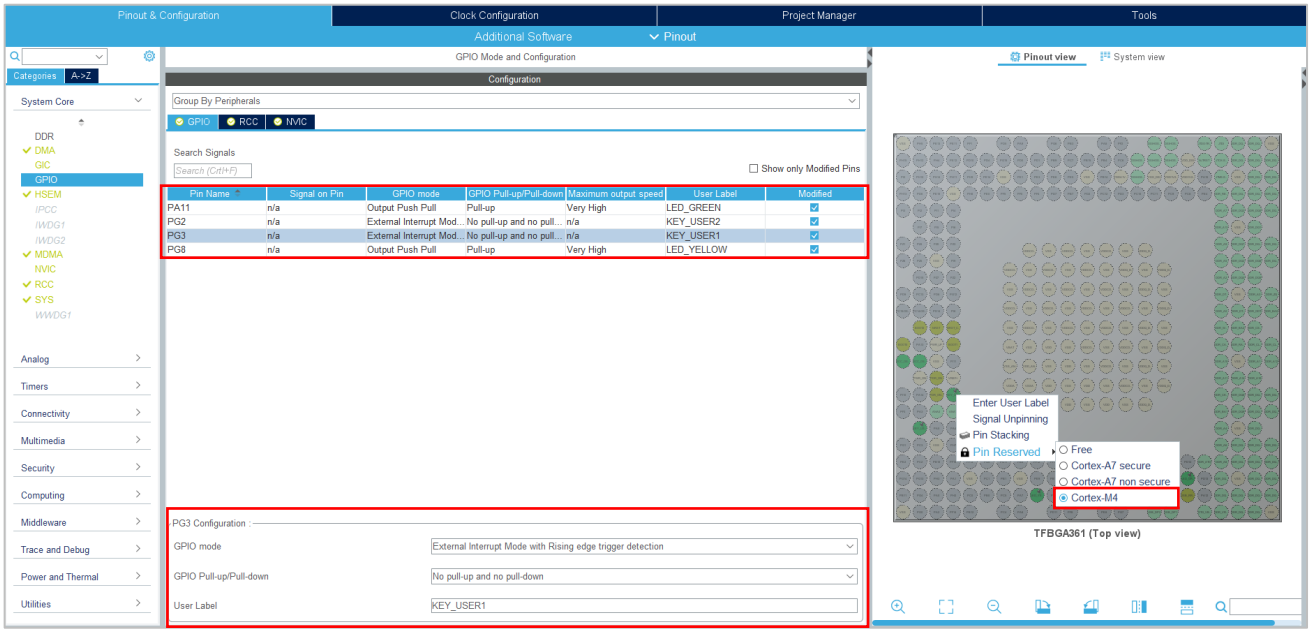


图 2.6.5 设置按键——按键中断

这里选择的是上升沿检测，即电平发生上升时，产生中断。参考前面图 2.5.2 可知，按键按下是由高电平变低的过程，松开按键才是低电平变高的过程。这里效果则是按下按键、松开按键时，才产生中断，得知有按键操作。另外，“External Interrupt Mode……”用在中断方式编程，才能够触发中断，而“External Event Mode……”用在查询方式，只设置中断标志位，不产生中断。

接着切到 NVIC 标签，使能中断，如图 2.6.6 所示。

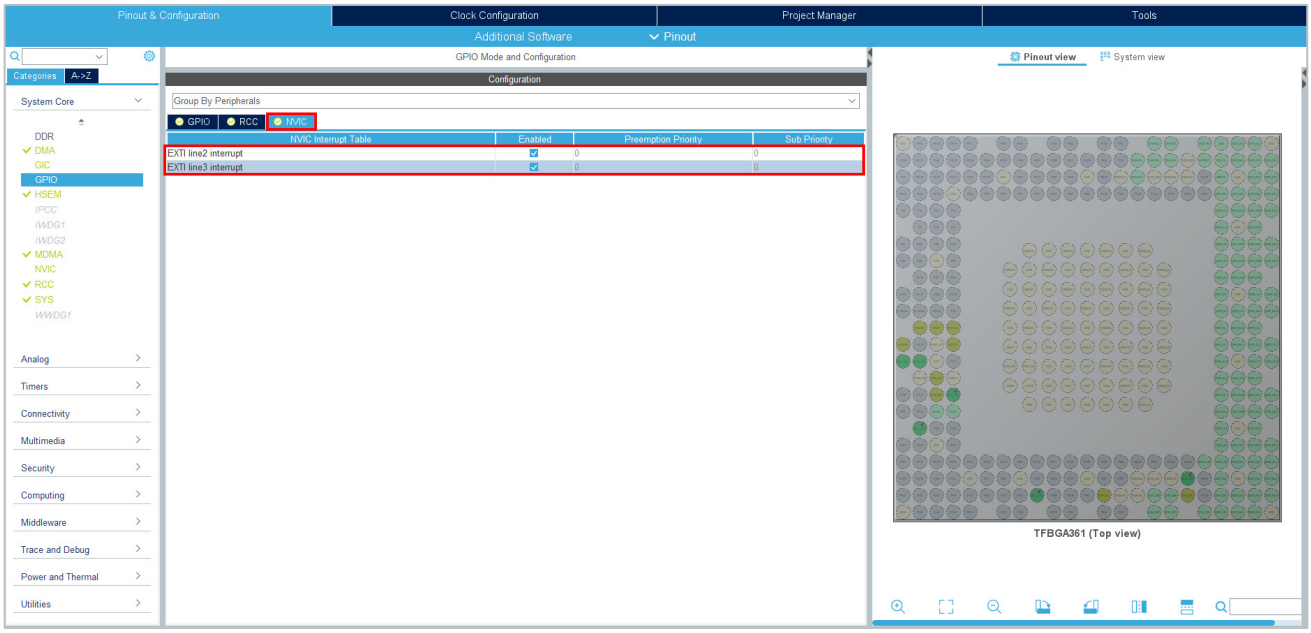


图 2.6.6 使能按键中断

如果还需调整中断优先级，需进入 NVIC 控制器设置界面，先选择分组，然后设置抢占优先级和子优先级。这里用到的是 PG2、PG3 引脚，对应的中断线就是 EXIT2 和 EXIT3。如果对中断优先级没什么特殊需求，如图 2.6.7 所示默认中断优先级即可。

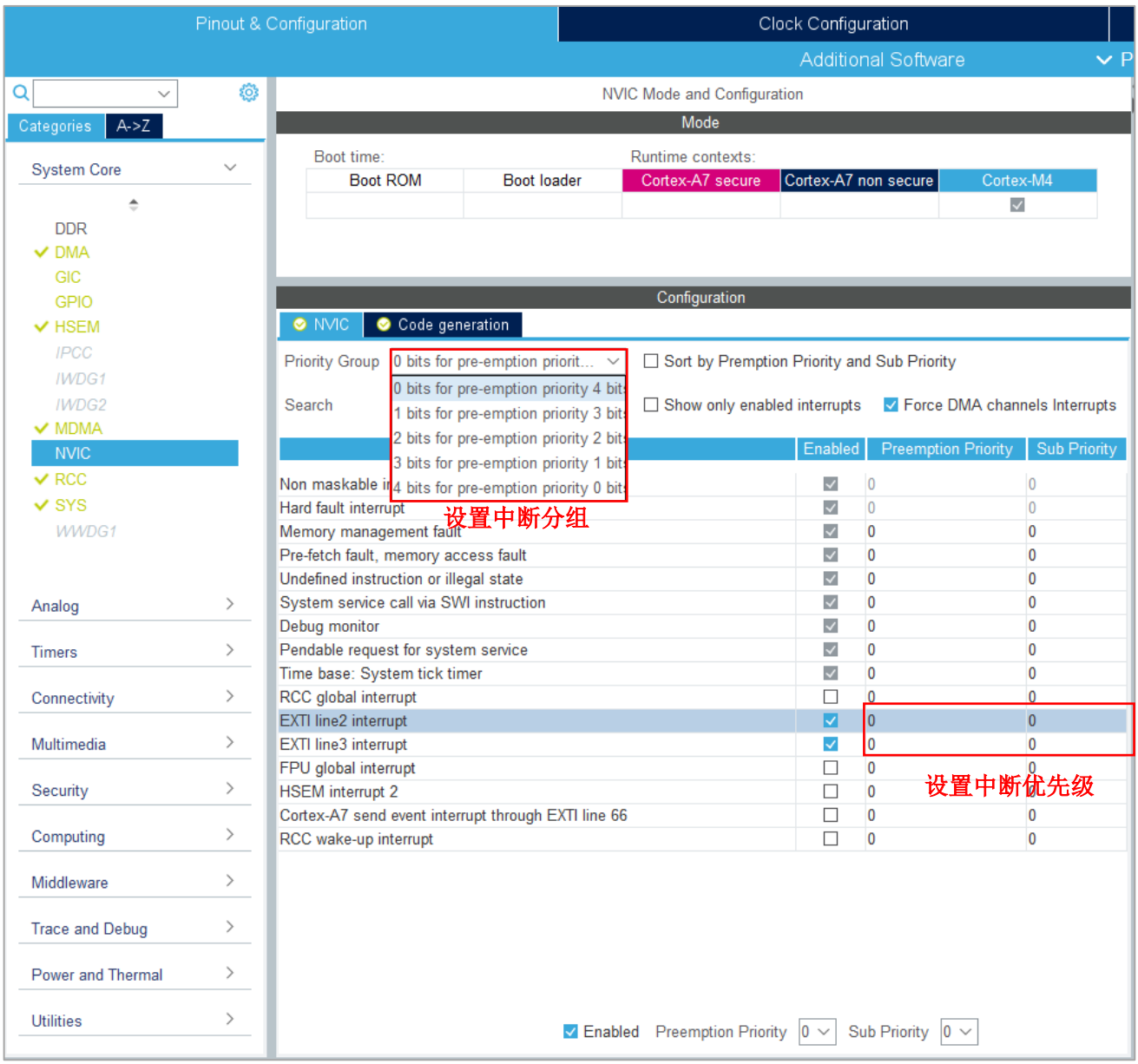


图 2.6.7 设置中断优先级

## 2.6.4 代码设计

前面设置 STM32CubeMX 时，设置哪个脚用于中断，检测哪种类型（上升沿）、中断分组、中断优先级、中断使能，这些都在自动生成的“gpio.c”里有体现，代码的编写就可以跳过了。

```
/** Configure pins as
        * Analog
        * Input
        * Output
        * EVENT_OUT
```

```
        * EXTI
*/
void MX_GPIO_Init(void)
{

  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();

  /*Configure GPIO pins : PGPin PGPin */
  GPIO_InitStruct.Pin = KEY_USER1_Pin|KEY_USER2_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING; // 设置上升沿触发中断
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOG, &GPIO_InitStruct);

  /* 省略 LED 灯部分 */

  /* EXTI interrupt init*/
  HAL_NVIC_SetPriority(EXTI2_IRQn, 0, 0);     // 设置中断线抢占优先级和子优先级
  HAL_NVIC_EnableIRQ(EXTI2_IRQn);             // 使能中断

  HAL_NVIC_SetPriority(EXTI3_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(EXTI3_IRQn);

}
```

而中断分组设置则在“HAL_Init()”里面。

```
HAL_StatusTypeDef HAL_Init(void)
{
  /* Set Interrupt Group Priority */
#if defined (CORE_CM4)
  HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_4);
#endif
……
}
```

真正用户需要编写的代码是实现 GPIO 外部中断上升沿回调函数“HAL_GPIO_EXTI_Rising_Callback()”，在“driver_gpio.c”中实现该函数。

```
/*
 *  函数名：void HAL_GPIO_EXTI_Callback(void)
 *  输入参数：无
 *  输出参数：无
```

```
 *  返回值：无
 *  函数作用：外部中断处理函数的回调函数，用以处理不同引脚触发的中断服务最终函数
*/
static volatile bool key1_flag = false;           // 定义一个全局静态标志，用以判断按键按下的次数
static volatile bool key2_flag = false;
void HAL_GPIO_EXTI_Rising_Callback(uint16_t GPIO_Pin)
{
  switch(GPIO_Pin)
  {
    case KEY_USER1_Pin:
    {
      key1_flag = !key1_flag;
      if(key1_flag)
      {
        LED_GREEN_ON();
      }
      else
      {
        LED_GREEN_OFF();
      }
      break;
    }

    case KEY_USER2_Pin:
    {
      key2_flag = !key2_flag;
      if(key2_flag)
      {
        LED_YELLOW_ON();
      }
      else
      {
        LED_YELLOW_OFF();
      }
      break;
    }
    default:break;
  }
}
```

当 GPIO 发生上升沿的外部中断时，就会最终调用到“HAL_GPIO_EXTI_Rising_Callback()”，根据传入的引脚，得知是哪个引脚发生了该中断，从而控制对应的 LED 灯，实现需求。我们主函数，这次无需添加任何代码。

M4 的一般外部中断实现流程如图 2.6.8 所示，前面三步都是通过配置 STM32CubeMX 修改，最后只需编写回调函数，即可使用中断。

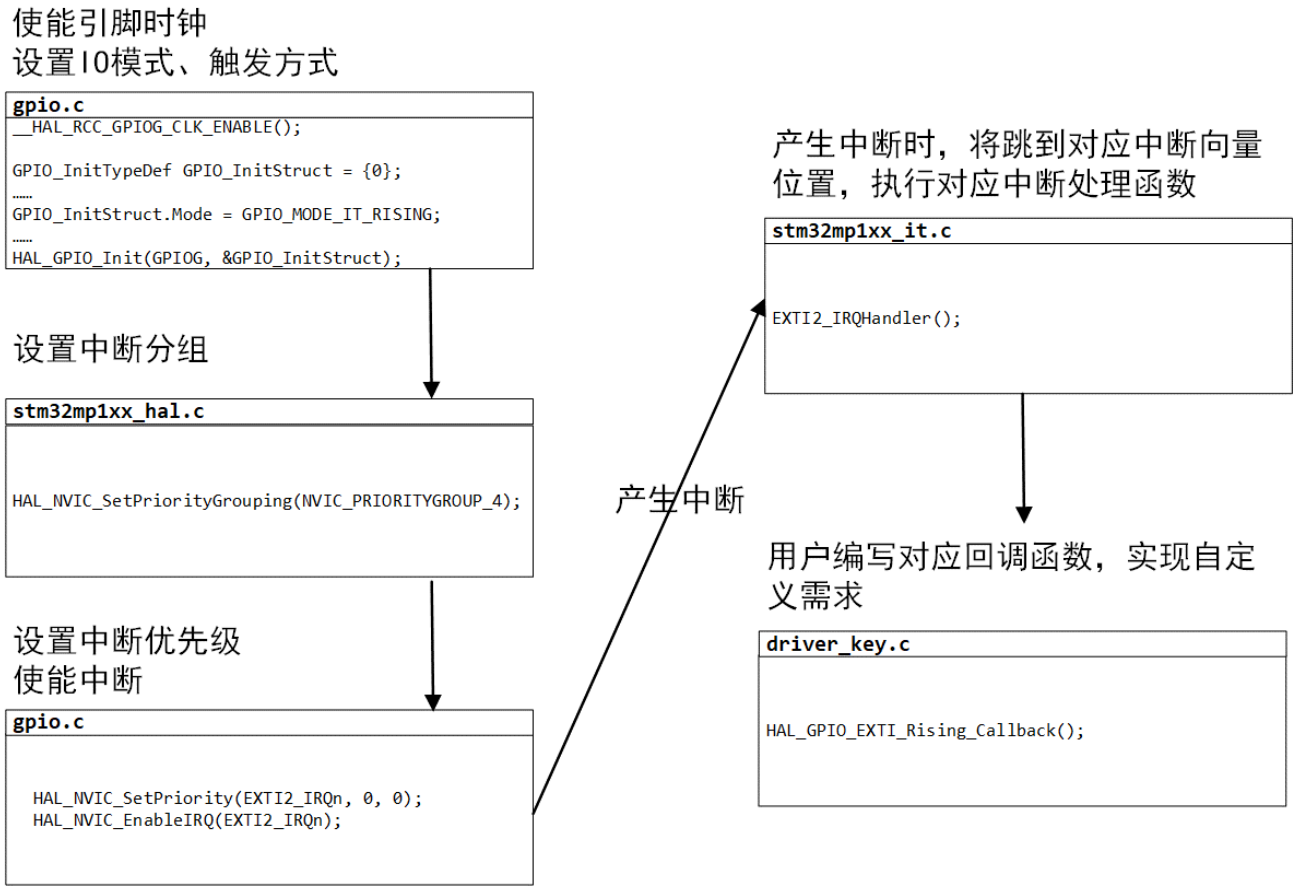


图 2.6.8 M4 中断流程示意

## 2.6.5 实验效果

编译程序，然后进入工程模式调试，按下 KEY_USER1（KEY1）按键，点亮 LED_GREEN，再次按下熄灭 LED_GREEN；按下 KEY_USER2（KEY2）按键，点亮 LED_YELLOW，再次按下熄灭 LED_YELLOW。

# 2.7 串行通信—串口打印

- 设计需求

使用串口打印调试信息。

## 2.7.1 基础知识

在嵌入式中，通信将主控芯片与传感器、存储芯片、外围控制芯片等连接了起来，使得功能不再受限于主控本身。主控既从其它设备获取信息，也将自己的信息传递给其它设备。如果通信没处理好，将直接影响整个系统的功能，由此可见通信技术的重要性。

常见的通信协议有 UART、I²C、SPI、1-Wire 等，这里先简单介绍一些通信的基本概念。

- **串行/并行**

按照数据传输的方式，通信可以分为串行通信和并行通信。串行通信简单的说就是数据依次传输，比如要传输 0x11111111，一位一位的发送，需要发送 8 次。并行通信则是几个数据一起传输，同样是 0x11111111，如果 8 位一起发送，只需要发送 1 次，如图 2.7.1 所示。

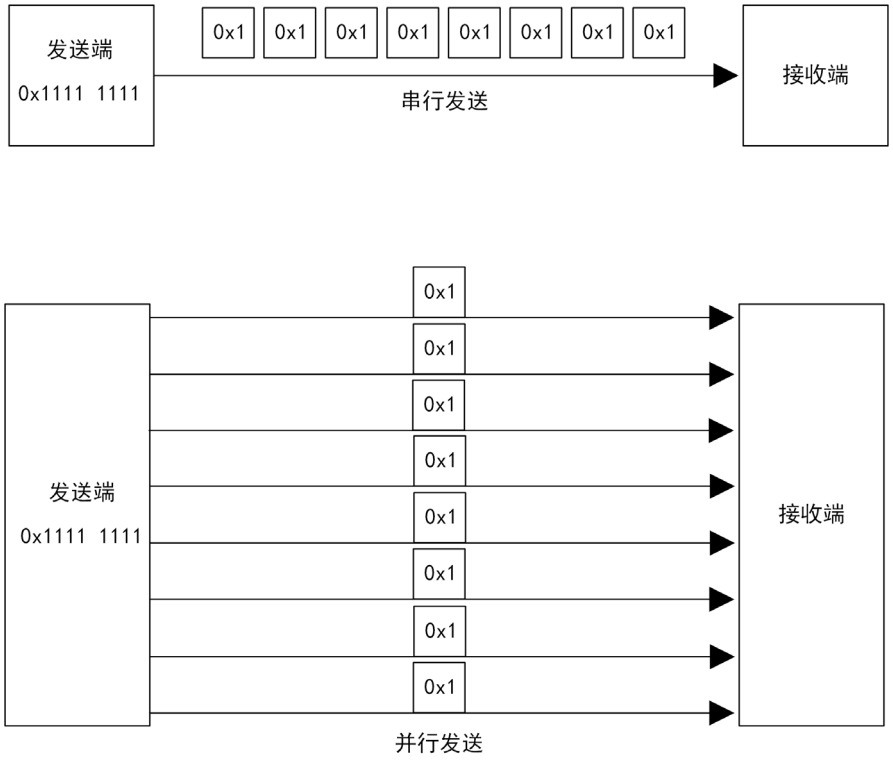


图 2.7.1 串行/并行传输示意图

由此可见，串行传输占用的通信线更少，成本低，通信速度相对较慢；并行传输占用的通信线多，成本高，通信速度相对更快。但随着对传输速度要求越来越高，并行传输开始出现信号之间的干扰，串行通信受干扰影响较小，之后又发展出差分传输等技术，极大的提高了串行传输速率，使得串行通信速度可能比并行通信速度更快。

串行通信就像单车道，行驶的车辆需要依次行驶。并行通信就像多车道，同时多辆汽车并排行驶。但当车速很快的时候，多车道上并列行驶的汽车之间会形成“气流”相互干扰，单车道则受影响较小，速度能够进一步提升。

- **全双工/半双工/单工**

按照数据传输的方向，通信可以分为全双工、半双工和单工。全双工指双方都可以同时收发信息；半双工双方都可以收发信息，但同一时刻只能一方发送信息；单工指只能一方发信息，一方接受信息，通信是单向的。

全双工就像双向车道，来往车辆互不影响；半双工就像乡间狭窄小道，同一时刻只能一方行驶；单工就像单向车道，从头到尾就只有向一个方向行驶，如图 2.7.2 所示。

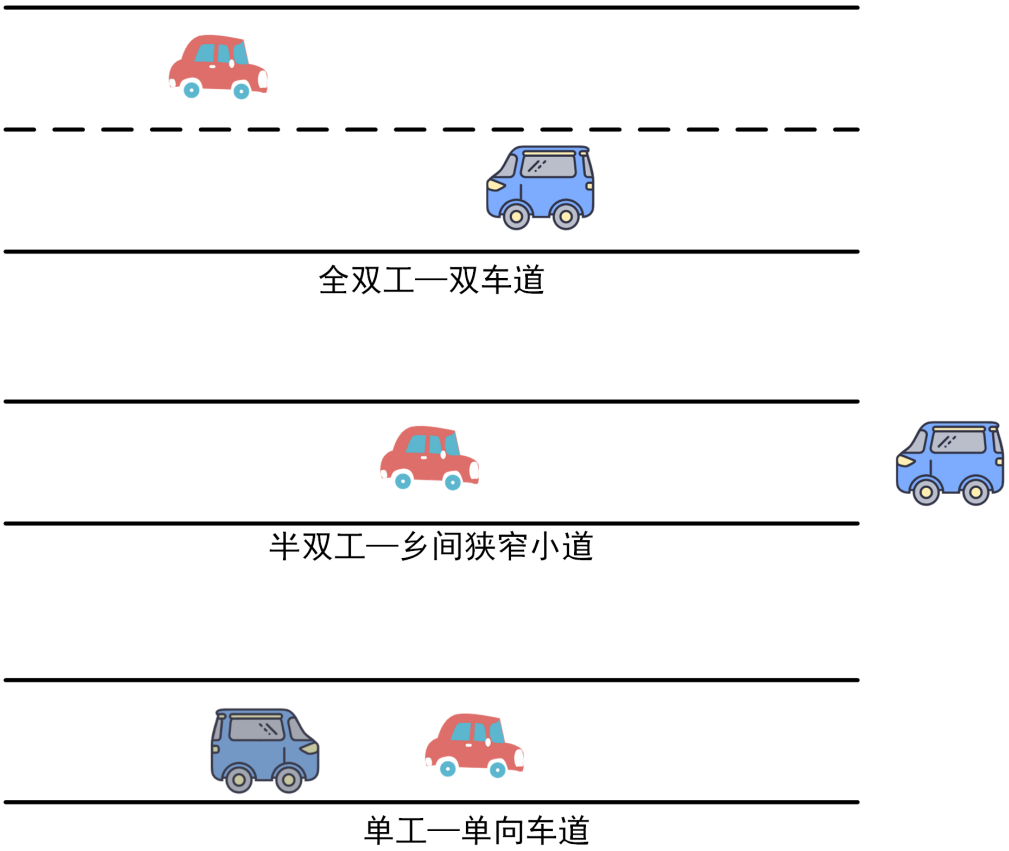


图 2.7.2 全双工/半双工/单工示意图

- **同步/异步**

按数据同步的方式，通信可以分为同步通信和异步通信。数据在双方之间传输时，需要制定规则保证数据传输的准确。同步通信的做法是加一个时钟信号，发送方和接收方在这个时钟的节拍下传输数据，比如常见的 SPI、$I^2C$。而异步通信的做法是对数据进行封装，在数据开头加上起始信号，在数据结尾加上终止信号，双方就按这个规则传输数据，比如 UART、1-Wire。

因此，可以通过是否有时钟信号，初步判断是何种数据同步方式。

- **通信速率**

对于同步通信，通信速率由时钟信号决定，时钟信号越快，传输速度就越快。

对于异步通信，需要收发双方提前统一通信速率，这也就是我们串口调试时，波特率不对显示乱码的原因。

时钟对通信的重要性，这里再举个例子：假设发送端时钟频率为 1Mhz，对应时钟周期则为 1us，接收端时钟频率为 10Mhz，对应时钟周期则为 0.1us。现在发送端发送一个数据 0x1，就会产生一个持续时间为 1us 高电平，接收端接收到这个 1us 的高电平时，会当作是 10 个 0.1us 的高电平，认为收到了 10 个 0x1。这时双方数据就乱套了，因此需要一个统一的时钟标准。

- **电平标准**

说到串口，经常提到 TTL、RS232、RS422、RS485，简单的说，就是为了适应不同的环境条件，使用了不同的电平标准。假如微处理器和板载的蓝牙透传模块通信时，一般就使用 TTL 电平，引脚直接连接即可。假如微处理器在工业现场，需要连接一个几十米外的装置，则应该考虑将 TTL 电平转为 RS232、RS422、RS485。

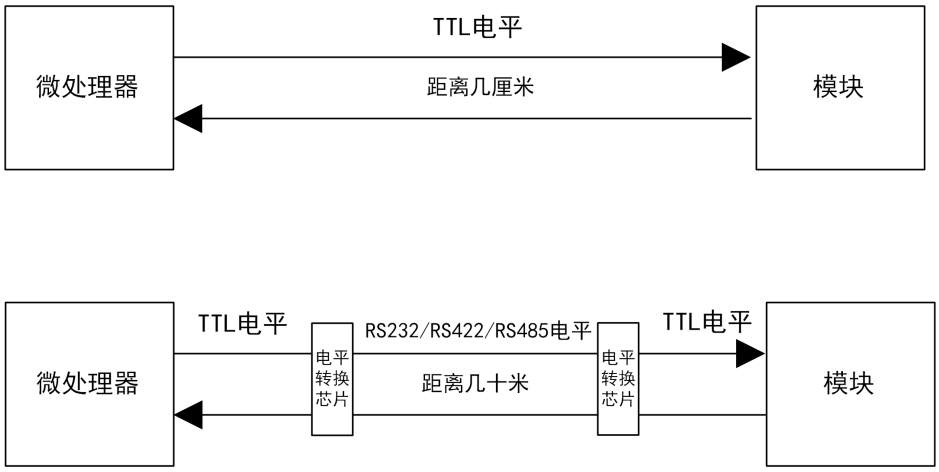


图 2.7.3 不同通信电平应用的场景

如下表 2.7.1 是几个通信接口标准总结。可以发现为了加大传输距离，人们依次尝试了增加电压、差分传输等方式。

表 2.7.1 常见通信接口标准

| 通信接口 | 电平标准 | 特点 |
|---|---|---|
| TTL | 逻辑1：2.4V至5V<br>逻辑0：0V至0.4V | 无需额外电路，直接引脚直连即可 |
| RS232 | 逻辑1：-3V至-15V<br>逻辑0：3V至15V | 需要电平转换电路，传输距离最大约15米 |
| RS422/RS485 | 逻辑1：两线压差为2V至6V<br>逻辑0：两线压差-2V至-6V | 需要电平转换电路，传输距离最大约1200米 |

回到 100ASK_STM32MP157 开发板，由前面图 2.1.1 可知，STM32MP157 有 8 个串口，其中 USART1 是 A7 独享；USART（Universal synchronous asynchronous receiver transmitter）通用同步异步收发器有三个，可以分给 A7 或 M4；UART（Universal asynchronous receiver transmitter）通用异步收发器有四个，也可以分给 A7 或 M4。USART 和 UART 的区别，USART 支持同步通信，可以主动提供时钟，常用的还是 UART。

有了前面的知识，现在就知道 STM32MP157 的 UART，是串行全双工传输，常用的是异步通信方式，需要设置好数据封装和通信速率，电平标准需要看电路图。

## 2.7.2 硬件设计

100ASK_STM32MP157 开发板主要用的了两个串口，一个用于调试，一个引出供用户使用。

如图 2.7.4 所示，UART4_RX（PA11）和 UART4_TX（PA12）经过缓冲器（U5）接到了串口转 USB 芯片（U6），得到的 USB 信号通过 Type-C（J5）连接电脑。

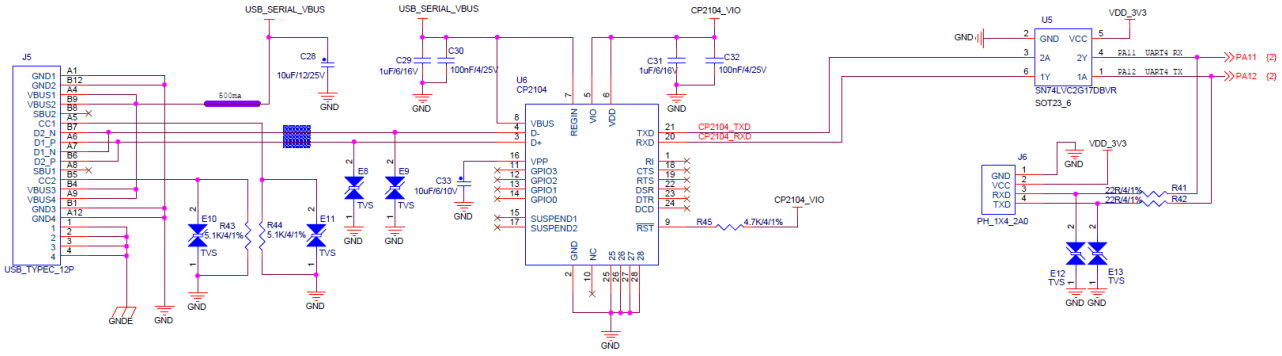


图 2.7.4 UART4 用于调试

如图 2.7.5 所示，提供了 UART8_TX（PE1）和 UART8_RX（PE0）供用户使用。

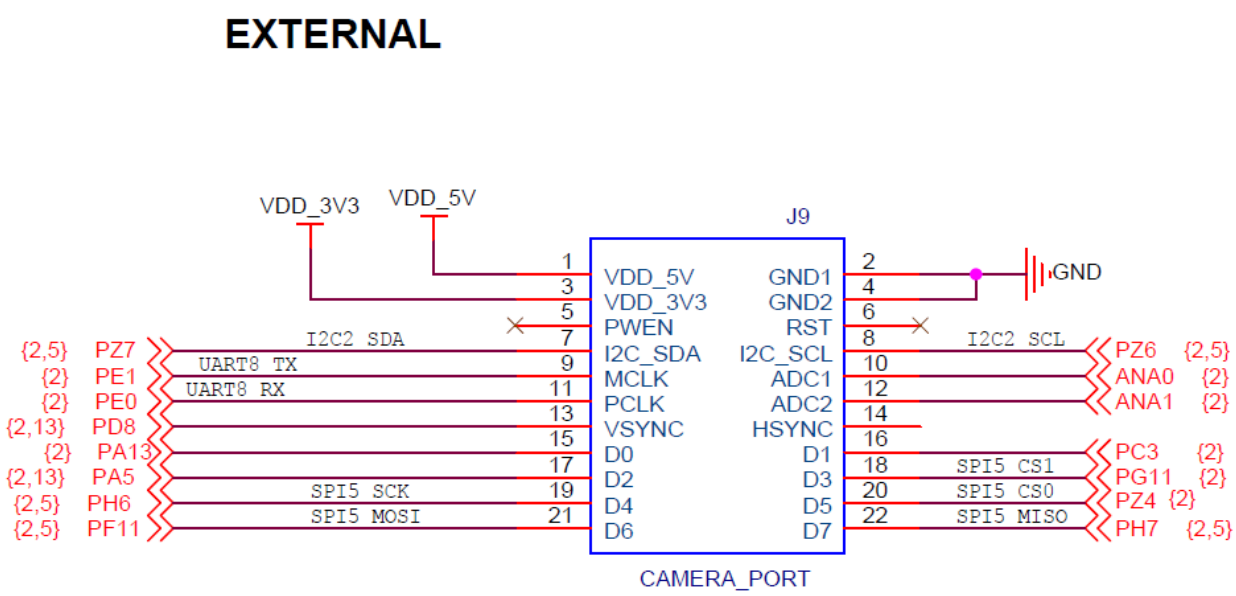


图 2.7.5 UART8 用户接口

正常开发时，UART4 默认用于 A7 的 Linux 信息输出，因此用户只能将 UART8 分给 M4，用于 M4 外接模块或者外接 TTL 转 USB 设备。但本结主要是介绍如何让 M4 串口打印，因此直接使用 UART4，后续高级开发的核之间通信里，会分别使用两个串口。

## 2.7.3 MX 设置

将 PA11、PA12 分别设置为 UART4_RX 和 UART4_TX，如图 2.7.6 所示。

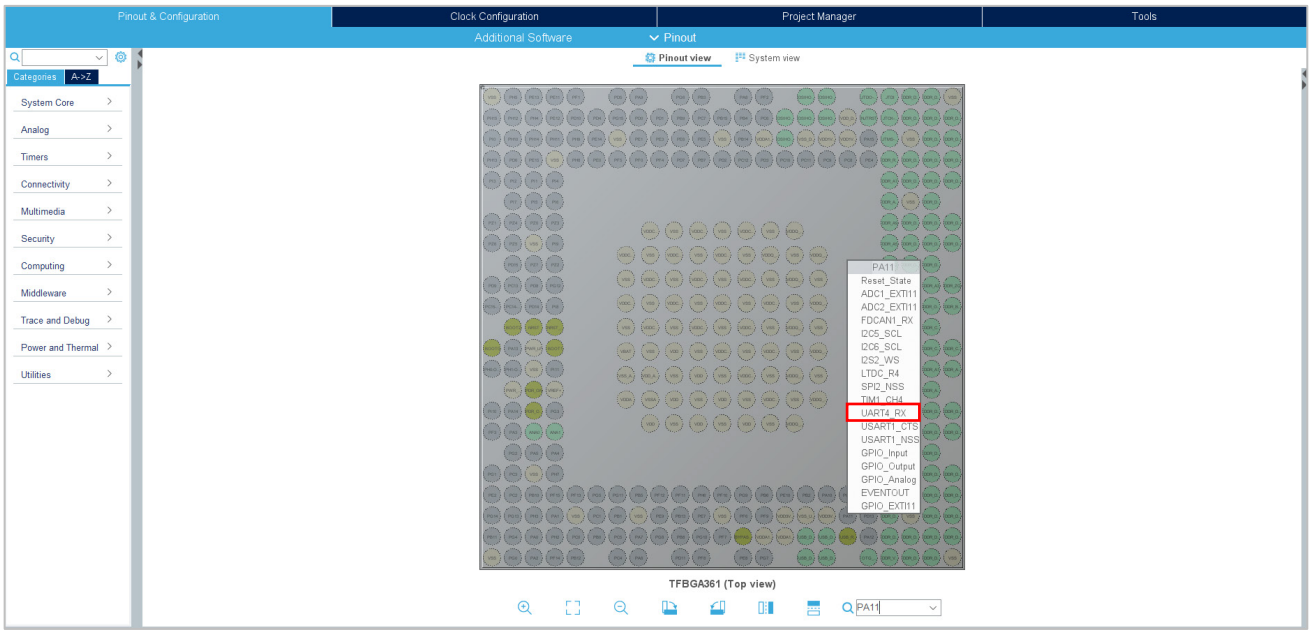


图 2.7.6 设置 UART4 引脚复用

然后切换到“Connectivity”选中“UART4”，在“Boot time”勾选给“Cortex-M4”,在“Mode”下拉栏选择异步“Aynchronous”。再选中下面的“Parameter Settings”标签，可以看到波特率设置、数据长度设置、极性设置、停止位设置等，如所示。也就是前面介绍提到的异步通信需要提前制定好规则，后面电脑端的串口设置也要设置一样，才能正常通信。

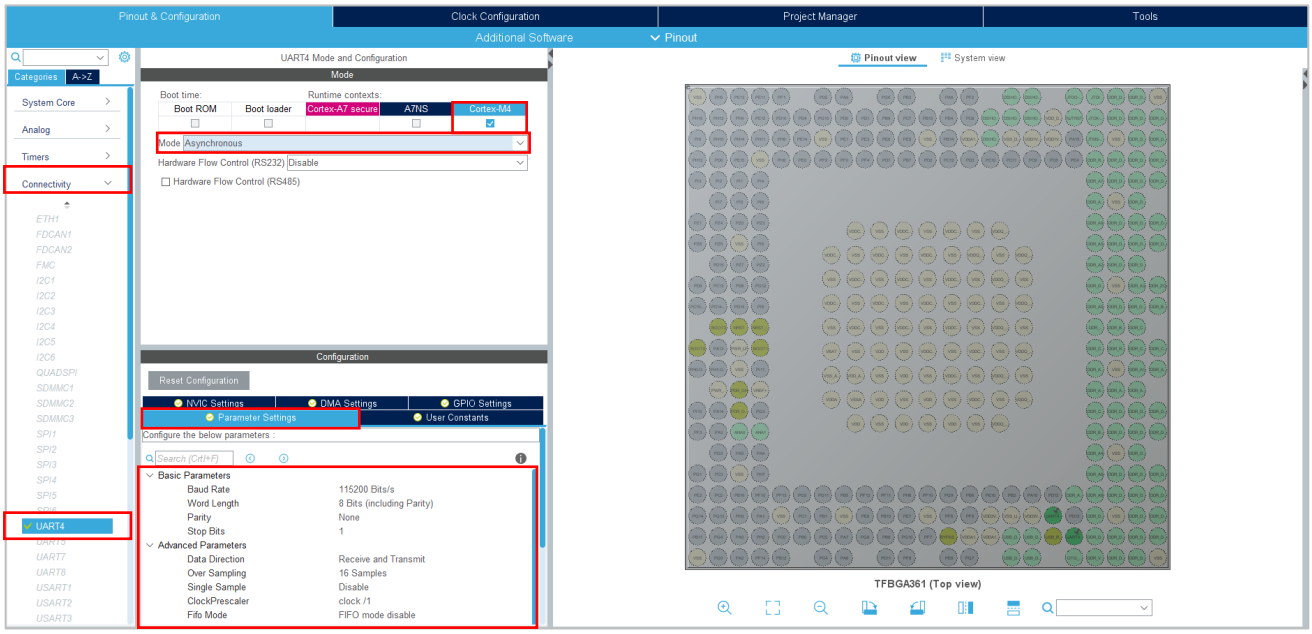


图 2.7.7 UART4 参数设置

然后参考前面的系统章节，先设置为 RCC 选择外部时钟输入。然后切换到“Clock Configuration”，将整个时钟配置为如图 2.7.8 所示。可以看到系统时钟工作在 209MHz，然后二分频为 104.5MHz 给 PCLK1，最后 UART4 选择这个 PCLK1 作为时钟源，工作在 104.5MHz。

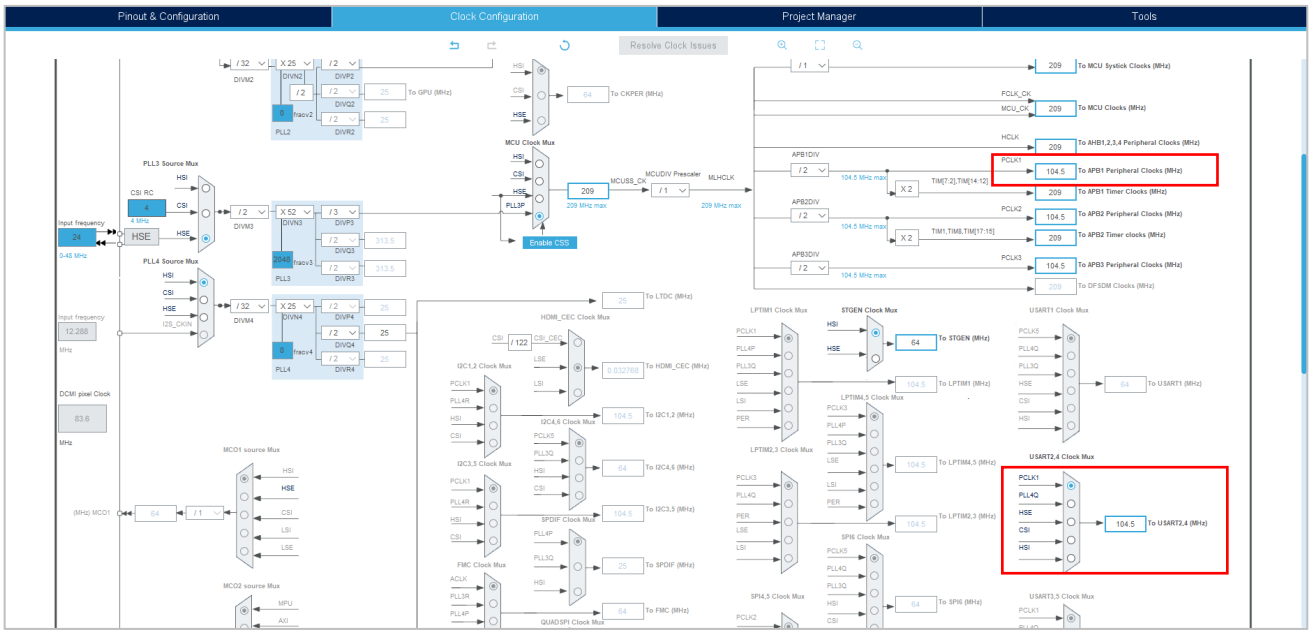


图 2.7.8 UART4 时钟设置

STM32CubeMX 生成代码后，进入 STM32CubeIDE，选择菜单栏的“Project”，选择“Properties”。为了后面串口打印时能够打印 float 类型，这里需要勾选上“Use float with printf from newlib-nano(-u _printf_float)”和“Use float with scanf from newlib-nano(-u _printf_float)”，如图 2.7.9 所示。

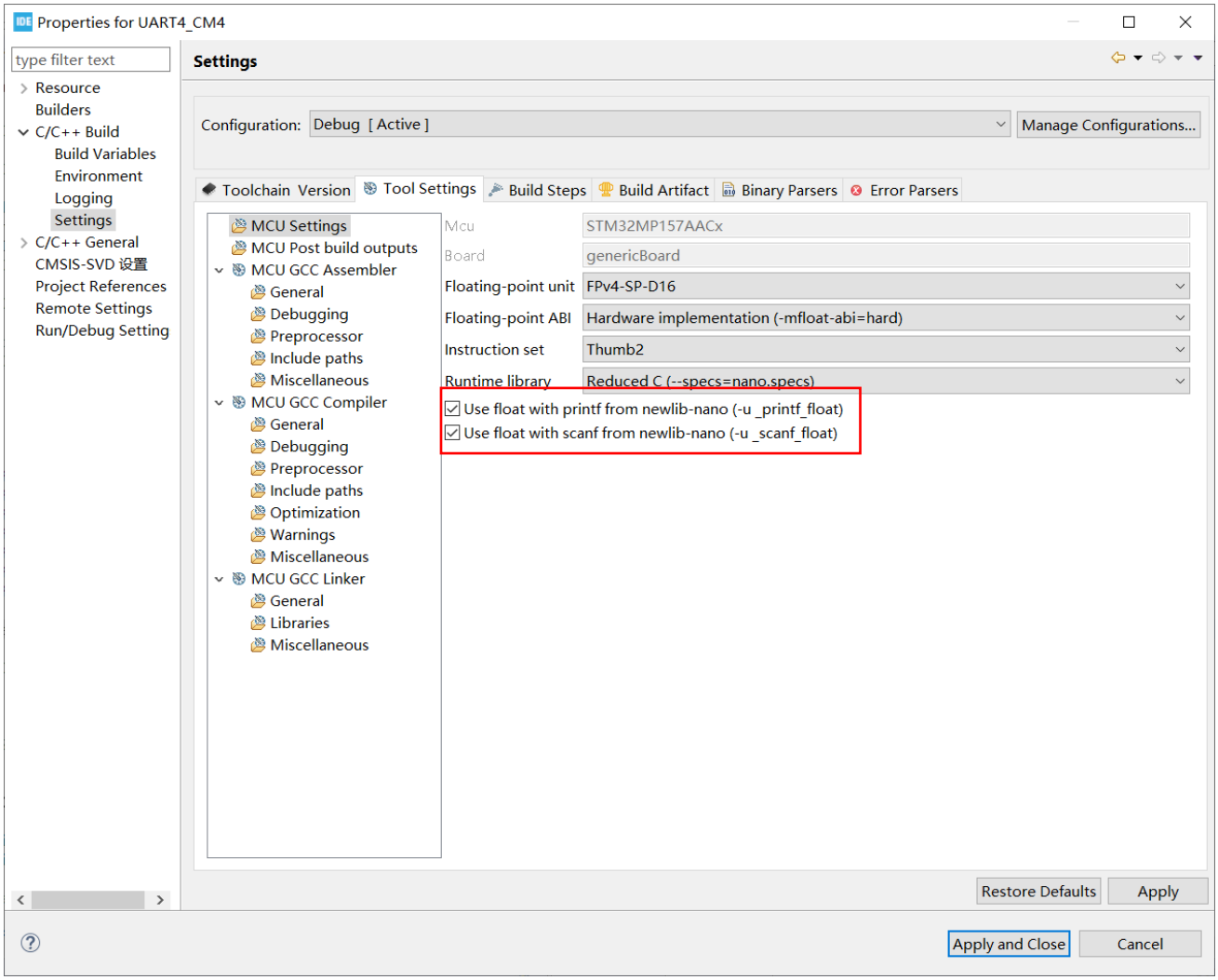


图 2.7.9 设置支持浮点类型打印

## 2.7.4 代码设计

前面在 STM32CubeMX 配置 UART4 的要点，STM32CubeMX 在“usart.c”文件里自动生成了代码，这里对重要部分进行了注释，供读者参考。

```
UART_HandleTypeDef huart4;

/* UART4 init function */
void MX_UART4_Init(void)
{

  huart4.Instance = UART4;                    // 选择串口
  huart4.Init.BaudRate = 115200;              // 设置波特率
  huart4.Init.WordLength = UART_WORDLENGTH_8B;  // 选择数据长度
  huart4.Init.StopBits = UART_STOPBITS_1;     // 选择停止位
  huart4.Init.Parity = UART_PARITY_NONE;      // 选择校验位
  huart4.Init.Mode = UART_MODE_TX_RX;         // 选择收发方向
```

```
  huart4.Init.HwFlowCtl = UART_HWCONTROL_NONE;      // 硬件流控制
  huart4.Init.OverSampling = UART_OVERSAMPLING_16; // 过采样位数
  huart4.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE; // 过采样不使能
  huart4.Init.ClockPrescaler = UART_PRESCALER_DIV1;// 时钟分频
  huart4.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
  if (HAL_UART_Init(&huart4) != HAL_OK)
  {
    Error_Handler();
  }

  // 设置发送 FIFO 缓存长度
  if (HAL_UARTEx_SetTxFifoThreshold(&huart4, UART_TXFIFO_THRESHOLD_1_8) != HAL_OK)
  {
    Error_Handler();
  }

  // 设置接收 FIFO 缓存长度
  if (HAL_UARTEx_SetRxFifoThreshold(&huart4, UART_RXFIFO_THRESHOLD_1_8) != HAL_OK)
  {
    Error_Handler();
  }

  // 不使能 FIFO 模式
  if (HAL_UARTEx_DisableFifoMode(&huart4) != HAL_OK)
  {
    Error_Handler();
  }

}
```

为了能支持 printf 打印，还需将 uart4 和 printf 绑定，在“usart.c”的“USER CODE BEGIN 1”后添加如下代码。

```
/* USER CODE BEGIN 1 */
// 加上此段才可以使用 printf()函数
#ifdef __GNUC__

#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)

PUTCHAR_PROTOTYPE
{

  HAL_UART_Transmit(&huart4, (uint8_t*)&ch, 1, HAL_MAX_DELAY);
  return ch;
}
```

```
#endif
/* USER CODE END 1 */
```

其作用就是使用我们定义的“HAL_UART_Transmit(&huart4, (uint8_t*)&ch, 1, HAL_MAX_DELAY);”代替系统的“__io_putchar(int ch)”编译。

最后，在主函数添加常用的测试打印语句，验证打印是否可用。

```
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */


    /* USER CODE BEGIN 3 */
      printf("=========This is printf test=========\n\r");
      printf("test char            = %c,%c\n\r", 'H', 'c');
      printf("test string1         = %s\n\r", "www.100ask.net");
      //中文打印需要将文件编码改为UTF-8,但会导致注释乱码
      //printf("test string2         = %s\n\r", "深圳百问网科技有限公司");
      printf("test decimal1 number = %d\n\r", 123456);
      printf("test decimal2 number = %d\n\r", -123456);
      printf("test hex1     number = 0x%x\n\r", 0x123456);
      printf("test hex2     number = 0x%08x\n\r", 0x123456);
      printf("test float           = %.5f\n\r", 3.1415);
      printf("test double          = %.10lf\n\r", 3.141592653);

      HAL_Delay(2000);
  }
  /* USER CODE END 3 */
}
```

## 2.7.5 实验效果

将 Type-C 线一端连接到板子调试串口位置，一端连接电脑 USB，如图 2.7.10 所示。

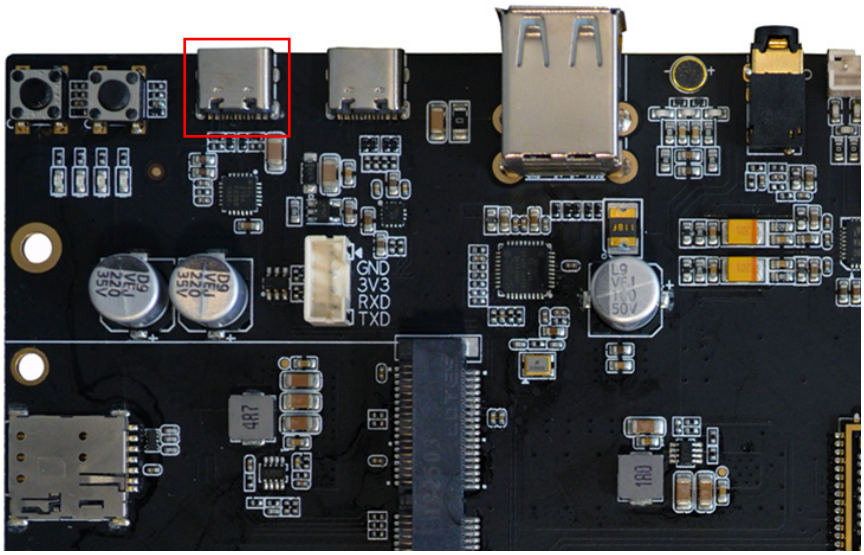


图 2.7.10 UART4 调试串口位置

打开 MobaXterm 之类的串口工具，选择对应端口，设置好波特率 115200、8 位数据位、1 位停止位、无极性、无流控，如图 2.7.11 所示。

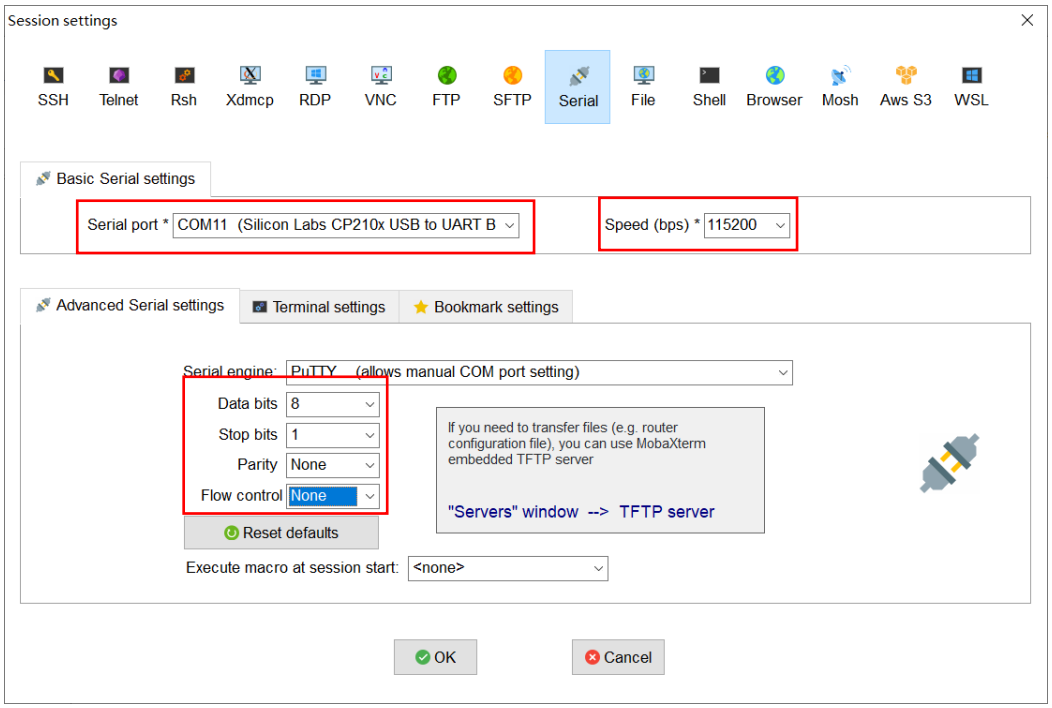


图 2.7.11 串口工具软件设置

串口打印如图 2.7.12，打印数据正常。

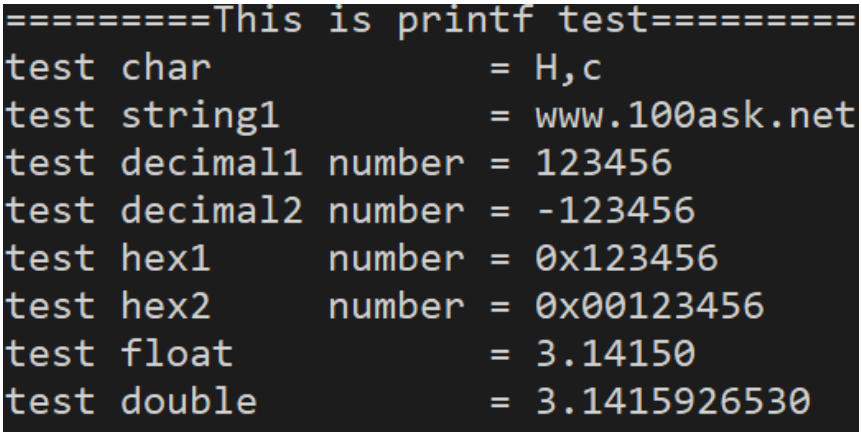


图 2.7.12 UART4 打印效果

# 2.8 串行通信—RS485

- 设计需求

使用 RS485 与另一块开发板的 RS485 通信。

## 2.8.1 基础知识

RS485 的本质还是串口，参考上一实验的介绍。

## 2.8.2 硬件设计

RS485 的电路如下，使用的 UART6_RX（PC7）和 UART6_TX（PC6），相对前面的实验多了一个收发控制引脚 PD13。RS485 是差分信号，无论是发送还是接收工都需要 A、B 同时进行，且同一时刻只能发或收，因此它是半双工。

RE 和 DE 端分别为接收和发送的使能端，当 RE 为逻辑 0 时，U16 处于接收状态；当 DE 为逻辑 1 时，U16 处于发送状态。

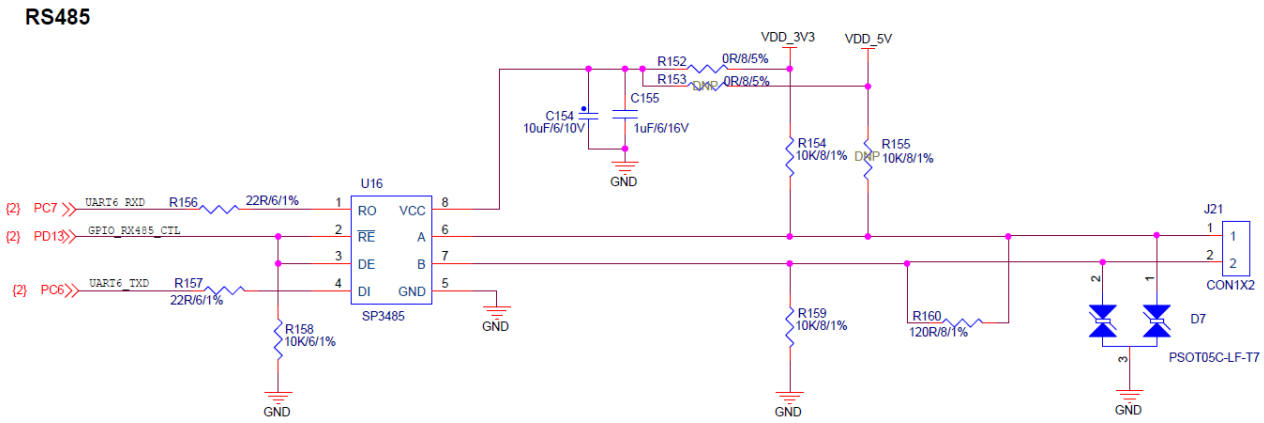


图 2.8.1 RS485 电路

## 2.8.3 MX 设置

STM32CubeMX 的设置和前面操作完全一致，只是这次改为了 UART6，如图 2.8.2 所示。

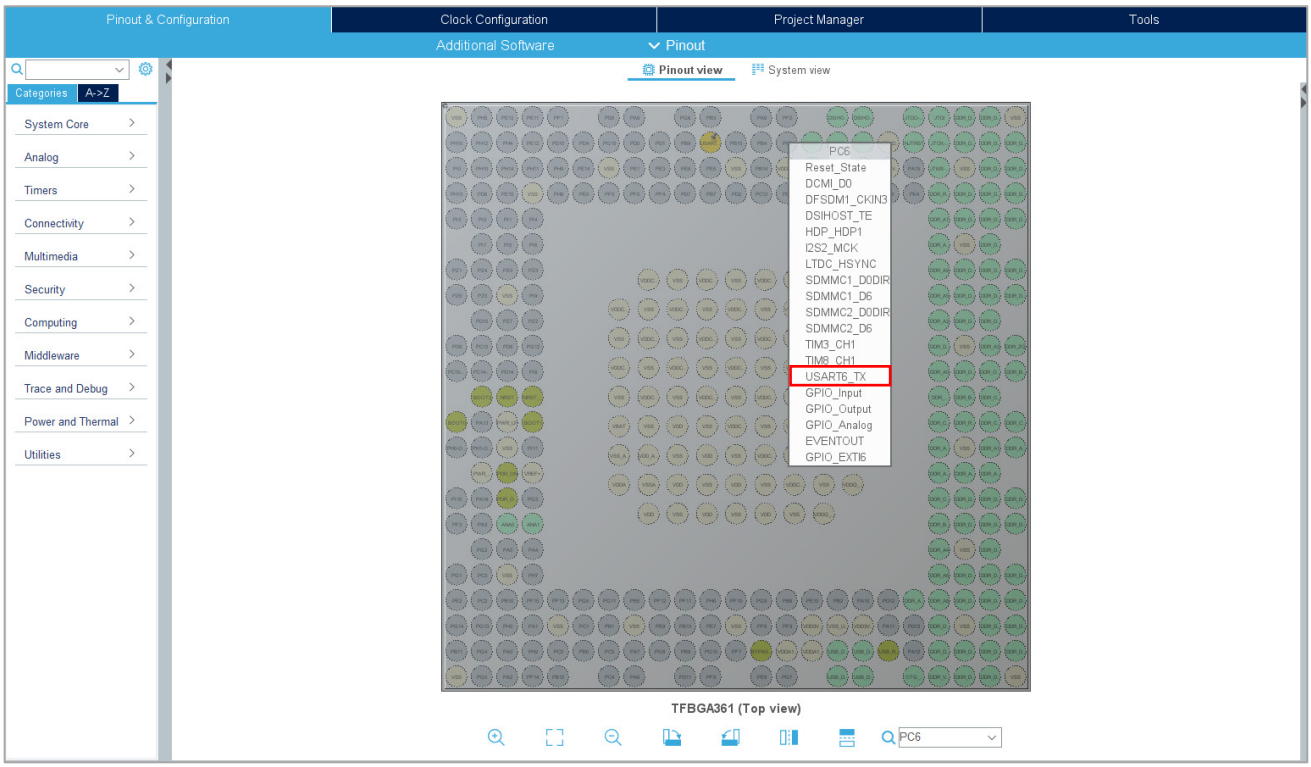


图 2.8.2 UART6 引脚设置

UART4 的设置也和前面一样，注意这里有个“Hardware Flow Control(RS485)”不用勾选，因为硬件上没有使用 RS485_RE 引脚来控制流控，使用的是其它 GPIO。

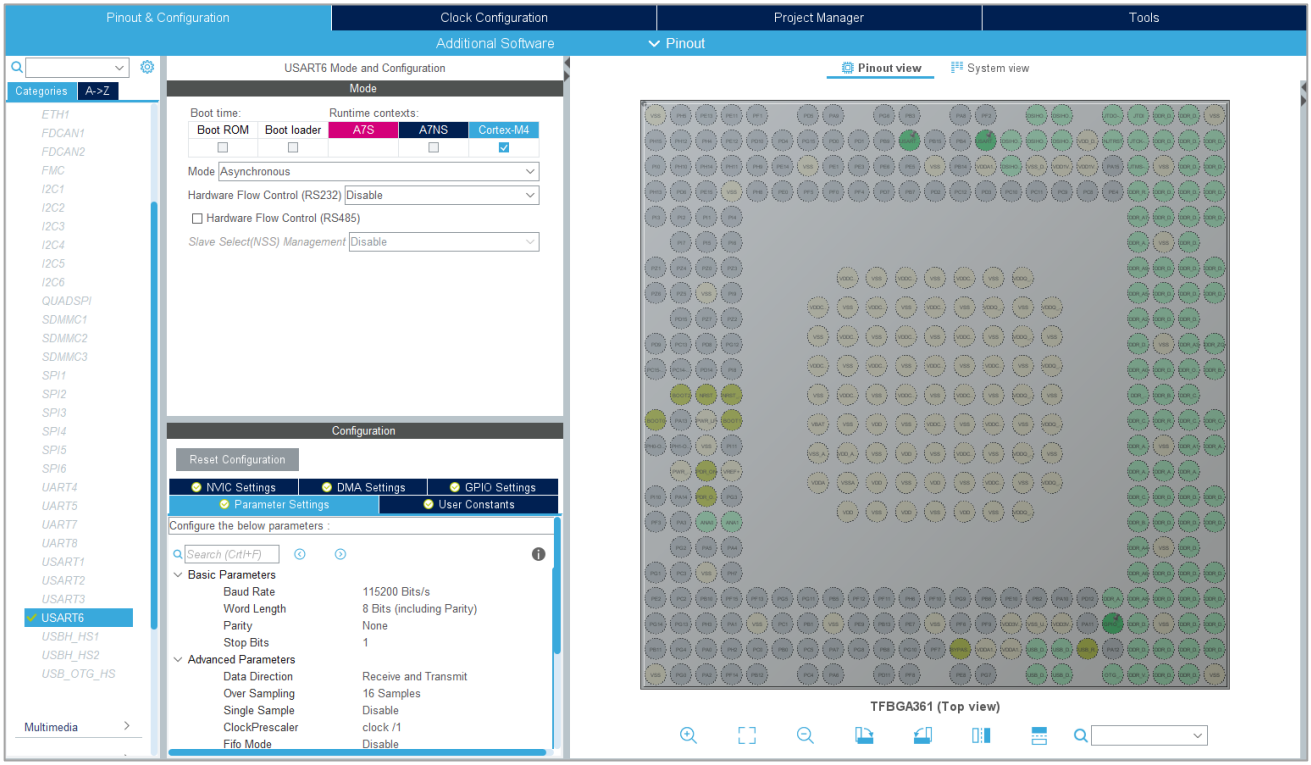


图 2.8.3 UART6 参数设置

与前面不同的是，这里需要设置 PD13 为普通 GPIO 输出，如图 2.8.4 所示。

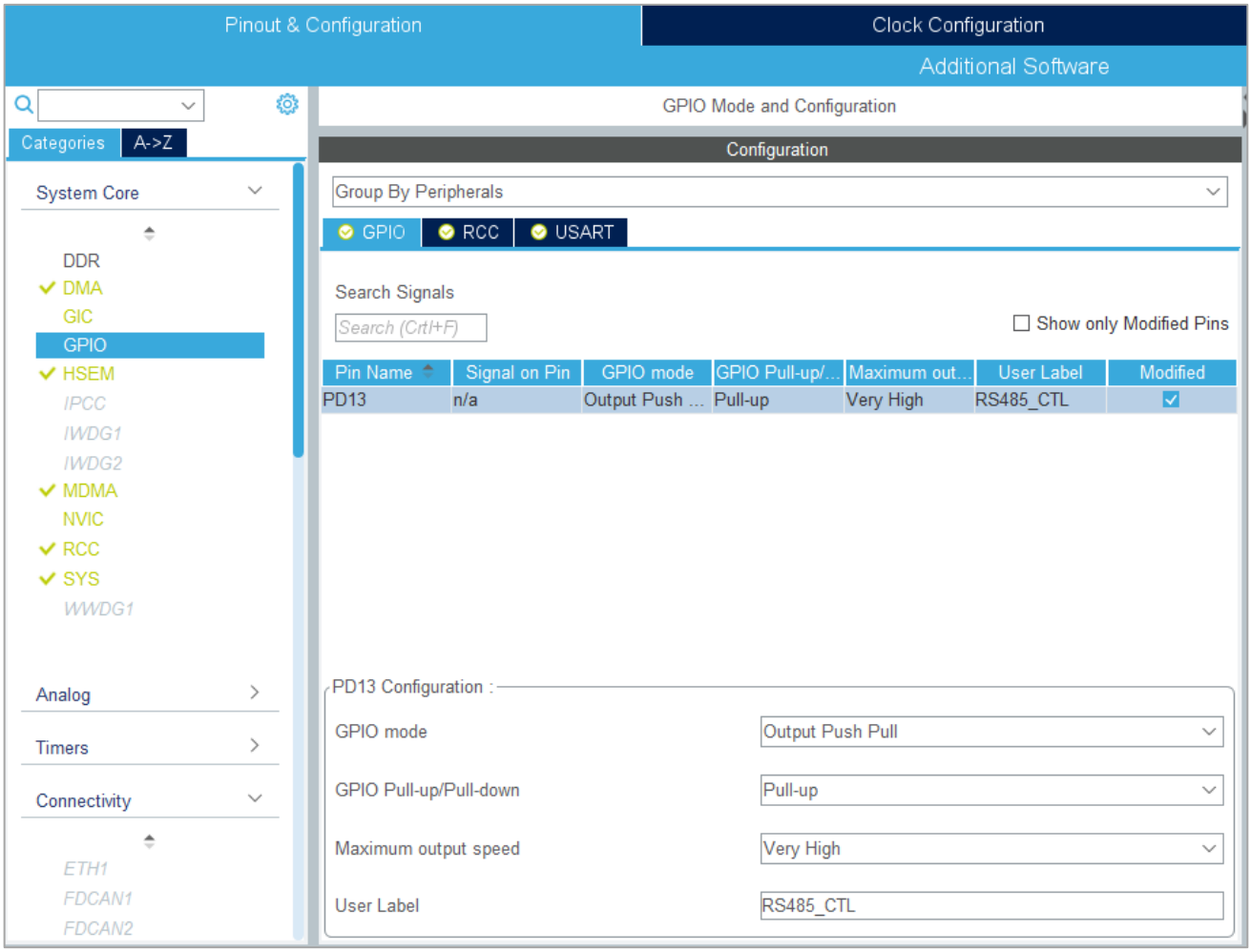


图 2.8.4 设置 RS485 流控引脚

最后就是参考前面设置对应时钟即可。

## 2.8.4 代码设计

代码部分主要就是注意下流控控制，然后调用 HAL 库函数发送数据即可。创建“driver_rs485.h”，定义流控引脚。

```
#define DE_TX() HAL_GPIO_WritePin(RS485_CTL_GPIO_Port, RS485_CTL_Pin, GPIO_PIN_SET)
#define DE_RX() HAL_GPIO_WritePin(RS485_CTL_GPIO_Port, RS485_CTL_Pin, GPIO_PIN_RESET)
```

主函数控制流控，调用“HAL_UART_Transmit()”发送数据，“HAL_UART_Receive()”接收数据。

```
/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  /* USER CODE BEGIN 1 */
```

```
  uint8_t txdata[4] = {1, 2, 3, 4};
/* USER CODE END 1 */

/* MCU Configuration--------------------------------------------------------*/

/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

if(IS_ENGINEERING_BOOT_MODE())
{
  /* Configure the system clock */
  SystemClock_Config();
}

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_USART6_UART_Init();
/* USER CODE BEGIN 2 */

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
    DE_TX();
    HAL_UART_Transmit(&huart6, (uint8_t*)&txdata, sizeof(txdata), HAL_MAX_DELAY);
    DE_RX();

    HAL_Delay(2000);
}
/* USER CODE END 3 */
}
```

### 2.8.5 实验效果

这里使用运行 Linux 的 IMX6ULL 作为 RS485 的接收对象，读者可以自行选择其它 RS485 接口设备。将 100ASK_STM32MP157 的 A、B 对应接到 IMX6ULL 的 A、B。启动 IMX6ULL 并运行 RS485 读取程序，同时 M4 启动，可以看到 IMX6ULL 收到 RS485 发过来的数据，如所示。

```
[root@imx6ull:~]# echo 128 > /sys/class/gpio/export
[root@imx6ull:~]# echo out > /sys/class/gpio/gpio128/direction
[root@imx6ull:~]# echo 0 > /sys/class/gpio/gpio128/value
[root@imx6ull:~]# rs485_read -d /dev/ttymxc2 -b 115200
RECV[  4]: 0x01 0x02 0x03 0x04
RECV[  4]: 0x01 0x02 0x03 0x04
```

图 2.8.5 RS485 实验效果

# 2.9 DMA—MemoryToMemory

- 设计需求

使用 DMA 将内存 A 处数据搬运到内存 B 处。

## 2.9.1 基础知识

DMA(Direct Memory Access)直接内存访问，可以大大减轻 CPU 工作量。CPU 根据代码内容执行指令，这些众多指令中，有用于计算、有控制程序、有转移数据等。其中转移数据的指令，尤其是转移大量数据，会占用大量 CPU。如果是把外设 A 的数据，传给外设 B，这种情况不需要 CPU 一直参与，只需在 A、B 之间创建个通道，让它们自己传输即可。这就是 DMA 设计的目的，减少大量数据转移指令消耗 CPU，DMA 专注数据转移，CPU 专注计算、控制。

DMA 主要实现将 A 处的数据直接搬运到 B 处，这里的 A、B 可以是内存，也可以是外设，因此所有的场景如如下四种：

- 内存到外设
- 外设到内存
- 内存到内存
- 外设到外设

无论是以上何种方式，都是先设置好 DMA 的数据源地址、数据目标地址、数据长度。设置好后，启动 DMA 就可以自动的把数据从源地址依次传输到目标地址。

由前面图 2.1.1 可以看到，STM32MP157 有四个 DMA，其中 DMA1 和 MDMA 是 A7 独享，DMAMUX 需要指定分配给 A7 或 M4，DMA2 是 M4 独享。

这里使用 DMA2，演示如何设置 DMA 搬运数据，后续在其它外设章节，还会使用到 DMA。

## 2.9.2 硬件设计

不涉及电路设计。

## 2.9.3 MX 设置

参考图 2.9.1 为 DMA2 添加通道。

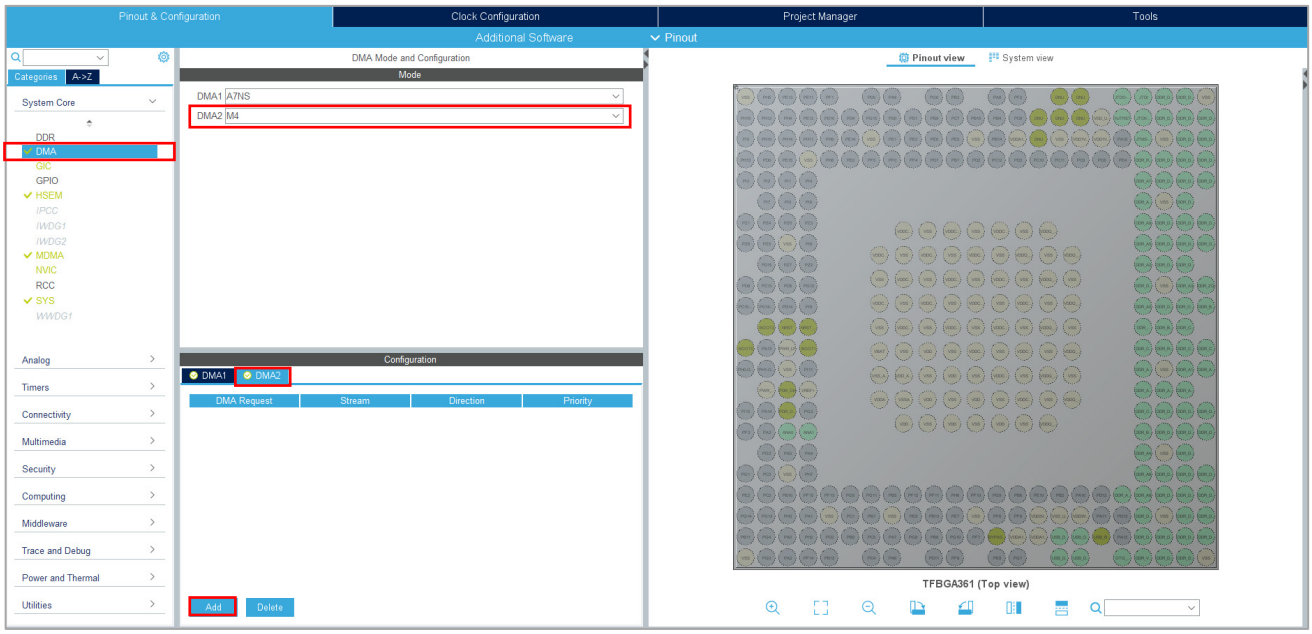


图 2.9.1 添加 DMA 通道

如图 2.9.2 为新增 DMA 通道选择请求源。

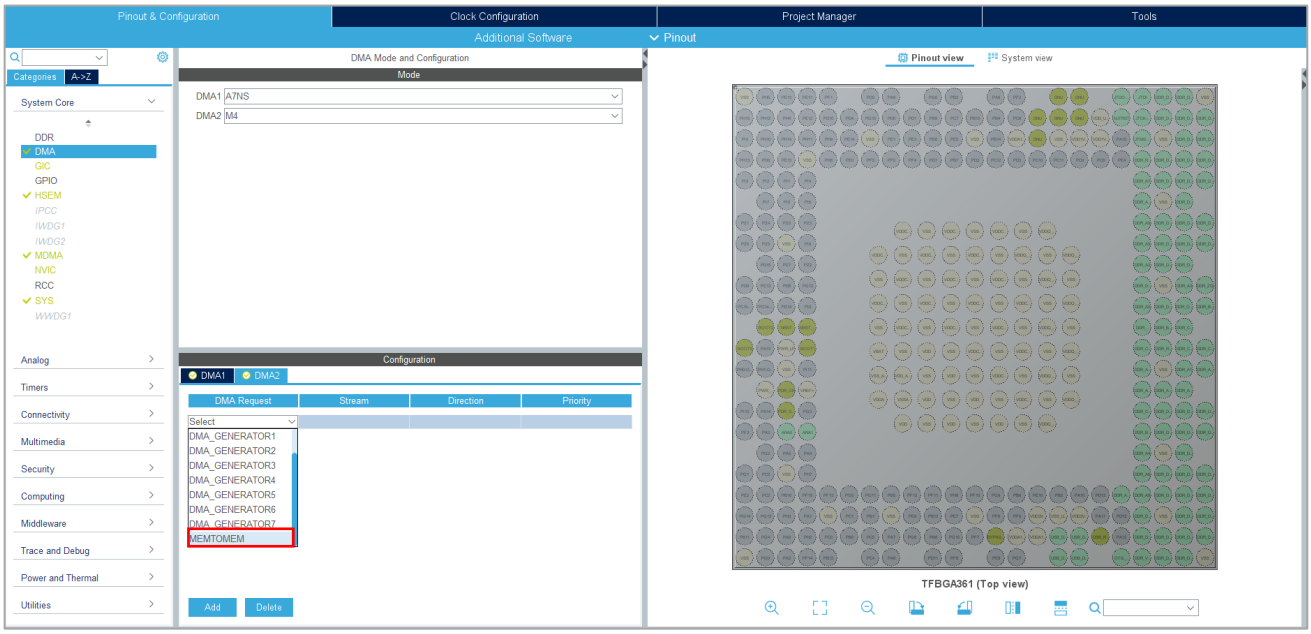


图 2.9.2 选择 DMA 请求源

接着会自动出现图 2.9.3 所示的 DMA 通道详细信息，可以大致分为三部分。第一部分是 DMA 请求类型，选择的通道(默认通道 0，可选 1-7，共 8 个通道)，DMA 通道描述、优先级，全都默认即可。接着是 DMA 请求的一些设置，这里“Mode”选择默认的“Normal”，即不循环传输，勾选上源内存和目标内存地址自动递增。第三部分是 FIFO 相关，默认使用 FIFO，且搬运的数据宽度为 Word，每次搬运一个。

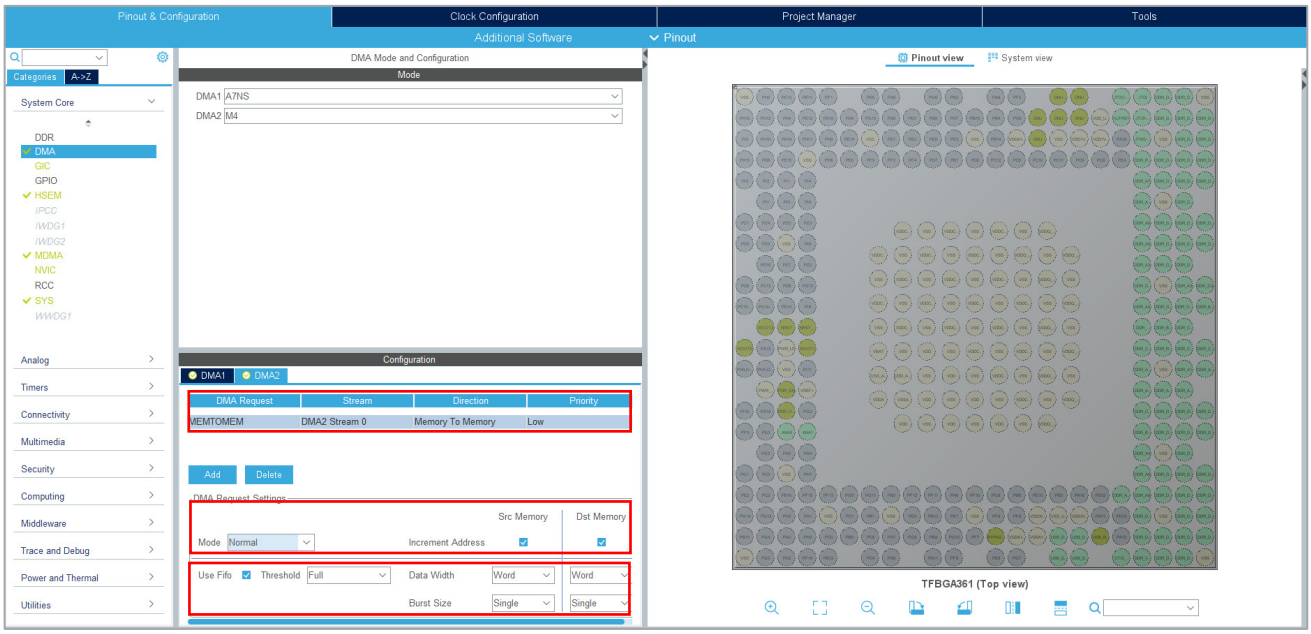


图 2.9.3 设置 DMA 通道详细信息

在最后 DMA 搬运完后，还需要产生中断告知搬运完成，因此这里还需要使能 DMA 中断，如图 2.9.4 所示。

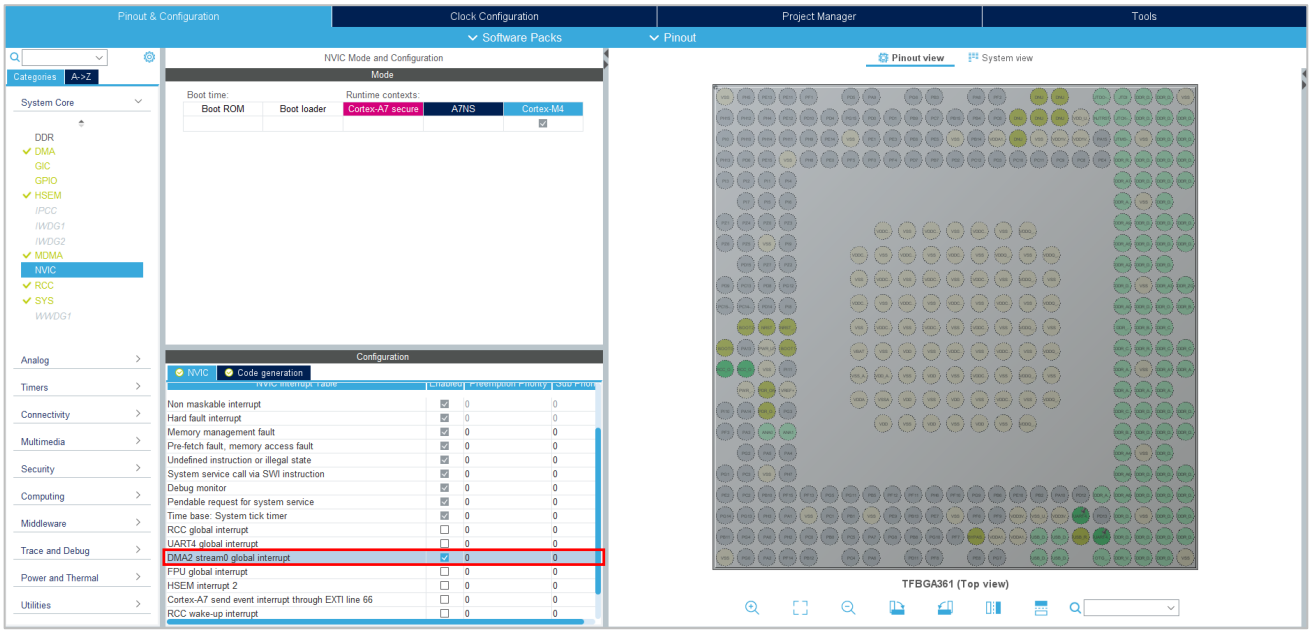


图 2.9.4 使能 DMA 中断

## 2.9.4 代码设计

配置完 STM32CubeMX，自动生成的 DMA 初始化和中断设置，都在“dma.c”里。接着还需要注册 DMA 传输完成和 DMA 传输错误回调函数，在回调函数里修改标志位，就可以得知传输是否完成或出错。新建“DemoDriver”文件夹创建“driver_dma.c”文件，内容如下：

```
__IO uint32_t transferErrorDetected;
```

```
__IO uint32_t transferCompleteDetected;

static void TransferComplete(DMA_HandleTypeDef *DmaHandle);
static void TransferError(DMA_HandleTypeDef *DmaHandle);

/*
 *  函数名：void RegisterCallbackFunction(void)
 *  输入参数：无
 *  输出参数：无
 *  返回值：无
 *  函数作用：注册回调函数
*/
void RegisterCallbackFunction(void)
{
      // 注册传输完成和传输错误回调函数
      HAL_DMA_RegisterCallback(&hdma_memtomem_dma2_stream0, HAL_DMA_XFER_CPLT_CB_ID, TransferComplete);
      HAL_DMA_RegisterCallback(&hdma_memtomem_dma2_stream0, HAL_DMA_XFER_ERROR_CB_ID, TransferError);

      // 配置 DMA 的中断优先级及其使能(在 dma.c 的 MX_DMA_Init 里自动生成)
      // HAL_NVIC_SetPriority(DMA2_Stream0_IRQn, 0, 0);
      // HAL_NVIC_EnableIRQ(DMA2_Stream0_IRQn);
}

/*
 *  函数名：void TransferComplete(DMA_HandleTypeDef *DmaHandle)
 *  输入参数：DmaHandle-DMA 句柄
 *  输出参数：无
 *  返回值：无
 *  函数作用：如果 DMA 传输完成且不发生错误，则在此函数将传输完成标志置一
*/
static void TransferComplete(DMA_HandleTypeDef *DmaHandle)
{
  transferCompleteDetected = 1;
}

/*
 *  函数名：void TransferError(DMA_HandleTypeDef *DmaHandle)
 *  输入参数：DmaHandle-DMA 句柄
 *  输出参数：无
 *  返回值：无
 *  函数作用：如果 DMA 传输过程中发生错误，则在此函数中将传输错误标志置一
*/
static void TransferError(DMA_HandleTypeDef *DmaHandle)
{
```

```
  transferErrorDetected = 1;
}
```

最后，还需要调用 HAL 的 API“HAL_DMA_Start_IT()”启动 DMA，封装如下：

```
/*
 *  函数名：void DMA_M2M_Start(uint32_t *srcAddr, uint32_t *dstAddr, uint16_t bufsz
 *  输入参数：srcAddr-数据原地址
 *            dstAddr-DMA 搬移的目的地址
 *            bufsz-DMA 搬移的数据大小，单位是初始化的时候选择的：byte,half-word,word
 *  输出参数：无
 *  返回值：无
 *  函数作用：初始化 DMA_Channel1，配置为内存-内存模式，每次搬移一个 word 即 4bytes
*/
void DMA_M2M_Start(uint32_t *srcAddr, uint32_t *dstAddr, uint16_t bufsz)
{
    if (HAL_DMA_Start_IT(&hdma_memtomem_dma2_stream0, (uint32_t)srcAddr, (uint32_t)dstAddr, bufsz) != HAL_OK)
    {
        Error_Handler();
    }
}
```

现在就可以在主函数，调用“DMA_M2M_Start()”，启动 DMA 传输数据。在主函数“main.c”里，先定义两个数据，用于存放 DMA 传输测试数据。

```
/* USER CODE BEGIN PV */
/*
 * 定义全局变量
*/
// 这两个数组是 DMA 测试使用
static uint32_t srcBuffer[20] =
{
    0x1234, 0x5678, 0x9876, 0x4586, 0xABCD,
    0x5678, 0xABCD, 0x4586, 0x4586, 0xABCD,
    0xABCD, 0x5678, 0x4586, 0x9876, 0x1234,
    0x1234, 0xABCD, 0x9876, 0x5678, 0xABCD,
};

static uint32_t dstBuffer[20] = {0};
/* USER CODE END PV */
```

然后设置传输完成标志、传输错误标志初值为 0，再启动 DMA 传输。

```
  /* USER CODE BEGIN 2 */
  printf("\rDMA  Memory To Memory Test\n");
  RegisterCallbackFunction();
  transferCompleteDetected = 0;
```

```
  transferErrorDetected = 0;
  DMA_M2M_Start(srcBuffer, dstBuffer, 20); // 启动 DMA 开始搬运数据
      /* USER CODE END 2 */
```

在主循环里，根据标志位的情况，得知 DMA 传输结果，并对传输结果的数据进行对比验证。

```
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    if(transferCompleteDetected==1) // 判断前面 DMA 搬运是否完成(标志在回调函数被修改)
    {
      transferCompleteDetected = 0; // 重新将传输完成标志清零
      for(i=0;i<20;i++)
      {
        if(dstBuffer[i] != srcBuffer[i]) // 挨个对比源数据和目标数据是否有差异
        {
          printf("\rDMA Data Error\n");
        }
      }
      if(i==20) // 没有差异则表示本次传输正常完成，再次启动传输测试
      {
        DMA_M2M_Start(srcBuffer, dstBuffer, 20);
        printf("\rDMA Test ok\n");
      }
    }
    else if(transferErrorDetected==1) // 判断 DMA 搬运是否出错
    {
      transferErrorDetected = 0;
       printf("\rDMA Tshezransmission Error\n");
    }
    HAL_Delay(1000);
  }
  /* USER CODE END 3 */
}
```

## 2.9.5 实验效果

编译代码，在工程模式运行，可以看到串口打印如图 2.9.5，表示使用 DMA 将数据从内存搬运到内存，实验成功。

```
DMA  Memory To Memory Test
DMA Test ok
DMA Test ok
DMA Test ok
DMA Test ok
DMA Test ok
DMA Test ok
DMA Test ok
DMA Test ok
```

图 2.9.5 DMA 测试结果

# 2.10 通用定时器—PWM

- 设计需求

这里假设需求为利用定时器产生 PWM，实现 LED 亮度渐变。

## 2.10.1 基础知识

PWM(Pulse Width Modulation),即脉冲宽 0 度调制。是一种利用微处理器的数字输出来对模拟电路进行控制的技术，广泛应用在测量、通信、功率控制等诸多领域。

举个最常见的例子，利用 PWM 控制显示屏亮度。屏幕背光可以看作是一个大灯，这个大灯只有亮、灭两种状态。如果把灯亮看作 100%，灯灭看作 0%，要实现 50%的亮度，可以在某个单位时间里亮灯 50%时间、灭灯 50%时间，只要这个单位时间够小，由于人眼具有视觉暂留效应，就会从宏观的感觉整个灯是一直亮着，且亮度只要原来的一半。本结就以 LED 灯类比屏幕背光灯，实现对其亮度的控制。

PWM 实质就是 GPIO 不断翻转输出高、低电平，这个效果可以写代码控制 GPIO 产生，但这样就会占用 CPU，CPU 就不方便做其它事情。MCU 里都有一个定时器，配置好这个定时器，就可以让它去控制 GPIO 自动翻转，无需 CPU 再参与。

STM32MP157 有 14 个定时器 TIM(各有差异)，5 个低功耗定时器 LPTIM。

## 2.10.2 硬件设计

参考前面图 2.2.2 开发板 LED 原理图可知，两个 LED 灯，分别使用 PA10 和 PG8，由前面图 1.5.15 提到的数据手册（Datasheet）可以查到这两个脚都支持 TIM 功能。

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| U16 | T16 | Y17 | Y17 | PA10 | I/O | FT_u | - | TIM1_CH3, SPI3_NSS/I2S3_WS, USART1_RX, MDIOS_MDIO, SAI4_FS_B, DCMI_D1, LCD_B1, EVENTOUT | OTG_FS_ID, OTG_HS_ID |
| V6 | U7 | AB9 | Y8 | PG8 | I/O | FT_e | - | TRACED15, TIM2_CH1/TIM2_ETR, ETH_CLK, TIM8_ETR, SPI6_NSS, SAI4_D2, USART6_RTS/USART6_DE, USART3_RTS/USART3_DE, SPDIFRX_IN3, SAI4_FS_A, ETH1_PPS_OUT, LCD_G7, EVENTOUT | - |

图 2.10.1 LED 对应引脚复用功能

这里以 PG8 为例，设置其复用功能位 TIM2_CH1 输出 PWM。

## 2.10.3 MX 设置

参考前面设置 MCU 系统时钟为 209MHz，此时 TIM2 的时钟也为 209MHz,如所示。

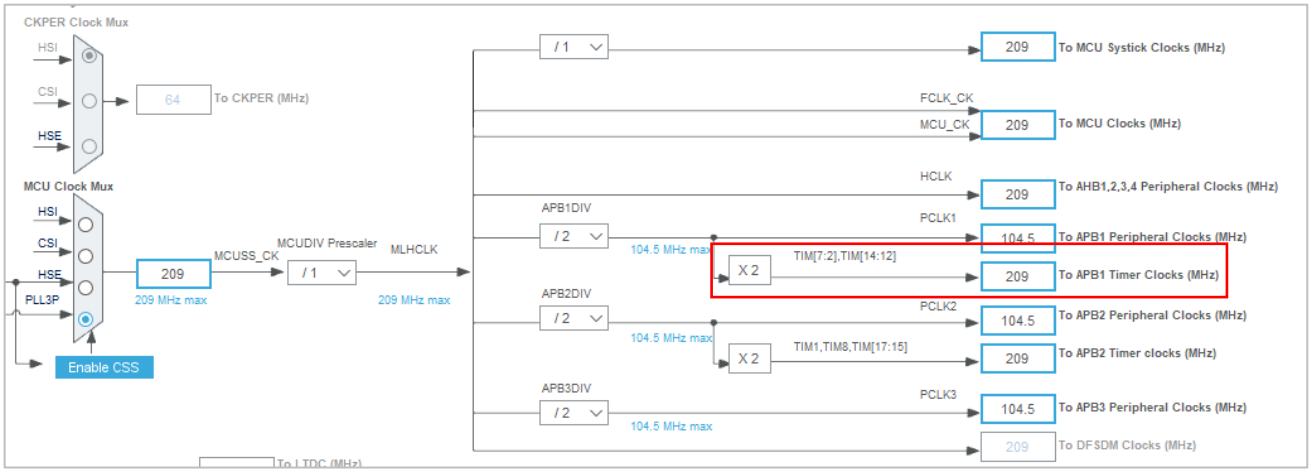


图 2.10.2 TIM2 时钟

然后如图 2.10.3 先设置 PG8 引脚复用功能为 TIM2_CH1。然后切换到“Timers”标签，选择“TIM2”，勾选给 Cortex-M4。在下面的通道 1“Channel1”选择其功能为产生 PWM“PWM Generation CH1”。再切换到下面的“Parameter Settings”标签，参考⑥、⑦处设置。

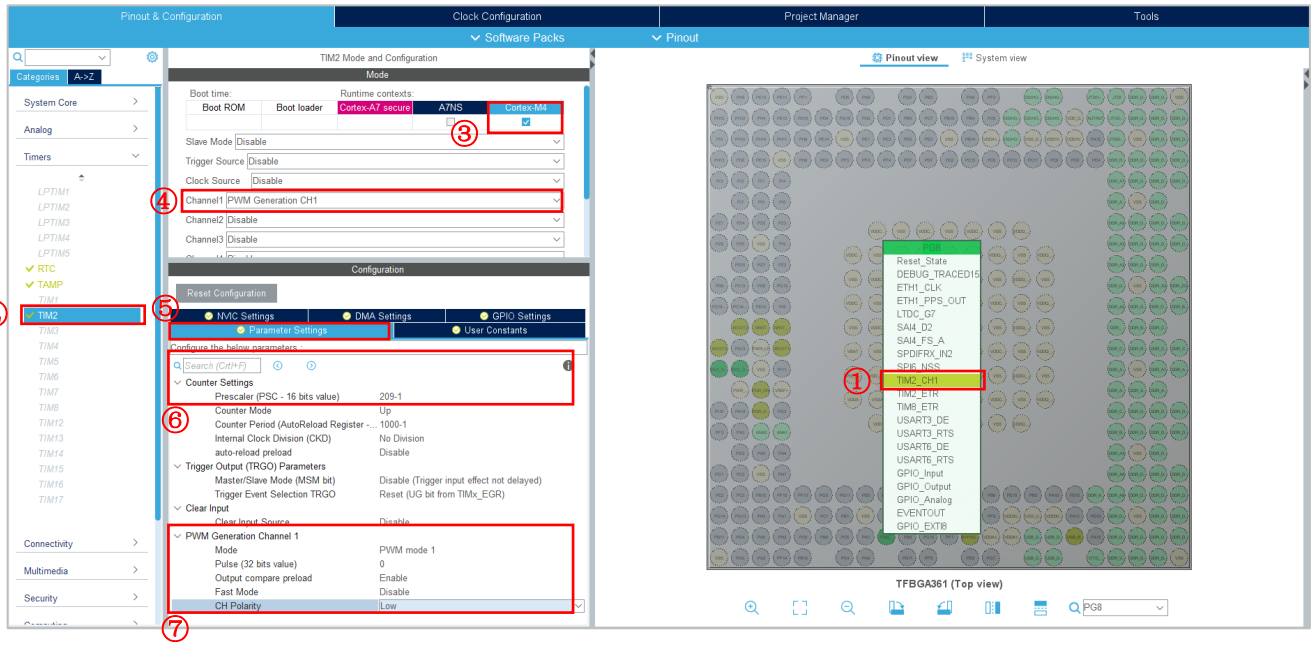


图 2.10.3 TIM2 配置

这里简单说明下⑥、⑦处设置的含义。

在⑥处主要设置分频系数“Perscaler”，这里 MCU 主频为 209MHz，这里为了方便计算，设置为“209-1”分频，这样定时器频率就工作在 209MHZ/209=1MHz。接着设置定时器计数周期“Counter Period”值为“1000-1”，这样定时器计数就会从 0 开始，计数到 1000，完成一次计数周期，然后再从 0 开始，周而往复，此时一个定时器频率为 1MHz/1000=1KHz，即 1 秒 GPIO 翻转 1000 次。

在⑦处大部分默认即可，通道极性“CH Polarity”可改为“Low”，因为 LED 是低电平亮。在代码里，修改“Pulse”的值，即可修改占空比。比如当“Pulse”值为 500 时，即 0-500 计数期间为低电平 LED 亮，500-1000 计数周期为高电平 LED 灭，LED 从宏观看就是半亮状态。

此外，还需要使能 TIM2 中断，这样完成一个计数周期才会进入中断回调函数，在中断回调函数实现修改占空比，不断改变 LED 亮度。TIM2 中断使能如图 2.10.4 所示。

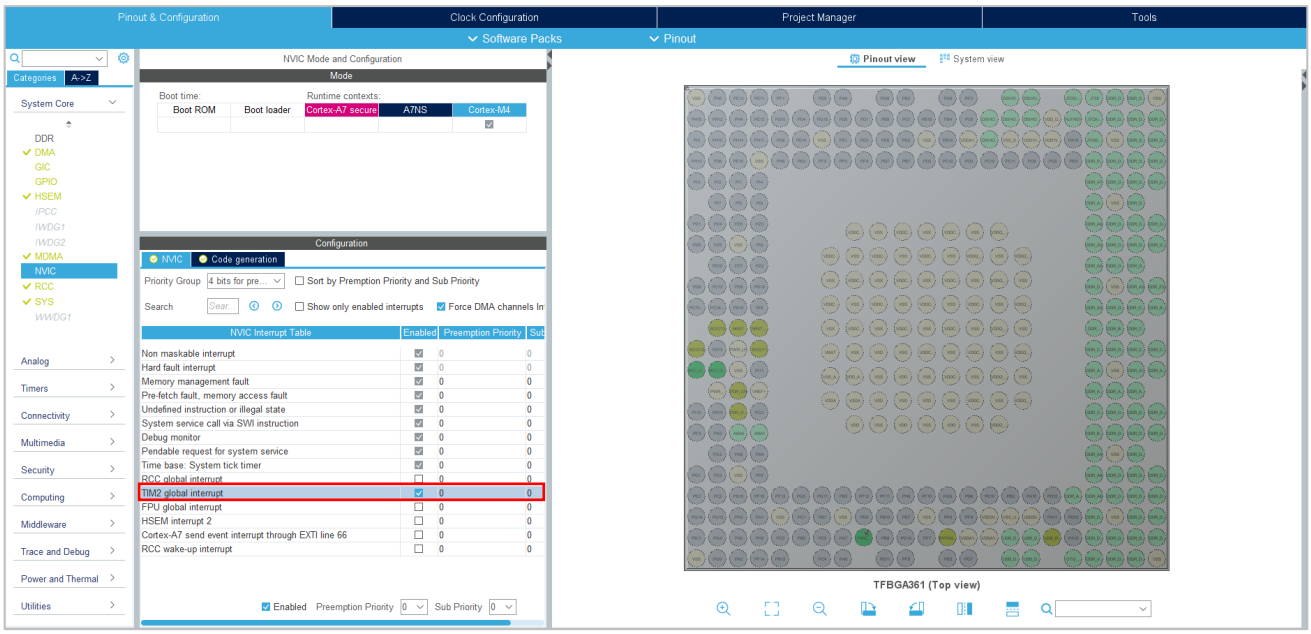


图 2.10.4 使能 TIM2 中断

## 2.10.4 代码设计

创建“DemoDriver”目录，在里面新建“driver_tim_pwm.c”。首先需要完善定时器中断回调函数，在该函数里，使用“__HAL_TIM_SET_COMPARE()”修改寄存器 CCR，实现不同的占空比。

```
/*
 *  函数名：void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
 *  输入参数：htim->定时器句柄
 *  输出参数：无
 *  函数作用：每当发生一次周期溢出时间会到此中断回调函数，在此函数内更新占空比
*/
static uint16_t pulse = 0;
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance == TIM2)
    {
        pulse = pulse + 1;
        if(pulse>=1000)
        {
            pulse = 0;
        }
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse);
    }
}
```

将定时器中断启动函数“HAL_TIM_Base_Start_IT()”和定时器 PWM 输出开始函数“HAL_TIM_PWM_Start()”封装在一起。

```
/*
 *  函数名：void TIM_LED3_PWM_Start(void)
 *  输入参数：无
 *  输出参数：无
 *  函数作用：开启定时器
*/
void TIM_LED3_PWM_Start(void)
{
    if(HAL_TIM_Base_Start_IT(&htim2) != HAL_OK)
    {
        Error_Handler();
    }
    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
   }
```

最后主函数调用“TIM_LED3_PWM_Start()”即可启动 TIM 中断和输出占空比不断增加的 PWM。

### 2.10.5 实验效果

使用工程模式调试，可以发现 LED YELLOW 逐渐变亮，再变暗，如此循环。注意，如果是实际应用，需要设备树覆盖，将 TIM2 分配给 M4。

# 2.11 通用定时器—us 延时

- 设计需求

这里假设需求为使用通用定时器，得到一个 us 级别的延时。

## 2.11.1 基础知识

HAL 库提供了一个毫秒级延时函数“HAL_Delay()”，它也是基于 SysTick。有些应用场合可能需要微秒级的延时，该函数就不能满足我们需求，因此这里利用定时器，自己创建一个 us 级延时。

STM32MP157 的 TIM 多达 10 几个，不同的定时器精度不同，这里使用 TIM5，精度位 32 位，可以从 0 计数到 $2^{32}$。

## 2.11.2 硬件设计

TIM 不涉及电路设计。

## 2.11.3 MX 设置

参考前面配置 LED 和时钟，为了方便计算，这里的把 MCU 系统时钟设置为 200MHz。将 TIM5 分配给 M4，时钟源选择“Internal Clock”，如图 2.11.1 所示。下面的参数设置，保持默认即可，预分频“Prescaler”为 0，表示 TIM5 的频率为 200MHz，意味着 1s 时间内，会计数 200000000 次。那么 1us 则计数 200000000/1000000=200 次。如果“Counter Period”值为 200-1，则计数时间为 1us，如果“Counter Period”值为 200*t-1，则计数时间为 t us。因此后面将通过代码修改“Counter Period”值即可实现不同长度的 us 延时。

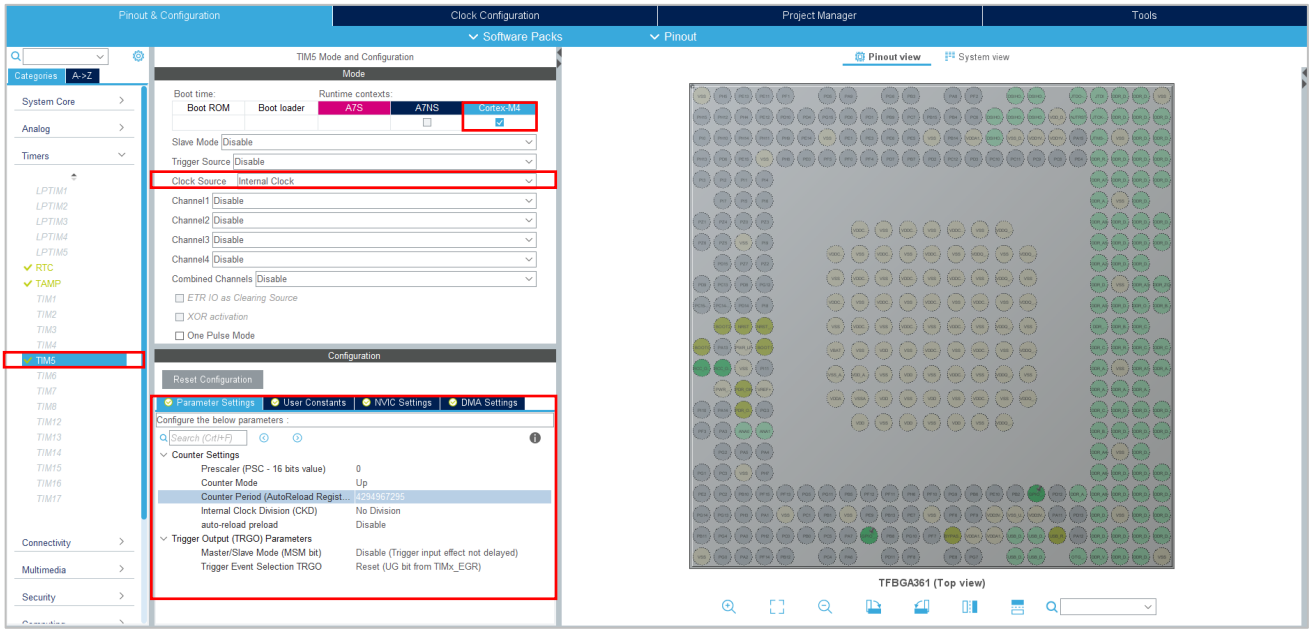


图 2.11.1 TIM5 配置

每个计数周期完成，将产生中断。在中断里修改标志位，即可得知一个计数周期完成。使能 TIM5 中断如图 2.11.2 所示。

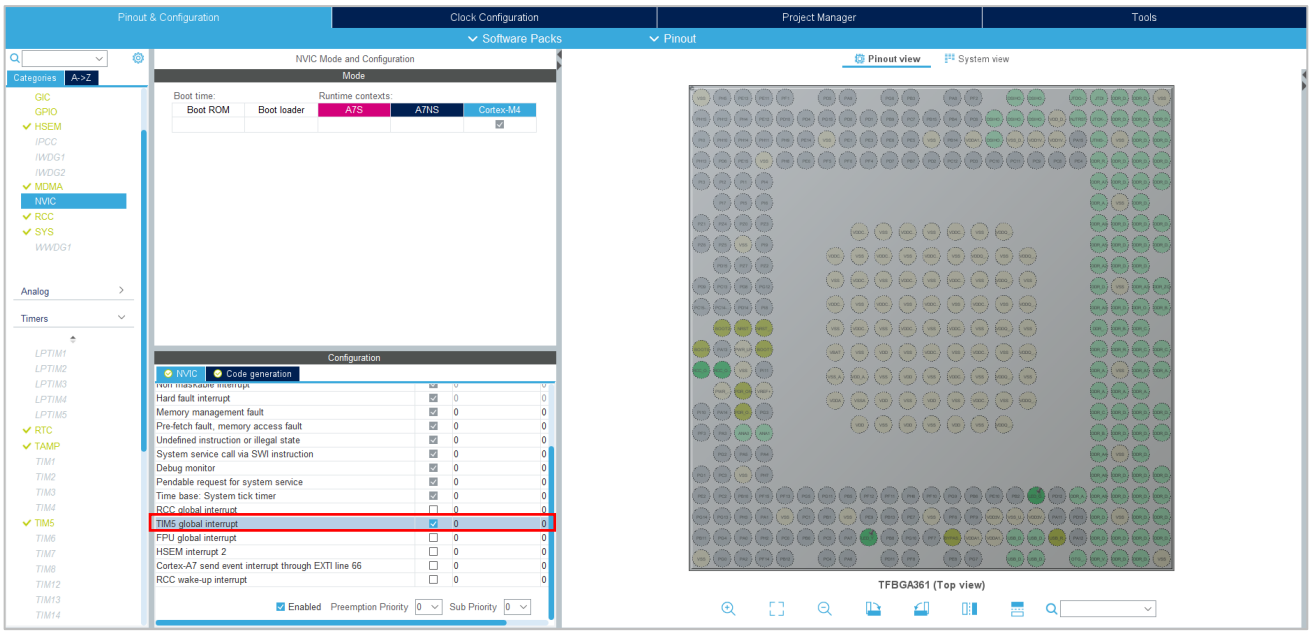


图 2.11.2 使能 TIM5 中断

## 2.11.4 代码设计

创建“DemoDriver”目录，在里面新建“driver_led.c”和“driver_tim.c”。“driver_led.c”参考前面的示例即可。

“driver_tim.c”里定义 us 延时函数，通过函数设置“autoreload”和“counter”，然后启动定时器，循环查询标志位，等待本次计数周期完成。

```
__IO uint16_t delay_end = 0; // 延时完成标志


/*
 *  函数名：void us_timer_delay(uint32_t t)
 *  输入参数：t-延时时间 us(范围:0~21,474,836us)
 *  输出参数：无
 *  返回值：无
 *  函数作用：定时器实现的延时函数，延时时间为 t us，为了缩短时间，函数体使用寄存器操作，用户可对照手册查看每个寄存器每位的含义
*/
void us_timer_delay(uint32_t t)
{
    uint32_t counter = 0;
    uint32_t autoreload = 200*t-1;

    delay_end = 0;
    HAL_TIM_Base_Stop_IT(&htim5); // 先停止 TIM5 计数

    __HAL_TIM_SET_AUTORELOAD(&htim5, autoreload); // 设置计数自动加载值,每 200 个为 1us
```

```
    __HAL_TIM_SET_COUNTER(&htim5, counter); // 设置计数初始值为 0,启动定时器后 counter 将增加到 autoreload

    HAL_TIM_Base_Start_IT(&htim5); // 启动 TIM5 计数
    while(!delay_end);
}
```

定时器将从 0（counter 初值设置为 0），一直增加到 autoreload（t 个 1us 对应的计数值），达到 autoreload 后，将回调“HAL_TIM_PeriodElapsedCallback()”，在该函数里关闭定时器和修改标志位。

首先需要完善定时器中断回调函数，在该函数里，关闭定时器和修改标志位。

```
/*
 *  函数名：void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
 *  输入参数：htim->定时器句柄
 *  输出参数：无
 *  函数作用：每当发生一次周期溢出时间会到此中断回调函数，在此函数内停止计数并修改标志位
*/
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance == TIM5)
    {
        HAL_TIM_Base_Stop_IT(&htim5); // 停止 TIM5 计数
        delay_end = 1;
    }
}
```

主函数调用 us_timer_delay()，即可实现 us 延时。这里为了方便观察，传入 1000000，即 1s 时间。

```
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
      LED_GREEN_ON();
      LED_YELLOW_OFF();
      us_timer_delay(1000000);

      LED_GREEN_OFF();
      LED_YELLOW_ON();
      us_timer_delay(1000000);
  }
  /* USER CODE END 3 */
   }
```

### 2.11.5 实验效果

调试运行后，可以看到 LED 灯基本按 1S 间隔进行闪烁。测试效果不严谨，有条件的读者可以修改测试代码，使用示波器或逻辑分析仪观察引脚翻转，验证实验效果。

# 2.12 SysTick 定时器-us 延时

- 设计需求

这里假设需求为使用 SysTick 定时器，得到一个 us 级别的延时。

## 2.12.1 基础知识

SysTick 定时器(又名系统滴答定时器)是存在于内核的一个定时器，只要是 ARM Cotex-M 系列内核的 MCU 都包含这个定时器。使用内核的 SysTick 定时器来实现延时，可以不占用系统定时器，节约资源。由于和 MCU 外设无关，因此可以在不同厂家的 Cortex-M 内核之间移植。

前面使用通用定时器，实现了一个 us 级别延时，这里再用 SysTick 实现一次。注意 SysTick 用于了 HAL 库的毫秒级延时函数“HAL_Delay()”，不建议日常使用 SysTick 去作为其它用途，这里只作为演示。

## 2.12.2 硬件设计

SysTick 定时器不涉及硬件，这里只需要一个 LED 灯去展示延时效果。

## 2.12.3 MX 设置

参考前面创建一个 STM32CubeIDE 工程，在 STM32CubeMX 里添加 LED 灯，设置 M4 时钟为 209MHz，其它无需操作。

## 2.12.4 代码设计

SysTick 定时器是一个 24 位递减定时器，即计数器可以从最大值 $2^{24}$ 开始每个时钟周期减 1，当减到 0 时，会产生一个中断，同时再自动重载定时初值，开始新一轮计数。而这个定时初值，则是我们可以设置的，就可以实现得到指定时间。如下图 2.12.1 所示，y 为定时器初值，然后随着时间增加，值逐渐减小，直至为 0，再重新加载初值，如此往复，x1、x2、x3 这些时间段，就是我们需要的。

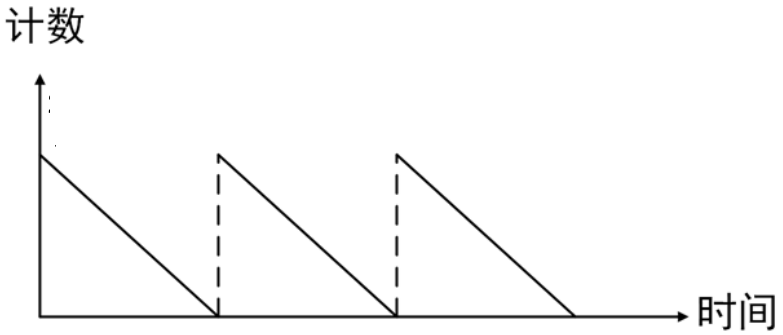


图 2.12.1 Systick 定时器工作示意图

M4 工作在 209MHz，即 209000000Hz，意味着 1s 时间内，会计数 209000000 次。那么 1us 则计数 209000000/1000000=209 次。这个 209 就是滴答定时器的初始值，将这个值写入滴答定时器，定时器在每个时钟周期减 1，减到 0 时，就刚好是 1us，同时产生中断通知，再次加载 209 如此反复。HAL 库提供“HAL_SYSTICK_Config()”函数去设置这个初始值。

在“DemoDriver”目录新建“driver_systick.c”和“driver_systick.h”。首先需要初始化 Systick，让其初始值为 209，再设置 SysTick 定时器的中断优先级，最后使能 SysTick 定时器中断。

```
/*
 *  函数名：void SysTickInit(void)
 *  输入参数：无
 *  输出参数：无
 *  返回值：无
 *  函数作用：初始化系统滴答时钟的频率和中断优先级
*/
void SysTickInit(void)
{
    uint32_t init_t = 0;

    init_t = SystemCoreClock/1000000;

/* 时间(单位:s)=1/频率(单位:HZ)
     * SystemCoreClock 频率：209MHz = 209,000,000
     * 即 MCU 1 秒会计数 209,000,000 次
     *       1us 则计数 209MHz/1000000 = 209 次
     * 209 就是滴答时钟的初始值,它向下计数 209 次,计数将变为 0,就会产生一次中断
     *
     * SystemCoreClock/1000:    1ms 中断一次
     * SystemCoreClock/100000:  10us 中断一次
     * SystemCoreClock/1000000: 1us 中断一次
     */
    if(HAL_SYSTICK_Config(init_t) != HAL_OK)
    {
        Error_Handler();
    }

    // 设置滴答定时器中断优先级：最高
    HAL_NVIC_SetPriority(SysTick_IRQn, 0, 0);
    // 使能滴答定时器中断
    HAL_NVIC_EnableIRQ(SysTick_IRQn);
}
```

配置完成后，每个 1us 就会产生一次中断，也就会调用“stm32mp1xx_it.c”里的“SysTick_Handler()”。为了不影响 HAL 库的“HAL_Delay()”，这里添加一个自己的中断处理函数“OwnSysTick_Handler()”。

```
      /**
  * @brief This function handles System tick timer.
  */
void SysTick_Handler(void)
{
  /* USER CODE BEGIN SysTick_IRQn 0 */
```

```
  /* USER CODE END SysTick_IRQn 0 */
  HAL_IncTick();
  /* USER CODE BEGIN SysTick_IRQn 1 */
  OwnSysTick_Handler();
  /* USER CODE END SysTick_IRQn 1 */
}
/*
 * 全局变量定义
*/

static volatile uint32_t systick_t = 0


……


/*
 *  函数名：void SysTickDelay(uint16_t u)
 *  输入参数：u-延时时间
 *  输出参数：无
 *  返回值：无
 *  函数作用：滴答定时器实现的延时函数
*/
void SysTickDelay(uint32_t u)
{
    systick_t = u;
    while(systick_t != 0);
}

/*
 *  函数名：void OwnSysTick_Handler(void)
 *  输入参数：无
 *  输出参数：无
 *  返回值：无
 *  函数作用：滴答定时器的中断服务函数
*/
void OwnSysTick_Handler(void)
{
    if(systick_t != 0)
    {
        systick_t--;
    }
}
```

回到“driver_systick.c”，我们定义个一个全局变量“systick_t”，用于记录需要多少个 1us，然后中断函数里每次将“systick_t”减 1，这样就循环了“systick_t”个 1us。

“main.c”即可调用“SysTickDelay()”实现 us 级别延时。

```
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    /* USER CODE BEGIN 2 */
    SysTickInit();
    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
      /* USER CODE END WHILE */

      /* USER CODE BEGIN 3 */
    LED_GREEN_ON();
    SysTickDelay(1000000);
    LED_GREEN_OFF();
    SysTickDelay(1000000);
    }
    /* USER CODE END 3 */
  }
```

## 2.12.5 实验效果

调试运行后，可以看到 LED 灯基本按 1S 间隔进行闪烁。测试效果不严谨，有条件的读者可以修改测试代码，使用示波器或逻辑分析仪观察引脚翻转，验证实验效果。

# 2.13 ADC—使用 DMA

- 设计需求

这里假设需求为使用 ADC，获取光敏电阻两端电压值，从而得知外部光照变化。

## 2.13.1 基础知识

ADC(Analog-to-Digital Converter)模数转换器，自然界的信号几乎都是模拟信号，比如光亮、温度、压力、声音；而为了方便存储、处理，计算机里面都是数字的 0/1 信号，将模拟信号转换成数字信号的装置就叫模数转换器(ADC)。

STM32MP157 有两个 ADC，都可以给分配给 M4，每个 ADC 有 19 个通道。

## 2.13.2 硬件设计

100ASK_STM32MP157 将 ADC 引脚引了出去，在 J9 的 External 接口上，如图 2.13.1 所示，ADC 分别是 10 脚和 12 脚。

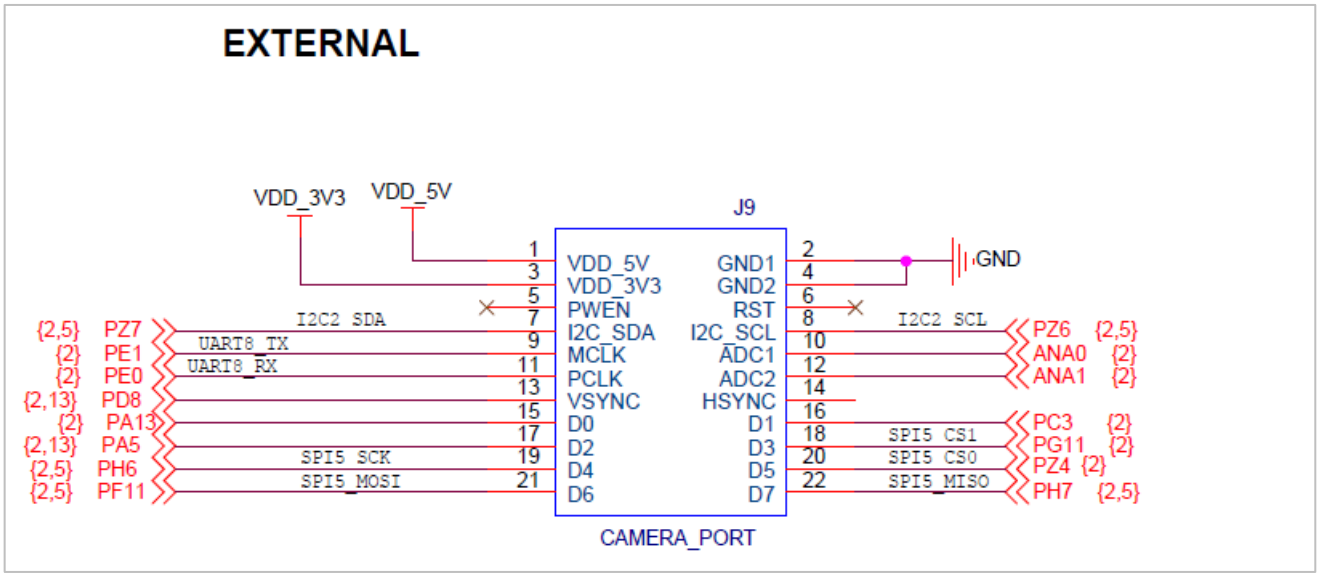


图 2.13.1 开发板扩展板接口

这个座子并不方便实验，因此做了一个转接板，将扩展引脚分类整理引了出来，如图 2.13.2 所示为 100ASK_STM32MP157 配套的扩展板。

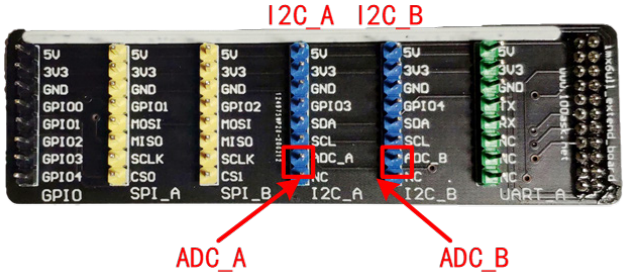


图 2.13.2 100ASK_STM32MP157 模块扩展板

为了方便观察 ADC 效果，这里制作一个模块，上面有一个光敏电阻，光敏电阻阻值受光照影响变化，反应到光敏电阻两端就是电压变化，就可以通过 ADC 测出变化。光敏电阻与 EEPROM 制作在了一起，直接将 EEPROM 模块接在 I2C 接口即可，各部分介绍如图 2.13.3 所示。为了防止用户接错方向，模块和扩展板都有一条长白线，连接时长白线在同一侧。

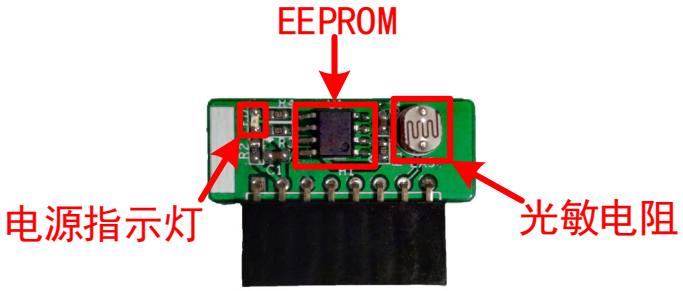


图 2.13.3 EEPROM 模块(含光敏电阻)各元件位置

如图 2.13.4 是光敏电阻模块的电路图，电路图中最上面是 3.3V 电源，往下是电阻 R1 和光敏电阻 LAS1，两个电阻之间，与 100ASK_STM32MP15 的 ADC 引脚相连，最下面是接地，一旁的电容 C2 可以暂时忽略(它起稳压作用，防止电压突然变化)。

当光敏电阻 LAS1 受到光照时，阻值会变小，此时它两端的电压会变小；当光照比较弱时，阻值会变大，导致它两端的电压也变大。假设上图中的 ADC_IN 接到某个 GPIO 引脚，通过读 GPIO 引脚只能得到 0 或 1 两个值之一，只能判断光照在某个阀值上下。如果使用 ADC 去读取光敏电阻两端电压，不仅可以知道是否有光照，还能知道光照的强度。

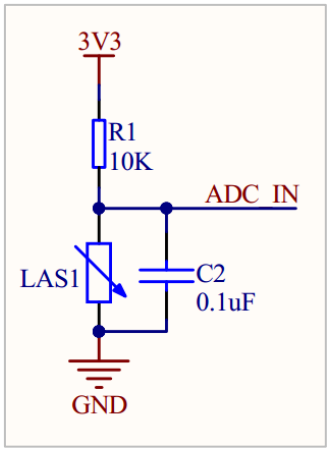


图 2.13.4 光敏电阻模块原理图

此外，ADC 还涉及参考电压，查看核心板电路图可知，参考电压为 3.3V，如所示。

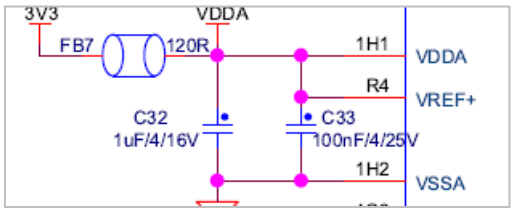


图 2.13.5 ADC 参考电压

## 2.13.3 MX 设置

先参考前面设置好 MCU 系统时钟 209MHz，设置好调试串口方便打印结果。这里假设将模块接到 ANA0，如图 2.13.6 所示配置 ADC。①搜索 ANA0，复用功能设置为 ADC1_INP0；②选择 ADC1；③勾选给 M4；④勾选通道 0“IN0 Single-ended”；⑤默认 ADC 分辨率为 16 位，使能连续 ADC 转换，转换数据管理模式这里暂时还不能选择 DMA，需要将后面的 DMA 设置后，再回来设置为“DMA One Shot Mode”。

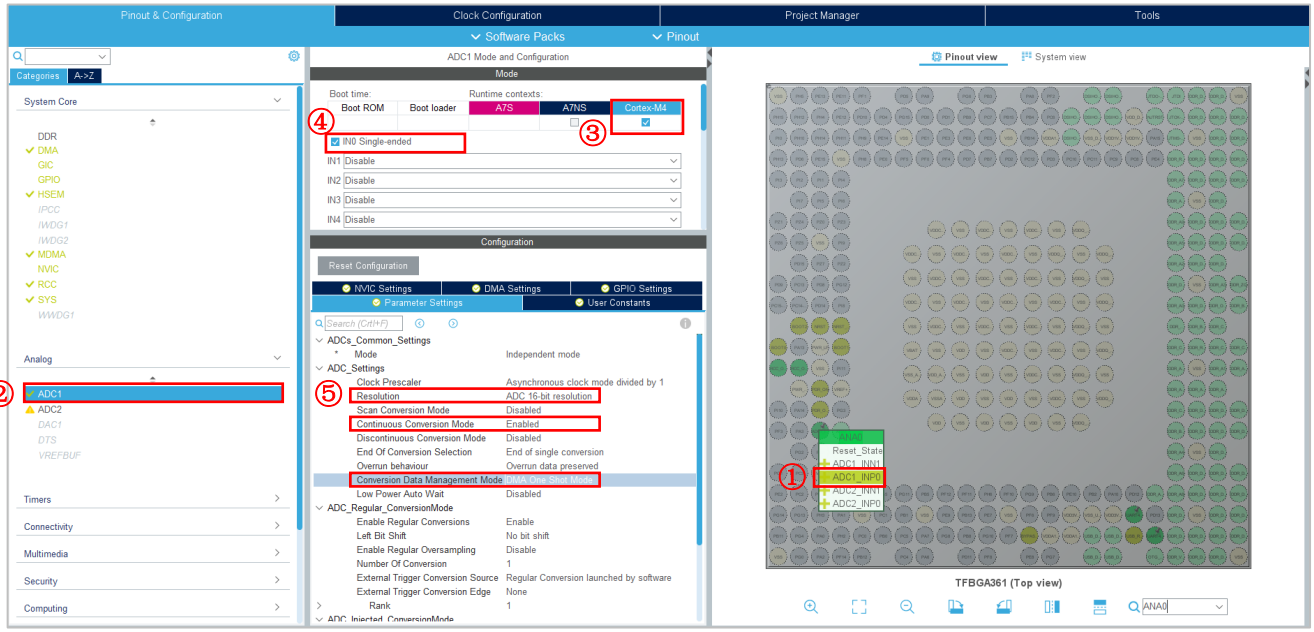

图 2.13.6 ADC 配置

切到 DMA 标签，在 DMA2 里“Add”选择“ADC1”，其它默认的配置就行，如图 2.13.7 所示。使用 DMA 就无需 CPU 参与数据的搬运，外设 ADC 产生数据，就通过 DMA 自动搬运到内存。在连续采集多次的情境下，将大大降低 CPU 的负担。

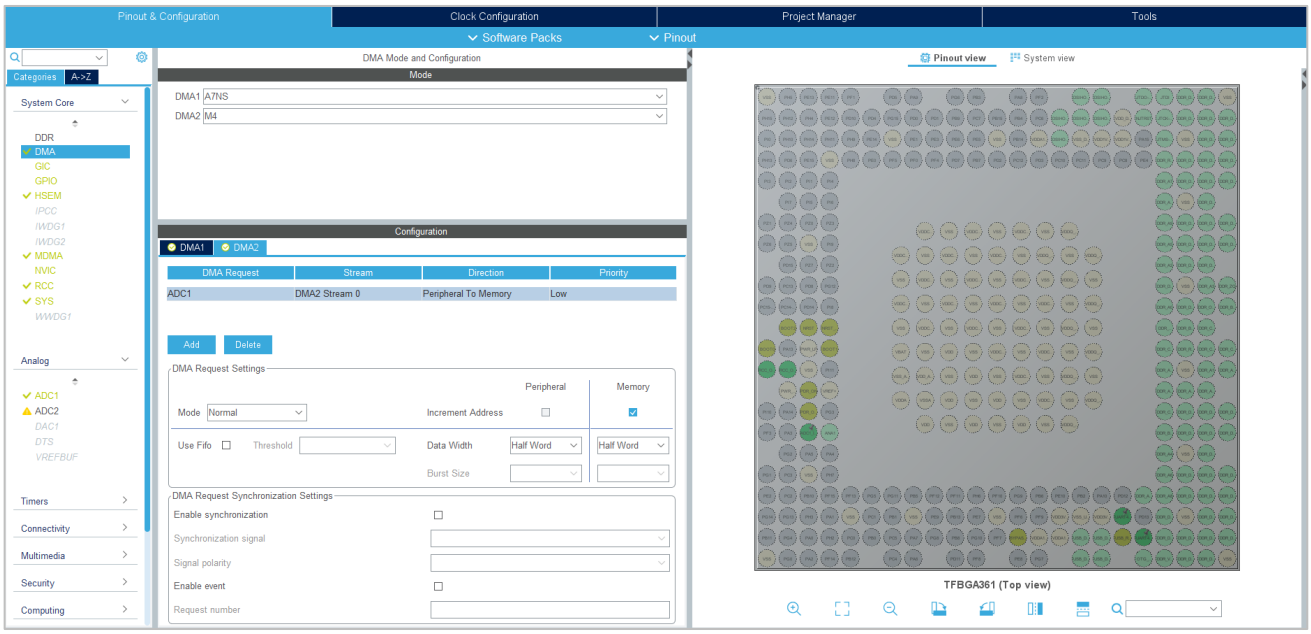


图 2.13.7 设置 ADC 的 DMA

最后勾选上 ADC 中断使能，这样就能通过中断得知转换完成，如图 2.13.8 所示。

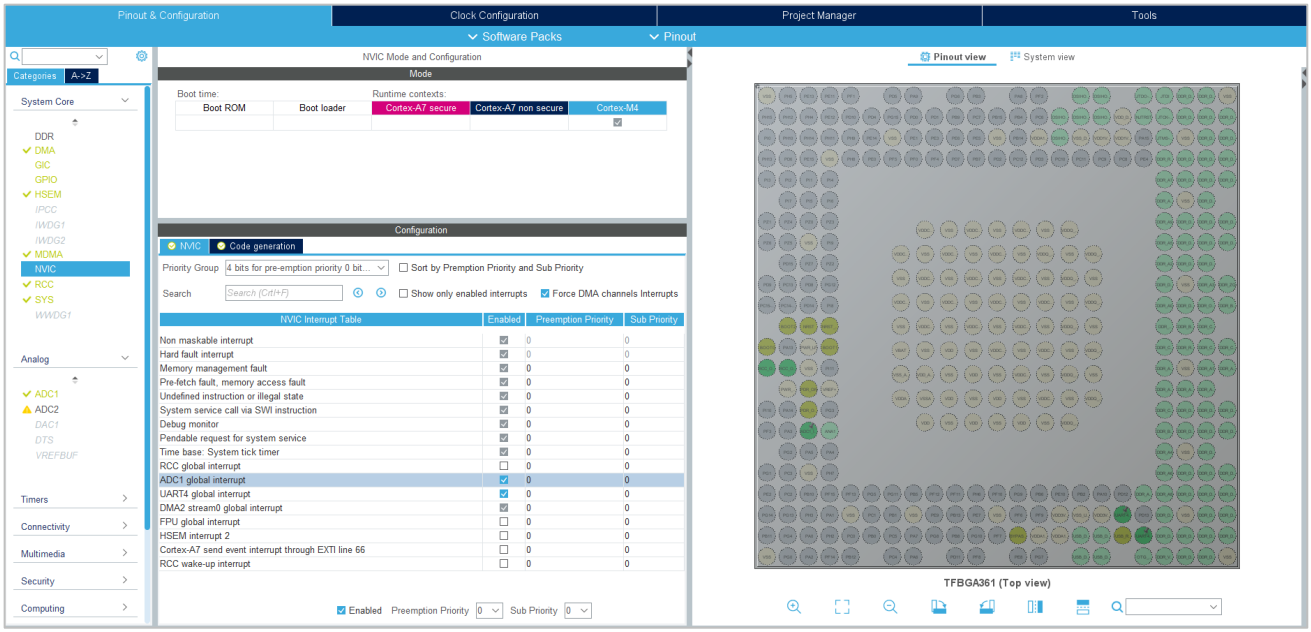


图 2.13.8 使能 ADC 中断

## 2.13.4 代码设计

创建“DemoDriver”目录，新建“driver_adc.c”，只需调用 HAL 库的“HAL_ADC_Start_DMA()”即可启动 ADC DMA 采样，需要传入数据保存位置和采样次数。然后等待中断修改标志位，得知转换完成，再将得到的数据转换成电压值。这里参考电压为 3.3V，ADC 分辨率为 $2^{16}$，因此对应的电压值为：ADC 采集值*3.3/65536。

```
/*
void GetAdcValue(uint16_t *pdatain, float *pdataout, uint16_t sz)
 *  输入参数：sz-采样点数
 *  输出参数：pdatain-采样的数据存储的首地址
 *       pdataout-电压转换至存储的首地址
 *  返回值：无
 *  函数作用：获取 ADC 的采样到的模拟电压
 *
 *  备注：此处参考电压为 3.3V
*/
void GetAdcValue(uint16_t *pdatain, float *pdataout, uint16_t sz)
{
  uint16_t i = 0;
  conv_flag = 0;
  HAL_ADC_Start_DMA(&hadc1, (uint32_t*)pdatain, sz);
  while(conv_flag != 1);

  for(i=0; i<sz; i++)
  {
    pdataout[i] = pdatain[i] * 3.3 / 65536;
  }
}
```

接着完善 ADC 完成回调函数，在里面更新完成标志。

```
/*
 *  函数名：void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
 *  输入参数：hadc->adc 句柄
 *  输出参数：无
 *  函数作用：每当 ADC 转换完成会到此中断回调函数，在此函数内更新完成标志
*/
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if(hadc->Instance==ADC1)
    {
        conv_flag = 1;
    }
}
```

在主函数里，调用前面封装的“GetAdcValue()”，一次性采集 256 次，然后将采集结果取平均值，提高准确率。

```
  /* USER CODE BEGIN 2 */
  printf("\r ADC DMA Test \n");
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    GetAdcValue((uint16_t*)AdcDigitalData, (float*)AdcAnolgaValue, 256); // 启动 ADC,采集 256 次,保存到 AdcAnolgaValue

    sum_value = 0;
    for(i=0; i<256; i++)
    {
      sum_value = sum_value + AdcAnolgaValue[i]; // 将 AdcAnolgaValue 所有值加起来
    }
    sum_value = (float)sum_value/256; // 得到电压值平均值

    printf("\r Res voltage:%.2f \n", sum_value); // 打印光敏电阻两端的电压值

    HAL_Delay(1000);
  }
  /* USER CODE END 3 */
}
```

## 2.13.5 实验效果

将扩展板接到开发板上，将模块接到扩展板 I2C_A（ANA0）上，如图 2.13.9 所示，注意模块白线、扩展板白线、开发板底板白线，三线朝向一致。。

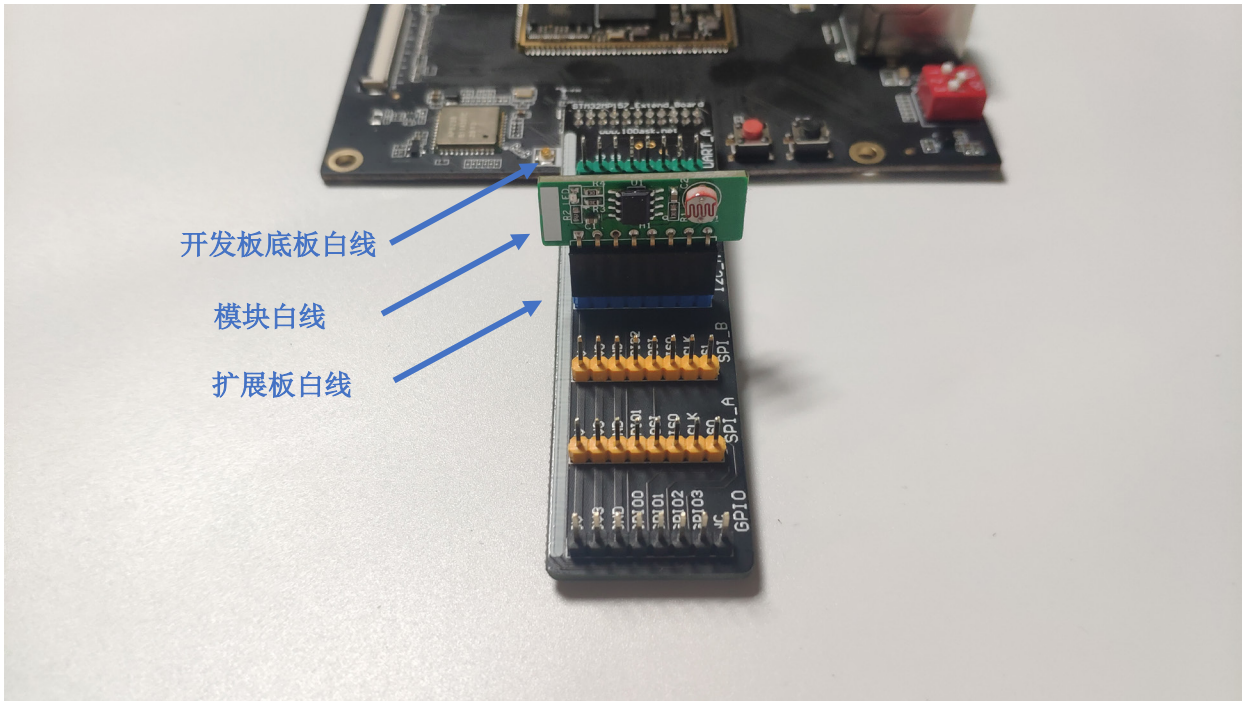


图 2.13.9 ADC 实验实物连接图

调试运行，可以看到如图 2.13.10 所示打印，改变光敏电阻光照环境，可以看到打印结果数值也发生变化。

```
ADC DMA Test
Res voltage:1.37
Res voltage:1.37
Res voltage:1.37
Res voltage:1.37
Res voltage:1.37
Res voltage:1.37
Res voltage:1.37
Res voltage:1.37
Res voltage:1.37
```

图 2.13.10 ADC 实验串口打印效果

# 2.14 ADC—内部温度

- 设计需求

这里假设需求为使用 ADC，获取芯片内部温度。

## 2.14.1 基础知识

ADC2 的通道 19 可以设置为获取芯片内部温度。

## 2.14.2 硬件设计

不涉及电路设计。

## 2.14.3 MX 设置

ADC 有两个时钟方案：一个是来自 adc_hclk（AHB 时钟），一个是来自 adc_ker_ck（ADC Kernel 时钟）。两个时钟源独立且异步。

先设置 MCU 系统时钟工作在 209MHz，ADC 默认由“PLL4R”提供时钟，然后设置 UART4 方便打印。最后切到 ADC2 标签，勾选给 M4，再勾选“Temperature Sensor Channel”，如图 2.14.1 所示。需要注意的是“Clock Prescaler”选择“Synchronous clock mode divided by 2”，“Rank”下的采样周期“Sampling Time”选择大一点，比如“810.5 Cycles”。

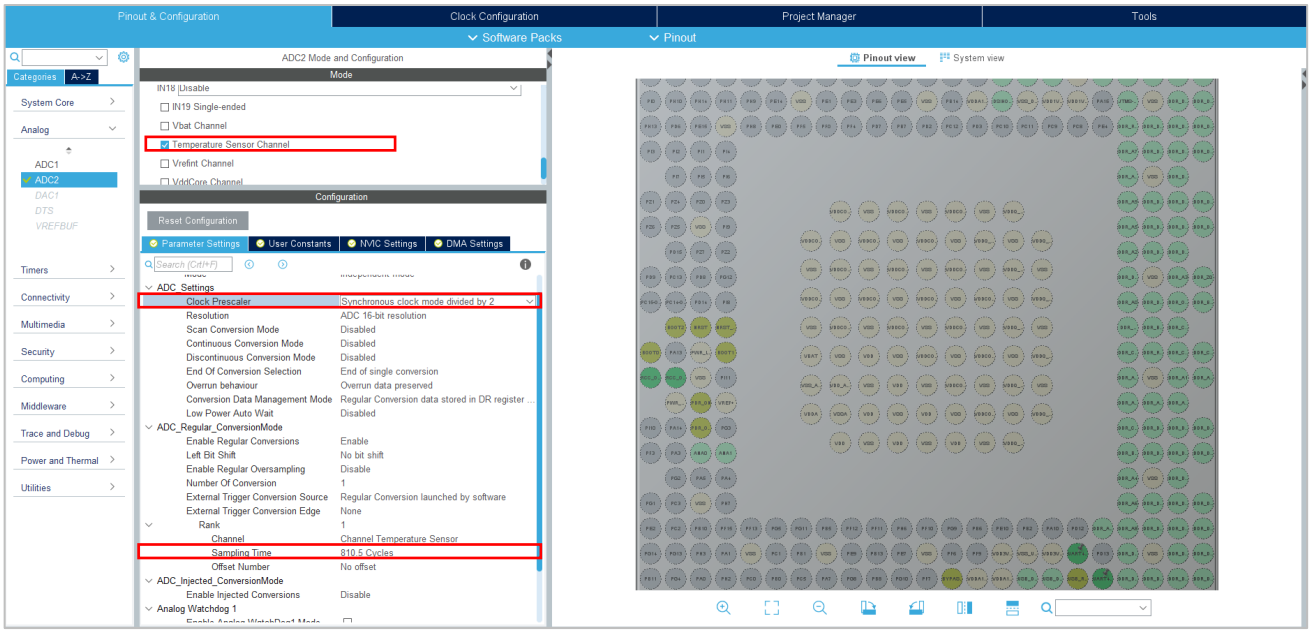


图 2.14.1 ADC 测量内部温度配置

## 2.14.4 代码设计

创建“DemoDriver”目录，在里面新建“driver_adc.c”，在该文件里调用“HAL_ADC_Start()”启动 ADC，调用“HAL_ADC_PollForConversion()”等待 ADC 转换结束，调用“HAL_ADC_GetValue()”获取 ADC 的结果，最后再将 ADC 值转换为温度值，转换公式在手册写有如下：

$$Temperature\ (in\ °C) = \frac{110\ °C - 30\ °C}{TS_CAL2 - TS_CAL1} \times (TS_DATA - TS_CAL1) + 30\ °C$$

其中：

- TS_CAL2 是在 110℃下获取的温度传感器校准值；
- TS_CAL1 是在 30℃下获取的温度传感器校准值；
- TS_DATA 是由 ADC 转换的实际温度传感器输出值；

HAL 库提供了一个带参数的宏“__HAL_ADC_CALC_TEMPERATURE()”实现对温度的转换。

```
/*
 *  函数名：int GetAdcAnlogValue(void)
 *  输入参数：无
 *  输出参数：无
 *  返回值：返回采样到的模拟电压
 *  函数作用：获取 ADC 的采样到的内部温度
 *
 *  备注：此处参考电压为 3.3V
*/
int GetTemptureValue(void)
{
  uint32_t nData = 0;
  uint16_t nValue = 0;

  HAL_ADC_Start(&hadc2);
  HAL_ADC_PollForConversion(&hadc2, 100);  // 等待转换完成，超时时间为 100ms
  nData = HAL_ADC_GetValue(&hadc2);

  // 157 的参考电压为 3.3V;采样位数是 16bit;
  nValue = __HAL_ADC_CALC_TEMPERATURE(3300UL, nData, ADC_RESOLUTION_16B);

  return nValue;
    }
```

最后主函数调用“GetTemptureValue()”打印温度值。

```
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    printf("\r Internal temprature: %d\n", GetTemptureValue());
    HAL_Delay(500);
  }
      /* USER CODE END 3 */
```

### 2.14.5 实验效果

调试运行，打印效果如所示。

```
Internal temprature: 75
Internal temprature: 75
Internal temprature: 75
Internal temprature: 75
Internal temprature: 75
Internal temprature: 75
Internal temprature: 75
Internal temprature: 75
Internal temprature: 75
Internal temprature: 75
Internal temprature: 75
```

图 2.14.2 ADC 获取内部温度实验效果

# 2.15 DAC—软件触发

- 设计需求

这里假设需求为使用 DAC 控制引脚输出不同电压。

## 2.15.1 基础知识

与前面 ADC 模数转换器相反，DAC(Digital to analog converter) 将数字信号转换为模拟信号，使得 MCU 不再只输出 0 或 1，适应更多领域。

STM32MP157 有一个 DAC，该 ADC 有两个通道，可以分配给 A7 或 M4。

## 2.15.2 硬件设计

由图 1.5.15 的数据手册（Datasheet）可知，DAC 两个输出通道分别接到 PA4 和 PA5，如图 2.15.1 所示。

| Pin Number | | | | Pin name (function after reset) | Pin type | I/O structure | Notes | Pin functions | |
|---|---|---|---|---|---|---|---|---|---|
| TFBGA257 | LFBGA354 | TFBGA361 | LFBGA448 | | | | | Alternate functions | Additional functions |
| 1H1 | P4 | V3 | U5 | PA5 | I/O | TT_ha | - | TIM2_CH1/TIM2_ETR, TIM8_CH1N, SAI4_CK1, SPI1_SCK/I2S1_CK, SPI6_SCK, SAI4_MCLK_A, LCD_R4, EVENTOUT | ADC1_INP19, ADC1_INN18, ADC2_INP19, ADC2_INN18, DAC_OUT2 |
| 1J1 | R4 | V4 | V6 | PA4 | I/O | TT_a | - | HDP0, TIM5_ETR, SAI4_D2, SPI1_NSS/I2S1_WS, SPI3_NSS/I2S3_WS, USART2_CK, SPI6_NSS, SAI4_FS_A, DCMI_HSYNC, LCD_VSYNC, EVENTOUT | ADC1_INP18, ADC2_INP18, DAC_OUT1 |

图 2.15.1 数据手册 DAC 引脚

其中 PA5 接在模块扩展接口，通过模块扩展板引出到 GPIO 位置，如图 2.15.2 所示，因此这里使用 PA5 进行测试。

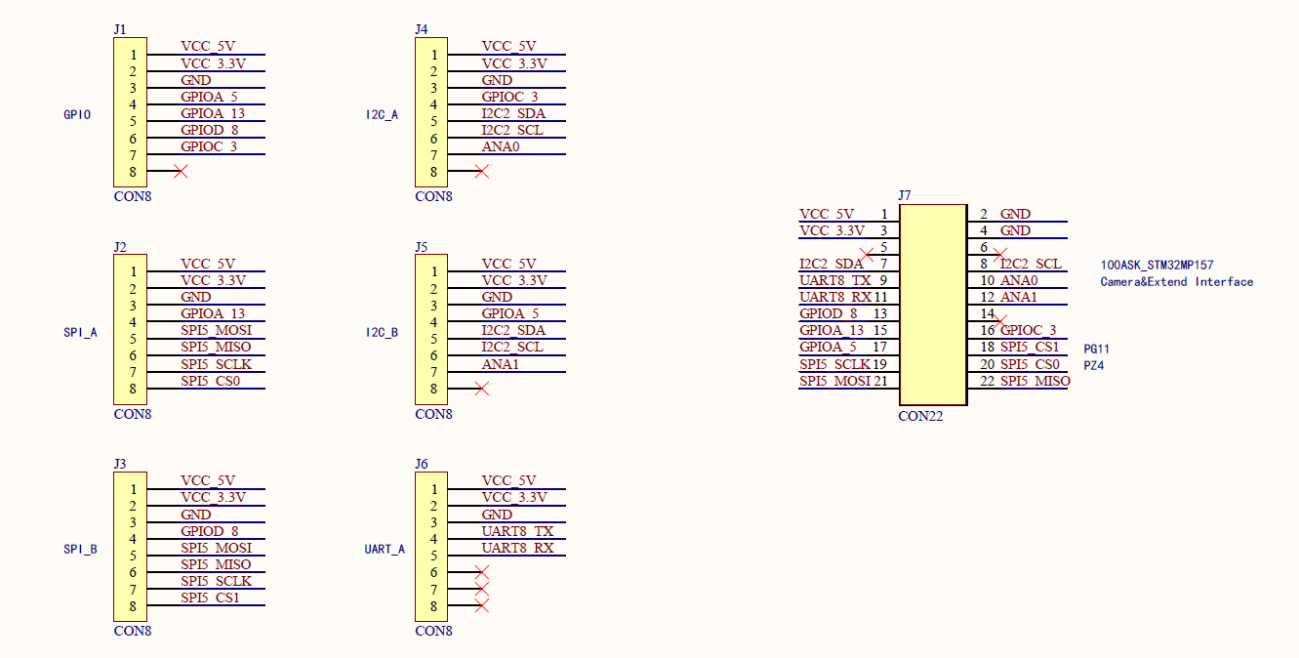


图 2.15.2 模块扩展板原理图

## 2.15.3 MX 设置

先设置好系统时钟、串口，然后搜索 PA5，设置为“DAC_OUT2”，切换到 DAC1 标签，勾选给“M4”。在下方的“OUT2 mode”选择“Connected to external pin only”以输出到引脚上，最后在下方触发方式“Trigger”选择软件触发“Software trigger”，如图 2.15.3 所示。

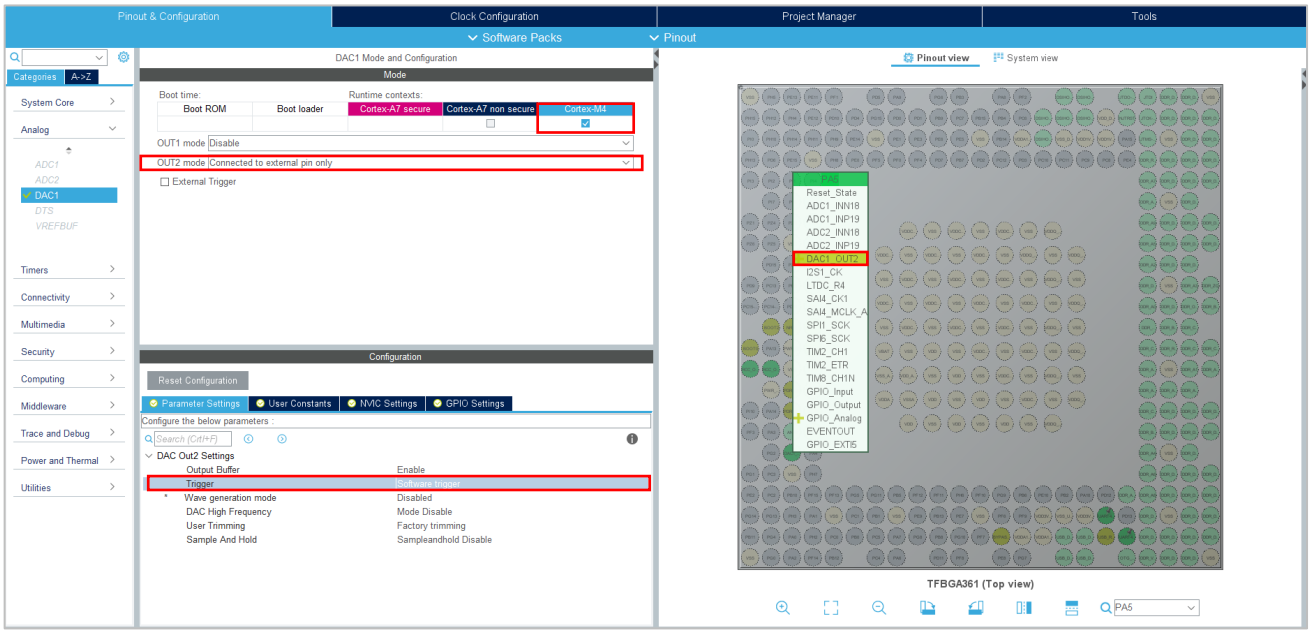


图 2.15.3 DAC 配置

## 2.15.4 代码设计

创建“DemoDriver”目录，在里面新建“driver_dac.c”，使用 HAL 库的“HAL_DAC_Start()”可以启动 DAC，使用“HAL_DAC_SetValue()”设置 DAC 输出值，前面我们选择的默认是右对齐 12 位。将这两个函数简单封装下，实现“DAC_SetVoltage()”传入需要的电压值，即可让 DAC 输出对应的电压值。

```
#include "driver_dac.h"
#include "dac.h"

/*
 *  函数名：void DAC_Start(void)
 *  输入参数：无
 *  输出参数：无
 *  返回值：无
 *  函数作用：启动 DAC_OUT2
 *
*/
void DAC_Start(void)
{
  HAL_DAC_Start(&hdac1, DAC_CHANNEL_2);
}

/*
 *  函数名：void DAC_Stop(void)
 *  输入参数：无
 *  输出参数：无
 *  返回值：无
 *  函数作用：停止 DAC_OUT2
 *
*/
void DAC_Stop(void)
{
  HAL_DAC_Stop(&hdac1, DAC_CHANNEL_2);
}

/*
 *  函数名：int8_t DAC_SetValue(uint16_t data)
 *  输入参数：无
 *  输出参数：无
 *  返回值：0:正确 -1:超过设置范围 -2:设置错误
 *  函数作用：设置 DAC 输出值
 *
 */
int8_t DAC_SetValue(uint16_t data)
{
  if(data>=4096)
```

```
  {
    return -1;
  }
  if(HAL_DAC_SetValue(&hdac1, DAC_CHANNEL_2, DAC_ALIGN_12B_R, data) != HAL_OK)
  {
    return -2;
  }
  DAC_Start(); // 软件触发,每次修改值都需要启动 DAC

  return 0;
}

/*
 *  函数名：void DAC_SetVoltage(float vol)
 *  输入参数：需要设置的输出电压值
 *  输出参数：无
 *  返回值：无
 *  函数作用：设置 DAC 输出值
 *
 *注意:参考电压为 VREF_VALUE 3.3V
 */
void DAC_SetVoltage(float vol)
{
  if (vol > VREF_VALUE)
  {
    printf("\r Set voltage out of range \n");
    return;
  }

  DAC_SetValue(vol*4095/VREF_VALUE);
}
```

主函数里调用“DAC_SetVoltage()”，循环修改传入的电压值，即可让 DAC 输出不断变化的电压。

```
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    printf("\rThe output voltage %.2f\n", out_vol);
    DAC_SetVoltage(out_vol); // 设置输出电压
```

```
    out_vol = out_vol + 0.1; //依次增加输出电压

    if(out_vol>3.3) //超过输出最大值重置为 0
      out_vol = 0;

    HAL_Delay(1000);
  }
    /* USER CODE END 3 */
```

## 2.15.5 实验效果

开发板连接串口，启动调试运行，可以看到如图 2.15.4 所示打印。此时用万用表测量 PA5 脚的电压，可以看到电压值和串口打印值一致。

```
DAC SoftTriger Test
The output voltage 0.00
The output voltage 0.10
The output voltage 0.20
The output voltage 0.30
The output voltage 0.40
The output voltage 0.50
The output voltage 0.60
The output voltage 0.70
The output voltage 0.80
The output voltage 0.90
The output voltage 1.00
```

图 2.15.4 DAC 输出实验串口打印

# 2.16 I2C—软件模拟

- 设计需求

这里假设需求为使用 GPIO 模拟 I²C 获取板子传感器 AP3216C 的数据。

## 2.16.1 基础知识

I²C（Inter-Integrated Circuit），常读作“I 方 C”，它是一种多主从架构串行通信总线。在 1980 年由飞利浦公司设计，为了让主板、嵌入式系统或手机连接低速周边设备。如今在嵌入式领域是非常常见通信协议，常用于 MPU/MCU 与外部设备连接通信、数据传输。

I²C 由两条线组成，一条双向串行数据线 SDA，一条串行时钟线 SCL。每个连接到总线的设备都有一个独立的地址，主机可以通过该地址来访问不同设备。因为 I²C 协议比较简单，常常用 GPIO 来模拟 I²C 时序，这种方法称为模拟 I²C。如果使用 MCU 的 I²C 控制器，设置好 I²C 控制器，I²C 控制器就自动实现协议时序，这种方式称为硬件 I²C。因为 I²C 设备的速率比较低，通常两种方式都可以，模拟 I²C 方便移植，硬件 I²C 工作效率相对较高。

关于 I²C 协议，通过下面例子进行一个形象的比喻方便大家理解，如图 2.16.1 所示，老师（MCU）将球（数据）传给众多学生中的一个（众多外设设备中的一个）。

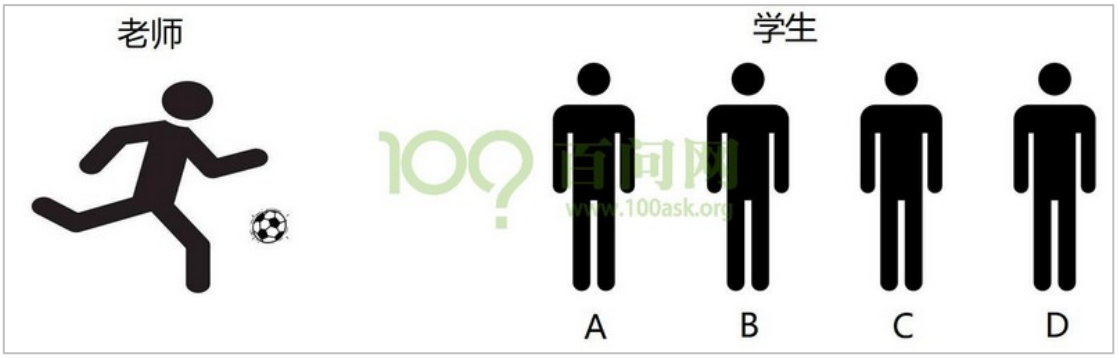


图 2.16.1 I²C 协议比喻

老师：可以把球发给学生，也可以把球从学生中接过来。

① 发球：

老师说：注意了(start)；

老师对 A 学生说，我要球发给你(A 就是地址)；

老师就把球发出去了（传输）；

A 收到球之后，应该告诉老师一声（回应）；

老师说下课（停止）。

② 接球：

老师说注意了(start)；

老师说：B 把球发给我(B 是地址)；

B 就把球发给老师（传输）；

老师收到球之后，给 B 说一声，表示收到球了（回应）；

老师说下课（停止）。

从上面这个例子，分析 I²C 的传输（发送和接收）需要准备哪些函数：
① 老师说注意了，表示开始信号(start)；
② 老师告诉某个学生，表示发送地址(address)；
③ 老师发球/接球，表示数据的发送/接收(send/read)；
④ 老师/学生收到球，回应/不回应/等待回应(ack/noack/getack)；
⑤ 老师说下课，表示 I²C 传输结束(stop)；

如果是软件模拟 I²C，就需要自己去实现每个操作，封装成函数。比如开始信号，就是先把 SCL 拉高，再 SDA 由高变低，这样就表示了一个开始信号。关于模拟 I²C 的实现函数，读者参考后面代码和相关时序文档。

AP3216C 芯片集成了光强传感器（ALS:AmbientLight Sensor）、接近传感器（PS: Proximity Sensor）、红外 LED（IR LED），最常见的应用就是手机。当我们接听电话时，耳朵靠近手机前置扬声器附近，也就靠近了该传感器，此时距离传感器就告诉 CPU 可以关闭屏幕显示，以防误触。光强传感器能识别当前环境光，告诉 CPU 对应调节屏幕亮度，手机常见传感器如所示。

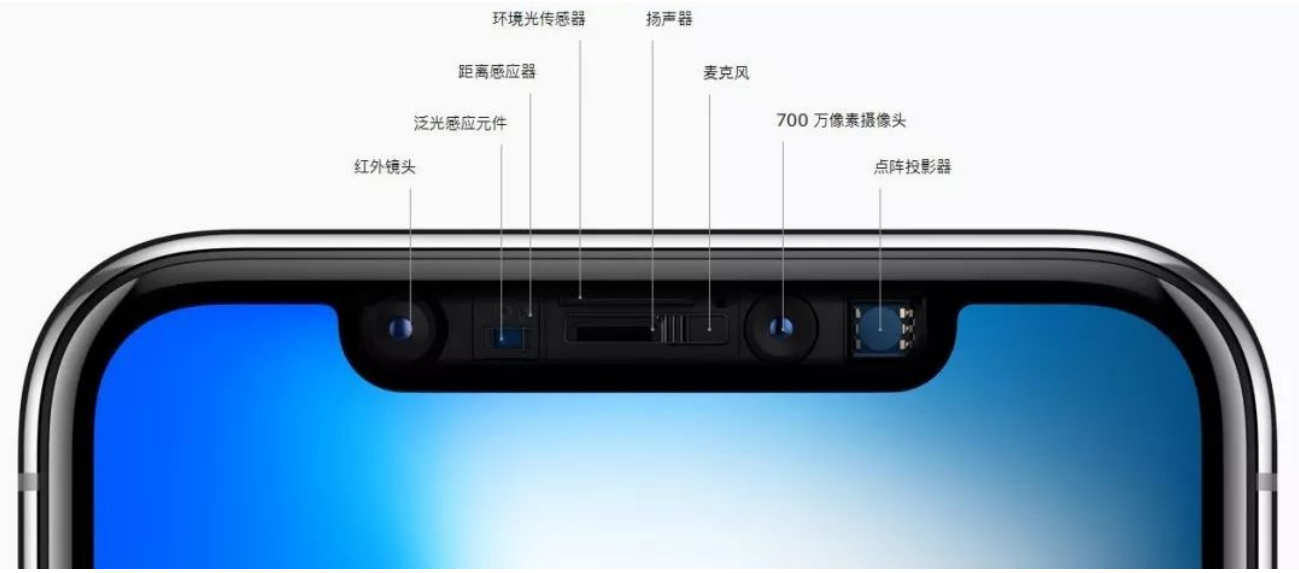


图 2.16.2 手机前置传感器示意

## 2.16.2 硬件设计

如图 2.16.3 所示，AP3216C 接在了 I²C1 上，这里使用 GPIO 模拟，可知 PF14 模拟作为 SCL 信号，PF15 作为 SDA 信号。

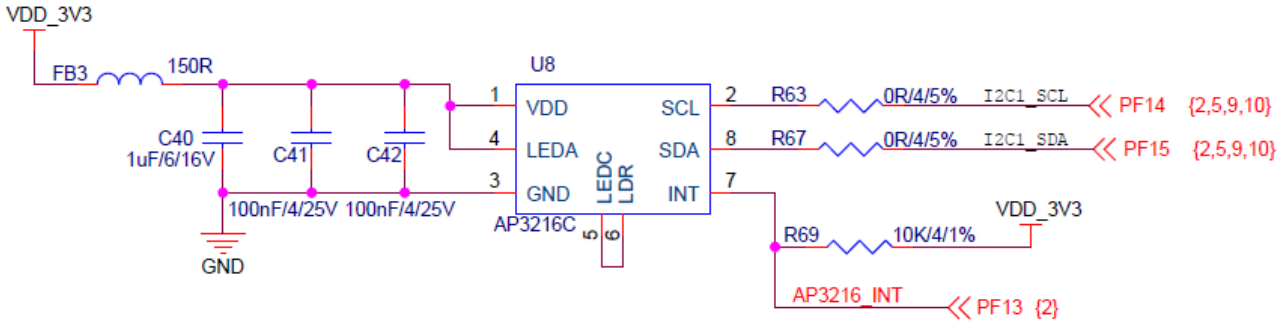


图 2.16.3 AP3216C 原理图

## 2.16.3 MX 设置

先设置好 MCU 系统时钟、调试串口，同时参考前面 2.11 通用定时器—us 延时，设置 TIM5 用于实现 us 级延时。接着搜索 PF14、PF15，设置为普通 GPIO 输出，其中 PF14 别名为 I2C1_SCL，PF15 别名为 I2C1_SDA，如图 2.16.4 所示。

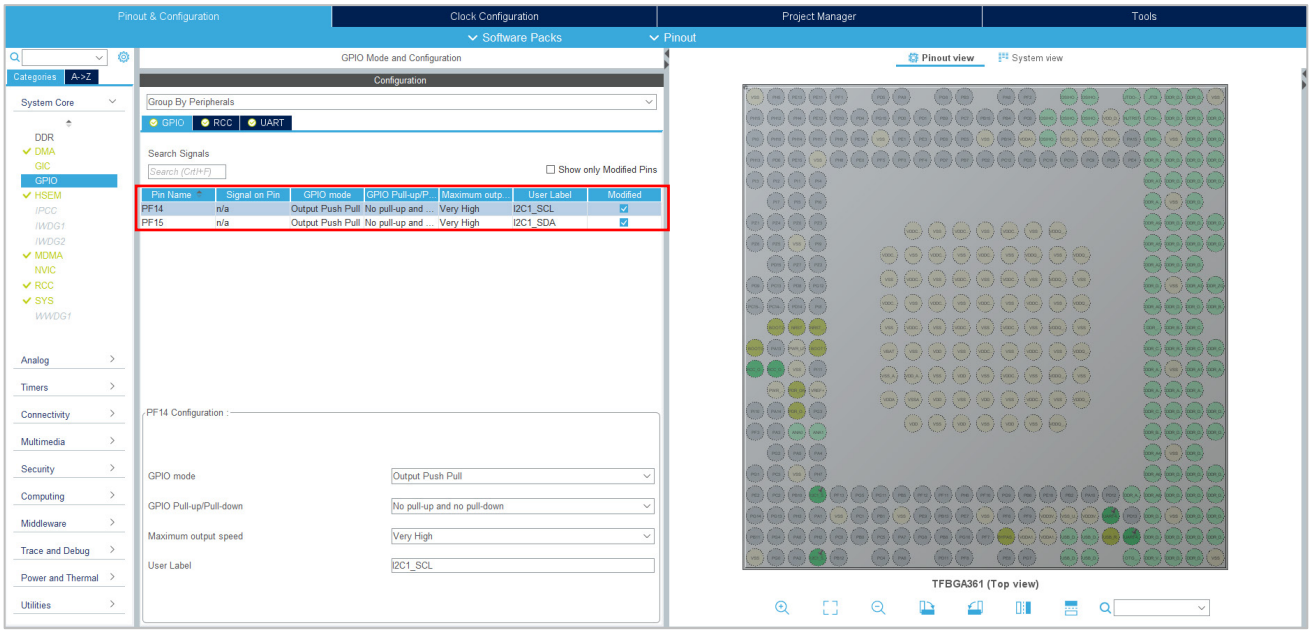


图 2.16.4 模拟 I2C 设置

## 2.16.4 代码设计

创建“DemoDriver”目录，将前面的“driver_tim.c”复制到该目录，然后新建“driver_i2c1.c”实现模拟 I²C 协议，创建“driver_ap3216c.c”实现对 AP3216C 的控制。

“driver_i2c1.c”实现了模拟 I²C 协议，代码如下：

```
#define I2C_Delay()     us_timer_delay(5)


/*
 *  函数名：void I2C_Init(void)
 *  输入参数：
```

```
 *  输出参数：无
 *  返回值：无
 *  函数作用：SCL/SDA 都初始为高电平
*/
void I2C_Init(void)
{
    SCL_H();
    SDA_H();
}

/*
 *  函数名：static void I2C_SDA_OUT(void)
 *  输入参数：
 *  输出参数：无
 *  返回值：无
 *  函数作用：配置 SDA 引脚为输出
*/
static void I2C_SDA_OUT(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    GPIO_InitStruct.Mode      = GPIO_MODE_OUTPUT_OD;
    GPIO_InitStruct.Pull      = GPIO_NOPULL;
    GPIO_InitStruct.Speed     = GPIO_SPEED_FREQ_HIGH;

    GPIO_InitStruct.Pin       = I2C1_SDA_Pin;
    HAL_GPIO_Init(I2C1_SDA_GPIO_Port, &GPIO_InitStruct);
}

/*
 *  函数名：static void I2C_SDA_IN(void)
 *  输入参数：
 *  输出参数：无
 *  返回值：无
 *  函数作用：配置 SDA 引脚为输入
*/
static void I2C_SDA_IN(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    GPIO_InitStruct.Mode      = GPIO_MODE_INPUT;
    GPIO_InitStruct.Speed     = GPIO_SPEED_FREQ_HIGH;

    GPIO_InitStruct.Pin       = I2C1_SDA_Pin;
```

```
    HAL_GPIO_Init(I2C1_SDA_GPIO_Port, &GPIO_InitStruct);
}

/*
 *  函数名：void I2C_Start(void)
 *  输入参数：
 *  输出参数：无
 *  返回值：无
 *  函数作用：I2C 开始信号
*/
void I2C_Start(void)
{
    I2C_SDA_OUT();

    SCL_H();
  I2C_Delay();

    SDA_H();
    I2C_Delay();

    SDA_L();
    I2C_Delay();

    SCL_L();
    I2C_Delay();
}

/*
 *  函数名：void I2C_Stop(void)
 *  输入参数：
 *  输出参数：无
 *  返回值：无
 *  函数作用：I2C 停止信号
*/
void I2C_Stop(void)
{
    I2C_SDA_OUT();

    SDA_L();
    I2C_Delay();

    SCL_H();
    I2C_Delay();
```

```
    SDA_H();
    I2C_Delay();
}

/*
 *  函数名：void I2C_ACK(void)
 *  输入参数：
 *  输出参数：无
 *  返回值：无
 *  函数作用：I2C 发出应答信号
*/
void I2C_ACK(void)
{
    I2C_SDA_OUT();

    SCL_L();
    I2C_Delay();

    SDA_L();
    I2C_Delay();

    SCL_H();
    I2C_Delay();

    SCL_L();
    I2C_Delay();
}

/*
 *  函数名：void I2C_NACK(void)
 *  输入参数：
 *  输出参数：无
 *  返回值：无
 *  函数作用：I2C 发出非应答信号
*/
void I2C_NACK(void)
{
    I2C_SDA_OUT();

    SCL_L();
    I2C_Delay();

    SDA_H();
    I2C_Delay();
```

```
    SCL_H();
    I2C_Delay();

    SCL_L();
    I2C_Delay();
}

/*
 *  函数名：uint8_t I2C_GetACK(void)
 *  输入参数：
 *  输出参数：无
 *  返回值：无
 *  函数作用：I2C 等待从机的应答信号
*/
uint8_t I2C_GetACK(void)
{
    uint8_t time = 0;
    I2C_SDA_IN();

    SCL_L();
  I2C_Delay();

    SDA_H();
  I2C_Delay();

  SCL_H();
  I2C_Delay();

    while(SDA_INPUT())
    {
        time++;
        if(time>250)
        {
            SCL_L();
            return 1;
        }
    }
    SCL_L();

    return 0;
}

/*
```

```
 *  函数名：void I2C_SendByte(uint8_t data)
 *  输入参数：data->发送的数据
 *  输出参数：无
 *  返回值：无
 *  函数作用：I2C 发送一个字节
*/
void I2C_SendByte(uint8_t data)
{
    uint8_t cnt = 0;

    I2C_SDA_OUT();

    for(cnt=0; cnt<8; cnt++)
    {
        SCL_L();
        I2C_Delay();

        if(data & 0x80)
        {
            SDA_H();
        }
        else
        {
            SDA_L();
        }
        data = data<<1;

        SCL_H();
        I2C_Delay();
    }

    SCL_L();
    I2C_Delay();
    I2C_GetACK();
}

/*
 *  函数名：uint8_t I2C_ReadByte(uint8_t ack)
 *  输入参数：ack->发送的应答标志，1 应答，0 非应答
 *  输出参数：无
 *  返回值：返回读到的字节
 *  函数作用：I2C 读出一个字节
*/
uint8_t I2C_ReadByte(uint8_t ack)
```

```
{
    uint8_t cnt;
    uint8_t data = 0xFF;
    I2C_SDA_IN();
    SCL_L();
    I2C_Delay();

    for(cnt=0; cnt<8; cnt++)
    {
        SCL_H();                    //SCL 高(读取数据)
        I2C_Delay();

        data <<= 1;
        if(SDA_INPUT())
        {
            data |= 0x01;           //SDA 高(数据有效)
        }
        SCL_L();
        I2C_Delay();
    }
    //发送应答信号，为低代表应答，高代表非应答
    if(ack == 0)
    {
        I2C_NACK();
    }
    else
    {
        I2C_ACK();
    }
    return data;                    //返回数据
}
```

“driver_ap3216c.c”调用前面 I²C 协议的函数，配置 AP3216C，读取 AP3216C 寄存器获取数据。首先初始化配置 AP3216C，相关寄存器如

表 2.16.1 所示，初始化流程如下：

① 复位:设置 SYS_CONFIG_ADDR(0x00)寄存器为 SYS_SW_RESET(0x04)；

② 设置工作模式：设置 SYS_CONFIG_ADDR(0x00)寄存器为 SYS_ALS_PS_IR_ACT(0x03)，IR+PS+ALS 三个都激活使用；

③ 设置中断(这里没使用中断)；

```
uint8_t AP3216_Init(void)
{
  uint8_t ret_value = 0;
  AP3216_WriteOneByte(SYS_CONFIG_ADDR, SYS_SW_RESET); // 系统软件复位
  HAL_Delay(50);
```

```
  AP3216_WriteOneByte(SYS_CONFIG_ADDR, SYS_ALS_PS_IR_ACT);  // IR+PS+ALS 三个都激活使用
  HAL_Delay(50);
  ret_value = AP3216_ReadOneByte(SYS_CONFIG_ADDR);  // 读取配置字
  if(ret_value != SYS_ALS_PS_IR_ACT)  // 判断是否与设置的一致
  {
    return 1;
  }
  printf("\r I2C Configuration register: 0x%x \n", SYS_CONFIG_ADDR);
  printf("\r I2C Configuration value: 0x%x \n", SYS_ALS_PS_IR_ACT);
  printf("\r I2C Read configuration value: 0x%x \n", ret_value);
  return 0;
}
```

表 2.16.1 AP3216C 部分系统寄存器

| ADDR (Hex) | REGISTER NAME | Bits | REGISTER COMMAND | FUNCTIONS/DESCRIPTION |
|---|---|---|---|---|
| 0x00 | System Configuration (Default : 0x00) | 2:0 | System Mode (Default : 0x0) | 0x0: Power down (Default) |
| | | | | 0x1: ALS function active |
| | | | | 0x2: PS+IR function active |
| | | | | 0x3: ALS and PS+IR functions active |
| | | | | 0x4: SW reset |
| | | | | 0x5: ALS function once |
| | | | | 0x6: PS+IR function once |
| | | | | 0x7: ALS and PS+IR functions once |

初始化配置完后，就可以读取 AP3216C 寄存器数据，如表 2.16.2 所示，分别是 IR、ALS、PS 的数据，每个数据占据相邻两位。以 IR 为例，分别读取 IR_DATA_LOW(0x0A)和 IR_DATA_HIGH(0x0B)寄存器的值，再合并就得到 IR 的数据，同理可以得到 ALS、PS 的数据。

表 2.16.2 AP3216C 部分数据寄存器

| | | | | |
|---|---|---|---|---|
| 0x0A | IR Data Low | 7 | IR overflow<br>(Read only) | 0x0: Valid IR and PS data<br>0x1: Invalid IR and PS data |
| | | 1:0 | (Read only) | IR lower byte of ADC output |
| 0x0B | IR Data High | 7:0 | (Read only) | IR higher byte of ADC output |
| 0x0C | ALS Data Low | 7:0 | (Read only) | ALS lower byte of ADC output |
| 0x0D | ALS Data High | 7:0 | (Read only) | ALS higher byte of ADC output |
| 0x0E | PS Data Low | 7 | Object detect<br>(Read only) | 0x0: The object leaving<br>0x1: The object closed |
| | | 6 | IR overflow<br>(Read only) | 0x0: Valid IR, PS data and object detected<br>0x1: Invalid IR, PS data and object detected |
| | | 3:0 | (Read only) | PS lower byte of ADC output |
| 0x0F | PS Data High | 7 | Object detect<br>(Read only) | 0x0: The object leaving<br>0x1: The object closed |
| | | 6 | IR overflow<br>(Read only) | 0x0: Valid IR, PS data and object detected<br>0x1: Invalid IR, PS data and object detected |
| | | 5:0 | (Read only) | PS higher byte of ADC output |

```
void AP3216_Read_IR_Data(uint16_t *pIR)
{
  uint8_t ir_l = 0, ir_h = 0;

  ir_l = AP3216_ReadOneByte(IR_DATA_LOW);
  ir_h = AP3216_ReadOneByte(IR_DATA_HIGH);

  if( (ir_l&0x80) == 0x80)  // Invalid IR and PS data
  {
    *pIR = 0;
  }
  else  // ir_l Bit[1:0] is data, ps_l bits[3:0] ans ps_h bits[5:0] is data
  {
    *pIR = (ir_h<<8) | (ir_l&0x03);
  }
}

void AP3216_Read_PS_Data(uint16_t *pPS)
{
  uint8_t ps_l = 0, ps_h = 0;

  ps_l = AP3216_ReadOneByte(PS_DATA_LOW);
  ps_h = AP3216_ReadOneByte(PS_DATA_HIGH);
```

```
  if( (ps_l&0x40)==0x40)  // Invalid IR and PS data
  {
    *pPS = 0;
  }
  else  // ir_l Bit[1:0] is data, ps_l bits[3:0] ans ps_h bits[5:0] is data
  {
    *pPS = ((ps_h&0x1F)<<8) | (ps_l&0x0F);
  }
}

void AP3216_Read_ALS_Data(uint16_t *pALS)
{
  uint8_t als_l = 0, als_h = 0;

  als_l = AP3216_ReadOneByte(ALS_DATA_LOW);
  als_h = AP3216_ReadOneByte(ALS_DATA_HIGH);

  *pALS = (als_h<<8) | (als_l);
}
```

主函数里，先分别初始化 $I^2C$ 和 AP3216，再调用“AP3216_ReadDataTest()”即可获取 IR、PS、ALS 的打印。

## 2.16.5 实验效果

AP3216C 板载在开发板上，位于启动拨码开关附近，背面丝印位号为 U8。调试运行，可以在串口看到如图 2.16.5 所示打印。将手靠近 AP3216C，可以看到 PS 值变化，用强光照射 AP3216C，可以看到 ALS 值变化。

```
I2C Soft AP3216C
I2C Configuration register: 0x0
I2C Configuration value: 0x3
I2C Read configuration value: 0x3
AP3216 Init OK
***************
IR = 0x0
PS = 0x0
ALS = 0x0
***************
***************
IR = 0x102
PS = 0x0
ALS = 0xc3
***************
***************
IR = 0x1
PS = 0x0
ALS = 0xc4
***************
```

图 2.16.5 模拟 I²C 读取 AP3216C 数据打印

# 2.17 I2C—硬件模式

- 设计需求

这里假设需求为使用硬件 I²C 获取板子传感器 AP3216C 的数据。

## 2.17.1 基础知识

I²C 协议简介和 AP3216C 简介，参考上一实验。STM32MP157 有 6 个硬件 I²C，其中 I²C4 和 I²C6 只能 A7 使用，余下可选择分配给 A7 或 M4。

## 2.17.2 硬件设计

硬件连接参考上一个实验，这里使用硬件 I²C1。

## 2.17.3 MX 设置

首先配置好 MCU 系统时钟和串口打印，然后如图 2.17.1 所示，将 PF14、PF15 设置为复用 I²C1 功能。

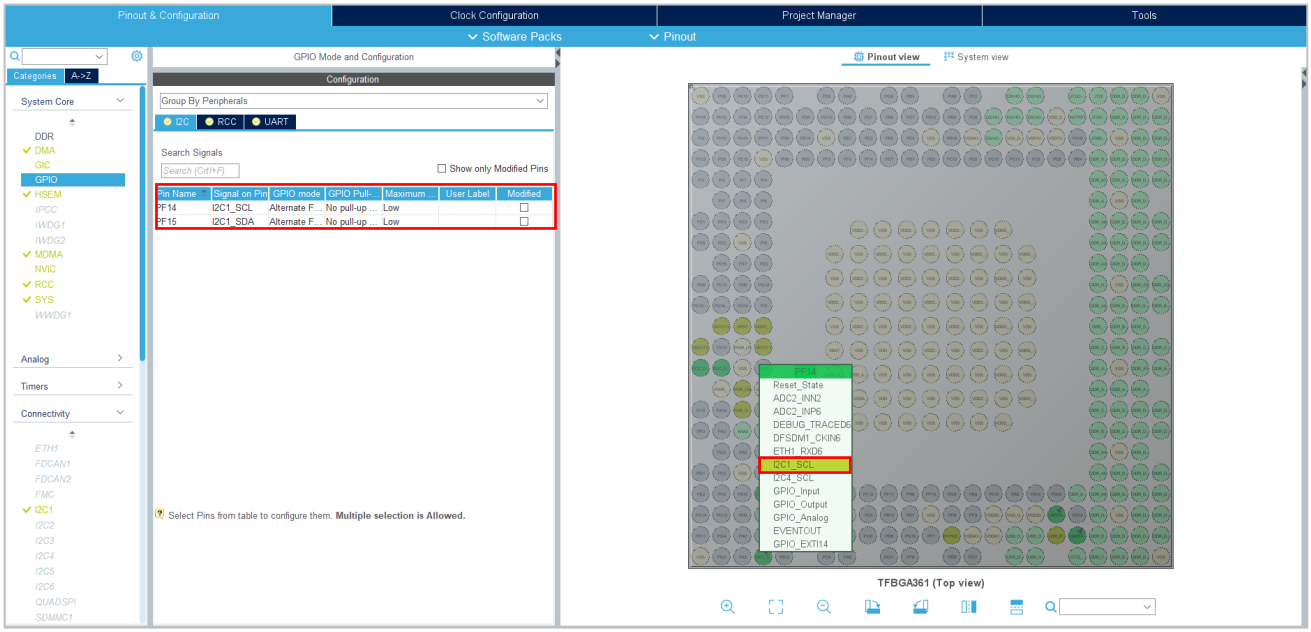


图 2.17.1 硬件 I²C 引脚复用设置

然后切换到 I²C1 标签，勾选给 M4，下方选择 I2C，其它默认配置即可。

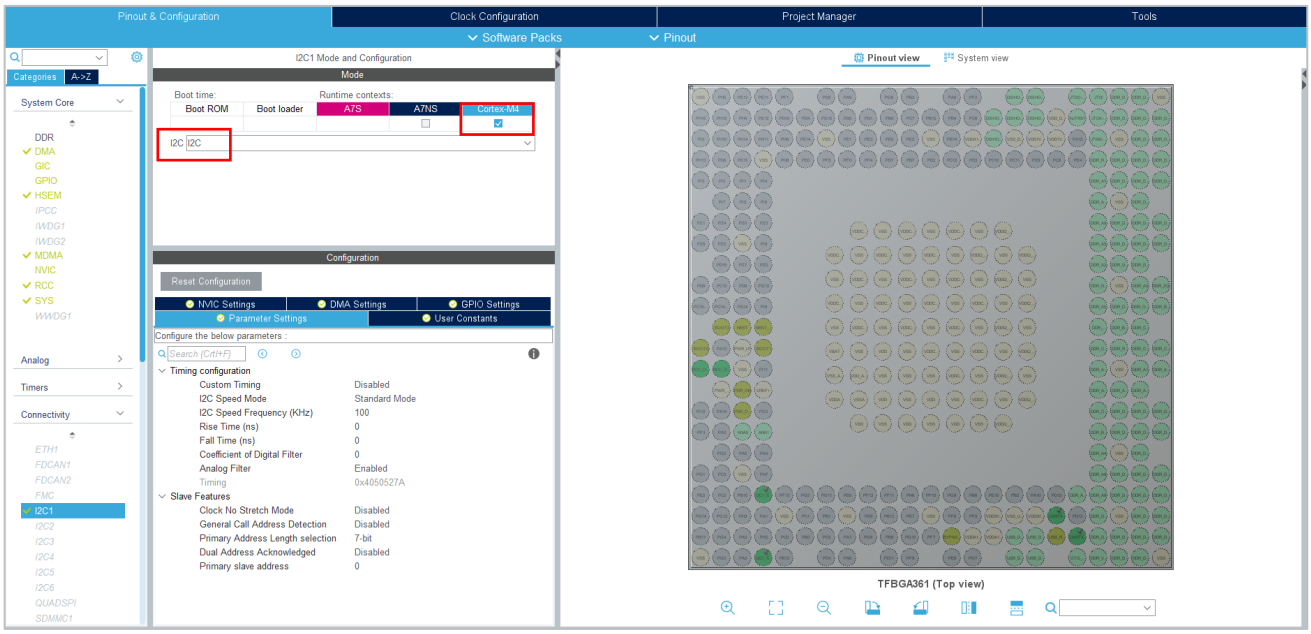


图 2.17.2 I²C1 配置

## 2.17.4 代码设计

因为使用的是硬件 I²C，就不需要自己实现 I²C 协议时序。创建“DemoDriver”目录，复制前面的“driver_ap3216c.c”，需要做的仅仅是用 HAL 提供的硬件 I²C 操作函数，替换原来的 GPIO 模拟的 I²C 操作函数。调用“HAL_I2C_Master_Transmit()”发送数据，“HAL_I2C_Master_Receive()”接收数据，需要修改部分如下。

```
uint8_t AP3216_WriteOneByte(uint8_t reg, uint8_t data)
{
  uint16_t write_data = reg | (data<<8);

  if(HAL_I2C_Master_Transmit(&hi2c1, AP3216_ADDR | AP3216_W , (uint8_t*)&write_data, 2, 300) != HAL_OK)
  {
    Error_Handler();
  }
  while (HAL_I2C_GetState(&hi2c1) != HAL_I2C_STATE_READY);

  return 0;
}

uint8_t AP3216_ReadOneByte(uint8_t reg)
{
  uint8_t read_data = 0;

  if(HAL_I2C_Master_Transmit(&hi2c1, AP3216_ADDR | AP3216_W , (uint8_t*)&reg, 1, 300) != HAL_OK)
  {
    Error_Handler();
```

```
  }
  while (HAL_I2C_GetState(&hi2c1) != HAL_I2C_STATE_READY);

  if(HAL_I2C_Master_Receive(&hi2c1, AP3216_ADDR | AP3216_R , (uint8_t*)&read_data, 1, 300) != HAL_OK)
  {
    Error_Handler();
  }

    return read_data;
}
```

## 2.17.5 实验效果

连接串口，调试、运行，可以看到如图 2.17.3 所示，将手靠近 AP3216C，可以看到 PS 值变化，用强光照射 AP3216C，可以看到 ALS 值变化。

```
IR = 0x401
PS = 0x0
ALS = 0x4b
***************
***************
IR = 0x302
PS = 0x0
ALS = 0x4f
***************
***************
IR = 0x500
PS = 0x0
ALS = 0x4e
***************
***************
IR = 0x303
PS = 0x104
ALS = 0x4e
***************
```

图 2.17.3 硬件 I²C 读取 AP3216C 数据打印

# 2.18 SPI—软件模拟

- 设计需求

这里假设需求为使用 GPIO 模拟 SPI 获取板子传感器 ICM20608 的数据。

## 2.18.1 基础知识

相较于 $I^2C$ 半双工的同步串行总线，SPI 是全双工的同步串行总线，理论有更高的传输带宽，但 SPI 占用的引脚更多，通常需要四根，分别是提供时钟的 SCLK，提供数据输出的 MOSI(Master Out Slave In)，提供数据输入的 MISO(Master In Slave Out)和提供片选信号的 CS。同一时刻只能有一个 SPI 设备处于工作状态，即多个 CS 信号中某时间只能有一个有效。为了适配不同的外设 ，SPI 支持通过寄存器来配置片选信号和时钟信号的极性和相位。

SPI 的速度也不是很高，通信协议也不是很复杂，因此在实际工程中，也不少使用 GPIO 模拟 SPI 的场景，关于 SPI 协议的详细分析，这里也不过多描述，读者参考后面的模拟 SPI 代码和其它资料。

ICM-20608-G 是 InvenSense 面向大众市场的一款六轴姿态传感器，包含三轴加速度传感器、三轴角速度传感器和温度传感器。三轴加速度传感器（Accelerometer、G-Sensor）也叫重力感应器，可以感知任意方向上的加速度（重力加速度则只是地表垂直方向加速度），加速度传感器主要用于测量设备的受力情况，相对外部参考物的运动。三轴角速度传感器（Gyroscope、GYRO-Sensor）也叫陀螺仪，通过测量三维坐标系内陀螺转子的垂直轴与设备之间的夹角，并计算角速度，角速度传感器主要用于测量设备的自身的旋转运动，擅长感知自身运动。通过六轴数据，基本可以计算出物体的运动状态，常用手机、手环、无人机、云台等领域。

ICM-20608-G 的内部框图如图 2.18.1 所示，六轴和温度传感器经过 ADC 转换保存到内部寄存器，最后通过 $I^2C$ 或 SPI 向外提供数据。

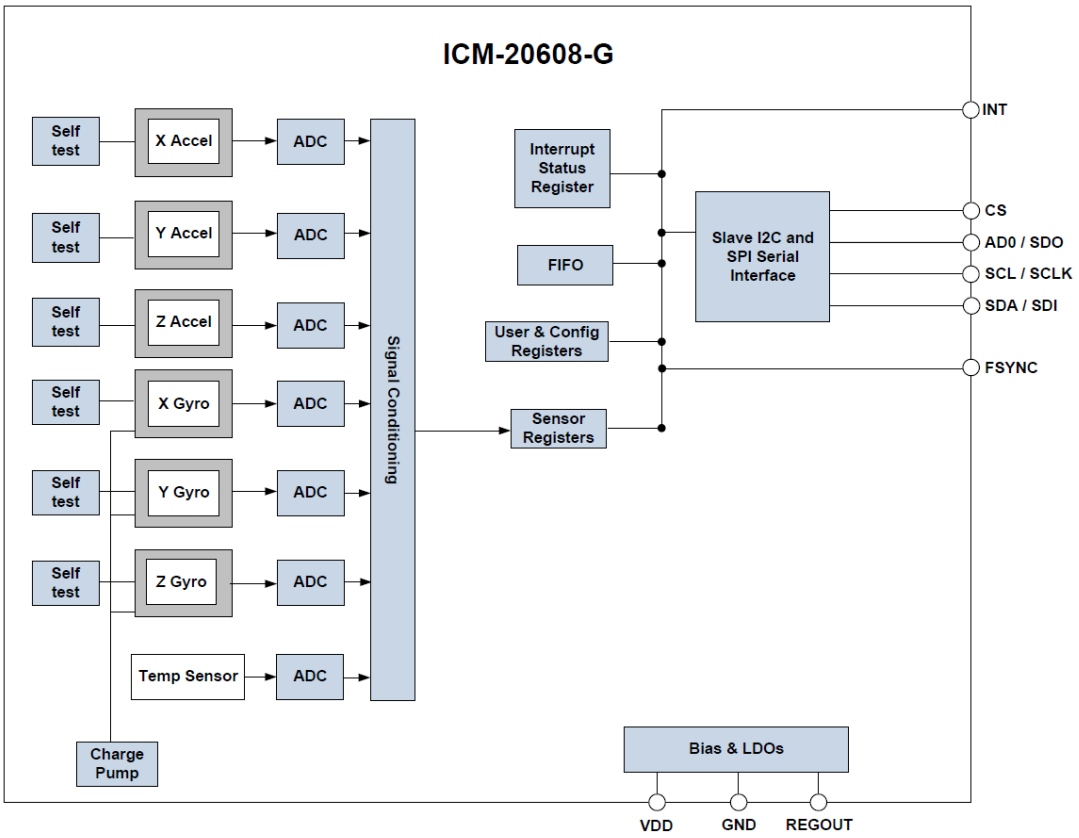


图 2.18.1 ICM-20608-G 结构框图

## 2.18.2 硬件设计

ICM-20608-G 在底板的电路图，如图 2.18.2 所示。接在了 SPI5 上，这里使用 GPIO 模拟，可知 PH6 模拟作为 SPI SCK 信号，PF11 作为 SPI MOSI 信号，PH7 作为 SPI MISO 信号,PH5 作为 SPI NSS 片选信号。

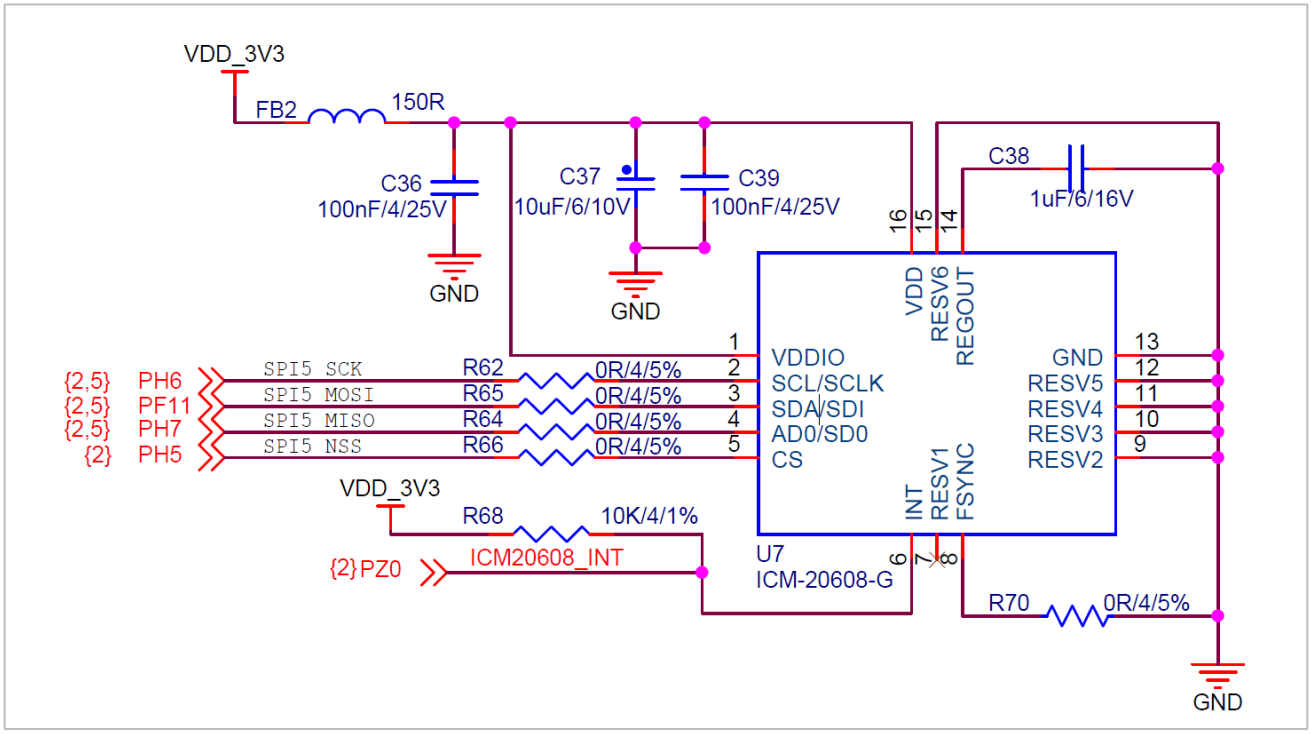


图 2.18.2 ICM20608 硬件电路图

## 2.18.3 MX 设置

先设置好 MCU 时钟、串口，接着设置 PH6、 PF11、PH7、PH5 作为普通 GPIO，注意 PH7 是 MISO，数据输入引脚，因此要设置为输入模式，其它为输出模式。

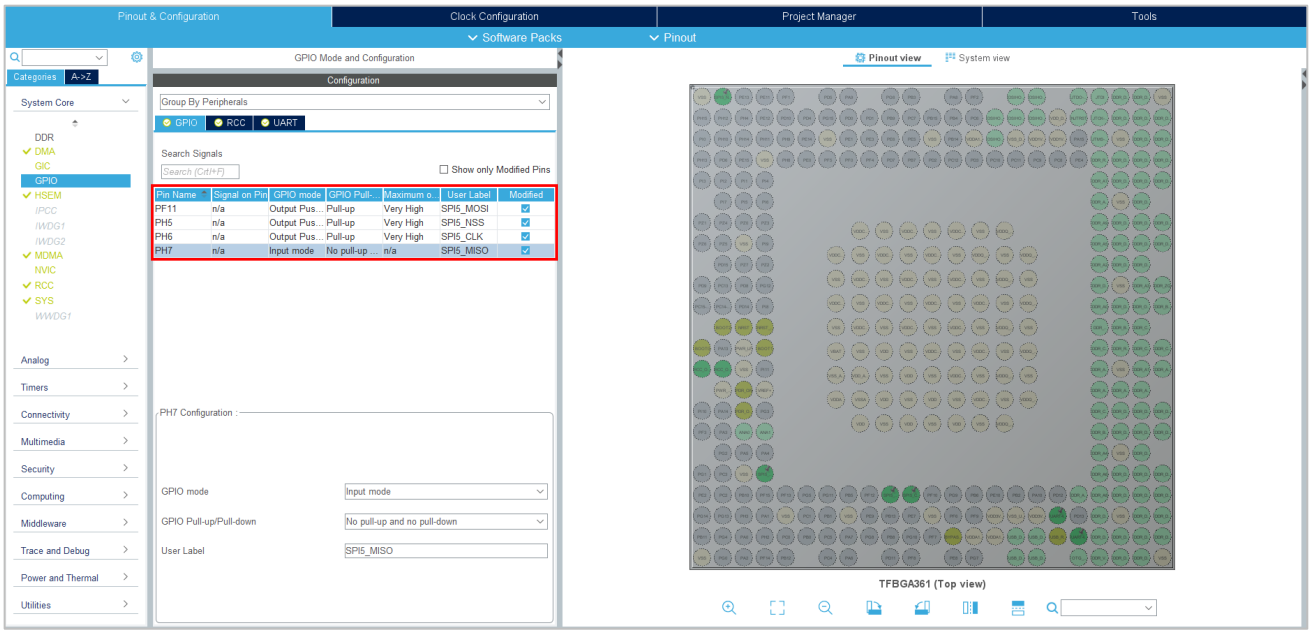


图 2.18.3 模拟 SPI 引脚设置

## 2.18.4 代码设计

创建“DemoDriver”目录，将前面的“driver_tim.c”复制到该目录，然后新建“driver_spi.c”实现模拟 SPI 协议，创建“icm_20608g.c”实现对 ICM_20608_G 的控制。

“driver_spi.c”实现了模拟 SPI 协议，代码如下：

```
/*
 *  函数名：void SPI_Init(void)
 *  输入参数：
 *  输出参数：无
 *  返回值：无
 *  函数作用：初始化 SPI 的四根引脚
*/
void SPI_Init(void)
{
     SPI_CS(1);       // CS 初始化高
     SPI_CLK(0);      // CLK 初始化高
}


/*
 *  函数名：void SPI_WriteByte(uint8_t data)
 *  输入参数：data -> 要写的数据
 *  输出参数：无
 *  返回值：无
 *  函数作用：模拟 SPI 写一个字节
*/
void SPI_WriteByte(uint8_t data)
{
    uint8_t i = 0;
    uint8_t temp = 0;

    for(i=0; i<8; i++)
    {
        temp = data&0x80;
        data = data<<1;

        SPI_CLK(0);
        SPI_MOSI(temp);
        SPI_Delay();
        SPI_CLK(1);
        SPI_Delay();
    }
}

/*
 *  函数名：uint8_t SPI_ReadByte(void)
```

```
 *  输入参数：
 *  输出参数：无
 *  返回值：读到的数据
 *  函数作用：模拟 SPI 读一个字节
*/
uint8_t SPI_ReadByte(void)
{
    uint8_t i = 0;
    uint8_t read_data = 0xFF;

    for(i=0; i<8; i++)
    {
        read_data = read_data << 1;

        SPI_CLK(0);
        SPI_Delay();
        SPI_CLK(1);
        SPI_Delay();
        if(SPI_MISO()==1)
        {
            read_data = read_data + 1;
        }
    }
    SPI_CLK(0);
    return read_data;
}

/*
 *  函数名：uint8_t SPI_WriteReadByte(uint8_t pdata)
 *  输入参数：pdata -> 要写的一个字节数据
 *  输出参数：无
 *  返回值：读到的数据
 *  函数作用：模拟 SPI 读写一个字节
*/
uint16_t SPI_WriteReadByte(uint16_t pdata)
{
    uint8_t i = 0;
    uint8_t temp = 0;
    uint16_t read_data = 0xFFFF;

    for(i=0;i<16;i++)
    {
        temp = ((pdata&0x8000)==0x8000)? 1:0;
        pdata = pdata<<1;
```

```
        read_data = read_data<<1;

        SPI_CLK(0);
        SPI_MOSI(temp);
        SPI_Delay();
        SPI_CLK(1);
        SPI_Delay();
        if(SPI_MISO()==1)
        {
            read_data = read_data + 1;
        }
    }

    return read_data;
}
```

接着就是利用 SPI 读写函数，对 ICM_20608_G 进行操作。在读取数据前，需要先初始化、配置 ICM_20608_G。初始化的内容比较多，依次为复位、选择时钟、获取 ID 测试、设置六轴详细参数（分辨率、量程、是否使用滤波器等）、使能、关闭低功耗模式和 FIFO 等。每个寄存器的功能、设置的值含义，读者可以参考芯片手册的寄存器对应理解，这里不再赘述。

```
void ICM_Init(void)
{
  uint8_t ret = 0;

  // 复位 ICM
  ICM_RW_Register(ICM_PWM_MGMT_1 | ICM_WRITE, 0x80);
  HAL_Delay(50);

  // 选择时钟
  ICM_RW_Register(ICM_PWM_MGMT_1 | ICM_WRITE, 0x01);
  HAL_Delay(50);
  // 获取设备 ID
  ret = ICM_RW_Register(ICM_WHO_AM_I | ICM_READ, Dummy_Byte);
  printf("\rICM ID: 0x%x\n", ret);

  ICM_CS_ENABLE();
  // 输出速率：内部采样率
  ICM_RW_Register(ICM_SMPLRT_DIV | ICM_WRITE, 0x00);
  // 陀螺仪 2000dps
  ICM_RW_Register(ICM_GYRO_CONFIG | ICM_WRITE, 0x18);
  // 加速度 16G 量程
  ICM_RW_Register(ICM_ACCEL_CONFIG | ICM_WRITE, 0x18);
  // 陀螺仪低通滤波 20Hz
  ICM_RW_Register(ICM_CONFIG | ICM_WRITE, 0x04);
```

```
    // 加速度低通滤波 21.2Hz
    ICM_RW_Register(ICM_ACCEL_CONFIG_2 | ICM_WRITE, 0x04);

    // 打开加速度计和陀螺仪所有轴
    ICM_RW_Register(ICM_PWM_MGMT_1 | ICM_WRITE, 0x00);

    // 关闭低功耗
    ICM_RW_Register(ICM_LP_MODE_CFG | ICM_WRITE, 0x00);

    // 关闭 FIFO
    ICM_RW_Register(ICM_FIFO_EN | ICM_WRITE, 0x00);
}
```

初始化完后，就可以读取数据，每个数据占两位，读取完后需要高低字节拼接处理。获取到寄存器原始值后，还需要做换算处理，比如温度，手册上写的换算公式为：

$$Temp\ ℃ = \left((Temp_\text{out} - RoomTemp_Offset) \div \text{Temp_Sensitivity}\right) + 25℃$$

而“Room Temperature Offset”可从手册查到典型值为 25℃，“Temp_Sensitivity”典型值为 326.8LSB/℃。

对于角速度，前面初始化的时候将量程设置了为±2000dps，读出来的原始数据是带符号的 16 位，则范围为-32767~+32767，+32767 对应+2000，即灵敏度为 16.4。

对于加速度，前面初始化的时候将量程设置了为±16g，出来的原始数据是带符号的 16 位，则范围为-32767~+32767，+32767 对应+16，即灵敏度为 2048。

```
void ICM_ReadGyroAccel(void)
  {
    int16_t temp = 0;
    float temp_value = 0;
    uint8_t temp_h = 0, temp_l = 0;

    int16_t xa = 0, ya = 0, za = 0;
    float xa_act = 0, ya_act = 0, za_act = 0;
    uint8_t xa_l = 0, ya_l = 0, za_l = 0;
    uint8_t xa_h = 0, ya_h = 0, za_h = 0;

    int16_t xg = 0, yg = 0, zg = 0;
    float xg_act = 0, yg_act = 0, zg_act = 0;
    uint8_t xg_l = 0, yg_l = 0, zg_l = 0;
    uint8_t xg_h = 0, yg_h = 0, zg_h = 0;

    temp_l = ICM_RW_Register(ICM_TEMP_OUT_L | ICM_READ, Dummy_Byte);
    temp_h = ICM_RW_Register(ICM_TEMP_OUT_H | ICM_READ, Dummy_Byte);
    temp = (temp_h<<8) | temp_l;
    temp_value = (temp - 25)/326.8 + 25;
```

```
    xa_l = ICM_RW_Register(ICM_ACCEL_XOUT_L | ICM_READ, Dummy_Byte);
    xa_h = ICM_RW_Register(ICM_ACCEL_XOUT_H | ICM_READ, Dummy_Byte);
    ya_l = ICM_RW_Register(ICM_ACCEL_YOUT_L | ICM_READ, Dummy_Byte);
    ya_h = ICM_RW_Register(ICM_ACCEL_YOUT_H | ICM_READ, Dummy_Byte);
    za_l = ICM_RW_Register(ICM_ACCEL_ZOUT_L | ICM_READ, Dummy_Byte);
    za_h = ICM_RW_Register(ICM_ACCEL_ZOUT_H | ICM_READ, Dummy_Byte);
    xa = xa_l + (xa_h<<8);  xa_act = xa / 2048.0;
    ya = ya_l + (ya_h<<8);  ya_act = ya / 2048.0;
    za = za_l + (za_h<<8);  za_act = za / 2048.0;

    xg_l = ICM_RW_Register(ICM_GYRO_XOUT_L | ICM_READ, Dummy_Byte);
    xg_h = ICM_RW_Register(ICM_GYRO_XOUT_H | ICM_READ, Dummy_Byte);
    yg_l = ICM_RW_Register(ICM_GYRO_YOUT_L | ICM_READ, Dummy_Byte);
    yg_h = ICM_RW_Register(ICM_GYRO_YOUT_H | ICM_READ, Dummy_Byte);
    zg_l = ICM_RW_Register(ICM_GYRO_ZOUT_L | ICM_READ, Dummy_Byte);
    zg_h = ICM_RW_Register(ICM_GYRO_ZOUT_H | ICM_READ, Dummy_Byte);
    xg = xg_l + (xg_h<<8);  xg_act = xg / 16.4;
    yg = yg_l + (yg_h<<8);  yg_act = yg / 16.4;
    zg = zg_l + (zg_h<<8);  zg_act = zg / 16.4;

    printf("\r-----------------------------------\n");

    printf("\rTemperature Original value: 0x%x \n", temp);
    printf("\rTemperature Converted value: %.2f\n", temp_value);
    printf("\r\n");
    printf("\rAccelerometer Original value: xa:0x%x ya:0x%x za:0x%x \n", xa, ya, za);
    printf("\rAccelerometer Converted value: xa_act:%.2fg ya_act:%.2fg za_act:%.2fg \n", xa_act, ya_act, za_act);
    printf("\r\n");
    printf("\rGyroscope Original value: xg:0x%x yg:0x%x zg:0x%x \n", xg, yg, zg);
    printf("\rGyroscope Converted value: xg_act:%.2fdeg/s yg_act:%.2fdeg/s zg_act:%.2fdeg/s \n", xg_act, yg_act, zg_
act);

    printf("\r-----------------------------------\n");
}
```

最后主函数调用“ICM_ReadGyroAccel()”打印结果。

## 2.18.5 实验效果

调试运行，串口输出如图 2.18.4 所示。先打印出 ICM_20608_G 的 ID 为 0xAF,再打印温度原始数据和转换数据。接着是三轴加速度传感器（重力感应器）的原始数据和转换数据，其中 Z 轴的值为 1g，即受重力影响，大小为 1g，其它两个方向无受力，数值为 0，用户将板子立放将会看到数值变化。最后是三轴角速度传感器（陀螺仪）的原始数据和转换数据，开发板目前静置，三个轴方向数值为 0，用户晃动开发板，将看到值对应变化。

```
SPI Soft ICM_20608G
ICM ID: 0xaf
---------------------------------
Temperature Original value: 0x96d
Temperature Converted value: 32.31

Accelerometer Original value: xa:0x8 ya:0xffffffd0 za:0x81d
Accelerometer Converted value: xa_act:0.00g ya_act:-0.02g za_act:1.01g

Gyroscope Original value: xg:0xfffffef yg:0xfffffff1 zg:0xfffffff9
Gyroscope Converted value: xg_act:-1.04deg/s yg_act:-0.91deg/s zg_act:-0.43deg/s
---------------------------------
---------------------------------
Temperature Original value: 0xfde
Temperature Converted value: 37.35

Accelerometer Original value: xa:0x3 ya:0xffffffd0 za:0x81d
Accelerometer Converted value: xa_act:0.00g ya_act:-0.02g za_act:1.01g

Gyroscope Original value: xg:0xfffffff1 yg:0xfffffef zg:0xfffffff9
Gyroscope Converted value: xg_act:-0.91deg/s yg_act:-1.04deg/s zg_act:-0.43deg/s
---------------------------------
```

图 2.18.4 模拟 SPI 读取 ICM_20608G 数据

# 2.19 SPI—硬件模式

- 设计需求

这里假设需求为使用硬件 SPI 获取板子传感器 ICM20608 的数据。

## 2.19.1 基础知识

SPI 简介和 ICM20608 简介，参考上一实验。STM32MP157 有 6 个硬件 SPI，其中 SPI6 只能 A7 使用，余下可选择分配给 A7 或 M4。

## 2.19.2 硬件设计

硬件连接参考上一个实验，这里使用硬件 SPI。

## 2.19.3 MX 设置

首先配置好 MCU 系统时钟和串口打印，然后如图 2.19.1 所示。因为使用的引脚是 SPI 的默认引脚，所以无需先设置引脚复用，直接切换到 SPI5 标签，勾选给 M4，“Mode”选择“Full-Duplex Master”，使用硬件片选，这里选择“Hardware NSS Output Signal”。接着下方属性需要注意的是数据大小“Data Size”常见的是 8 位，MSB（Most Significant Bit）高位在前，时钟极性（CPOL）和时钟相位（CPHA）根据设备芯片手册需求设置，最后还需要关闭“NSSP Mode”。

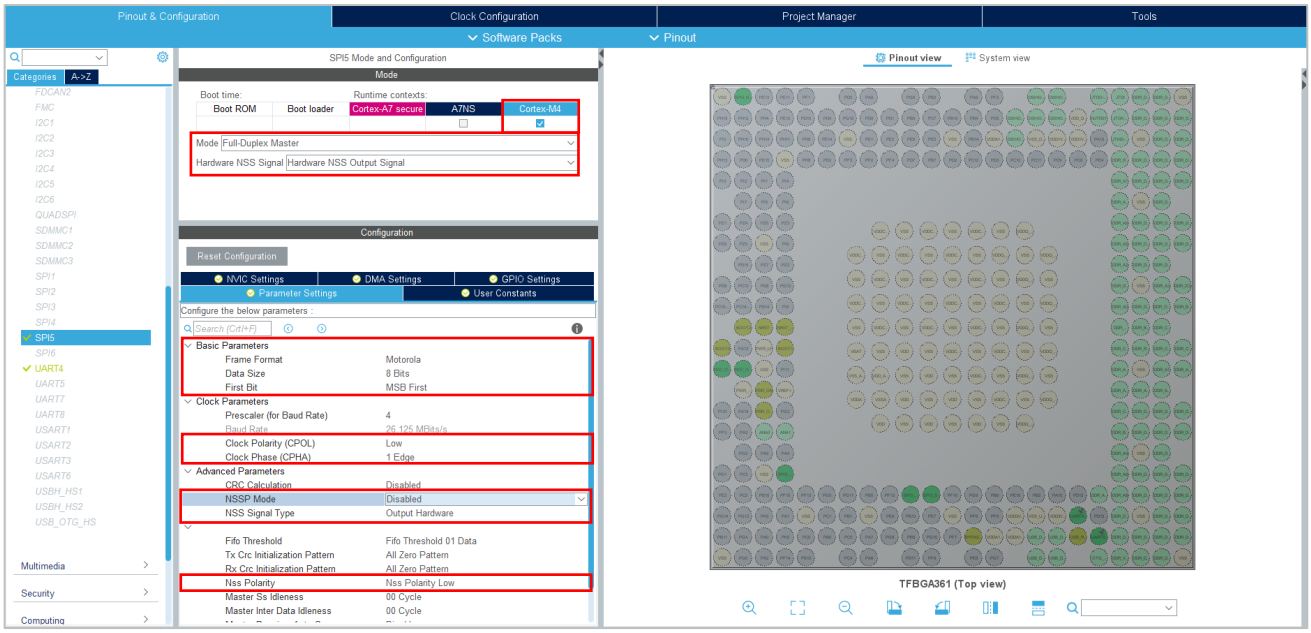


图 2.19.1 硬件 SPI 配置

## 2.19.4 代码设计

因为使用的是硬件 SPI，就不需要自己实现 SPI 协议时序。创建“DemoDriver”目录，复制前面的“icm_20608g.c”，需要做的仅仅是用 HAL 提供的硬件 SPI 操作函数，替换原来的 GPIO 模拟的 SPI 操作函

数。调用“HAL_SPI_TransmitReceive()”收发送数据，重新编写“ICM_RW_Register()”即可，需要修改部分如下。

```
uint8_t ICM_RW_Register(uint8_t reg, uint8_t data)
{
    uint16_t rxdata = 0;
    uint16_t txdata = (data<<8) | reg;

    if(HAL_SPI_TransmitReceive(&hspi5, (uint8_t*)&txdata, (uint8_t*)&rxdata, sizeof(rxdata), 300) != HAL_OK)
    {
        Error_Handler();
    }

    return (rxdata>>8);
}
```

## 2.19.5 实验效果

调试运行，串口输出如图 2.19.2 所示。先打印出 ICM_20608_G 的 ID 为 0xAF,再打印温度原始数据和转换数据。接着是三轴加速度传感器（重力感应器）的原始数据和转换数据，其中 Z 轴的值为 1g，即受重力影响，大小为 1g，其它两个方向无受力，数值为 0，用户将板子立放将会看到数值变化。最后是三轴角速度传感器（陀螺仪）的原始数据和转换数据，开发板目前静置，三个轴方向数值为 0，用户晃动开发板，将看到值对应变化。

```
SPI Hardware ICM_20608G
ICM ID: 0xaf
-----------------------------------
Temperature Original value: 0x1090
Temperature Converted value: 37.90

Accelerometer Original value: xa:0x110 ya:0xffffff60 za:0x3fa4
Accelerometer Converted value: xa_act:0.13g ya_act:-0.08g za_act:7.96g

Gyroscope Original value: xg:0xffffff61 yg:0x5 zg:0xffffffd5
Gyroscope Converted value: xg_act:-9.70deg/s yg_act:0.30deg/s zg_act:-2.62deg/s
-----------------------------------
-----------------------------------
Temperature Original value: 0x1094
Temperature Converted value: 37.91

Accelerometer Original value: xa:0x17 ya:0xffffffdb za:0x81a
Accelerometer Converted value: xa_act:0.01g ya_act:-0.02g za_act:1.01g

Gyroscope Original value: xg:0xfffffff0 yg:0xffffffec zg:0xfffffff9
Gyroscope Converted value: xg_act:-0.98deg/s yg_act:-1.22deg/s zg_act:-0.43deg/s
-----------------------------------
```

图 2.19.2 硬件 SPI 读取 ICM_20608G 数据

# 2.20 CAN 通信

- 设计需求

这里假设需求为使用百问网的 CAN-RS485 互转模块，实现 RS485、CAN 接口互相发送、接收数据。

## 2.20.1 基础知识

控制器局域网（Controller Area Network，CAN），是由德国 BOSCH（博世）公司开发，是目前国际上应用最为广泛的现场总线之一。其特点是可拓展性好，可承受大量数据的高速通信，高度稳定可靠，因此常应用于汽车电子领域、工业自动化、医疗设备等高要求环境。

CAN 总线有两个 ISO 国际标准：ISO11519 和 ISO11898。

- ISO11519 定义了通信速率为 10～125 kbps 的低速 CAN 通信标准，属于开环总线，传输速率为 40kbps 时，总线长度可达 1000 米；
- ISO11898 定义了通信速率为 125 kbps～1 Mbps 的高速 CAN 通信标准，属于闭环总线，传输速率可达 1Mbps，总线长度≤40 米；

高速 CAN 主要应用在发动机、变速箱等对实时性、传输速度要求高的场景。低速 CAN 主要应用在车身控制系统等可靠性要求高的场景，低速 CAN 在断掉其任一导线后，仍可以继续接收数据，因此在汽车发生交通事故时，使用低速 CAN 能更大提高设备正常接收数据工作的可能性，提高安全性。

如图 2.20.1 所示，是低速 CAN 的拓扑结构图，如图 2.20.2 是高速 CAN 的拓扑结构图。低速 CAN 总线为开环，高速 CAN 总线为闭环，总线由 CAN_H 和 CAN_L 两根线组成，总线上可以挂多个节点设备。每个节点设备由 CAN 控制器和 CAN 收发器组成，CAN 控制器通常作为外设集成在 MPU/MCU 上，而 CAN 收发器则需要外围添加芯片电路。

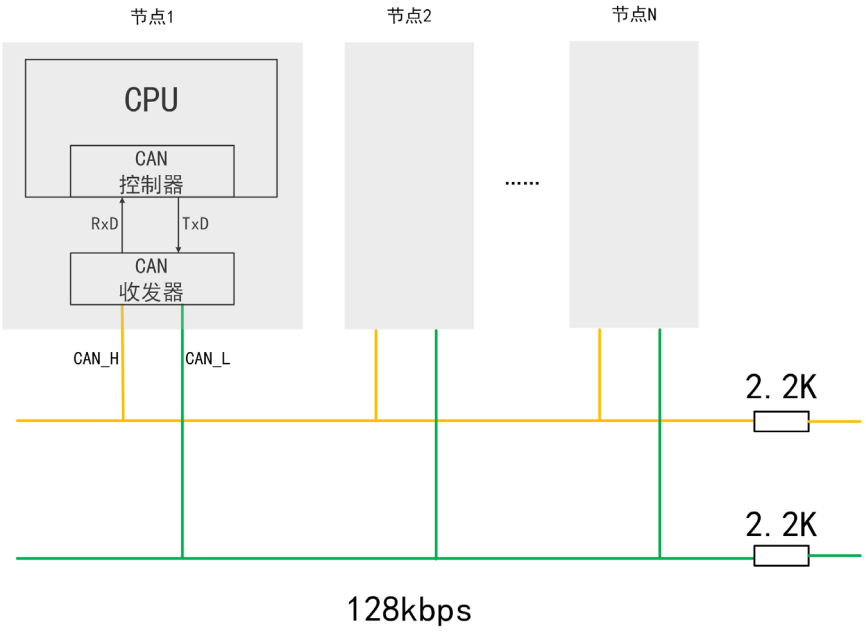


图 2.20.1 低速 CAN 拓扑结构

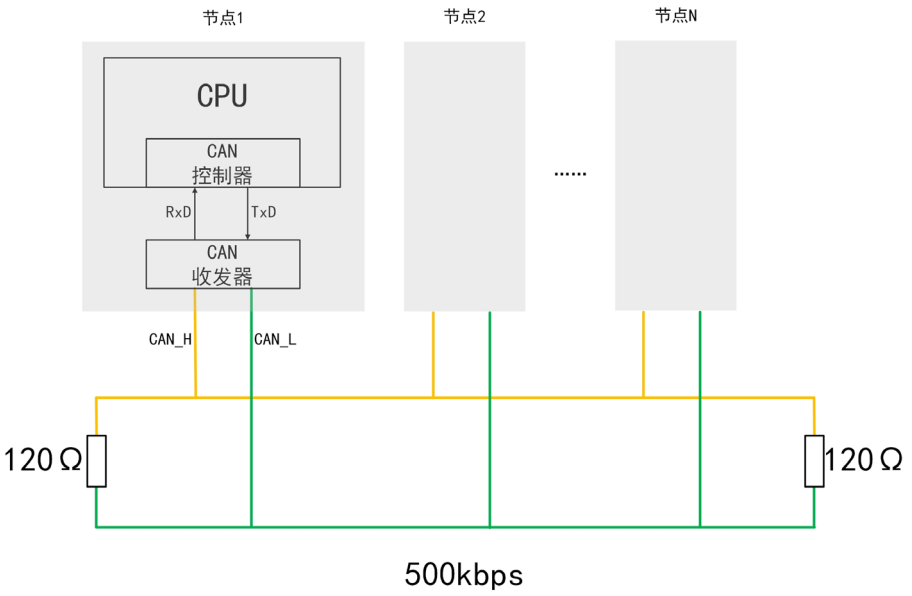


图 2.20.2 高速 CAN 拓扑图

类似 RS485，CAN 也使用差分信号传输数据。CAN 总线使用 CAN_H 和 CAN_L 的电位差来表示数据结构。电位差分为显性电平和隐性电平，分别表示逻辑 0 和 1。如图 2.20.3 所示，是低速 CAN（ISO11519 标准）的电平定义，图 2.20.4 是高速 CAN（ISO11898 标准）的电平定义，两者物理层电气特性不一样，因此不能将它们连接在一起。可以看到当 CAN_H 和 CAN_L 电压相近，则表示隐性电平，对应逻辑 1，当两个电压相差较大，表示显性电平，对应逻辑 0。

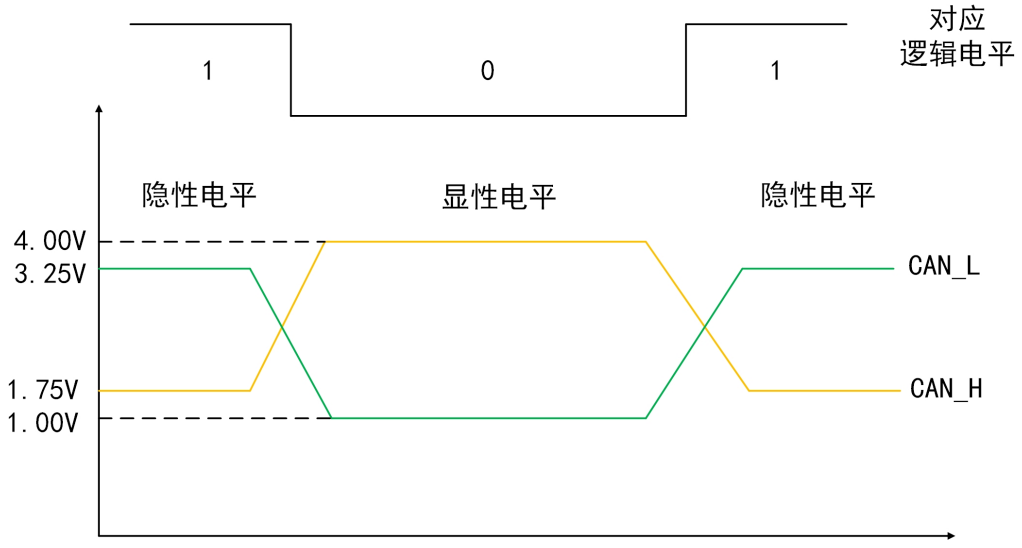


图 2.20.3 低速 CAN 电平定义

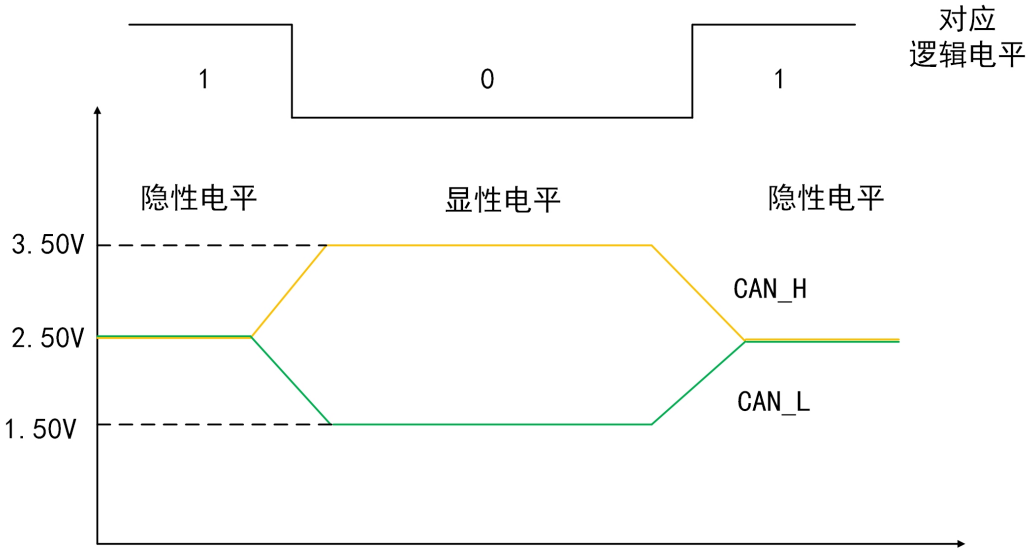


图 2.20.4 高速 CAN 电平定义

CAN 总线以“帧”（Frame）的形式进行通信。CAN 总线协议规定了 5 种帧，分别是数据帧、远程帧、错误帧、超载帧以及帧间隔，其中数据帧的应用最为频繁，表 2.20.1 是各个帧的用途。

表 2.20.1 CAN 帧种类及其作用

| | |
|---|---|
| 数据帧 | 用于发送节点向接收节点传输数据的帧 |
| 远程帧 | 用于接收节点向具有相同ID的发送节点请求数据的帧 |
| 错误帧 | 用于当检测出错误时，向其他节点通知错误的帧 |
| 超载帧 | 用于接收节点通知其尚未做好接收准备的帧 |
| 帧间隔 | 用于将数据帧及远程帧与前面的帧分离开来的帧 |

这其中，数据帧是最常用的，也是后面实验需要用到的。数据帧由七段组成，如图 2.20.5 所示。数据帧又分为标准帧和扩展帧，主要体现在在仲裁段和控制段上。

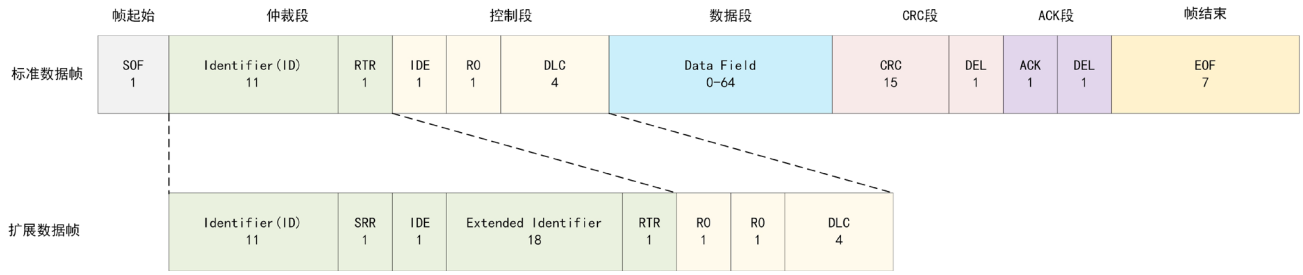


图 2.20.5 CAN 标准数据帧和扩展数据帧结构

- 帧起始(Start Of Frame-SOF)：1bit，显性信号，表示数据帧（或远程帧）的开始；
- 仲裁段(Arbitration Field)：包括标识符位（Identifier field-ID）和远程发送请求位（Remote Transfer Request，RTR）；

标准帧的 ID 位是 11 位，即范围是 0x000~0x7FF，而扩展帧的 ID 是 11+18=29 位；在 CAN 协议中，ID 决定报文的优先级高低，也决定这拓扑结构的节点是否接收此 ID 的帧数据；
远程发送请求位，用于区分该帧是数据帧还是远程帧，显性信号（0）代表数据帧（Data Frame），隐性信号（1）代表远程帧（Remote Frame）；

- 控制段（Control Field）：标准帧中由扩展标识符位（Identifier Extension bit-IDE，1 bit）、保留位 0（Reseved bit0-r0，1 bit）、数据长度编码位（Data Length Code-DLC，4 bits）组成；扩展帧用由两个保留位（Reseved bit，2 bit）、数据长度编码位（Data Length Code-DLC，4 bits）组成；
- 数据段（Data Field）：发送数据的内容，最多 8 个字节（64bit），它的实际长度会写到前面的数据长度编码位 DLC 里。
- 循环校验段（CRC Field）：包括循环校验序列（CRC Sequence）和界定符（Delimiter，DEL）；循环校验序列用于校验传输是否正确；界定符用于表示循环校验序列是否结束；
- 确认段（ACK Field）：包括确认位（ACK SLOT）和界定符（Delimiter，DEL）；确认位在节点收到正确的 CRC 序列时，发送端的 ACK 位被置位；界定符表示确认是否正常接收；
- 帧结束（End of Frame-EOF）：7 位长度，隐性信号，表示帧的结束；

CAN 协议也在不断发展，从 CAN 2.0A（经典 CAN），发展出了 CAN 2.0B（CAN-FDCAN）。CAN 2.0A，就算高速 CAN，前面提到最大传输率也不过是 1Mbps，而 CAN2.0 B 最高可达 8Mbps；CAN 2.0A，从前面可以看到数据段为 64 位，CAN 2.0B 提升到了 512 位。

STM32MP157 的 CAN 支持 CAN 2.0A 和 CAN 2.0B，都可以分配给 A7 或 M4，开发板使用的是 FDCAN2，实验为了简化，使用 CAN2.0A 标准。

## 2.20.2 硬件设计

如图 2.20.6 是 100ASK_STM32MP157 的 CAN 接口电路图， PB5、PB6 来自于集成了 CAN 控制器 MCU，通过 U17，也就是前面提到的 CAN 收发控制器，生成 CAN_H 和 CAN_L。

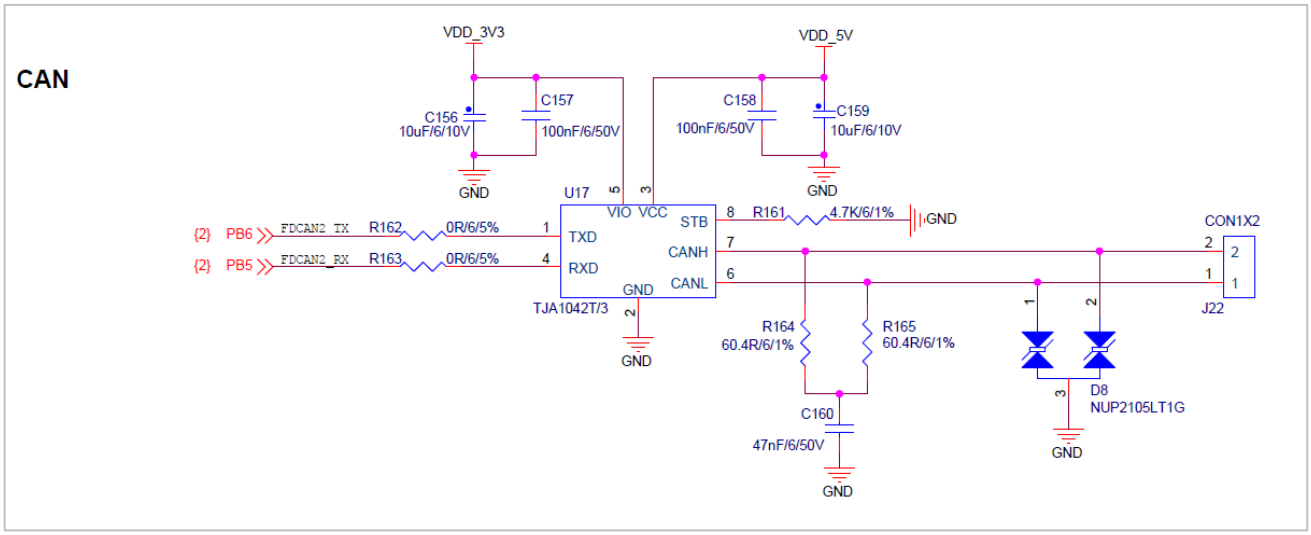


图 2.20.6 CAN 接口电路图

另外，CAN 和 RS485 都是半双工的差分信号，需要两个设备连接测试。百问网制作了一个 CAN/RS485 互转模块，可以直接连接到 100ASK_STM32MP157 上，实现 RS485 的 CAN 的透传，同时验证、学习两个接口，该模块外形如图 2.20.7 所示。

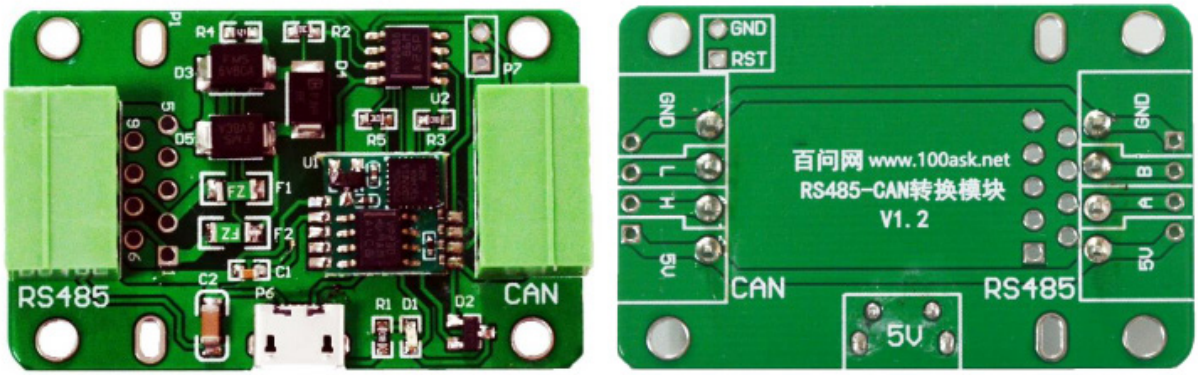


图 2.20.7 RS485_CAN 互转模块

本实验除了会用到 RS485 和 CAN 以外，还会使用调试串口打印发送情况，按键控制收发，串口打印和按键电路部分参考前面，

## 2.20.3 MX 设置

首先配置 MCU 系统时钟在 209MHz，然后配置调试串口。再参考前面 RS485 小结添加 RS485，注意 RS485_CAN 互转模块的默认波特率为 9600Bits/s，因此 USART6 的波特率要设置为 9600，如图 2.20.8 所示。

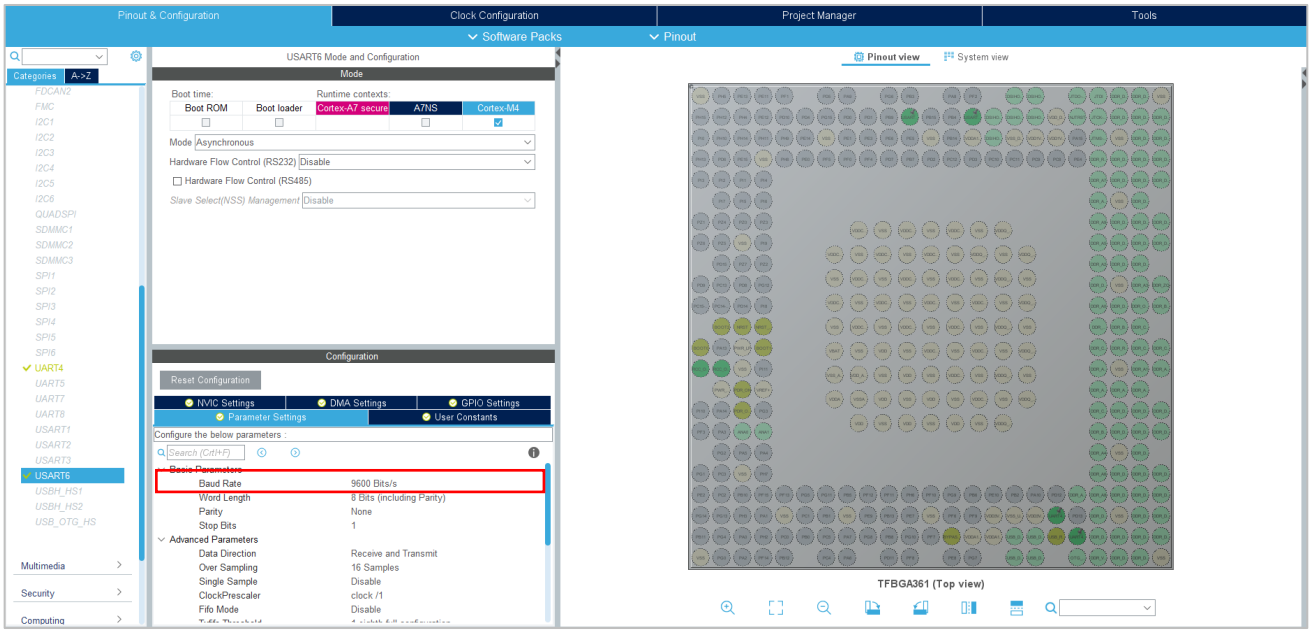


图 2.20.8 RS485 配置

此外还要添加 RS485 收发控制引脚，这个和前面的配置一样。接着参考前面按键中断实验，添加板子上的两个按键，如图 2.20.9 所示。记得这个三个 GPIO 都要设置“Pin Reservation”给“Cortex-M4”，否则不会生成初始化代码。

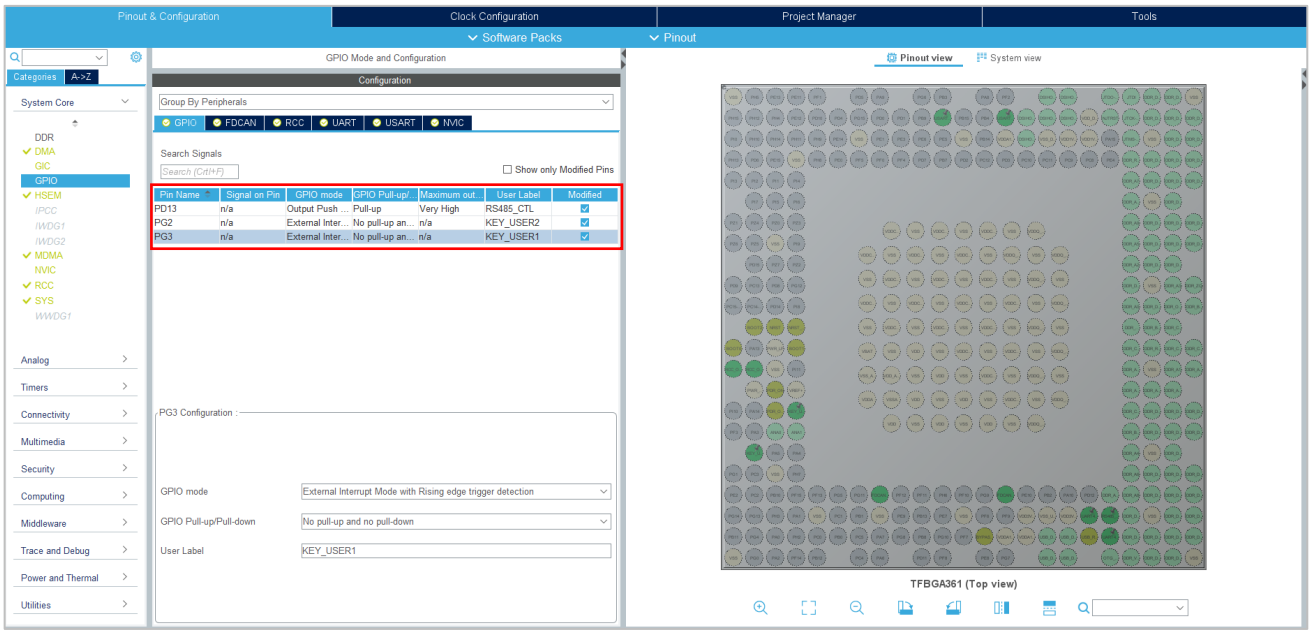


图 2.20.9 按键和 RS485 控制引脚配置

切到 FDCAN2 标签，勾选给“M4”，模式选择经典 CAN“Classic”，如图 2.20.10 所示。从时钟树可以看到 CAN 的时钟直接来自外部晶振 HSE，也就是 24MHz。由参考手册 FDCAN 部分可知，CAN 的波特率等于标称位时间的导数。

$$baudrate = 1 / bit\ time$$

$$bit\ time = tSyncSeg + tBS1 + tBS2$$

时钟预分频为 8，因此当前 CAN 波特率为 24/8/(1+3+2)=500。下方还需设置收发 FIFO，如图设置即可。

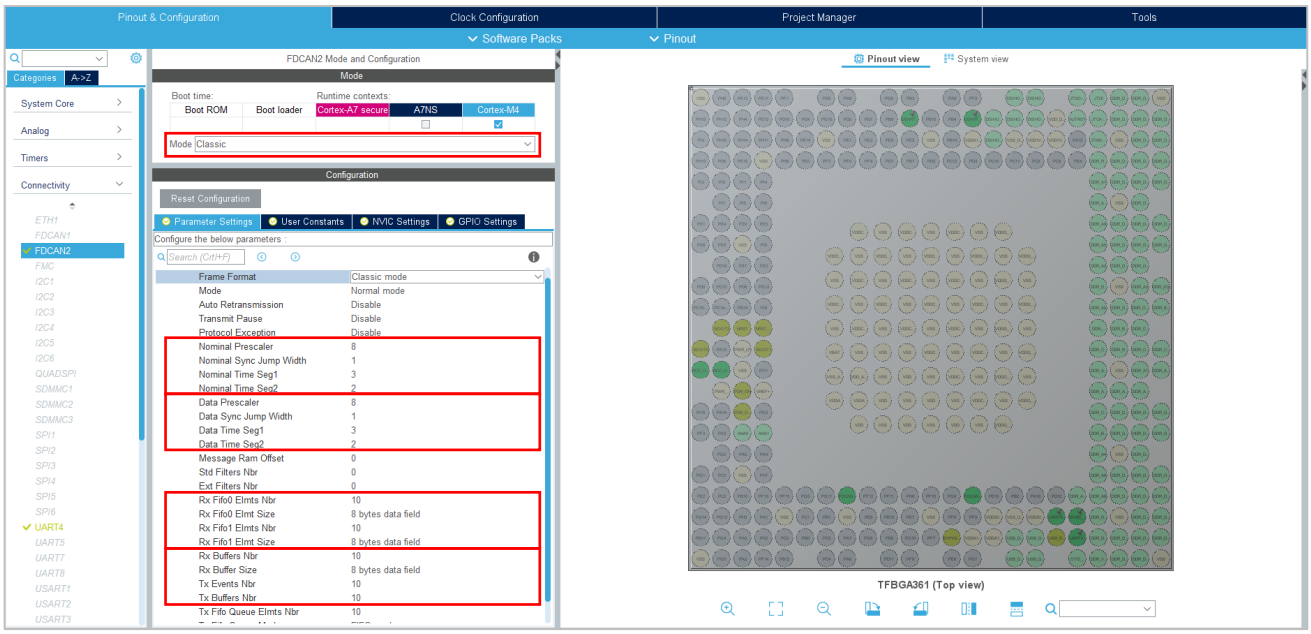


图 2.20.10 CAN 参数设置

最后使能按键中断和 FDCAN 中断，如图 2.20.11 所示。

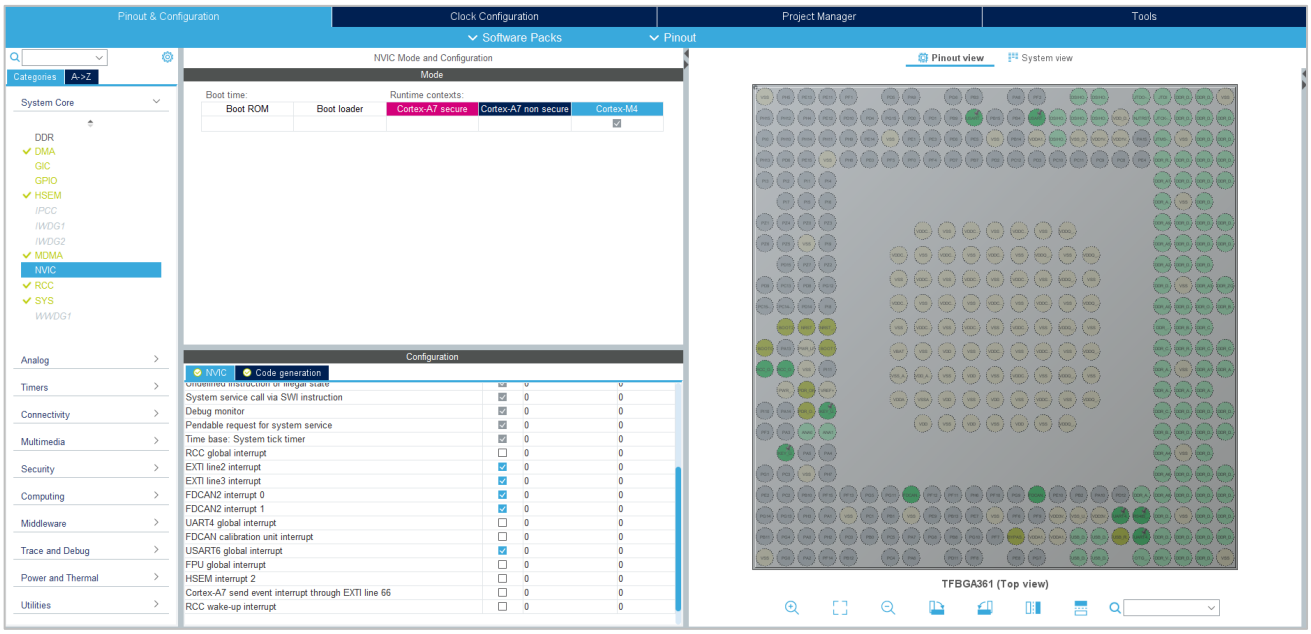


图 2.20.11 使能中断

## 2.20.4 代码设计

生成代码后，先将之前按键实验和 RS485 的“DemoDriver”目录文件复制到本工程。打开“driver_rs485.c”，调用“HAL_UART_Transmit_IT()”发送数据，调用“HAL_UART_Receive_IT()”接收数据，并在中断回调函数里修改收发标志，用于判断是否收发完成。

```
static uint8_t rs485_rx_flag = 0;
volatile uint8_t rs485_tx_flag = 0;

void RS485_TX(uint8_t *pdata, uint8_t sz)
{
  rs485_tx_flag = 0;

  // RS485 设置为输出
  DE_TX();

  // 发送数据
  HAL_UART_Transmit_IT(&huart6, (uint8_t*)pdata, sz);

  while(!RS485_GetTxFlag());
  // RS485 设置为输入
  DE_RX();
}

void RS485_RX(uint8_t *pdata, uint8_t sz)
{
```

```
  rs485_rx_flag = 0;
  HAL_UART_Receive_IT(&huart6, pdata, sz);
}

uint8_t RS485_GetTxFlag(void)
{
  return rs485_tx_flag;
}

uint8_t RS485_GetRxFlag(void)
{
  return rs485_rx_flag;
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  if(huart->Instance == USART6)
  {
    rs485_rx_flag = 1;
  }
}

void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
  if(huart->Instance == USART6)
  {
    rs485_tx_flag = 1;
  }
}
```

新增“driver_fdcan.c”文件，对于 CAN，首先要设置过滤，也就是识别仲裁段的 ID，这样就有选择的接收 CAN 总线上自己需要的数据，这里设置为全部接收，不做过滤。

```
/*
 * 此函数设置接收的过滤器
*/
void HAL_FDCAN_SetFilter(void)
{
  FDCAN_FilterTypeDef  sFilterConfig;
  /* 配置 CAN 的过滤器，此处全部接收，不做过滤 */
  sFilterConfig.IdType     = FDCAN_STANDARD_ID;       // 标准帧
  sFilterConfig.FilterIndex   = 0;
  sFilterConfig.FilterType  = FDCAN_FILTER_DUAL;
  sFilterConfig.FilterConfig  = FDCAN_FILTER_TO_RXFIFO0;
  sFilterConfig.FilterID1 = 0;  // ID1 和 ID2 都等于 0 表示都进行接收不做过滤
```

```
  sFilterConfig.FilterID2 = 0;
  if (HAL_FDCAN_ConfigFilter(&hfdcan2, &sFilterConfig) != HAL_OK)
  {
    Error_Handler();
  }

  sFilterConfig.FilterConfig = FDCAN_FILTER_TO_RXFIFO1;
  if (HAL_FDCAN_ConfigFilter(&hfdcan2, &sFilterConfig) != HAL_OK)
  {
    Error_Handler();
  }

  if (HAL_FDCAN_Start(&hfdcan2) != HAL_OK)
  {
    Error_Handler();
  }

  // 激活接收 FIFO
  if(HAL_FDCAN_ActivateNotification(&hfdcan2, FDCAN_IT_RX_FIFO0_NEW_MESSAGE, FDCAN_IT_TX_COMPLETE) != HAL_OK)
  {
    Error_Handler();
  }
}
```

这里用到了 FIFO，需要 CAN 发送时，把要发送的消息放入 FIFO 即可。

```
/*
 *  函数名：void CAN_Transmit(uint16_t ID, uint8_t *pdata, uint8_t length)
 *  输入参数：ID     -> CAN 发送报文的 ID
 *            pdata -> 发送报文的首地址
 *            length-> 发送报文的个数，最多 8 个字节
 *  输出参数：无
 *  返回值：无
 *  函数作用：CAN 发送函数
*/
void CAN_Transmit(uint16_t ID, uint8_t *pdata, uint8_t length)
{
  TxHeader.Identifier   = ID;          // 发送 ID
  TxHeader.IdType     = FDCAN_STANDARD_ID;  // 发送帧为标准 ID
  TxHeader.TxFrameType  = FDCAN_DATA_FRAME;   // 是数据帧
  TxHeader.DataLength   = FDCAN_DLC_BYTES_8;  // 长度是 8 个字节
  TxHeader.ErrorStateIndicator  = FDCAN_ESI_PASSIVE;
  TxHeader.BitRateSwitch      = FDCAN_BRS_OFF;
  TxHeader.FDFormat       = FDCAN_CLASSIC_CAN;// 传统 CAN
  TxHeader.TxEventFifoControl = FDCAN_NO_TX_EVENTS;
```

```
  TxHeader.MessageMarker = 0;
  if (HAL_FDCAN_AddMessageToTxFifoQ(&hfdcan2, &TxHeader, pdata) != HAL_OK)
  {
    Error_Handler();
  }
}
```

当 CAN 收到数据，就会调用 CAN 接收回调函数，在回调函数里调用“HAL_FDCAN_GetRxMessage()”获取数据。

```
/*
 *  函数名：void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *CanHandle)
 *  输入参数：CanHandle -> CAN 句柄
 *  输出参数：无
 *  返回值：无
 *  函数作用：CAN 接收回调函数
*/
void HAL_FDCAN_RxFifo0Callback(FDCAN_HandleTypeDef *hfdcan, uint32_t RxFifo0ITs)
{
  /* Get RX message */
  if (HAL_FDCAN_GetRxMessage(hfdcan, FDCAN_RX_FIFO0, &RxHeader, (uint8_t*)&can_rx_data.RX_Data[0]) != HAL_OK)
  {
    Error_Handler();
  }
  can_rx_data.ID = RxHeader.Identifier;
  can_rx_data.length = RxHeader.DataLength>>16;
  rx_flag = 1;
}
```

此外，想通过按键实现收发开始，这里在按键中断回调函数修改按键标志，主函数查询该标志就可知道是否有按键按下。

```
extern uint8_t test_flag;
/*
 *  函数名：void HAL_GPIO_EXTI_Callback(void)
 *  输入参数：无
 *  输出参数：无
 *  返回值：无
 *  函数作用：外部中断处理函数的回调函数，用以处理不同引脚触发的中断服务最终函数
*/
void HAL_GPIO_EXTI_Rising_Callback(uint16_t GPIO_Pin)
{
  switch(GPIO_Pin)
  {
    case KEY_USER1_Pin:
    {
```

```
      break;
    }

    case KEY_USER2_Pin:
    {
      test_flag = 1;
      break;
    }
    default:break;
  }
}
```

最后，主函数里设计控制流程。首先，让 RS485 发送 ID，CAN 接收 ID，然后 RS485 发送数据，CAN 接收数据。再反过来，CAN 发送 ID 和数据，RS485 接收。主函数的代码如下：

```
  /* USER CODE BEGIN 2 */
  HAL_FDCAN_SetFilter();
  printf("CAN Send And Receive test\r\n");
  printf("\r\n");
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    if(test_flag==1)
    {
      test_flag = 0;
      // RS485 Send ID
      // RS485->CAN
      RS485_TX((uint8_t*)&rs485_id, 2);
      printf("\r\n");
      printf("Step 0: RS485 Send ID to CAN\r\n");
      printf("RS485 Send ID 0x%x\r\n", rs485_id);

      step = 1;
    }

    switch(step)
    {
      case 0:
      {
```

```
    break;
  }
  case 1:
  {
    // Wait for CAN receive finish
    if(CAN_Receive() == 1)
    {
      CAN_SetReceiveFlag(0);
      printf("\r\n");
      printf("Step 1: CAN wait for Receive ID from RS485\r\n");
      printf("CAN Receive Message: \r\n");
      printf("\tID: 0x%x\r\n", can_rx_data.ID);
      printf("\tLength: 0x%x\r\n", can_rx_data.length);
      printf("\tData:");
      for(i=0; i<can_rx_data.length; i++)
      {
        printf("\t0x%x", can_rx_data.RX_Data[i]);
      }
      printf("\r\n");

      step = 2;
    }

    break;
  }
  case 2:
  {
    RS485_TX((uint8_t*)rs485_tx_data, 8);
    printf("\r\n");
    printf("Step 2: RS485 Send Data to CAN\r\n");
    printf("RS485 Send Data: \r\n");
    printf("\tData:");
    for(i=0; i<8; i++)
    {
      printf("\t0x%x", rs485_tx_data[i]);
    }
    printf("\r\n");

    step = 3;
  }
  case 3:
  {
    // Wait for CAN receive finish
    if(CAN_Receive() == 1)
```

```
    {
      CAN_SetReceiveFlag(0);
      printf("\r\n");
      printf("Step 3: CAN wait for Receive Data from RS485\r\n");
      printf("CAN Receive Message: \r\n");
      printf("\tID: 0x%x\r\n", can_rx_data.ID);
      printf("\tLength: 0x%x\r\n", can_rx_data.length);
      printf("\tData:");
      for(i=0; i<can_rx_data.length; i++)
      {
        printf("\t0x%x", can_rx_data.RX_Data[i]);
      }
      printf("\r\n");
      step = 4;
    }
    break;
  }
  case 4:
  {
    // CAN->RS485
    printf("\r\n");
    printf("Step 4: CAN Send Frame Data to RS485, and RS485 start receive data from CAN\r\n");
    CAN_Transmit(fdcan_id, fdcan_tx_data, 8);
    RS485_RX(rs485_rx_data, 8);
    step = 5;
    break;
  }
  case 5:
  {
    if(RS485_GetRxFlag())
    {
      step = 0;
      printf("\r\n");
      printf("Step 5: RS485 Received data from CAN finished.\r\n");
      printf("CAN Send Message:\r\n");
      printf("\tID:0x%x\r\n", fdcan_id);
      printf("\tLength:8\r\n");
      printf("\tData:");
      for(i=0; i<8; i++)
      {
        printf("\t0x%x", fdcan_tx_data[i]);
      }
      printf("\r\n");
      printf("RS485 Receive Data:\r\n");
```

```
          printf("\tData: ");
          for(i=0; i<8; i++)
          {
            printf("\t0x%x", rs485_rx_data[i]);
          }
          printf("\r\n");
        }
      }
      default:break;
      }/*End switch(step)*/
  }/* End while(1) */
  /* USER CODE END 3 */
}
```

## 2.20.5 实验效果

首先如图 2.20.12 所示连接好硬件，注意 CAN 和 RS485 图中的位置，以及线的连接情况。

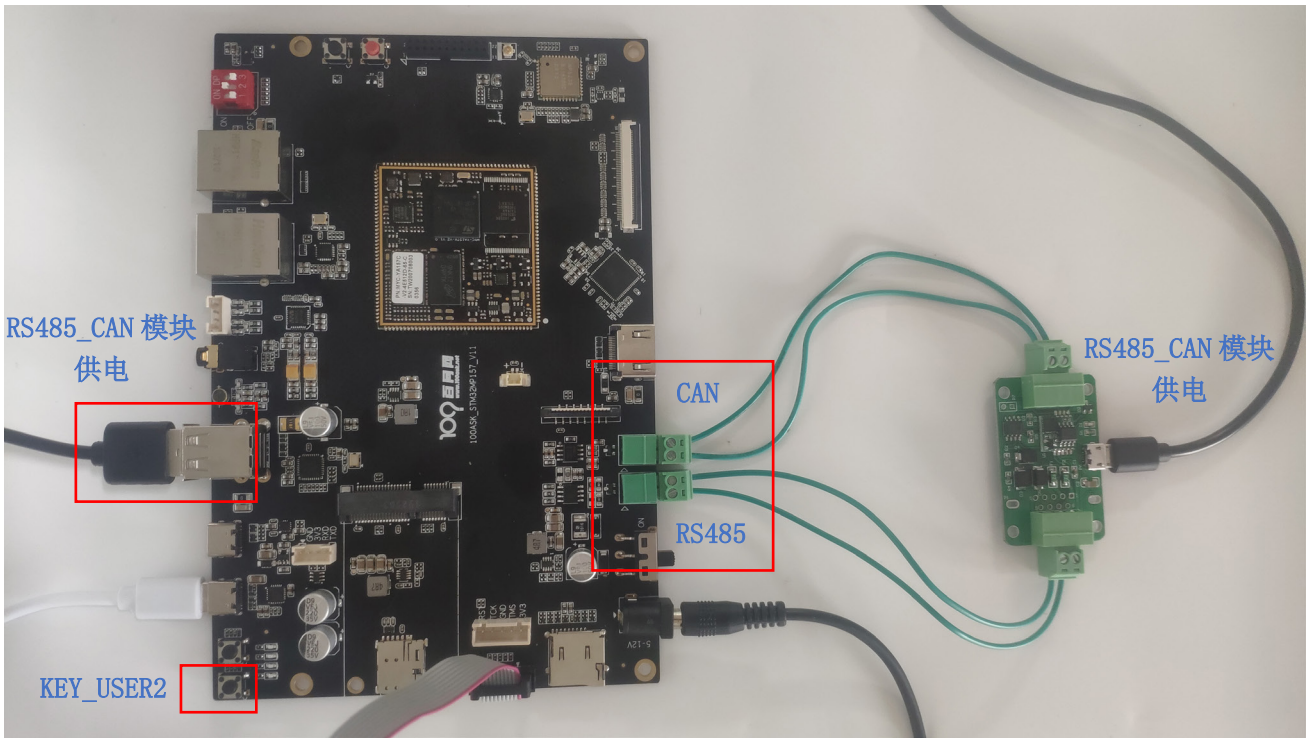


图 2.20.12 CAN 实验连接实物图

工程模式启动调试，按下按键 KEY_USER2，可以看到串口如图 2.20.13 打印。

Setp 0-1：RS485 想发送 ID，但 RS485 中没有 ID 概念，这里相当于直接发了一个数据，经过转换模块，CAN 实际收到的 ID 为模块发出的 0x00，收到两位数据，也就是 RS485 发送的 ID。

Setp 2-3：RS485 发送了 8 位数据，CAN 接到了该 8 位数据。

Setp 4-5：CAN 发送一个数据，这个数据包含 ID 和数据，经过模块转换，ID 被省略，RS485 只收到数据。

可以看出，该透传模块忽略的 ID 的发送，经过转换，实现了数据的透传。

```
Step 0: RS485 Send ID to CAN
RS485 Send ID 0x485

Step 1: CAN wait for Receive ID from RS485
CAN Receive Message:
        ID: 0x0
        Length: 0x2
        Data:   0x85    0x4

Step 2: RS485 Send Data to CAN
RS485 Send Data:
        Data:   0x1     0x2     0x3     0x4     0x5     0x6     0x7     0x8

Step 3: CAN wait for Receive Data from RS485
CAN Receive Message:
        ID: 0x0
        Length: 0x8
        Data:   0x1     0x2     0x3     0x4     0x5     0x6     0x7     0x8

Step 4: CAN Send Frame Data to RS485, and RS485 start receive data from CAN

Step 5: RS485 Received data from CAN finished.
CAN Send Message:
        ID:0x584
        Length:8
        Data:   0xa1    0xb2    0xc3    0xd4    0xe5    0xf6    0x17    0x28
RS485 Receive Data:
        Data:   0xa1    0xb2    0xc3    0xd4    0xe5    0xf6    0x17    0x28
```

图 2.20.13 CAN 实验串口打印

# 2.21 WWDG—窗口看门狗

- 设计需求

这里假设需求为使用窗口看门狗，展示“喂狗”和“不喂狗”的效果。

## 2.21.1 基础知识

MCU 可能工作在一些复杂环境，受到某些干扰可能出现程序跑飞导致死循环，看门狗的作用就是为了避免这种情况。看门狗启动后，需要在一定时间内(通过定时器计数)再给它一个信号，俗称“喂狗”，如果按时没有“喂狗”，说明 MCU 可能处于非正常状态，这时看门狗就向 MCU 发送个复位信号，使整个系统重启，进入正常的工作状态。

STM32MP157 有三个看门狗，两个独立看门狗 IWDG，只能 A7 使用，一个窗口看门狗，只能 M4 使用，因此本结使用窗口看门狗 WWDG 实验。

独立看门狗，需要在 0-t 之间喂狗，而窗口看门狗，需要在 t1-t2 之间喂狗，也就是说喂狗时间在一个上下限范围之内，不能过早，也不能过晚。

窗口看门狗是一个 7 位的递减计数器，规定它的范围为 80-127。如图 2.21.1 所示，假设设置这个递减计数器初值为 127，随着时间的递增，它的值将逐渐减少。我们还需要定一个窗口上边界，这个上边界位于下边界和递减计数器初值之间，假设为 80。那么计数器从 127 递减到 80 这个时间段，是不允许喂狗的，一旦喂狗将复位重启；80 递减到 64 这个时间段，必须喂狗；64 这个下边界是固定的，一旦递减计数器小于这个值，系统将复位。可以看到，整个过程，我们需要定义两个值，一个是递减计数器初值，一个是窗口上边界值。

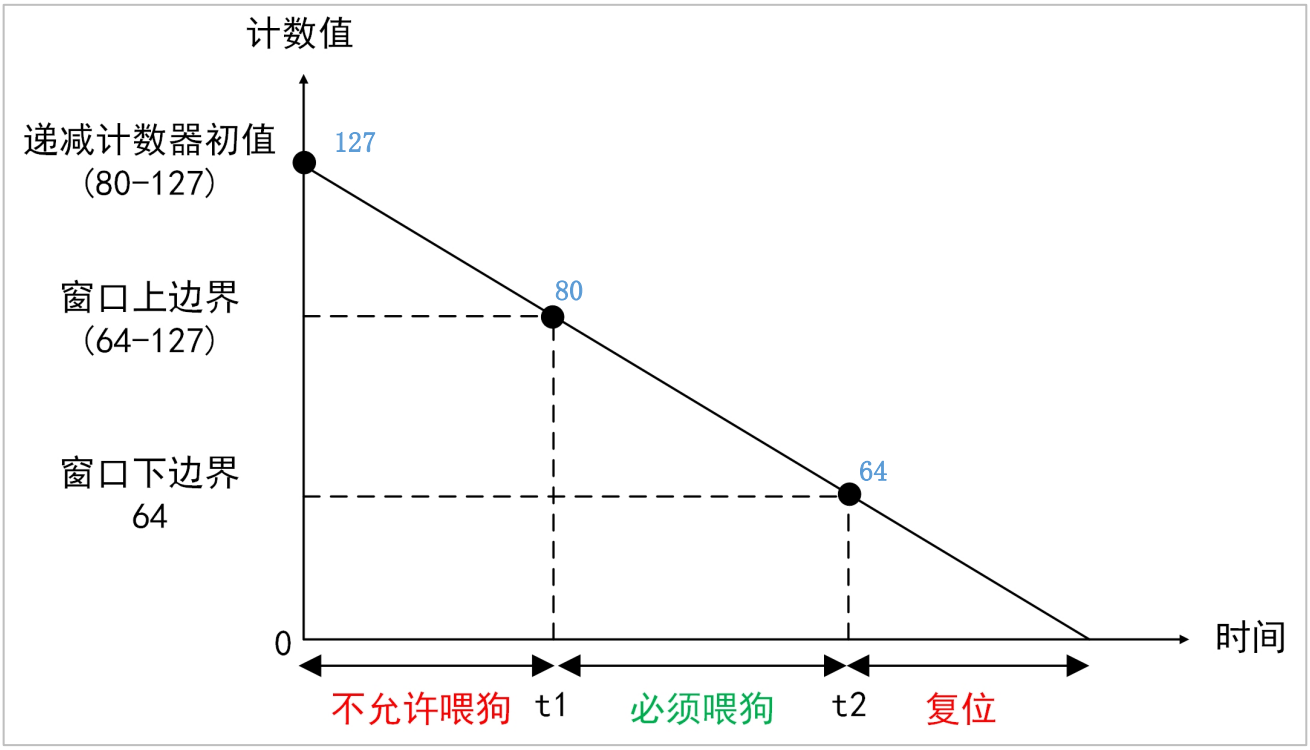


图 2.21.1 窗口看门狗工作示意图

计数器递减的速度由看门狗时钟决定，时钟越快，计数递减就越快，时钟越慢，计数递减就越慢。由前面图 2.3.4 可知，WWDG 挂在 APB1 总线上，APB 时钟给 WWDG 后，还要 4096 分频和 $2^{WDGTB}$ 分频，如图 2.21.2 所示。

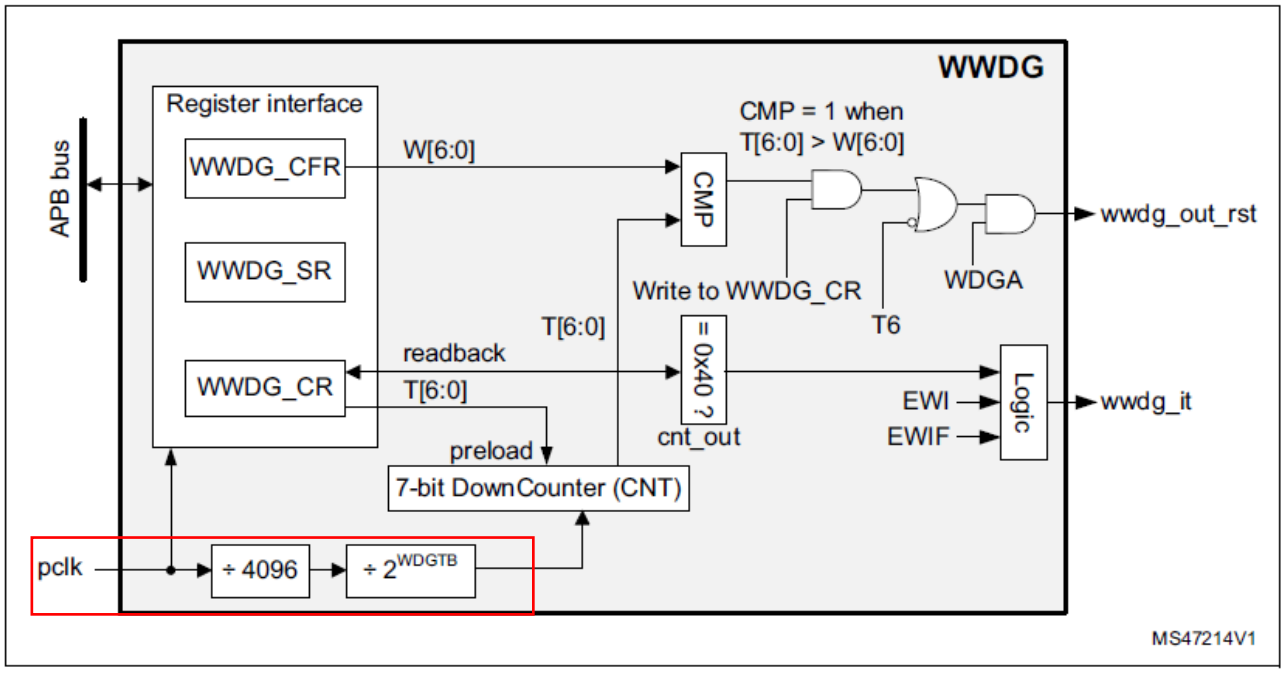


图 2.21.2 窗口看门狗时钟框图

## 2.21.2 硬件设计

不涉及电路设计。

## 2.21.3 MX 设置

先设置好 MCU 系统时钟和串口。此时 APB1 时钟为 104.5MHz,如图 2.21.3 所示。

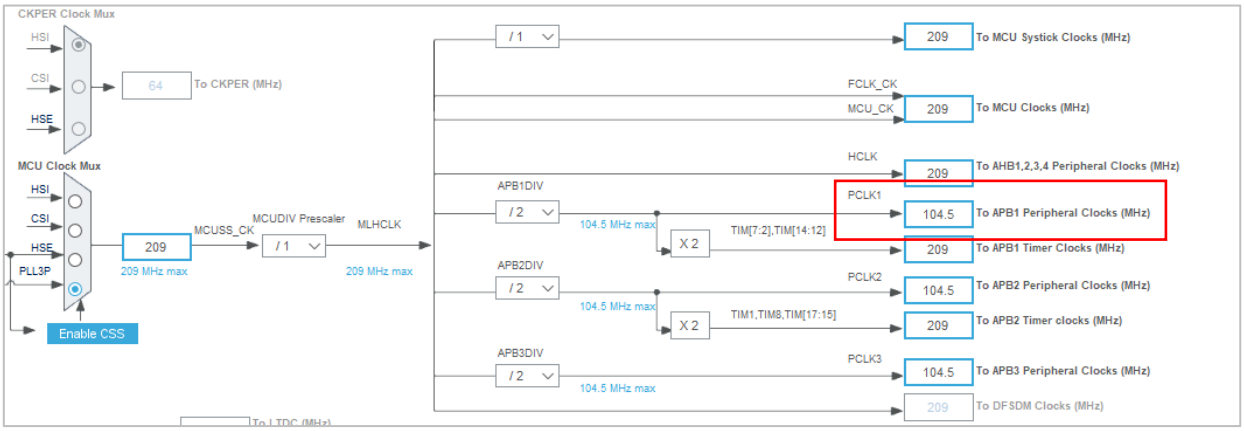


图 2.21.3 APB1 时钟

然后切到 WWDG1 标签，勾选给 M4，勾选“Activated”激活。在下面先是设置 WWDG 时钟分频，也就是前面提到的 $2^{WDGTB}$ 分频，这里设置为 128 分频，则此时 WWDG 实际时钟为 104.5MHz/4096/128≈199.3Hz，也就是每秒计数 199 个数，每个计数花费 5 毫秒。

接着是设置窗口上边界数值，这里设置为 80，最后是递减计数器初值为 127，如图 2.21.4 所示。此时（127-80）*5ms=235ms，这个时间段不能喂狗；（80-64）*5ms=80ms，这个时间段必须喂狗。

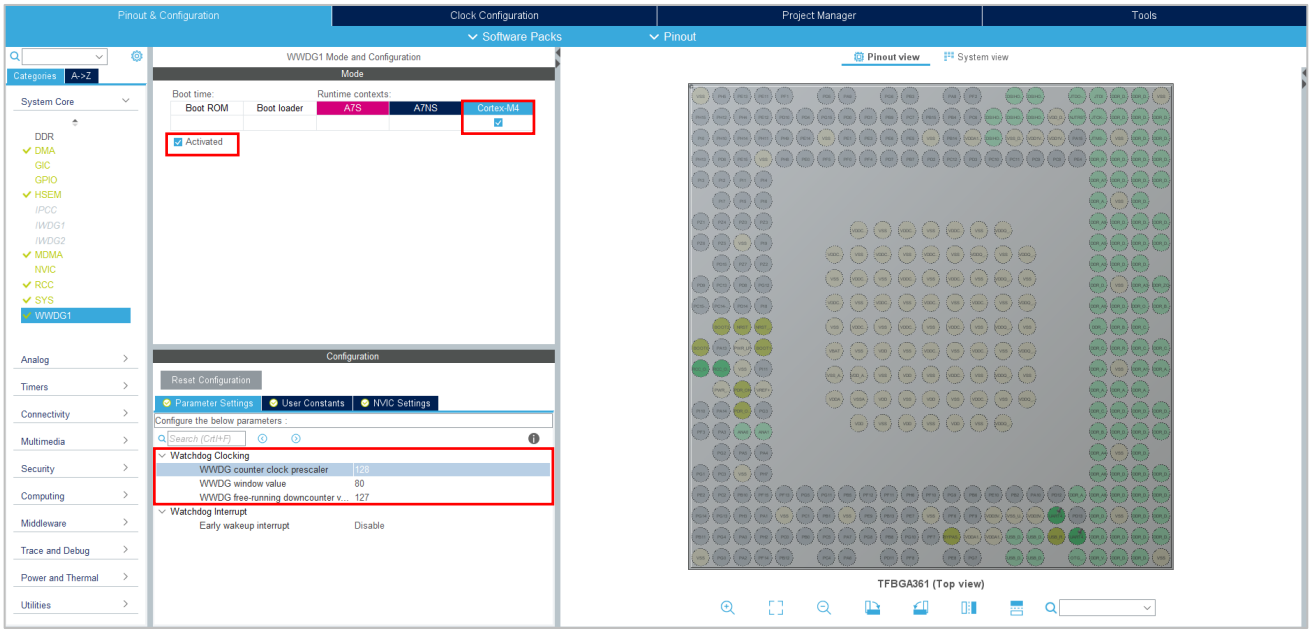


图 2.21.4 WWDG 配置

## 2.21.4 代码设计

创建“DemoDriver”目录，在里面新建“driver_wwdg.c”，调用 HAL 库的“HAL_WWDG_Refresh()”刷新 WWDG 的值，也就是喂狗。

```
/*
 *  函数名：void ClearIWDG(void)
 *  输入参数：无
 *  输出参数：无
 *  返回值：无
 *  函数作用：刷新独立看门狗的计数器，俗称"喂狗"
*/
void ClearWWDG(void)
{
    if (HAL_WWDG_Refresh(&hwwdg1) != HAL_OK)
    {
        Error_Handler();
    }
}
```

再实现一个计算窗口上边界时间的函数，验证前面的理论计算。

```
uint32_t TimeoutCalculation(uint32_t timevalue)
```

```
{
  uint32_t timeoutvalue = 0;
  uint32_t pclk1 = 0;
  uint32_t wdgtb = 0;

  /* considering APB divider is still 1, use HCLK value */
  pclk1 = HAL_RCC_GetPCLK1Freq();

  /* get prescaler */
  wdgtb = (1 << ((hwwdg1.Init.Prescaler) >> WWDG_CFR_WDGTB_Pos)); /* 2^WDGTB[1:0] */

  /* calculate timeout */
  timeoutvalue = ((4096 * wdgtb * timevalue) / (pclk1 / 1000));

  return timeoutvalue;
}
```

最后，主函数先获取窗口上边界时间，然后延时该段时间不喂狗，再之后喂狗。

```
  /* USER CODE BEGIN 2 */
  printf("\r WWDG Test\n");
  delay = TimeoutCalculation(hwwdg1.Init.Counter-hwwdg1.Init.Window); // 获取窗口上边界时间
  printf("\r delay=%ld \n", delay);
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
   HAL_Delay(delay); // 不允许喂狗
   HAL_Delay(1); // 之后的 80ms 内必须喂狗

   ClearWWDG(); //喂狗,重新计数
  }
    /* USER CODE END 3 */
```

## 2.21.5 实验效果

调试运行，串口打印如图 2.21.5 所示，显示窗口上边界时间为 235，和前面理论计算一致。此外，读者可以尝试再小于 235ms 内喂狗，或者 235+80ms 后喂狗，测试系统是否会复位。

图 2.21.5 WWDG 串口打印效果

# 第3章 M4 高级开发示例

## 3.1 FreeRTOS

- 设计需求

这里假设需求为M4运行FreeRTOS，执行两个闪烁LED任务，互不干扰。

### 3.1.1 基础知识

在嵌入式领域，操作系统分两类：实时操作系统(Real Time Operating System，RTOS)和通用操作系统(General Purpose Operating System)。

常见的实时操作系统有VxWorks、QNX、uCOS、FreeRTOS、RT-Thread、LiteOS等，实时操作系统最大的特点就是**给定任务(Task)可以在给定的时间(Deadline)内完成**，在一些场景，比如汽车安全气囊、军工航天，对任务完成时间都有明确要求，没按时完成就会导致严重后果。

常见的通用操作系统有Linux、Windows、macOS等，作为通用操作系统将延时降低到一个用户可以接受的范围，提供更丰富的功能。比如日常使用电脑，当电脑运行程序比较多的时候，鼠标卡顿一下，用户是可以接受的，而实时操作系统不允许出现这种情况，也就限制了实时操作系统的应用场景。

FreeRTOS是一个小巧简易的实时操作系统内核，整个核心代码只有3到4个C文件，但它却实现了多线程（threads）、多任务（task）、互斥锁（mutex）、信号量（semaphore）和软件定时器（software timer）等。FreeRTOS操作系统是完全免费的操作系统，具有源码公开、可移植、可裁减、调度策略灵活的特点，可以方便地移植到各种MCU上运行。

STM32CubeMX已经移植好了FreeRTOS，现在只需要在软件里勾选上，即可运行FreeRTOS。

### 3.1.2 硬件设计

FreeRTOS不涉及硬件，为了展示效果，会用到开发板的外部时钟HSE、UART4以及两个LED灯，相关硬件电路参考前面实验。

### 3.1.3 MX 设置

配置好MCU时钟、UART4、LED灯后，切换到中间件“Middleware”的“FREERTOS”，默认自动勾选给M4了，然后接口“Interface”选择“CMSIS_V2”。初次接触，下面的参数大部分默认即可，为了运行其它任务，这里需要切换到“Tasks and Queues”标签，可以看到默认有个“defaultTask”任务，我们再点击“Add”再新增一个任务，如所示。

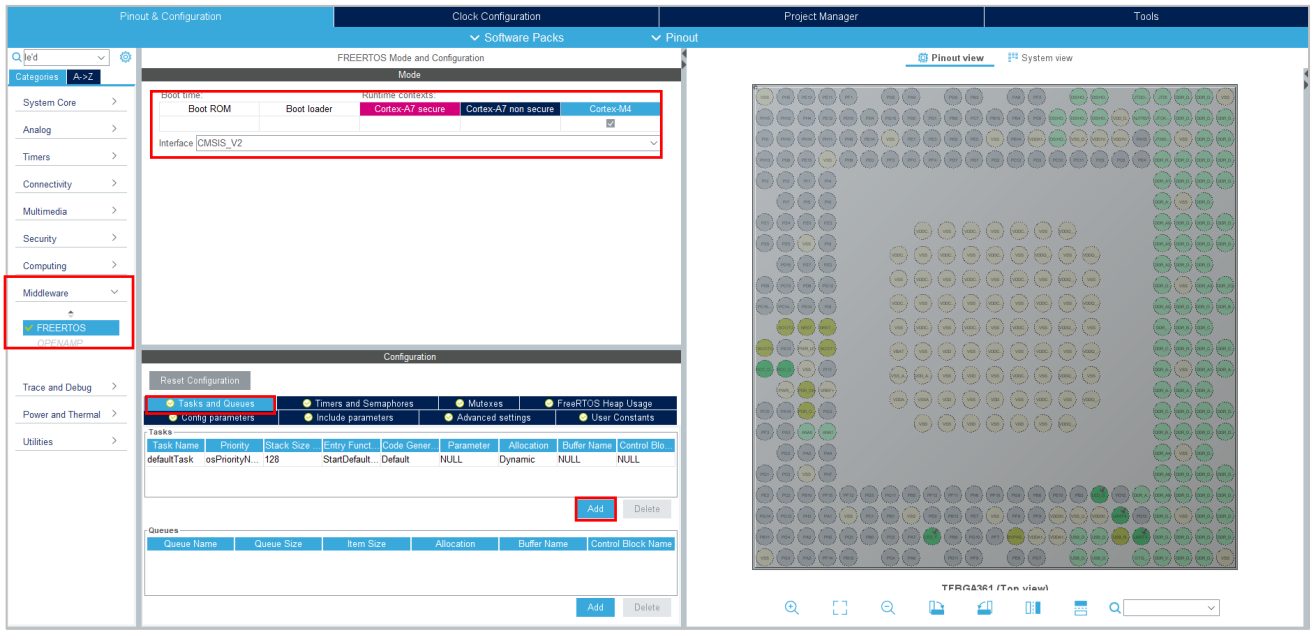


图 3.1.1 FreeRTOS 配置

随后弹出编辑任务界面，如图 3.1.2 所示，默认任务名字为“myTask02”，优先级默认低，栈大小 128 字节，进入的函数名为“StartTask02”，代码生成选项等后面全默认即可。

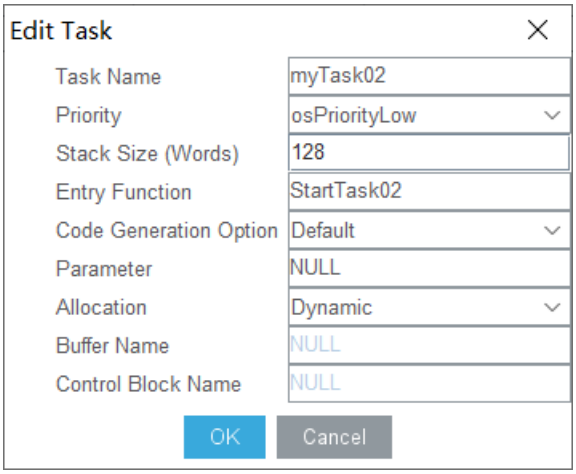


图 3.1.2 编辑新增任务

随后生成初始化代码，会弹出 Systick 被 FreeRTOS 占用，建议 HAL 库时钟基准源换成其它，如图 3.1.3 所示。

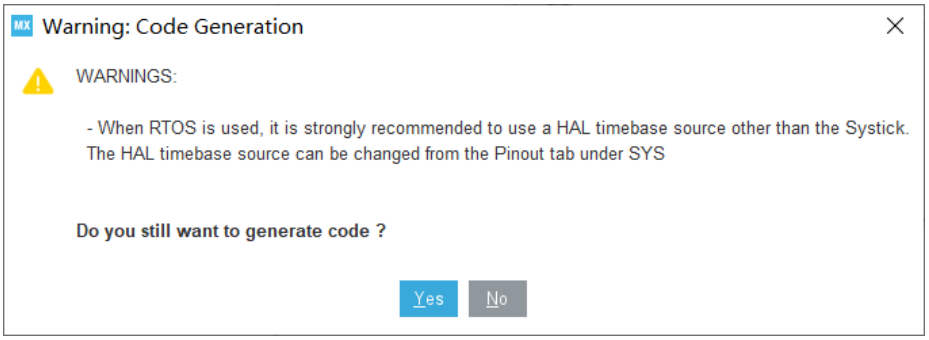


图 3.1.3 代码生成警告

### 3.1.4 代码设计

生成代码后，找到“main.c”，结尾部分会有两个任务函数，分别是“StartDefaultTask()”和“StartTask02()”。两个任务宏观看同时、独立运行，互不干扰，这里在两个任务里加入打印和加入不同时间间隔的闪烁灯。

```
/* USER CODE BEGIN Header_StartDefaultTask */
/**
  * @brief  Function implementing the defaultTask thread.
  * @param  argument: Not used
  * @retval None
  */
/* USER CODE END Header_StartDefaultTask */
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN 5 */
    printf("\rFreeRTOS Task1 run\n");
  /* Infinite loop */
  for(;;)
  {
    LED_GREEN_ON();
    osDelay(1000);
    LED_GREEN_OFF();
    osDelay(1000);
  }
  /* USER CODE END 5 */
}

/* USER CODE BEGIN Header_StartTask02 */
/**
* @brief Function implementing the myTask02 thread.
* @param argument: Not used
* @retval None
*/
/* USER CODE END Header_StartTask02 */
```

```
void StartTask02(void *argument)
{
  /* USER CODE BEGIN StartTask02 */
    printf("\rFreeRTOS Task2 run\n");
  /* Infinite loop */
  for(;;)
  {
    LED_YELLOW_ON();
    osDelay(2000);
    LED_YELLOW_OFF();
    osDelay(2000);
  }
  /* USER CODE END StartTask02 */
   }
```

### 3.1.5 实验效果

调试运行，串口打印如图 3.1.4 所示。串口同时打印了两个任务信息，同时开发板两个 LED 灯，闪烁速度不一致，可以看出它们各自独立运行。

图 3.1.4 FreeRTOS 运行效果

# 3.2 A7_M4 用户态通信

- 设计需求

这里假设需求为 A7 和 M4 同时运行，然后 A7 控制 M4 操作 LED 灯，A7 和 M4 的相互唤醒。

## 3.2.1 基础知识

开放式非对称多处理（Open Asymmetric Multi-Processing，OpenAMP）是一种通用的抽象框架，提供一系列 API，实现了构成系统的异构处理器之间上电、加载固件、断电及共享信息（通信）。

OpenAMP 包含了两个关键组件：

- Remoteproc（Remote Processor Framework）：管理框架；
- RPMsg（Remote Processor Messaging Framework）：消息传递框架；

Remoteproc 主要作用就是对远程处理器的生命周期进行管理（life cycle management，LCM），即启动、停止远程处理器。前面加载运行 M4 固件已经体验过了，Cortex-A 内核先启动，然后使用 Linux RemoteProc 框架进行加载 Cortex-M4 固件，启动 M4 内核。STM32MP1 的 Remoteproc 框架如图 3.2.1 所示。

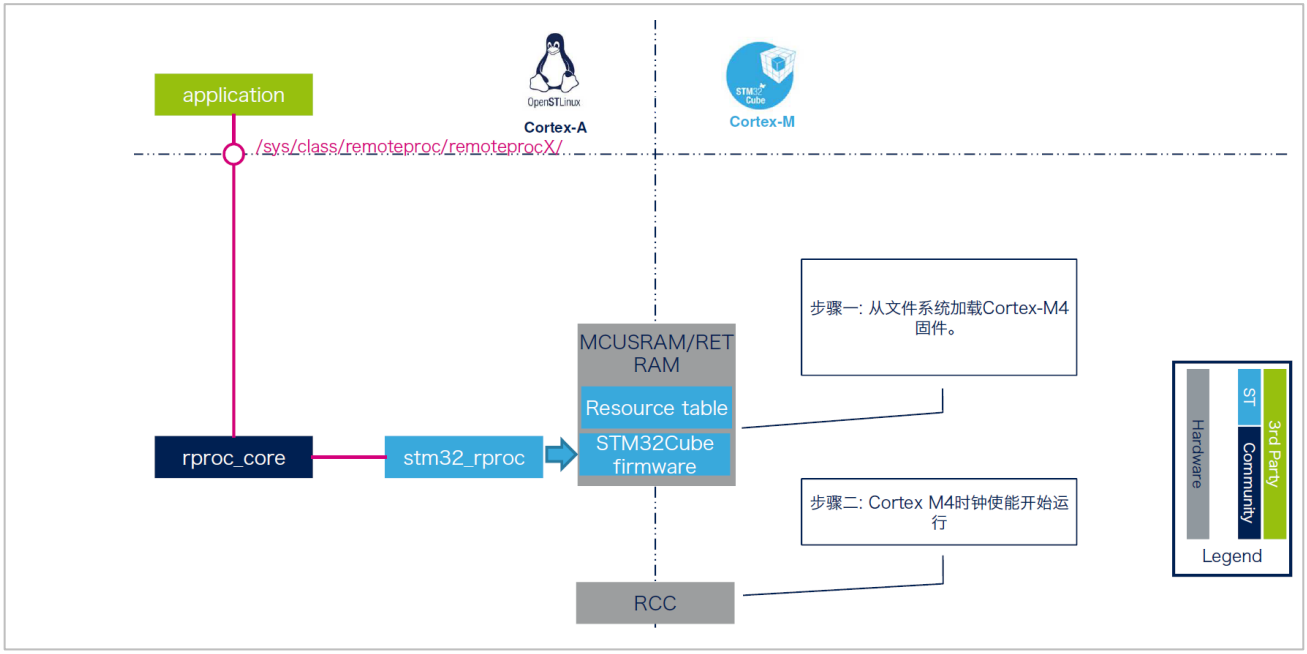


图 3.2.1 Remoteproc 框架

ST 官方提供的内核已经默认配置了 Remoteproc 驱动，进入系统后，首先将要运行的 M4 固件放在“/lib/firmware/”目录下，然后将固件名字写到“/sys/class/remoteproc/remoteproc0/firmware”，再操作“/sys/class/remoteproc/remoteproc0/state”启动、停止 M4 处理器。

```
[root@100ask:~]$ ls /lib/firmware/  //elf固件存放目录
DEMO_LED_CM4.elf
[root@100ask:~]$ echo GPIO_LED_CM4.elf > /sys/class/remoteproc/remoteproc0/firmware  //加载固件
[root@100ask:~]# cat /sys/class/remoteproc/remoteproc0/state  //查看固件状态
Offline  //固件离线状态
[root@100ask:~]# echo start > /sys/class/remoteproc/remoteproc0/state  //启动M4
```

```
[22683.222322] remoteproc remoteproc0: powering up m4
[22683.229097] remoteproc remoteproc0: Booting fw image GPIO_LED_CM4.elf, size 1899976
[22683.235549] remoteproc remoteproc0: header-less resource table
[22683.241235] remoteproc remoteproc0: not resource table found for this firmware
[22683.248749] remoteproc remoteproc0: header-less resource table
[22683.254414] remoteproc remoteproc0: remote processor m4 is now up  //远程M4已启动
[root@100ask:~]# echo stop > /sys/class/remoteproc/remoteproc0/state  //停止M4
[22709.281733] remoteproc remoteproc0: warning: remote FW shutdown without ack
[22709.287325] remoteproc remoteproc0: stopped remote processor m4  //远程M4已停止
```

除了在 Linux 的用户态控制 M4 内核的生命周期，还能在 Linux 内核态使用 API 控制(参考 linux-origin_master/Documentation/remoteproc.txt)，甚至 U-boot 中控制。

RPMsg 是处理器间的消息传输总线，其中每个处理器都是总线上的器件。它允许内核驱动程序与系统上的远程处理器进行通信，同时，驱动程序可以根据需要公开适当的用户空间接口(参考 linux-origin_master/Documentation/rpmsg.txt)。RPMsg 使用虚拟 I/O（Virtio）组件，Virtio 可提供虚拟 I/O 服务，以支持主处理器与远程处理器之间的通信。

STM32MP1 多核通信框架如图 3.2.2 所示。一边是运行 Linux 的 A7，基于 RPMsg 服务面向 Linux 用户空间提供“/dev/ttyRPMSG”串口节点收发数据；另一边是运行裸机或 RTOS 的 M4，基于 OpenAMP 库调用虚拟串口函数“virtual_hal_uart()”收发数据。从两边应用角度来看，如同串口透传，类似 A7 和 M4 通过串口连接；从两边硬件角度来看，信号通知(Mailbox)服务基于内部 IPCC(Inter-Processor communication controller)，数据传输基于共享内存。

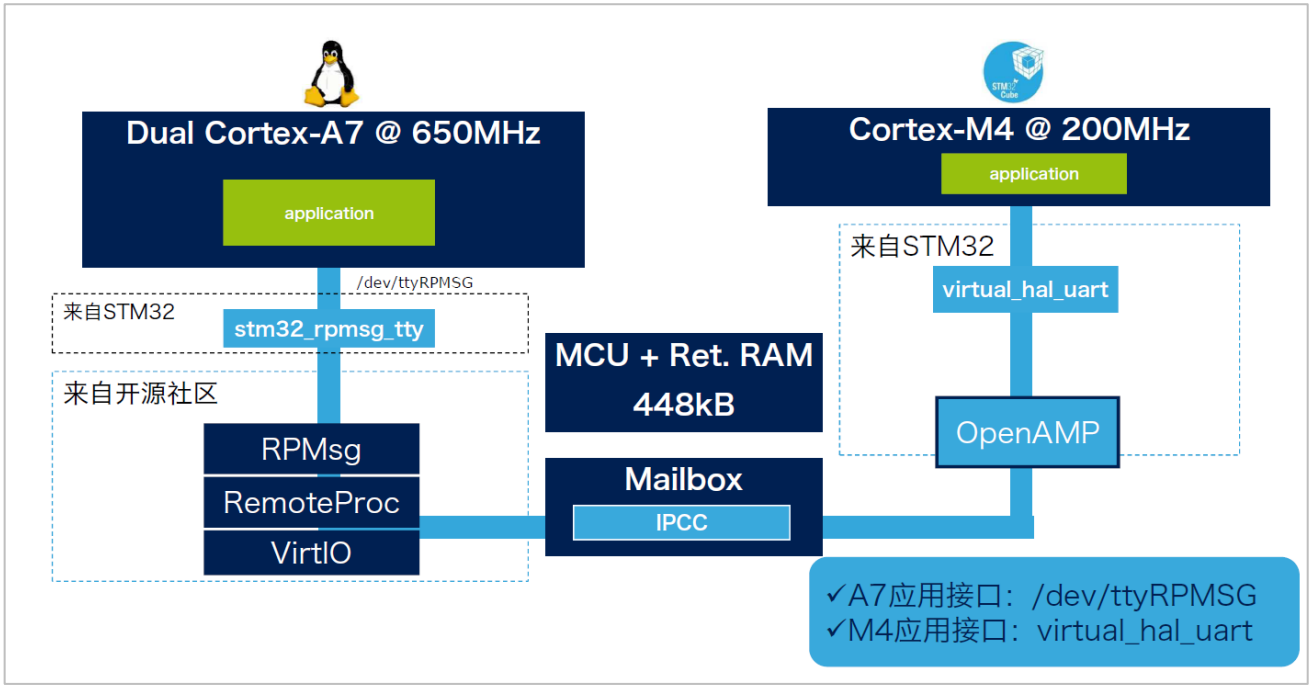


图 3.2.2 STM32MP1 多核通信框架图

### 3.2.2 硬件设计

硬件外设需要用到一个 LED 灯给 M4，A7 使用 UART4，M4 使用 UART8。UART 是开发板默认调试串口，有串口转 USB 芯片，用户可以直接通过 Type-C 连接电脑，UART8 是在扩展接口，对外提供的是 TTL 电平，需要用户自行准备 TTL 转 USB 模块连接电脑。

### 3.2.3 MX 设置

首先配置 MCU 系统时钟为 209MHz，然后配置 LED_YELLOW 给 M4，如图 3.2.3 所示配置。

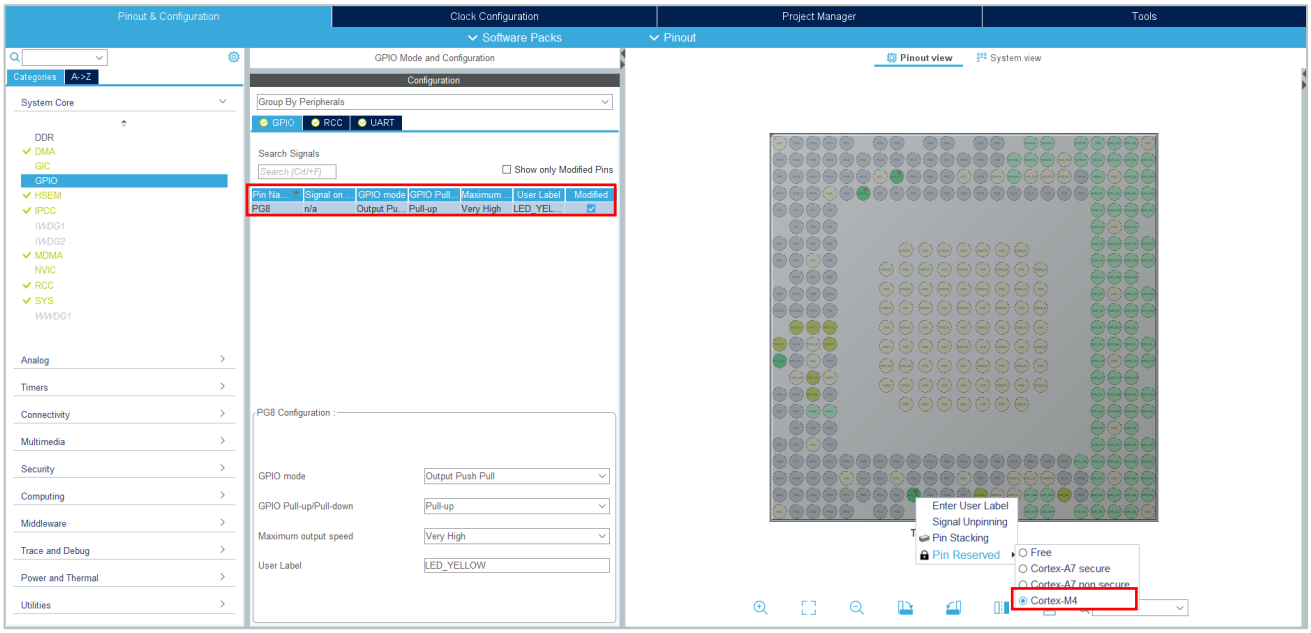


图 3.2.3 LED 灯配置

然后将 UART8(PE0、PE1)分配给 M4，用于 M4 打印，如图 3.2.4 所示。

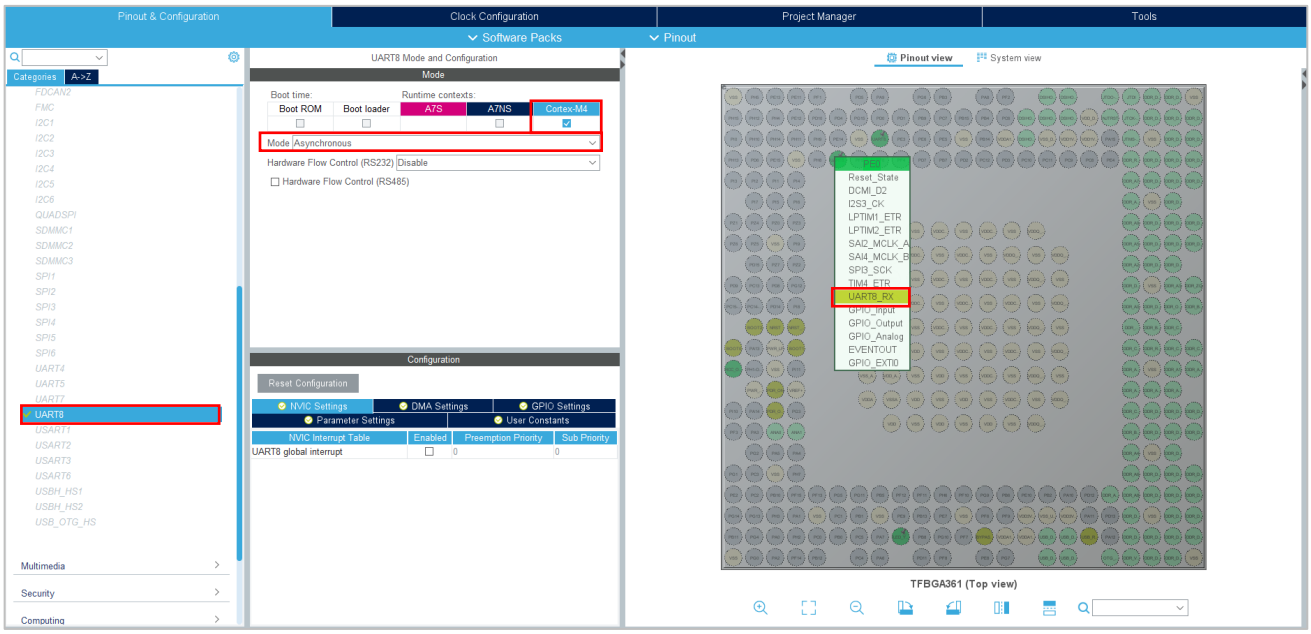


图 3.2.4 UART8 配置

然后配置 IPPC，同时给 A7 和 M4，并且使能中断，如图 3.2.5 所示。

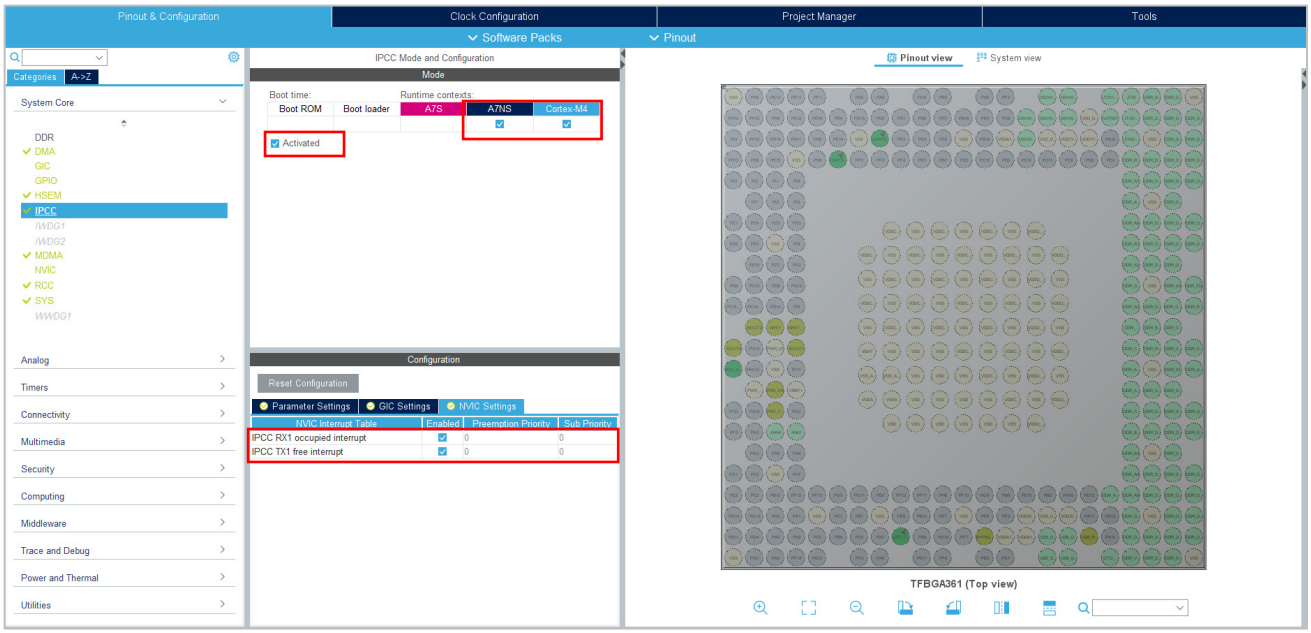


图 3.2.5 IPCC 配置

最后，勾选 OPENAMP 给 M4，如图 3.2.6 所示。

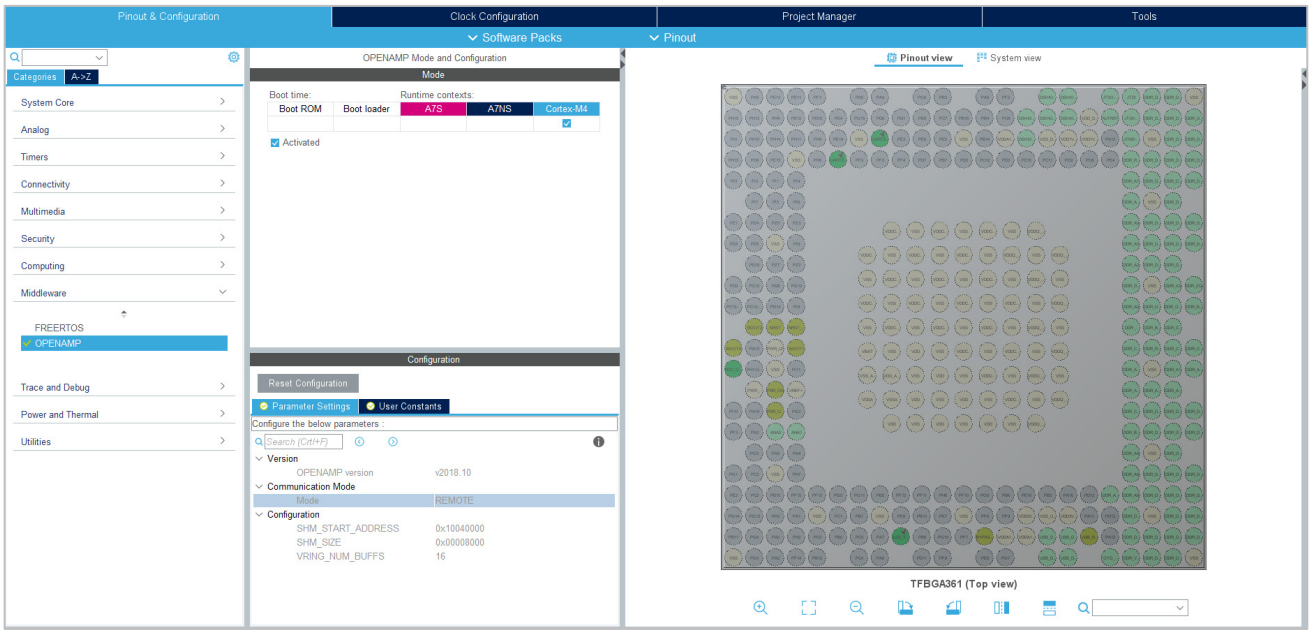


图 3.2.6 OPENAMP 配置

## 3.2.4 代码设计

生成初始化代码后，在主函数末尾附近，找到“USER CODE BEGIN 4”和“USER CODE END 4”，在之间添加 printf 的重定向函数，让 UART8 与 printf 绑定。

```
/* USER CODE BEGIN 4 */
#ifdef __GNUC__

#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)

PUTCHAR_PROTOTYPE
{

  HAL_UART_Transmit(&huart8, (uint8_t*)&ch, 1, HAL_MAX_DELAY);
  return ch;
}
#endif/* USER CODE END 4 */
```

这里计划创建两个 RPMsg tty 通道，一个用来 LED 控制命令，一个用来传输唤醒命令。

- Setp 1：初始化两个 RPMsg tty 虚拟串口

```
  if (VIRT_UART_Init(&huart0) != VIRT_UART_OK) {
     printf("VIRT_UART_Init UART0 failed.\r\n");
    Error_Handler();
  }
  if (VIRT_UART_Init(&huart1) != VIRT_UART_OK) {
     printf("VIRT_UART_Init UART1 failed.\r\n");
    Error_Handler();
```

```
}
```

- Setp 2：注册回调函数以按通道接收消息

```
if(VIRT_UART_RegisterCallback(&huart0, VIRT_UART_RXCPLT_CB_ID, VIRT_UART0_RxCpltCallback) != VIRT_UART_OK)
{
 Error_Handler();
}
if(VIRT_UART_RegisterCallback(&huart1, VIRT_UART_RXCPLT_CB_ID, VIRT_UART1_RxCpltCallback) != VIRT_UART_OK)
{
 Error_Handler();
 }
```

- Setp 3：编写虚拟串口回调函数

当 RPMsg 收到数据后，将调用该回调函数。在此函数里，需要将接收的数据复制到用户内存，并修改接收标志位，通知用户完成数据接收。

```
void VIRT_UART0_RxCpltCallback(VIRT_UART_HandleTypeDef *huart)
{
    printf("Msg received on VIRTUAL UART0 channel:  %s \r\n", (char *) huart->pRxBuffPtr);
    /* copy received msg in a variable to sent it back to master processor in main infinite loop*/
    VirtUart0ChannelRxSize = huart->RxXferSize < MAX_BUFFER_SIZE? huart->RxXferSize : MAX_BUFFER_SIZE-1;
    memcpy(VirtUart0ChannelBuffRx, huart->pRxBuffPtr, VirtUart0ChannelRxSize);
    VirtUart0RxMsg = SET;
}
void VIRT_UART1_RxCpltCallback(VIRT_UART_HandleTypeDef *huart)
{
    printf("Msg received on VIRTUAL UART1 channel:  %s \r\n", (char *) huart->pRxBuffPtr);
    /* copy received msg in a variable to sent it back to master processor in main infinite loop*/
    VirtUart1ChannelRxSize = huart->RxXferSize < MAX_BUFFER_SIZE? huart->RxXferSize : MAX_BUFFER_SIZE-1;
    memcpy(VirtUart1ChannelBuffRx, huart->pRxBuffPtr, VirtUart1ChannelRxSize);
    VirtUart1RxMsg = SET;
}
```

- Setp 4：主函数轮询 RPMsg 消息

“OPENAMP_check_for_message()”查询 MailBox 状态。

当收到数据时，“VIRT_UARTx_RxCpltCallback()”会保存好收到数据，然后修改 VirtUartxRxMsg 标志位。主函数里发现 VirtUartxRxMsg 标志位发生变化时，即可获取接收的数据。

```
while (1)
{
   OPENAMP_check_for_message();
  /* USER CODE END WHILE */
  /* USER CODE BEGIN 3 */
  if (VirtUart0RxMsg)
  {
    VirtUart0RxMsg = RESET;
```

```
    /*VirUART0 收到数据*/
  }
  if (VirtUart1RxMsg)
  {
    VirtUart1RxMsg = RESET;
    /*VirUART1 收到数据*/
  }
}
```

- Setp 5：VirUART0 接收控制 LED 指令

每次 VirtUart0RxMsg 发生变化，说明 VirUART0 收到了数据,然后比较收到的数据内容，执行对应的操作。

这里,M4 收到“MSG_LED_ON(*led_on)”则打开 LED 灯,并发送消息给 A7;M4 收到“MSG_LED_OFF(*led_off)”则关闭 LED 灯，并发送消息给 A7。

```
    if (VirtUart0RxMsg)
    {
      VirtUart0RxMsg = RESET;
      if (!strncmp((char *)VirtUart0ChannelBuffRx, MSG_LED_ON, strlen(MSG_LED_ON)))
      {
          strcpy((char *)BuffTx, "m4:led on\n");
          printf("%s\r", BuffTx);
          VIRT_UART_Transmit(&huart0, BuffTx, strlen((const char *)BuffTx));
          HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_RESET);
      }
      if (!strncmp((char *)VirtUart0ChannelBuffRx, MSG_LED_OFF, strlen(MSG_LED_OFF)))
      {
          strcpy((char *)BuffTx, "m4:led off\n");
          printf("%s\r", BuffTx);
          VIRT_UART_Transmit(&huart0, BuffTx, strlen((const char *)BuffTx));
          HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_SET);
      }
      memset(VirtUart0ChannelBuffRx, 0 ,VirtUart0ChannelRxSize);
      memset(BuffTx, 0 ,strlen((const char *)BuffTx));
    }
```

- Setp 6：VirUART1 接收休眠唤醒指令

每次 VirtUart1RxMsg 发生变化，说明 VirUART1 收到了数据，然后比较收到的数据内容，执行对应的操作。

这里，M4 收到“MSG_STOP(*stop)”则进入 CStop 模式，中途 A7 再发任意数据给 M4，由于 IPCC 也可设置为中断唤醒源，将 M4 唤醒；M4 收到“MSG_DELAY(*delay)”则等待 20S 后发数据给 A7，在这 20S 内，将 A7 先休眠，随后将被 M4 唤醒。

```
    if (VirtUart1RxMsg)
    {
      VirtUart1RxMsg = RESET;
```

```
if (!strncmp((char *)VirtUart1ChannelBuffRx, MSG_STOP, strlen(MSG_STOP)))
{
    strcpy((char *)BuffTx, "m4:stop\n");
    printf("%s\r", BuffTx);
    VIRT_UART_Transmit(&huart1, BuffTx, strlen((const char *)BuffTx));
    //RCC_backupClocks();
    /* Clear the MCU flags before going into CSTOP */
    SET_BIT(PWR->MCUCR, PWR_MCUCR_CSSF);
    printf("Going to CStop mode\r\n");
    /* (C)STOP protection mechanism
     * Only the IT with the highest priority (0 value) can interrupt.
     * RCC_WAKEUP_IRQn IT is intended to have the highest priority and to be the
     * only one IT having this value
     * RCC_WAKEUP_IRQn is generated only when RCC is completely resumed from
     * CSTOP */
     __set_BASEPRI(1 << (8 - __NVIC_PRIO_BITS));
     HAL_PWR_EnterSTOPMode(PWR_MAINREGULATOR_ON, PWR_STOPENTRY_WFI);
     /* To allow Systick to increment after CSTOP (Eg.: to not block during
      * TIMEOUT routines), TICK_INT_PRIORITY < BASEPRI
      * For this example as TICK_INT_PRIORITY = 1, BASEPRI should be 2  */
     __set_BASEPRI(2 << (8 - __NVIC_PRIO_BITS));
     printf("Leaving CStop mode\r\n");
     /* Test if system was on STOP mode */
     if( (PWR->MCUCR & PWR_MCUCR_STOPF) == PWR_MCUCR_STOPF)
     {
        printf("System was on STOP mode\r\n");
        /* Clear the MCU flags */
        SET_BIT(PWR->MCUCR, PWR_MCUCR_CSSF);
        /* Restore clocks */
        /*
        if (RCC_restoreClocks() == HAL_OK)
        {
            printf("CM4 restored clocks successfully\r\n");
        }
        */
      }
      /* All level of ITs can interrupt */
      __set_BASEPRI(0U);
}
if (!strncmp((char *)VirtUart1ChannelBuffRx, MSG_DELAY, strlen(MSG_DELAY)))
{
    printf("Waiting 20 secs before sending the answer message\r\n");
    HAL_Delay(20 *1000);
    strcpy((char *)BuffTx, "m4:wakeup A7\n");
```

```
        printf("%s\r", BuffTx);
        VIRT_UART_Transmit(&huart1, BuffTx, strlen((const char *)BuffTx));
    }
    memset(VirtUart1ChannelBuffRx, 0 ,VirtUart1ChannelRxSize);
    memset(BuffTx, 0 ,strlen((const char *)BuffTx));
  }
```

为了 A7 能发消息将 M4 唤醒，还需要 IPCC 作为 M4 的中断唤醒源。

```
EXTI_ConfigTypeDef EXTI_ConfigStructure;
EXTI_HandleTypeDef hexti62;
/*
 * Set configuration of Exti line 62 (IPCC interrupt CPU2). It could be used to wakeup the
 * M4 from CStop mode when RPMsg received from Cortex-A7
 */
EXTI_ConfigStructure.Line = EXTI_LINE_62;
EXTI_ConfigStructure.Mode = EXTI_MODE_C2_INTERRUPT;
//PERIPH_LOCK(EXTI);
HAL_EXTI_SetConfigLine(&hexti62, &EXTI_ConfigStructure);
//PERIPH_UNLOCK(EXTI);
/*
 *     Enable RCC_IT_WKUP to exit M4 from CStop mode.
 *     Indeed, due to SOC issue, M4 firmware shall make sure
 *     RCC_WAKEUP interrupt is the first one used to exit M4 from CStop mode.
 *     Therefore, M4 masks all NVIC interrupts with priority higher than 0
 *     before entering CStop mode and unmasks them when moving from WFI.
 *     (in HAL_PWR_EnterSTOPMode function)
 *     Note: All other NVIC interrupts shall be set to a different value
 *     from 0 to make sure that this workaround works well.
 */
    __HAL_RCC_ENABLE_IT(RCC_IT_WKUP);
```

### 3.2.5 实验效果

编译程序，如图 3.2.7 所示连接开发板。

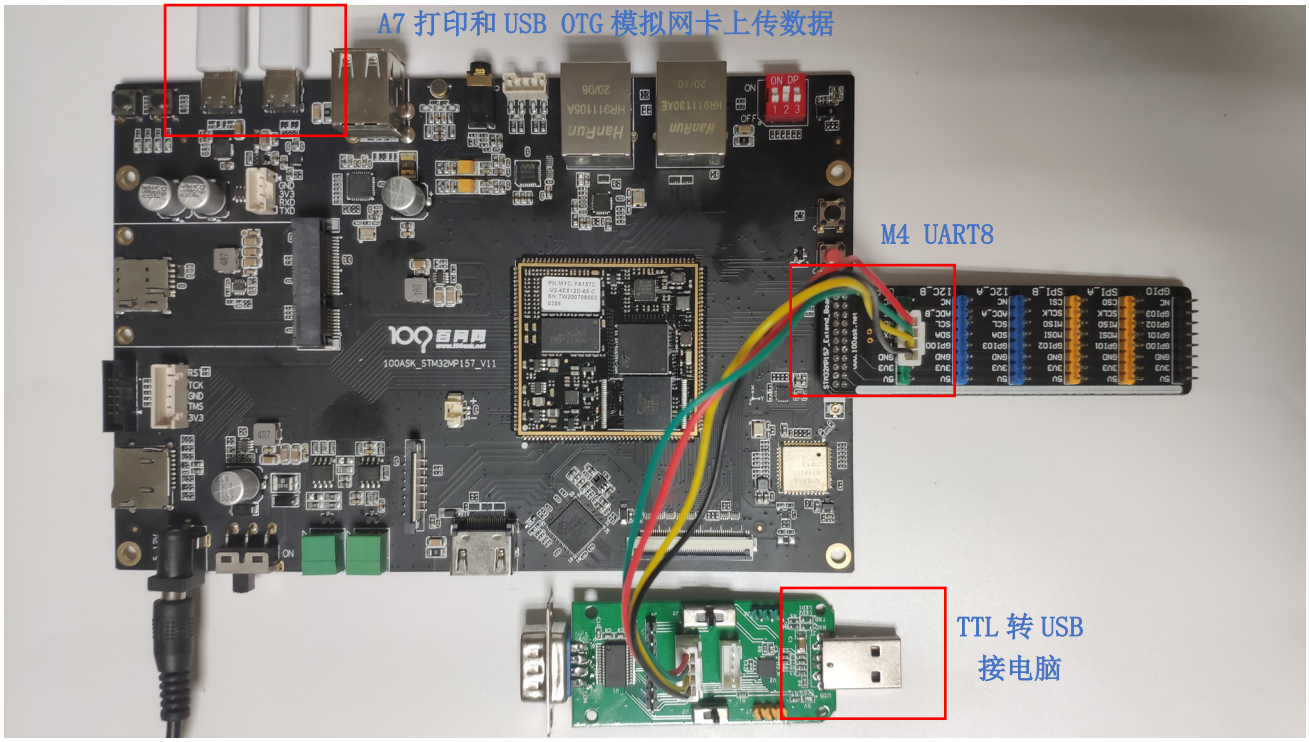


图 3.2.7 连接实物图

然后将前面生成“rpmsg_user_CM4.elf”，拖动上传到开发板 Linux 系统里，并执行“cp rpmsg_user_CM4.elf /lib/firmware/”复制到“/lib/firmware/”目录里，如图 3.2.8 所示。

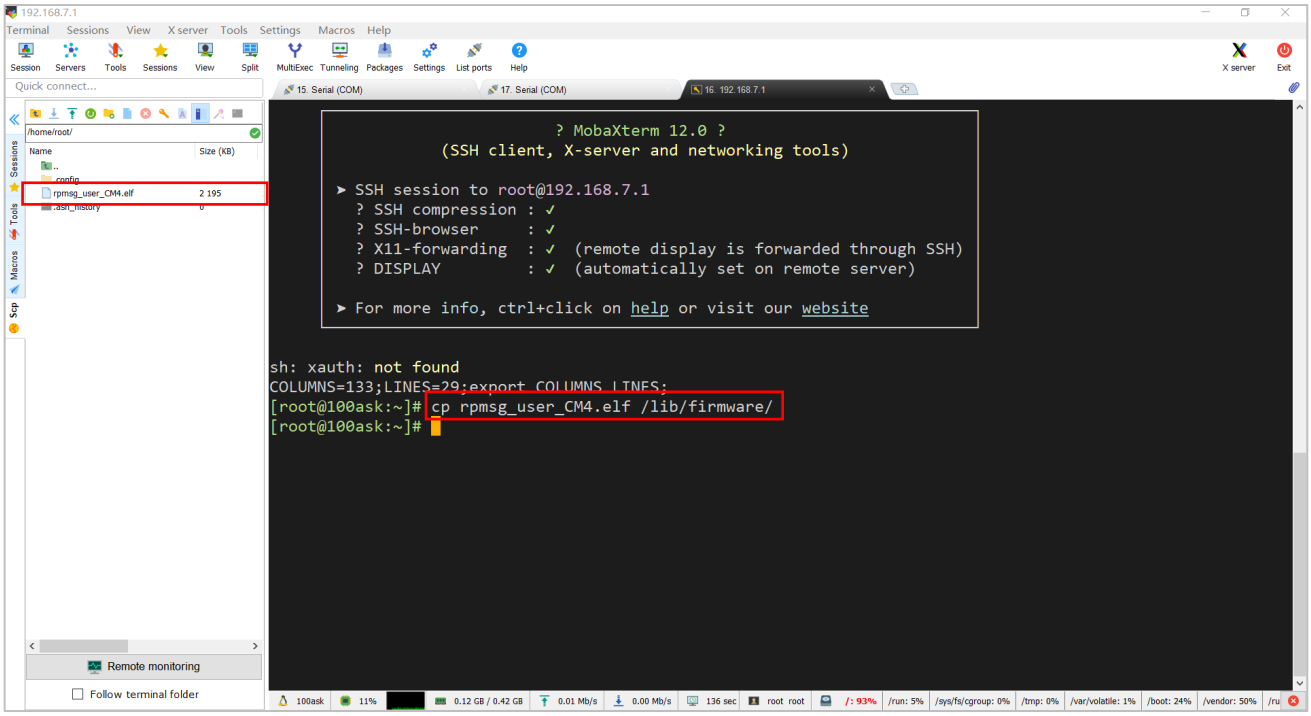


图 3.2.8 上传、ELF 文件到指定目录

然后再打开 M4 对应的串口，为了方便同时查看 A7 和 M4 的打印信息，在 MobaXterm 上方选择 2 分屏显示，如图 3.2.9 所示。

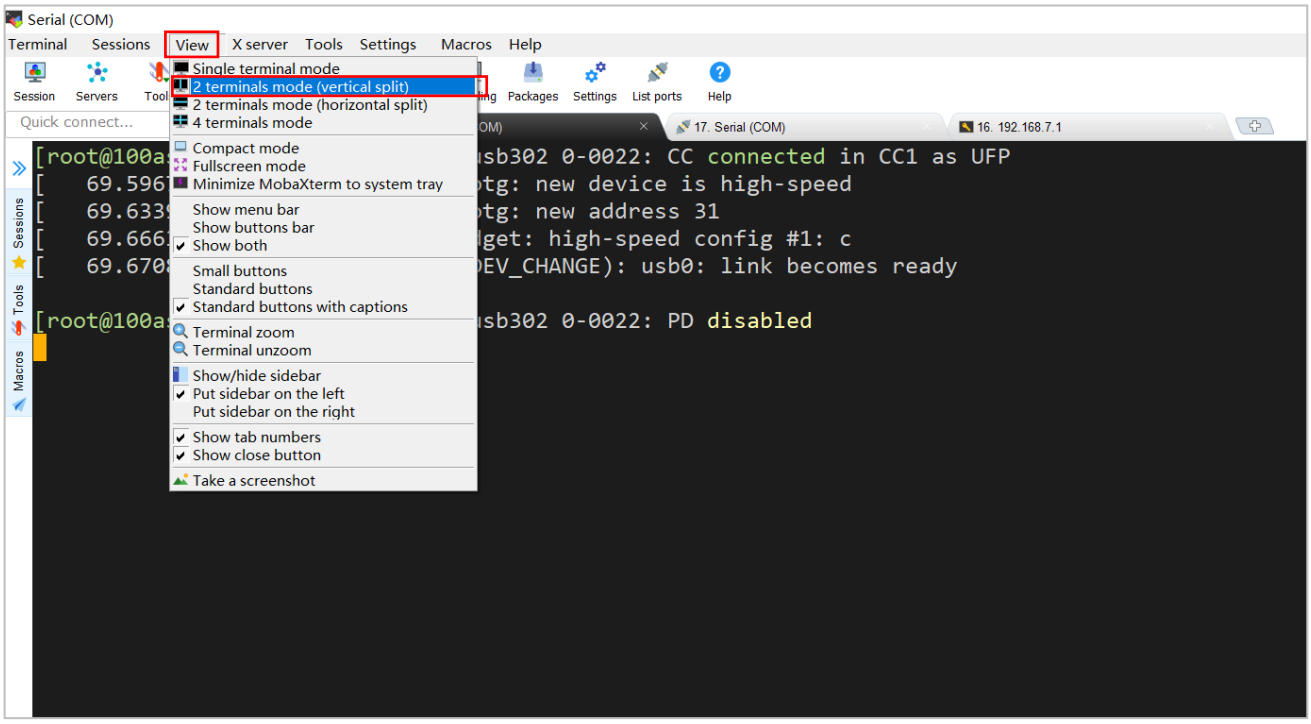


图 3.2.9 MobaXterm 设置二分屏幕

效果如图 3.2.10 所示，左边是 Linux 的串口终端，右边是 M4 对应的 UART8 调试串口。

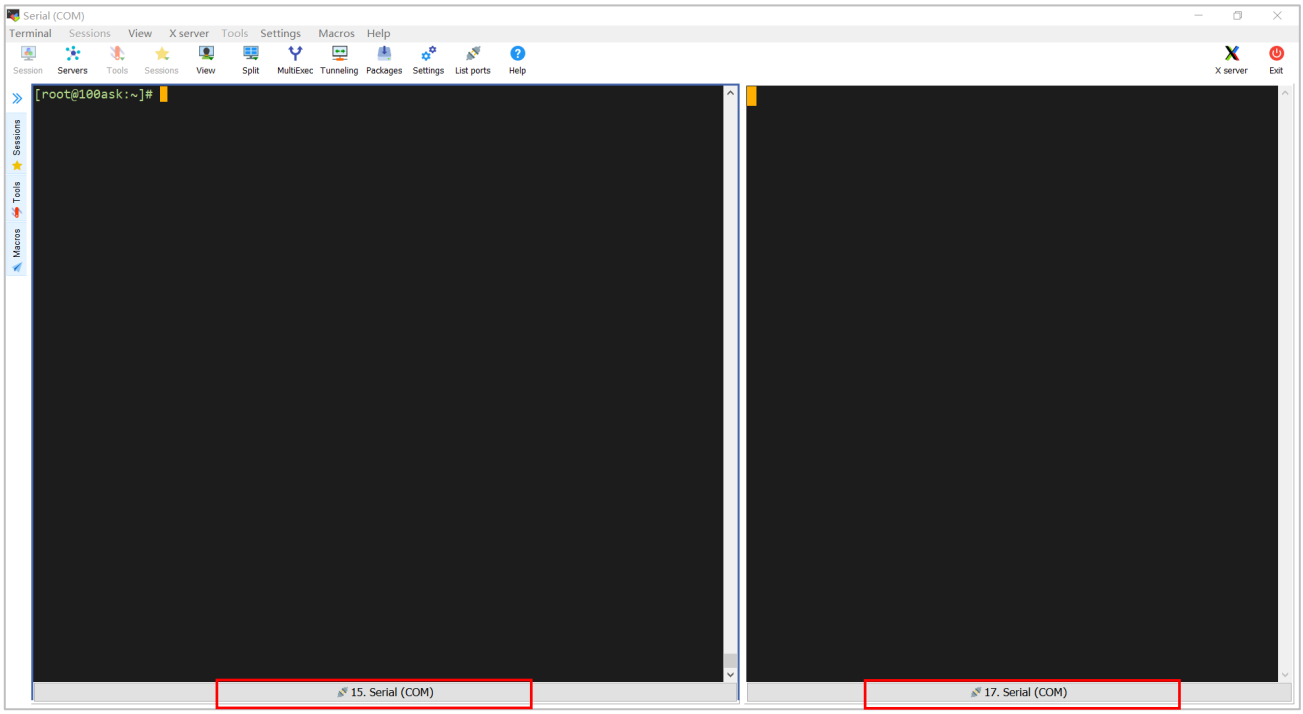


图 3.2.10 同时查看 A7 和 M4 打印

执行以下命令加载 M4：

```
[root@100ask:~]$ echo rpmsg_user_CM4.elf > /sys/class/remoteproc/remoteproc0/firmware
[root@100ask:~]$ echo start > /sys/class/remoteproc/remoteproc0/state
```

随后打印可以看到启动了 M4，并创建两个虚拟串口通道，生成了新节点“/dev/ttyRPMSG0”和“/dev/ttyRPMSG1”，右边 M4 也打印了启动信息，如图 3.2.11 所示。

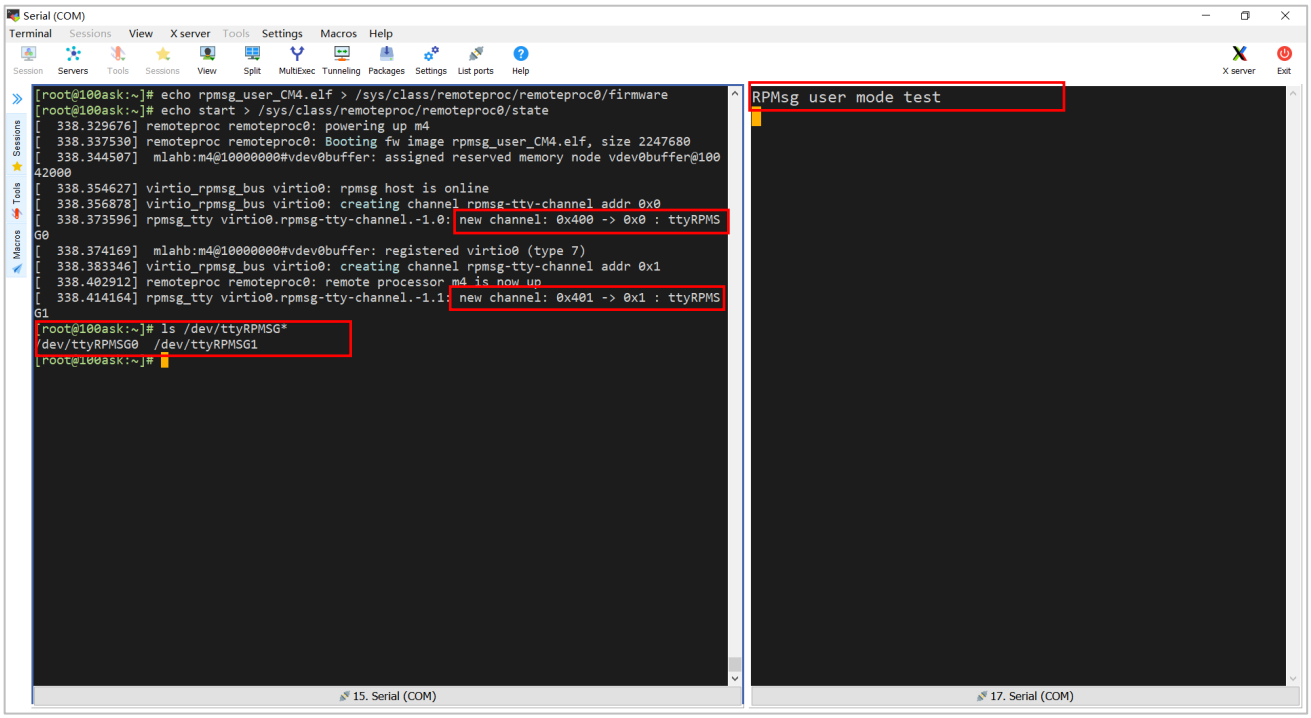


图 3.2.11 启动 M4

接着需要输入以下两个命令设置下生成的虚拟串口“/dev/ttyRPMSG0”。

```
[root@100ask:~]$ stty -onlcr -echo -F /dev/ttyRPMSG0
[root@100ask:~]$ cat /dev/ttyRPMSG0 &
```

第一条命令里的“-onlcr”是不将 NL 字符映射为 CR-NL 字符，就是说发送给 M4 的数据，不会自动加上回车，不然这里发送“led_on”，M4 收到的为“led_on\n\r”， “-echo”是禁止回显，以方便查看接收的字符。

第二条命令是后台运行查看“/dev/ttyRPMSG0”，一旦“/dev/ttyRPMSG0”收到数据，将立刻打印到当前终端。

接着是向“/dev/ttyRPMSG0”写入预定义的指令，M4 收到指令便会控制 LED 灯，并发送执行结果给 A7，如图 3.2.12 所示。

```
[root@100ask:~]$ echo "*led_on" > /dev/ttyRPMSG0
[root@100ask:~]$ echo "*led_off" > /dev/ttyRPMSG0
```

执行以上两个命令，观察开发板 LED，可以看到对应 LED 亮灭。

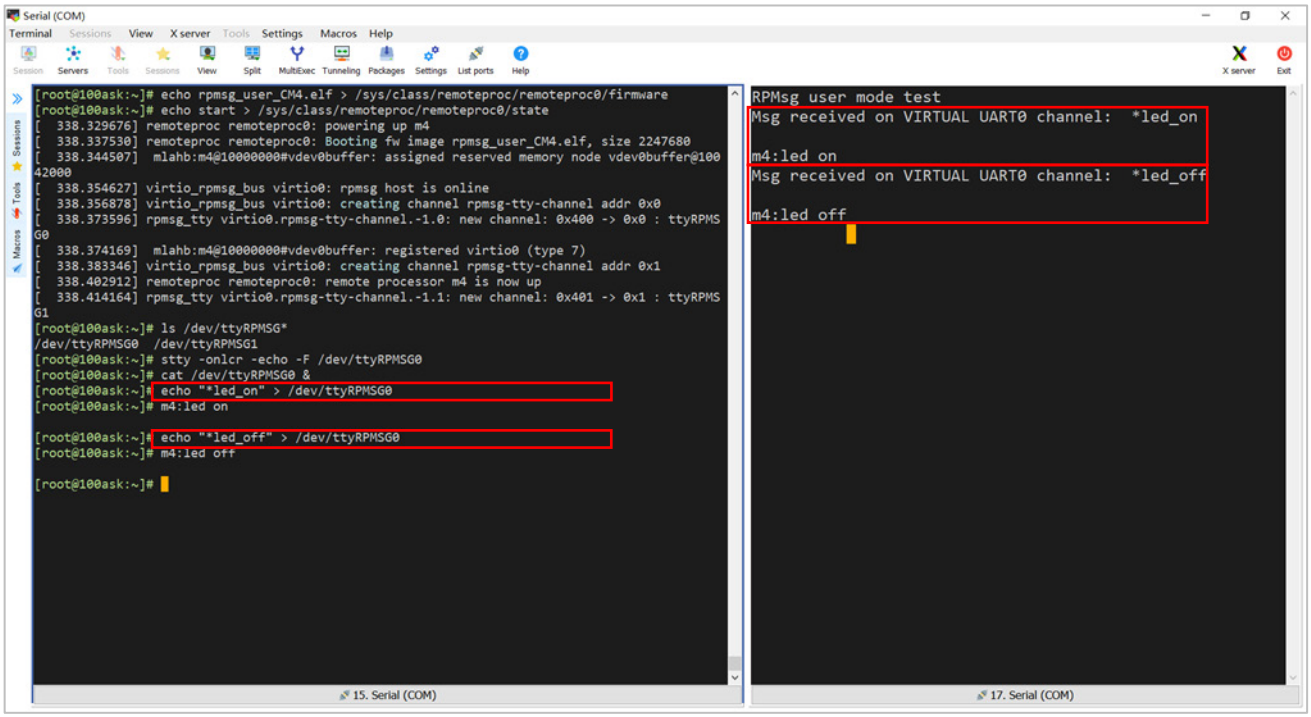


图 3.2.12 A7 控制 M4 的 LED 灯

然后再测试 A7 和 M4 相互唤醒，先设置“/dev/ttyRPMSG1”。

```
[root@100ask:~]$ stty -onlcr -echo -F /dev/ttyRPMSG1
[root@100ask:~]$ cat /dev/ttyRPMSG1 &
```

然后向“/dev/ttyRPMSG1”写入“*stop”，M4 随后卡在打印“Going to CStop mode”后，接着向“/dev/ttyRPMSG1”写入：“wakeup”(任意字符即可)，M4 随后打印“Leaving CStop mode”，即实现了 A7 控制 M4 的休眠和唤醒，效果如图 3.2.13 所示。

```
[root@100ask:~]$ echo "*stop" > /dev/ttyRPMSG1
[root@100ask:~]$ echo "wakeup" >/dev/ttyRPMSG1
```

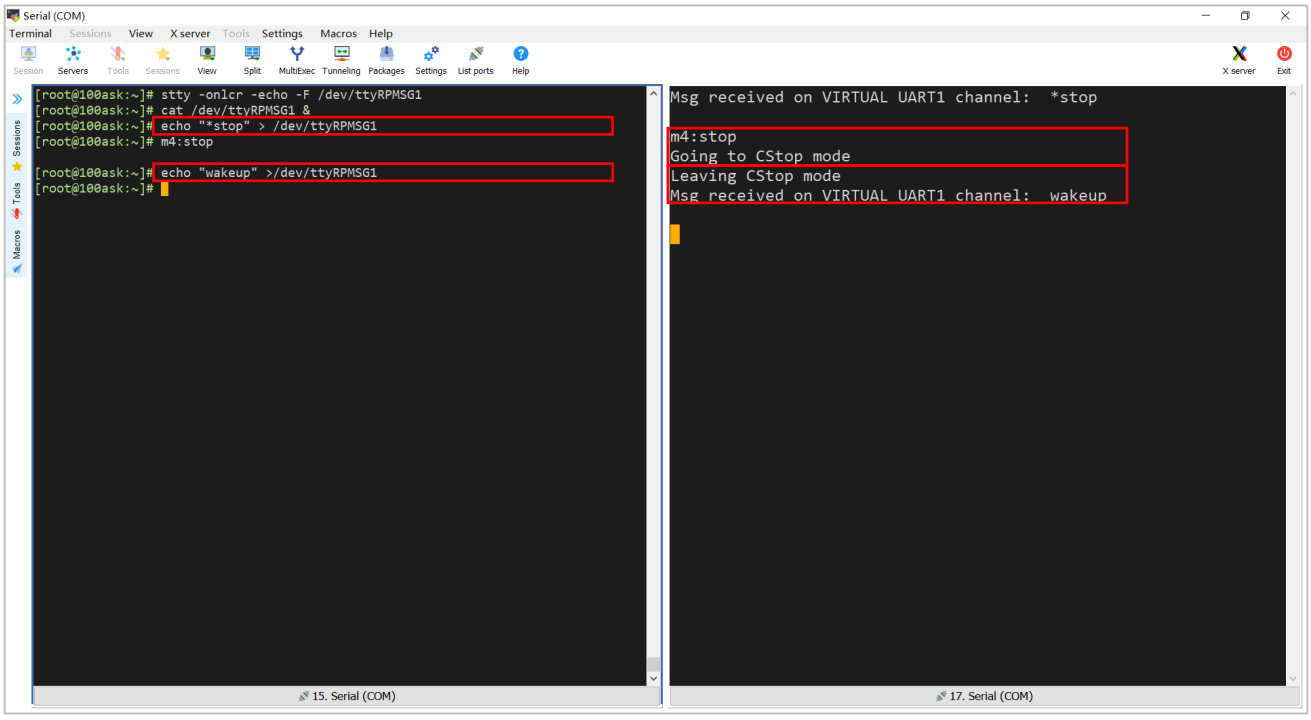


图 3.2.13 A7 控制 M4 休眠和唤醒

再测试 M4 唤醒 A7。先使能 A7 唤醒，然后向“/dev/ttyRPMSG1”写入“*delay”，M4 收到指令后，20S 后会向 A7 发数据，从而唤醒 A7。此时控制 A7 进入休眠状态，20S 后，A7 被唤醒。

```
[root@100ask:~]$ echo enabled > /sys/devices/platform/soc/4c001000.mailbox/power/wakeup //使能唤醒
[root@100ask:~]$ echo "*delay" > /dev/ttyRPMSG1  //通知M4一会唤醒自己
[root@100ask:~]$ echo mem > /sys/power/state  //A7进入休眠
```

注意，由于当前 Linux 休眠唤醒驱动 bug，A7 唤醒会复位重启，这里后期解决后，用户可再进行本实验，但这不影响本小结主题，A7 和 M4 通信完成。

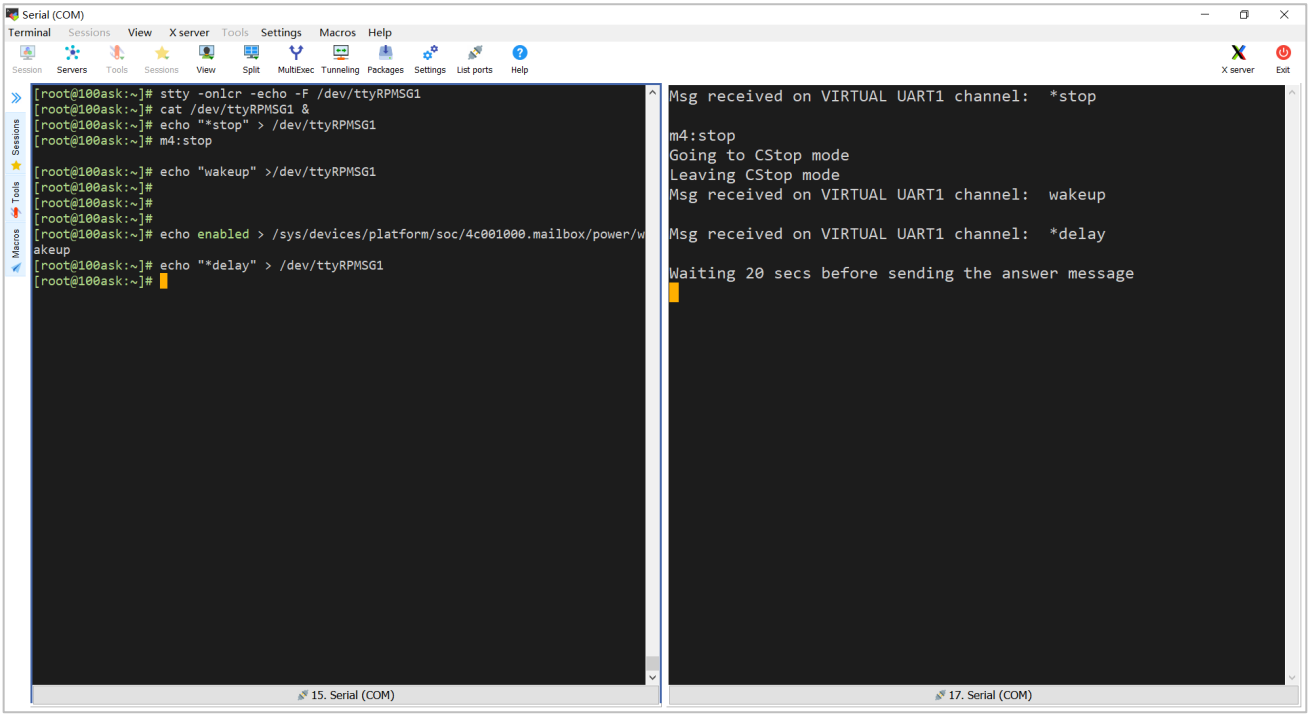


图 3.2.14 M4 唤醒 A7(未成功)

# 3.3 A7_M4 内核态通信

- 设计需求

这里假设需求为 A7 和 M4 直接发送大量数据。

## 3.3.1 基础知识

前面将 A7 和 M4 通信，封装成了虚拟串口。如果需要传输大量数据，且对性能有一定要求，可尝试爸呢小结示例，在 Linux 驱动层和 M4 数据传输。

## 3.3.2 硬件设计

和上一小结一致。

## 3.3.3 MX 设置

参考前面的示例，依次在 STM32CubeMX 配置 UART8、IPCC 和 OPENAMP，然后生成初始化代码。

## 3.3.4 代码设计

Linux 内核提供了一个驱动示例“linux-origin_master/samples/rpmsg/rpmsg_client_sample.c”，在此驱动上简单修改了打印内容，方便查看。

```
/*
 * Remote processor messaging - sample client driver
 *
 * Copyright (C) 2011 Texas Instruments, Inc.
 * Copyright (C) 2011 Google, Inc.
 *
 * Ohad Ben-Cohen <ohad@wizery.com>
 * Brian Swetland <swetland@google.com>
 *
 * This software is licensed under the terms of the GNU General Public
 * License version 2, as published by the Free Software Foundation, and
 * may be copied, distributed, and modified under those terms.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 */

#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/rpmsg.h>
```

```
#define MSG     "hello world! A7->M4"
#define MSG_LIMIT   100

struct instance_data {
    int rx_count;
};

static int rpmsg_sample_cb(struct rpmsg_device *rpdev, void *data, int len,
                           void *priv, u32 src)
{
    int ret;
    struct instance_data *idata = dev_get_drvdata(&rpdev->dev);

    //dev_info(&rpdev->dev, "incoming msg %d (src: 0x%x)\n", ++idata->rx_count, src);
    //print_hex_dump(KERN_DEBUG, __func__, DUMP_PREFIX_NONE, 16, 1, data, len,  true);

    printk(KERN_DEBUG "received_rpmsg: %p %d \n", data, ++idata->rx_count);


    /* samples should not live forever */
    if (idata->rx_count >= MSG_LIMIT) {
        //dev_info(&rpdev->dev, "goodbye!\n");
        return 0;
    }

    /* send a new message now */
    ret = rpmsg_send(rpdev->ept, MSG, strlen(MSG));
    if (ret)
        dev_err(&rpdev->dev, "rpmsg_send failed: %d\n", ret);
    else
        printk(KERN_DEBUG "send_rpmsg: %s \n", MSG);

    return 0;
}

static int rpmsg_sample_probe(struct rpmsg_device *rpdev)
{
    int ret;
    struct instance_data *idata;

    dev_info(&rpdev->dev, "new channel: 0x%x -> 0x%x!\n",
                    rpdev->src, rpdev->dst);

    idata = devm_kzalloc(&rpdev->dev, sizeof(*idata), GFP_KERNEL);
```

```
    if (!idata)
        return -ENOMEM;

    dev_set_drvdata(&rpdev->dev, idata);

    /* send a message to our remote processor */
    ret = rpmsg_send(rpdev->ept, MSG, strlen(MSG));
    if (ret) {
        dev_err(&rpdev->dev, "rpmsg_send failed: %d\n", ret);
        return ret;
    }
    else
        printk(KERN_DEBUG "send_rpmsg: %s \n", MSG);


    return 0;
}

static void rpmsg_sample_remove(struct rpmsg_device *rpdev)
{
    dev_info(&rpdev->dev, "rpmsg sample client driver is removed\n");
}

static struct rpmsg_device_id rpmsg_driver_sample_id_table[] = {
    { .name = "rpmsg-client-sample" },
    { },
};
MODULE_DEVICE_TABLE(rpmsg, rpmsg_driver_sample_id_table);

static struct rpmsg_driver rpmsg_sample_client = {
    .drv.name   = KBUILD_MODNAME,
    .id_table   = rpmsg_driver_sample_id_table,
    .probe      = rpmsg_sample_probe,
    .callback   = rpmsg_sample_cb,
    .remove     = rpmsg_sample_remove,
};
module_rpmsg_driver(rpmsg_sample_client);

MODULE_DESCRIPTION("Remote processor messaging sample client driver");
MODULE_LICENSE("GPL v2");
```

驱动比较简单，核心是“rpmsg_sample_cb()“，rpmsg 收到数据后将回调该函数。从“data”获得 M4 发来的数据，通过“rpmsg_send()”将数据发给 M4。

将该驱动和 Makefile 上传到 Linux 主机，根据内核源码路径，修改 Makefile，编译即可得到“rpmsg_client_sample.ko”，驱动的开发流程参考 Linux 开发手册，这里不再赘述。同时，本示例也提供“rpmsg_client_sample.ko”文件，用户可以直接使用。

使用 STM32CubeMX 生成初始化代码后，下面将编写程序从 A7 接收数据，并发送数据。

- Step 1：建一个 rpmsg 通道

```
/* Create an rpmsg channel to communicate with the Master processor CPU1(CA7) */
OPENAMP_create_endpoint(&resmgr_ept, RPMSG_SERVICE_NAME, RPMSG_ADDR_ANY, rx_callback, NULL);
```

注意这里的“RPMSG_SERVICE_NAME”和驱动里“rpmsg_device_id”结构体的“.name”的名字一样。之后，一旦 rpmsg 通道数据，将回调 rx_callback()函数。

- Step 2：编写接收回调函数

此函数里，需要将接收的数据复制到用户内存，并修改接收标志位。

```
static int rx_callback(struct rpmsg_endpoint *rp_chnl, void *data, size_t len, uint32_t src, void *priv)
{
  /* copy received msg, and raise a flag  */
  memcpy(received_rpmsg, data, len > sizeof(received_rpmsg) ? sizeof(received_rpmsg) : len);
  printf("received_rpmsg=%s\r\n", received_rpmsg);
  rx_status = SET;
  return 0;
}
```

- Step 3:主函数轮询 RPMsg 消息

“OPENAMP_check_for_message()”查询 MailBox 状态。当收到数据时，“rx_callback()”会保存好收到数据，然后修改“rx_status”标志位。主函数里发现“rx_status”标志位发生变化时，即可获取接收的数据。

```
while (1)
{
   OPENAMP_check_for_message();
   /* USER CODE END WHILE */
   if (rx_status == SET)
   {
   /* Message received: send back a message anwser */
   rx_status = RESET;
   /*rpmsg 收到数据*/
   }
      /*
```

- Step 3:向 M4 发送数据

使用“OPENAMP_send()”向 A7 发送数据。

```
   if (rx_status == SET)
   {
     /* Message received: send back a message anwser */
     rx_status = RESET;
     if (++count < 100)
```

```
    sprintf((char *)msg, "hello world! M4->A7 %02ld", count);
  else
    strcpy((char *)msg, "goodbye!");
  if (OPENAMP_send(&resmgr_ept, msg, strlen((char *)msg) + 1) < 0)
  {
    printf("Failed to send message\r\n");
    Error_Handler();
  }
  else
    printf("send_rpmsg=%s\r\n", msg);
  }
```

### 3.3.5 实验效果

硬件连接和上一实验保持不变。编译得到“rpmsg_kernel_CM4.elf”，将它和驱动“rpmsg_client_sample.ko”一起上传到开发板 Linux。然后加载驱动，将“rpmsg_kernel_CM4.elf”复制到“/lib/firmware/”目录，为了查看驱动的打印信息，这里还要修改内核的打印等级，设置如所示。

```
[root@100ask:~]$ insmod rpmsg_client_sample.ko
[root@100ask:~]$ cp rpmsg_kernel_CM4.elf /lib/firmware/
[root@100ask:~]$ echo 8 > /proc/sys/kernel/printk
```

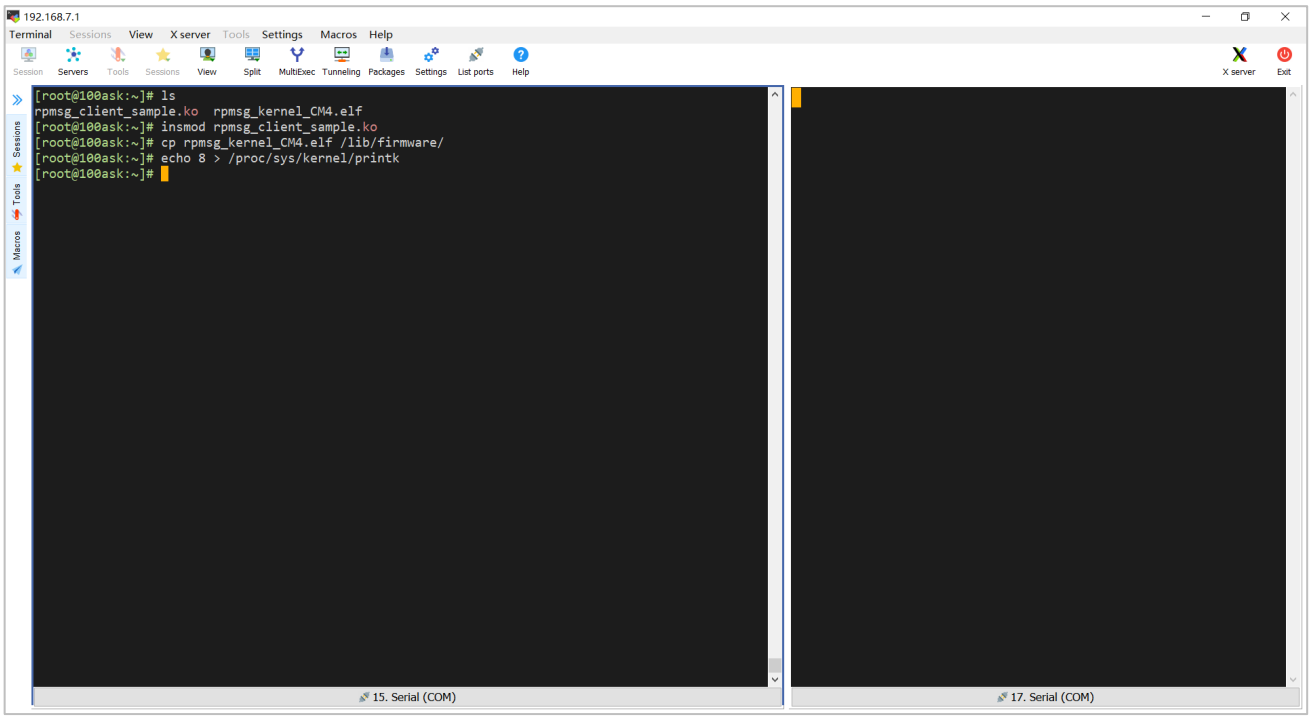


图 3.3.1 准备工作

接着启动固件，可以看到如图 3.3.2 所示打印。可以看到启动 M4 后，A7 向 M4 发送“hello world! A7->M4”，同时收到 M4 发过来的“hello world! M4->A7”。M4 收到 A7 发过来的“hello world! A7->M4”，同时向 A7 发送“hello world! M4->A7”，在极短时间里相互之间发送了 100 次。

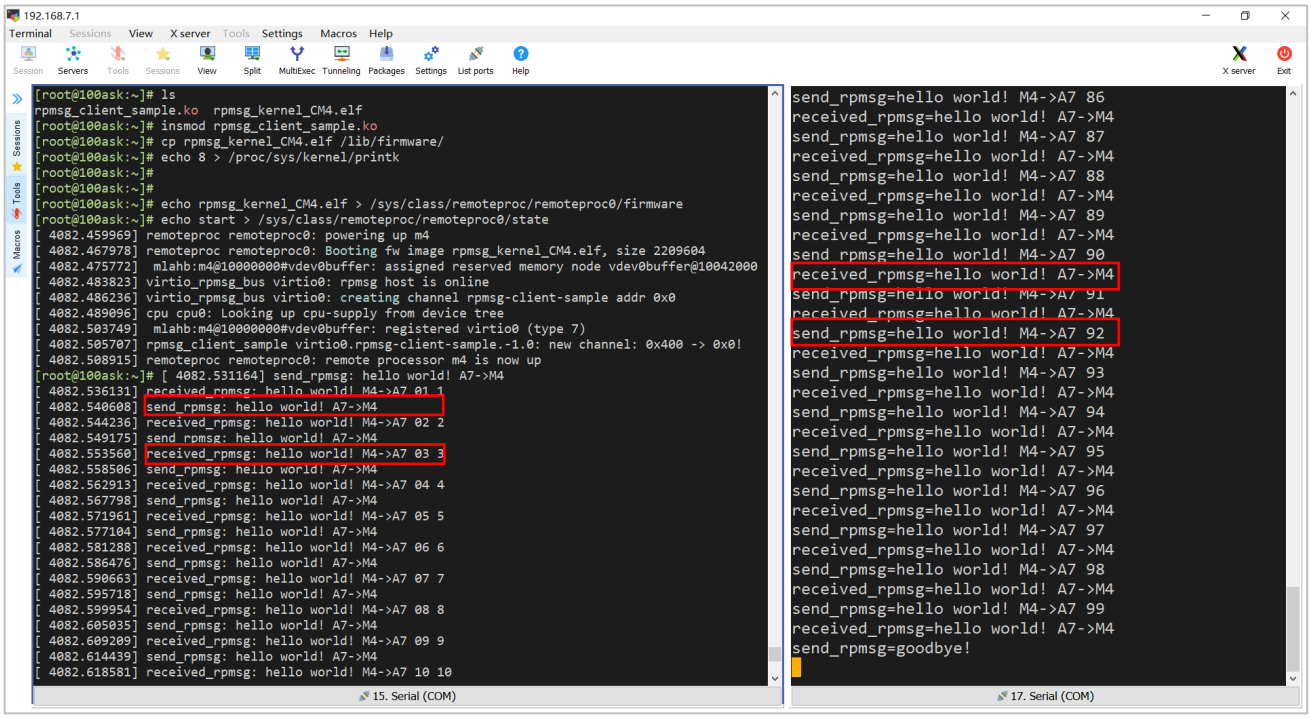
192.168.7.1
Terminal Sessions View X server Tools Settings Macros Help
[root@100ask:~]# ls
rpmsg_client_sample.ko rpmsg_kernel_CM4.elf
[root@100ask:~]# insmod rpmsg_client_sample.ko
[root@100ask:~]# cp rpmsg_kernel_CM4.elf /lib/firmware/
[root@100ask:~]# echo 8 > /proc/sys/kernel/printk
[root@100ask:~]#
[root@100ask:~]#
[root@100ask:~]# echo rpmsg_kernel_CM4.elf > /sys/class/remoteproc/remoteproc0/firmware
[root@100ask:~]# echo start > /sys/class/remoteproc/remoteproc0/state
[ 4082.459969] remoteproc remoteproc0: powering up m4
[ 4082.467978] remoteproc remoteproc0: Booting fw image rpmsg_kernel_CM4.elf, size 2209604
[ 4082.475772] mlahb:m4@10000000#vdev0buffer: assigned reserved memory node vdev0buffer@10042000
[ 4082.483823] virtio_rpmsg_bus virtio0: rpmsg host is online
[ 4082.486236] virtio_rpmsg_bus virtio0: creating channel rpmsg-client-sample addr 0x0
[ 4082.489096] cpu cpu0: Looking up cpu-supply from device tree
[ 4082.503749] mlahb:m4@10000000#vdev0buffer: registered virtio0 (type 7)
[ 4082.505707] rpmsg_client_sample virtio0.rpmsg-client-sample.-1.0: new channel: 0x400 -> 0x0!
[ 4082.508915] remoteproc remoteproc0: remote processor m4 is now up
[root@100ask:~]# [ 4082.531164] send_rpmsg: hello world! A7->M4
[ 4082.536131] received_rpmsg: hello world! M4->A7 01 1
[ 4082.540608] send_rpmsg: hello world! A7->M4
[ 4082.544236] received_rpmsg: hello world! M4->A7 02 2
[ 4082.549175] send_rpmsg: hello world! A7->M4
[ 4082.553560] received_rpmsg: hello world! M4->A7 03 3
[ 4082.558506] send_rpmsg: hello world! A7->M4
[ 4082.562913] received_rpmsg: hello world! M4->A7 04 4
[ 4082.567798] send_rpmsg: hello world! A7->M4
[ 4082.571961] received_rpmsg: hello world! M4->A7 05 5
[ 4082.577104] send_rpmsg: hello world! A7->M4
[ 4082.581288] received_rpmsg: hello world! M4->A7 06 6
[ 4082.586476] send_rpmsg: hello world! A7->M4
[ 4082.590663] received_rpmsg: hello world! M4->A7 07 7
[ 4082.595718] send_rpmsg: hello world! A7->M4
[ 4082.599954] received_rpmsg: hello world! M4->A7 08 8
[ 4082.605035] send_rpmsg: hello world! A7->M4
[ 4082.609209] received_rpmsg: hello world! M4->A7 09 9
[ 4082.614439] send_rpmsg: hello world! A7->M4
[ 4082.618581] received_rpmsg: hello world! M4->A7 10 10
15. Serial (COM)
send_rpmsg=hello world! M4->A7 86
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 87
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 88
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 89
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 90
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 91
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 92
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 93
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 94
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 95
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 96
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 97
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 98
received_rpmsg=hello world! A7->M4
send_rpmsg=hello world! M4->A7 99
received_rpmsg=hello world! A7->M4
send_rpmsg=goodbye!
17. Serial (COM)

图 3.3.2 通信打印